CORE JAVA

Complete Java Notes

From JVM fundamentals to Java 25: OOP, collections, concurrency, streams, design patterns and modern APIs

18 phases + appendix	Merged from Java_Programming_Notes + Java_Short_Notes
Verified	Code examples run on JDK 21 (except Java 22+ features and snippets needing external libraries like Jackson or Appium)
Current to	Java 25 LTS (features from 8, 11, 17, 21 and 25 marked)
Corrections	Inaccuracies in the original notes fixed and logged (Appendix)
Format	Interview Q&A, Predict-the-Output and Quick Revision in every phase

Prepared for Rohit · November 2026

Contents

Phase 1: Java Foundations & the JVM 8

1.1 Where Java Fits in Full-Stack Development 8
1.2 History, Editions and Versions 9
1.3 Features of Java (WORA: Write Once, Run Anywhere) 10
1.4 JDK vs JRE vs JVM 11
1.5 Environment Setup (Windows) 11
1.6 Your First Java Program 12
1.7 The main() Method Explained 12
1.8 Structure of a Java Program 13
1.9 How a Java Program Runs 13
1.10 JVM Architecture 14
1.11 Common Beginner Errors and Fixes 16
1.12 Interview Questions 16
1.13 Quick Revision 17

Phase 2: Language Basics 18

2.1 Variables 18
2.2 Data Types 19
2.3 Type Conversion and Casting 21
2.4 Reading Console Input 23
2.5 Comments 23
2.6 Identifiers and Keywords 24
2.7 Naming Conventions 25
2.8 Operators 26
2.9 Operator Precedence 28
2.10 Predict the Output 28
2.11 Interview Questions 28
2.12 Quick Revision 29

Phase 3: Control Flow & Arrays 30

3.1 Control Statements Overview 30
3.2 if, if-else and the else-if Ladder 31
3.3 The switch Statement (classic) 31
3.4 Modern switch: Expressions and Pattern Matching 33
3.5 Loops 34
3.6 break, continue, return and Labels 35
3.7 Nested Loops and Pattern Programs 35
3.8 Reading Input from the Keyboard 36
3.9 Arrays: Basics 36
3.10 Iterating and Printing Arrays 37
3.11 The java.util.Arrays Utility Class 38
3.12 Copying Arrays 39
3.13 Multidimensional and Jagged Arrays 39
3.14 Arrays vs ArrayList 40
3.15 Classic Array Programs 40
3.16 Predict the Output 41
3.17 Interview Questions 42
3.18 Quick Revision 42

Phase 4: Strings, StringBuilder & Regex 44

4.1 What Is a String? 44
4.2 Creating Strings and the String Constant Pool 44
4.3 Immutability 45

4.4 Comparing Strings 46
4.5 The String API 47
4.6 Formatting Strings 49
4.7 Text Blocks (Java 15+) 49
4.8 StringBuilder and StringBuffer 49
4.9 StringJoiner and String.join 51
4.10 Regular Expressions (java.util.regex) 51
4.11 Classic String Programs 53
4.12 Predict the Output 54
4.13 Interview Questions 55
4.14 Quick Revision 55

Phase 5: Methods, Classes & Objects 57

5.1 Procedural vs Object-Oriented Programming 57
5.2 Classes and Objects 57
5.3 Objects in Memory 59
5.4 Garbage Collection Basics 59
5.5 Variables in Depth: Instance, Static and Local 60
5.6 Methods 61
5.7 Java Is Always Pass-by-Value 63
5.8 Variable Arguments (Varargs) 63
5.9 Recursion 64
5.10 Constructors 64
5.11 The this Keyword 66
5.12 The static Keyword 66
5.13 Initialization Blocks and Execution Order 67
5.14 Predict the Output 68
5.15 Interview Questions 68
5.16 Quick Revision 69

Phase 6: OOP I: Encapsulation, Packages & Inheritance 70

6.1 Encapsulation 70
6.2 Access Modifiers 72
6.3 Non-Access Modifiers at a Glance 74
6.4 Packages 74
6.5 Inheritance 76
6.6 Constructors and Inheritance (Constructor Chaining) 78
6.7 The super Keyword 79
6.8 IS-A vs HAS-A (Inheritance vs Composition) 80
6.9 Predict the Output 80
6.10 Interview Questions 80
6.11 Quick Revision 81

Phase 7: OOP II: Polymorphism, Abstraction & Interfaces 83

7.1 Polymorphism 83
7.2 Method Overloading 83
7.3 Method Overriding 84
7.4 Dynamic Binding (Runtime Polymorphism) 86
7.5 Reference Casting: Upcasting and Downcasting 87
7.6 The final Keyword 88
7.7 Abstraction and Abstract Classes 88
7.8 Interfaces 89
7.9 Abstract Class vs Interface 92
7.10 Marker and Functional Interfaces 92
7.11 Predict the Output 93
7.12 Interview Questions 93

7.13 Quick Revision 94

Phase 8: Object Class, Wrappers, Math & Immutability 95

8.1 java.lang.Object: The Root of Every Class 95
8.2 toString() 96
8.3 equals(): Logical Equality 96
8.4 hashCode(): Must Agree with equals() 97
8.5 getClass() and a Glimpse of Reflection 98
8.6 clone(): Shallow vs Deep Copy 99
8.7 finalize() and Resource Cleanup 100
8.8 Wrapper Classes 100
8.9 The Math Class 102
8.10 BigDecimal and BigInteger 103
8.11 Immutable Classes 103
8.12 Predict the Output 105
8.13 Interview Questions 105
8.14 Quick Revision 106

Phase 9: Exception Handling 107

9.1 What Is an Exception? 107
9.2 The Throwable Hierarchy 108
9.3 Checked vs Unchecked Exceptions 109
9.4 try and catch 109
9.5 The finally Block 110
9.6 try-with-resources (Java 7) 112
9.7 throw: Raising an Exception Yourself 113
9.8 throws: Declaring Exceptions 113
9.9 Exception Propagation 114
9.10 Useful Throwable Methods 115
9.11 Custom (User-Defined) Exceptions 115
9.12 Exception Chaining (Wrapping) 116
9.13 Exceptions and Overriding (Recap) 117
9.14 Common Exceptions: Cause and Fix 118
9.15 Important Errors 119
9.16 Best Practices 120
9.17 Predict the Output 120
9.18 Interview Questions 121
9.19 Quick Revision 122

Phase 10: Nested Classes, Enums & Generics 123

10.1 Nested Classes: The Four Kinds 123
10.2 Member Inner Class 124
10.3 Static Nested Class 124
10.4 Local Inner Class 125
10.5 Anonymous Inner Class 125
10.6 Enums 127
10.7 Generics 129
10.8 Type Erasure 132
10.9 Predict the Output 133
10.10 Interview Questions 134
10.11 Quick Revision 135

Phase 11: Multithreading & Concurrency 136

11.1 Multitasking, Processes and Threads 136
11.2 Creating Threads 137
11.3 Thread Lifecycle 138

11.4 Important Thread Methods 139
11.5 Race Conditions: Why Synchronization Is Needed 140
11.6 synchronized 140
11.7 volatile and Visibility 142
11.8 Atomic Variables 142
11.9 Inter-Thread Communication: wait(), notify(), notifyAll() 142
11.10 Deadlock, Livelock and Starvation 143
11.11 Explicit Locks: ReentrantLock and ReadWriteLock 144
11.12 The Executor Framework and Thread Pools 145
11.13 CompletableFuture (Java 8): Asynchronous Pipelines 146
11.14 Concurrent Collections and Synchronizers 147
11.15 Virtual Threads (Java 21) 147
11.16 Best Practices 148
11.17 Predict the Output 149
11.18 Interview Questions 149
11.19 Quick Revision 150

Phase 12: File I/O, NIO.2 & Serialization 152

12.1 Java I/O Overview 152
12.2 The File Class (java.io) 153
12.3 Byte Streams 154
12.4 Character Streams and Buffering 154
12.5 Choosing the Right Class 155
12.6 Other Useful java.io Classes 155
12.7 NIO.2: Path and Files (Java 7+) 156
12.8 Properties Files: External Configuration 158
12.9 Serialization 158
12.10 Best Practices 160
12.11 Predict the Output 160
12.12 Interview Questions 160
12.13 Quick Revision 161

Phase 13: Collections I: List, Set, Queue & Sorting 163

13.1 Why Collections? 163
13.2 The Collection Hierarchy 163
13.3 List: ArrayList 164
13.4 List: LinkedList, Vector and Stack 165
13.5 Iterating Collections 166
13.6 Set: HashSet, LinkedHashSet, TreeSet 168
13.7 Queue and Deque 169
13.8 Comparable vs Comparator 170
13.9 The Collections Utility Class 171
13.10 Predict the Output 172
13.11 Interview Questions 172
13.12 Quick Revision 173

Phase 14: Collections II: Maps & Concurrent Collections 174

14.1 The Map Interface 174
14.2 HashMap Internals 175
14.3 Custom Objects as Keys 177
14.4 LinkedHashMap: Predictable Order and an LRU Cache 177
14.5 TreeMap: Sorted Keys and Range Queries 177
14.6 Other Map Implementations 178
14.7 Map Implementations Compared 179
14.8 Thread-Safe Collections 179
14.9 Sequenced Collections (Java 21) 180

14.10 Choosing the Right Collection 181
14.11 Big-O Cheat Sheet 181
14.12 Classic Map Programs 182
14.13 Predict the Output 182
14.14 Interview Questions 183
14.15 Quick Revision 184

Phase 15: JVM Deep Dive & Modern Java (8 → 25) 185

15.1 Heap Structure and Object Lifecycle 185
15.2 How Garbage Collection Works 186
15.3 Class Loading in Depth 187
15.4 Lambda Expressions (Java 8) 188
15.5 Built-in Functional Interfaces (java.util.function) 189
15.6 Method References 189
15.7 Streams API 190
15.8 Optional (Java 8) 192
15.9 Date and Time API 193
15.10 Records (Java 16) 195
15.11 Sealed Classes (Java 17) and Pattern Matching 195
15.12 Modern Java Feature Map (8 → 25) 196
15.13 Rapid-Fire Interview Round (all phases) 197
15.14 Quick Revision 198

Phase 16: Annotations & Reflection 199

16.1 What Are Annotations? 199
16.2 Built-in Annotations 199
16.3 Meta-Annotations 200
16.4 Writing Custom Annotations 201
16.5 Processing Annotations at Runtime 201
16.6 The Reflection API 203
16.7 Costs and Risks of Reflection 204
16.8 Interview Questions 205
16.9 Quick Revision 206

Phase 17: SOLID Principles & Design Patterns 207

17.1 Why Design Principles? 207
17.2 SOLID Principles 207
17.3 Design Pattern Categories 209
17.4 Creational Patterns 209
17.5 Structural Patterns 212
17.6 Behavioural Patterns 213
17.7 Patterns in the JDK and Frameworks 215
17.8 Patterns in Test Automation (Appium / Selenium) 215
17.9 Interview Questions 216
17.10 Quick Revision 217

Phase 18: HTTP Client, Networking & Internationalization 218

18.1 Networking Basics 218
18.2 TCP Sockets (java.net) 219
18.3 The Java 11 HttpClient (java.net.http) 219
18.4 A Local HTTP Server for Mocks and Tests 222
18.5 Internationalization (i18n) and Localization (l10n) 222
18.6 Interview Questions 224
18.7 Quick Revision 225

Appendix: Leftovers, Corrections Log & Revision Plan 226

A.1 Leftover Topics from the Original Notes 226

A.2 Corrections Log 227
A.3 How to Revise This Book 228

PHASE 1 OF 18

Java Foundations & the JVM

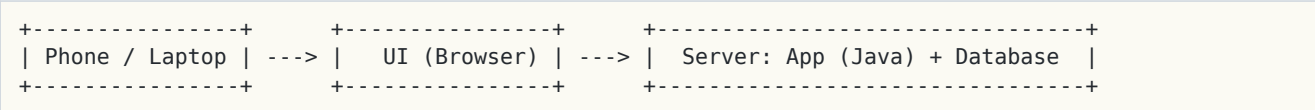
Where Java fits, history and versions, JDK/JRE/JVM, first program, main(), compilation flow and JVM architecture

IN THIS PHASE

1.1 Where Java Fits in Full-Stack Development
1.2 History, Editions and Versions
1.3 Features of Java (WORA: Write Once, Run Anywhere)
1.4 JDK vs JRE vs JVM
1.5 Environment Setup (Windows)
1.6 Your First Java Program
1.7 The main() Method Explained
1.8 Structure of a Java Program
1.9 How a Java Program Runs
1.10 JVM Architecture
1.11 Common Beginner Errors and Fixes
1.12 Interview Questions
1.13 Quick Revision

1.1 Where Java Fits in Full-Stack Development

A full-stack application has a **frontend** (what the user sees and clicks), a **backend** (hidden business logic that bridges the UI and the database), a **database**, and a **server/runtime** that hosts the application. When a user performs an action on the frontend, backend logic runs to handle that operation.



Layer	Responsibility	Common technologies
Frontend	User interface	HTML (structure), CSS, JavaScript (dynamic behaviour), Bootstrap (responsive), React (library), Angular (framework)
Backend	Business logic, APIs, security	**Java (Spring Boot)**, .NET, Python, Node.js, PHP
Database	Persistent data	Oracle, MySQL, PostgreSQL, SQL Server, MongoDB, Cassandra, HBase, Hive
Server / runtime	Runs the application	Tomcat, Jetty, WildFly/JBoss, WebLogic, WebSphere (IIS for .NET)

The Java project tool ecosystem

Category	Tools
Build & dependencies	Maven, Gradle
Version control	Git, GitHub, Bitbucket
Testing	JUnit 5, TestNG, Mockito (mocking), JaCoCo (coverage), JMeter (performance), Postman (API)
Code quality	SonarQube (static analysis / code review)
CI/CD	Jenkins, GitHub Actions (build → test → deploy)
Logging	SLF4J (facade) + Logback or Log4j2
API docs	Swagger / OpenAPI
Containers & orchestration	Docker, Kubernetes
Messaging & caching	Apache Kafka, Redis
Productivity	Project Lombok (removes boilerplate)
Monitoring & ops	Splunk, PuTTY, WinSCP
Project management	Jira (tasks, bugs, sprints)
Cloud	AWS, Azure, GCP, Oracle Cloud, IBM Cloud

1.2 History, Editions and Versions

- Developed at **Sun Microsystems** by a team led by **James Gosling**, starting in **1991** under the name **Oak**.
- Released publicly as **Java in 1995** (JDK 1.0 shipped January 1996).
- **Oracle acquired Sun in 2010**; Oracle now leads Java's development.
- **OpenJDK** is the open-source reference implementation. Free, production-ready OpenJDK builds include Eclipse Temurin, Amazon Corretto, Microsoft Build of OpenJDK and Azul Zulu. Oracle JDK has its own licence terms.
- Official API docs: `https://docs.oracle.com/en/java/javase/` (pick your version).

Editions

Edition	Old name	Used for
Java SE (Standard Edition)	J2SE	Core Java: language, collections, I/O, threads, networking. Base for everything else.
Jakarta EE (Enterprise Edition)	J2EE / Java EE	Enterprise web apps: Servlets, JPA, JAX-RS. Moved to the Eclipse Foundation in 2017; packages renamed `javax.*` → `jakarta.*` from Jakarta EE 9.
Java ME (Micro Edition)	J2ME	Small/embedded devices. Largely legacy.

> **NOTE**
> Android apps can be written in Java, but Android does **not** use Java ME or the standard JVM. It compiles to its own format and runs on the Android Runtime (ART).

Release cadence and LTS versions

Since Java 9 (2017), a new feature release ships **every six months**, and a **Long-Term Support (LTS)** release arrives every two years. Companies standardise on LTS versions.

LTS version	Year	Headline features you should know
Java 8	2014	Lambdas, Streams API, functional interfaces, `Optional`, `java.time`, default/static interface methods

LTS version	Year	Headline features you should know
Java 11	2018	`var` in lambda params, standard `HttpClient`, run single-file source (`java Hello.java`), `String.isBlank()/lines()/strip()/repeat()`
Java 17	2021	Records, sealed classes, pattern matching for `instanceof`, text blocks, switch expressions
Java 21	2023	Virtual threads, pattern matching for `switch`, record patterns, sequenced collections
Java 25	2025	Compact source files & instance `main`, flexible constructor bodies, scoped values, module import declarations

INTERVIEW

Spring Boot 3.x requires **Java 17 or later**. Many enterprise projects (especially in banking) still run Java 8 or 11, so know what changed in each LTS.

1.3 Features of Java (WORA: Write Once, Run Anywhere)

Feature	What it really means
Simple	Familiar C-style syntax, but no pointer arithmetic, no manual memory freeing, no multiple inheritance of classes, no operator overloading.
Platform independent	`javac` produces **bytecode** (`.class`), which runs on any OS that has a JVM.
Object-oriented	Programs are built from classes and objects; supports encapsulation, inheritance, polymorphism and abstraction. Code reuse through inheritance and composition.
Robust	Strong static typing, compile-time checks, exception handling, automatic garbage collection, array bounds checking.
Secure	No direct memory access, bytecode verifier, class-loader isolation, strong typing.
Multithreaded	Built-in threads, `java.util.concurrent`, and virtual threads (Java 21).
Distributed / networked	Built-in networking (`java.net`, `HttpClient`), RMI, used for distributed systems.
Portable	Primitive sizes are fixed (an `int` is always 32-bit) on every platform.
High performance	The JIT compiler turns frequently-run bytecode into native machine code.
Dynamic	Classes are loaded on demand at runtime; reflection lets code inspect classes at runtime.

CORRECTED FROM ORIGINAL NOTES

A few lines in the original notes were inaccurate:

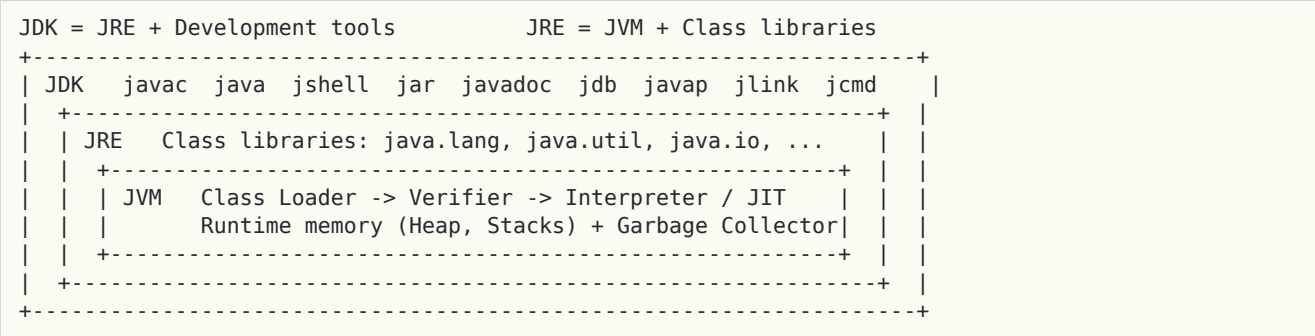
- "Secure: viruses can attack object code but not bytecode": Java's security comes from the verifier, no pointers, sandboxing and class-loader isolation, not from bytecode being immune.
- "Distributed: code need not execute sequentially": that describes **multithreading**. Distributed means built-in support for networked applications.
- "Java is an interpreted language": Java is **both compiled and interpreted**. `javac` compiles to bytecode; the JVM interprets it and JIT-compiles hot code to native code.

INTERVIEW

Is Java 100% object-oriented? **No.** It has primitive types (`int`, `char`, ...) that are not objects, and `static` members that belong to no object. Wrapper classes exist to treat primitives as objects.

1.4 JDK vs JRE vs JVM

- **JVM (Java Virtual Machine):** executes bytecode. Loads classes, verifies them, manages memory (heap, stacks, GC) and runs code via the interpreter and JIT.
- **JRE (Java Runtime Environment):** JVM **plus** the standard class libraries (`java.lang`, `java.util`, `java.io`, ...). Enough to **run** Java programs.
- **JDK (Java Development Kit):** JRE **plus** development tools (`javac`, `jar`, `javadoc`, `jdb`, ...). Needed to **develop** Java programs.



Useful JDK tools

Tool	Purpose
`javac`	Compiles `.java` source into `.class` bytecode
`java`	Launches the JVM to run a class, JAR or (Java 11+) a single source file
`jshell`	Interactive REPL (Java 9+): try snippets without writing a class
`jar`	Packages classes and resources into a `.jar` archive
`javadoc`	Generates HTML API documentation from `/** ... */` comments
`jdb`	Command-line debugger
`javap`	Disassembles a `.class` file (`javap -c` shows bytecode)
`jlink`	Builds a minimal custom runtime image (Java 9+)
`jcmd`, `jstack`, `jmap`, `jconsole`	Diagnostics: thread dumps, heap dumps, monitoring
`keytool`	Manages keystores and certificates (common in banking/TLS setups)

NOTE

Since **Java 11**, OpenJDK and Oracle no longer ship a separate JRE download. You install the JDK, or build a slim runtime with `jlink`.

INTERVIEW

Is the JVM platform independent? **No.** Bytecode is platform independent; each OS needs its own JVM implementation. That is what makes the bytecode portable.

1.5 Environment Setup (Windows)

1. Download and install a JDK (for example Eclipse Temurin 21 LTS).
2. Create a system variable `JAVA_HOME` pointing at the JDK folder (not the `bin` folder).
3. Add `%JAVA_HOME%\bin` to the system `Path` so `java` and `javac` work from any folder.
4. Open a **new** Command Prompt and verify:

```
C:\> java -version
openjdk version "21.0.x" ...
C:\> javac -version
javac 21.0.x
C:\> echo %JAVA_HOME%
C:\Program Files\Eclipse Adoptium\jdk-21.0.x-hotspot
```

TIP

Maven, Gradle, IntelliJ and many tools read `JAVA_HOME`, not just `Path`. With multiple JDKs installed, switching `JAVA_HOME` (and reopening the terminal) switches versions.

1.6 Your First Java Program

```java
// File name MUST be Hello.java because the public class is Hello
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
```

```
javac Hello.java      -> compiles, creates Hello.class
java Hello            -> runs (class name, NOT Hello.class)
java Hello.java       -> Java 11+: compile + run a single file in one step
```

Rules to remember

- Java is **case sensitive**: `Main`, `main` and `MAIN` are different.
- A file can hold **at most one** `public` **top-level class**, and the file name must match it.
- A file may contain several non-public classes; `javac` creates one `.class` per class.
- `java.lang` (`String`, `System`, `Math`, ...) is imported automatically.
- Every statement ends with `;` and blocks are delimited by `{ }`.

`System.out.println()` explained: `System` is a class in `java.lang`, `out` is a static `PrintStream` field of `System`, and `println()` is a method of `PrintStream`. `print()` keeps the cursor on the same line; `println()` moves to the next line.

MODERN JAVA

Java 25 finalised *compact source files and instance main methods*. Beginners can now write a program without declaring a class or `static`:

```java
void main() {
    IO.println("Hello from Java 25!");
}
```

1.7 The main() Method Explained

```java
public static void main(String[] args)
```

Part	Why it is there
`public`	The JVM calls `main` from outside the class, so it must be accessible.
`static`	The JVM calls it **without creating an object** (no instance exists yet).
`void`	It returns nothing to the JVM. Use `System.exit(code)` to return an exit status.
`main`	The method name the JVM looks for as the entry point.
`String[] args`	Command-line arguments, always received as Strings.

Valid variations

```
public static void main(String[] args)       // standard
public static void main(String args[])       // C-style array declaration
public static void main(String... args)      // varargs
static public void main(String[] args)       // modifier order doesn't matter
public static final void main(final String[] args)  // final is allowed
```

INTERVIEW

Common questions about `main`:

- *Can we overload* `main`*?* Yes, but the JVM only calls `main(String[])`.
- *Can we run a class without* `main`*?* Not as a standalone application (static blocks alone stop working from Java 7).
- *What if* `static` *is missing?* Before Java 21 the program compiles but fails at launch ("main method is not static"). Newer launchers can use instance main methods (finalised in Java 25).
- *Is* `args` *ever null?* No. With no arguments, it is an empty array (`args.length == 0`).

1.8 Structure of a Java Program

```
package com.bank.payments;                  // 1. package: optional, at most one, must be first

import java.util.List;                      // 2. imports: optional, zero or more
import java.time.LocalDate;

public class PaymentService {               // 3. type declarations (class/interface/enum/record)

    private static final String CURRENCY = "INR";   // constants
    private double balance;                          // fields (data)

    public PaymentService(double opening) {          // constructor
        this.balance = opening;
    }

    public void pay(double amount) {                 // methods (behaviour)
        balance -= amount;
    }
}
```

Order is fixed: **package → imports → types**. Comments can appear anywhere.

1.9 How a Java Program Runs

```
 Hello.java --javac--> Hello.class (bytecode, platform independent)
                            |
                            v
 +------------------------- JVM ------------------------------+
 | Class Loader  ->  Bytecode Verifier  ->  Execution Engine  |
 |                                          (Interpreter + JIT)|
 +-------------------------------------------------------------+
                            |
                            v
                 Native machine code -> OS + CPU -> Output
```

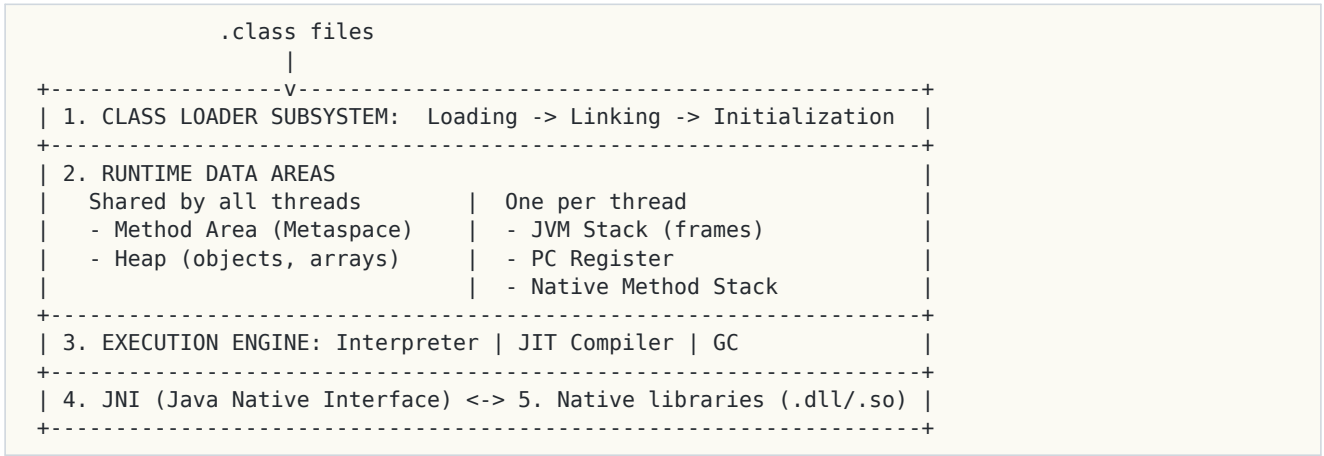
Translators

Translator	How it works	Speed
Interpreter	Translates and runs code line by line	Slower; starts immediately
Compiler	Translates the whole program ahead of time	Fast execution
Assembler	Converts assembly language into machine code	n/a
JIT compiler	Compiles frequently used ("hot") bytecode to native code **at runtime**	Fast after warm-up

Java uses a **compiler** at build time (`javac` → bytecode) and an **interpreter plus JIT** at runtime. The HotSpot JVM starts by interpreting bytecode and counts how often each method runs. Hot methods are compiled to optimised native code (C1 then C2, called *tiered compilation*). That is why Java apps get faster after warm-up.

> **TIP**
> Run `javap -c Hello` to see the actual bytecode instructions inside `Hello.class`.

1.10 JVM Architecture



1. Class Loader Subsystem

Loading reads `.class` files and creates `Class` objects. Three built-in loaders:

Class loader	Loads	Example
Bootstrap	Core platform classes (`java.base` module)	`java.lang.String`, `java.util.List`
Platform (called *Extension* before Java 9)	Other platform modules	`java.sql`, `java.xml`
Application (System)	Your classes and JARs on the classpath	`com.bank.PaymentService`

Loaders follow the **parent delegation model**: a loader first asks its parent to load a class, and only loads it itself if the parent cannot. This stops anyone from replacing core classes such as `java.lang.String` with a fake.

Linking has three steps: **Verify** (bytecode is valid and safe), **Prepare** (memory for static variables, set to default values), and **Resolve** (symbolic references become direct references).

Initialization assigns real values to static variables and runs `static { }` blocks, top to bottom. This happens once, when the class is first used. (Class loading is covered in depth in Phase 15.)

2. Runtime Data Areas

Area	Stores	Scope	Error when exhausted
Method Area (Metaspace since Java 8)	Class metadata, method bytecode, runtime constant pool	Shared	`OutOfMemoryError: Metaspace`
Heap	All objects and arrays, instance variables, String pool (since Java 7)	Shared; managed by GC	`OutOfMemoryError: Java heap space`
JVM Stack	One **frame** per method call: local variables, operand stack, return address	Per thread	`StackOverflowError`
PC Register	Address of the bytecode instruction currently executing	Per thread	n/a
Native Method Stack	State of native (C/C++) method calls	Per thread	`StackOverflowError`

```
class Account {
    static String bank = "ABC Bank";  // static: class-level, one copy per class
    double balance;                   // instance field: lives inside the object (Heap)

    void deposit(double amount) {     // bytecode for this method: Method Area
        double fee = 1.5;             // local variable: in this call's Stack frame
        balance += amount - fee;
    }
}

Account acc = new Account();   // 'acc' reference: Stack | the Account object: Heap
acc.deposit(500);              // new frame pushed on the Stack, popped when it returns
```

> **NOTE**
> Before Java 8, class metadata lived in **PermGen**, which had a fixed size and often caused `OutOfMemoryError: PermGen space`. Java 8 replaced it with **Metaspace**, which uses native memory and grows as needed. Static variables are conceptually class-level; in HotSpot they are physically stored with the `Class` object on the heap.

The heap is divided into **generations**: the Young Generation (Eden, Survivor S0 and S1) for new objects, and the Old Generation for long-lived objects. Garbage collection is covered in Phase 15.

3. Execution Engine

- **Interpreter:** executes bytecode instruction by instruction. Quick to start, slower to run.
- **JIT compiler:** compiles hot methods to native code and caches it (C1 = quick, C2 = highly optimised).
- **Garbage Collector:** automatically frees heap objects that are no longer reachable. **G1** is the default since Java 9; alternatives include ZGC, Shenandoah, Parallel and Serial.

4 & 5. JNI and Native Libraries

The **Java Native Interface** lets Java call C/C++ code and vice versa. Native libraries are `.dll` on Windows, `.so` on Linux and `.dylib` on macOS.

```
public class NativeDemo {
    static { System.loadLibrary("crypto"); }  // loads crypto.dll / libcrypto.so
    public native int encrypt(byte[] data);   // body implemented in C/C++
}
```

Common JVM options

Option	Meaning	Example
`-Xms`	Initial heap size	`-Xms512m`
`-Xmx`	Maximum heap size	`-Xmx2g`
`-Xss`	Stack size per thread	`-Xss1m`
`-XX:MaxMetaspaceSize`	Upper limit for Metaspace	`-XX:MaxMetaspaceSize=256m`
`-XX:+UseG1GC`	Select a garbage collector	`-XX:+UseZGC`
`-Xlog:gc`	Log garbage collection (Java 9+)	`java -Xlog:gc -jar app.jar`
`-XX:+HeapDumpOnOutOfMemoryError`	Write heap dump on OOM	Useful in production

1.11 Common Beginner Errors and Fixes

Error message	Cause	Fix
`'javac' is not recognized ...`	JDK `bin` not on `Path`	Add `%JAVA_HOME%\bin` to `Path`, reopen the terminal
`class Hello is public, should be declared in a file named Hello.java`	File name and public class name differ	Rename the file or the class
`Could not find or load main class Hello`	Wrong name, wrong folder/classpath, or class is in a package	Run from the right folder; for packages use `java com.bank.Hello`
`Main method not found in class Hello`	Wrong `main` signature	Use `public static void main(String[] args)`
`UnsupportedClassVersionError ... class file version 65.0`	Compiled with a newer JDK than the runtime	Run on a newer JDK, or compile with `--release 17`
`StackOverflowError`	Infinite or very deep recursion	Add a base case; refactor to iteration
`OutOfMemoryError: Java heap space`	Too many live objects, or a memory leak	Fix the leak; increase `-Xmx` if truly needed

> **TIP**
> Class-file major versions to recognise: **52** = Java 8, **55** = 11, **61** = 17, **65** = 21, **69** = 25.

1.12 Interview Questions

Q. What is the difference between JDK, JRE and JVM?

The JVM executes bytecode. The JRE is the JVM plus the class libraries needed to run programs. The JDK is the JRE plus development tools such as `javac`, `jar` and `javadoc`. Developers need the JDK; running an app needs only a runtime.

Q. Why is Java platform independent?

`javac` compiles source into bytecode rather than OS-specific machine code. Any OS with a JVM can run that bytecode, and the JVM translates it into native instructions.

Q. Is Java compiled or interpreted?

Both. Source is compiled to bytecode by `javac`. At runtime the JVM interprets bytecode and JIT-compiles hot code into native machine code.

Q. What is the JIT compiler?

The Just-In-Time compiler inside the JVM. It detects frequently executed methods and compiles them to optimised native code at runtime, improving performance after warm-up.

Q. What are the JVM memory areas?

Method Area/Metaspace and Heap (shared by all threads), plus the JVM Stack, PC Register and Native Method Stack (one per thread).

Q. What is the difference between Stack and Heap memory?

The stack holds method frames (local variables and references), is per thread, and is freed automatically when a method returns. The heap holds all objects, is shared by all threads, and is cleaned by the garbage collector. Stack overflow gives `StackOverflowError`; heap exhaustion gives `OutOfMemoryError`.

Q. What is the parent delegation model?

A class loader first delegates loading to its parent (Application → Platform → Bootstrap). It loads the class itself only if the parent cannot. This prevents duplicate loading and protects core classes from being replaced.

Q. What replaced PermGen in Java 8, and why?

Metaspace. PermGen had a fixed maximum size and caused frequent out-of-memory errors. Metaspace uses native memory and can grow dynamically (cap it with `-XX:MaxMetaspaceSize`).

Q. Why is `main` static?

The JVM must call it before any object of the class exists. A static method can be invoked using just the class name.

Q. What happens when you run `java Hello`?

The JVM starts, the Application class loader loads `Hello.class` (delegating to its parents), the class is verified, prepared and initialized (static blocks run), and then the execution engine invokes `main` on the main thread.

1.13 Quick Revision

- Java: Sun Microsystems, James Gosling, 1991 (Oak), released 1995; owned by Oracle since 2010.
- WORA: source → `javac` → bytecode → any OS's JVM.
- LTS versions: **8, 11, 17, 21, 25**. Spring Boot 3 needs Java 17+.
- **JDK = JRE + tools; JRE = JVM + libraries.** The JVM is platform dependent; bytecode is not.
- Java is **compiled and interpreted**; the JIT makes hot code native.
- `main` is `public static void main(String[] args)`; one public class per file, matching the file name.
- JVM = Class Loader (Load → Link → Initialize) + Runtime Data Areas + Execution Engine + JNI.
- Shared memory: **Heap** and **Metaspace**. Per thread: **Stack**, **PC Register**, **Native Stack**.
- Class loaders: Bootstrap → Platform → Application, using parent delegation.
- GC cleans the heap; G1 is the default collector since Java 9.

PHASE 2 OF 18

Language Basics

Variables, data types, literals, type casting and conversion, comments, identifiers, keywords, naming conventions and operators

IN THIS PHASE

2.1 Variables
2.2 Data Types
2.3 Type Conversion and Casting
2.4 Reading Console Input
2.5 Comments
2.6 Identifiers and Keywords
2.7 Naming Conventions
2.8 Operators
2.9 Operator Precedence
2.10 Predict the Output
2.11 Interview Questions
2.12 Quick Revision

2.1 Variables

A **variable** is a named memory location that stores a value. Java is **statically typed**: every variable has a type, fixed at compile time.

Storing data in a variable takes two steps, which can be combined:

```
byte age;          // 1. Declaration: type + name
age = 20;          // 2. Initialization: store a value
byte age2 = 20;    // both in one statement

int x = 10, y = 20, z;    // several variables of the same type in one line
```

Three kinds of variables (detailed in Phase 5)

Kind	Declared	Lives in	Default value?
Local	Inside a method, constructor or block	Stack frame	**No**. Must be initialised before use.
Instance	Inside a class, outside methods (no `static`)	Inside each object (Heap)	Yes (0, false, null, ...)
Static (class)	Inside a class with `static`	One copy per class	Yes

```
public class Demo {
    int count;                // instance: default 0
    static String bank;       // static: default null

    void show() {
        int local;
        // System.out.println(local);  // COMPILE ERROR: variable might not be initialized
        local = 5;
        System.out.println(local + count);
    }
}
```

Constants with final

```
final int MIN_AGE = 21;              // cannot be reassigned
// MIN_AGE = 25;                     // COMPILE ERROR
static final double PI = 3.14159;    // class-level constant (typical style)
```

CORRECTED FROM ORIGINAL NOTES

The original notes show `int MIN_AGE = 21;` as a constant. Naming it in capitals does **not** make it constant; only `final` does.

Local variable type inference: var (Java 10+)

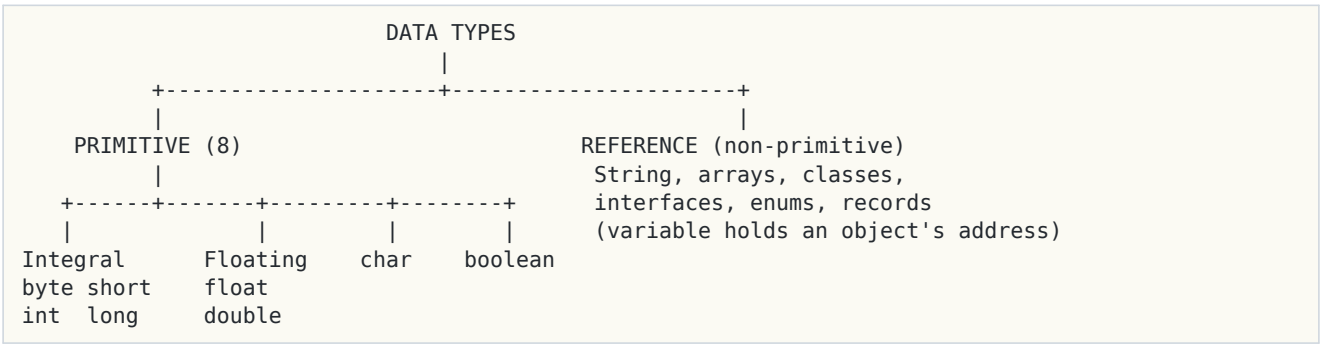
```
var count = 10;                 // inferred as int
var name = "Rohit";             // inferred as String
var list = new ArrayList<String>();   // inferred as ArrayList<String>
for (var entry : map.entrySet()) { }  // handy for long generic types
```

REMEMBER

Rules for `var`:

- Only for **local** variables (including loop variables), never fields, parameters or return types.
- Must be initialised on the same line: `var x;` and `var x = null;` do not compile.
- Type is still static: `var x = 10; x = "hi";` is a compile error.
- `var` is a reserved type name, not a keyword, so a variable can still be called `var`.

2.2 Data Types



The eight primitive types

Type	Size	Range	Default	Literal example
`byte`	8 bits (1 byte)	-128 to 127	0	`byte b = 100;`
`short`	16 bits (2 bytes)	-32,768 to 32,767	0	`short s = 30000;`
`int`	32 bits (4 bytes)	about -2.1 billion to 2.1 billion (-2^31 to 2^31-1)	0	`int i = 1_000_000;`
`long`	64 bits (8 bytes)	-2^63 to 2^63-1 (about 9.2 x 10^18)	0L	`long l = 30000000000L;`
`float`	32 bits	about ±3.4 x 10^38, **6–7 significant digits**	0.0f	`float f = 25.01f;`
`double`	64 bits	about ±1.8 x 10^308, **15–16 significant digits**	0.0	`double d = 25.01;`
`char`	16 bits (unsigned)	`'\u0000'` to `'\uffff'` (0 to 65,535)	`'\u0000'`	`char c = 'A';`
`boolean`	not specified by the JVM spec	`true` / `false`	false	`boolean ok = true;`

CORRECTED FROM ORIGINAL NOTES

Fixes to the original notes:

- `float` / `double` precision is about 6–7 / 15–16 **significant digits** in total, not "decimal places".
- `boolean` does not occupy "1 bit". The JVM specification leaves its size unspecified; in practice a `boolean` field usually takes 1 byte, and `boolean` locals are handled as `int`s.
- A `char` stores a **UTF-16 code unit** (Unicode), not an ASCII value. ASCII is only the first 128 characters of Unicode.

TIP

Use the wrapper constants instead of memorising ranges: `Integer.MAX_VALUE`, `Long.MIN_VALUE`, `Byte.MAX_VALUE`, `Double.MAX_VALUE`, `Character.MAX_VALUE`.

Literals

Literal	Example	Notes
Integer (decimal)	`int a = 255;`	An integer literal is `int` by default
Long	`long l = 9_000_000_000L;`	`L` suffix is required beyond the int range; prefer uppercase `L` (lowercase `l` looks like `1`)
Hexadecimal	`int h = 0xFF;`	255
Binary (Java 7+)	`int bin = 0b1111_1111;`	255
Octal	`int o = 0377;`	255. A leading `0` means octal, a common trap!
Underscores (Java 7+)	`long card = 1234_5678_9012_3456L;`	Readability only; not at the start or end, or next to `.`
Double	`double d = 3.14; 1.5e3`	A decimal literal is `double` by default
Float	`float f = 3.14f;`	`f`/`F` suffix is mandatory
Char	`'A'`, `'\n'`, `'\u0041'`	Single quotes, exactly one character
String	`"Hello"`	Double quotes; a reference type
Boolean	`true`, `false`	Not `1`/`0` as in C
Null	`null`	Default for every reference type

```
float a = 25.01;    // COMPILE ERROR: 25.01 is a double (8 bytes) -> possible lossy conversion
float b = 25.01f;   // OK
char rank = '1';    // OK: one character
char bad = '10';    // COMPILE ERROR: unclosed / too many characters in char literal
```

Escape sequences

Escape	Meaning	Escape	Meaning
`\n`	New line	`\\`	Backslash
`\t`	Tab	`\'`	Single quote
`\r`	Carriage return	`\"`	Double quote
`\b`	Backspace	`\uXXXX`	Unicode character (hex)
`\s`	Space (Java 15+)	`\0`	Null character (octal escape)

char is a number underneath

```
char c = 'A';
System.out.println(c + 1);          // 66   (char promoted to int)
System.out.println((char) (c + 1)); // B
System.out.println((int) 'a');      // 97
char u = '\u0041';                  // 'A' written as a Unicode escape
char next = c; next++;              // 'B' (++ on a char works)
```

Character	Code	Character	Code
'0' – '9'	48 – 57	'a' – 'z'	97 – 122
'A' – 'Z'	65 – 90	space	32

TIP

Convert a digit character to its number with `ch - '0'` (for example `'7' - '0'` is `7`), or `Character.getNumericValue(ch)`.

Floating-point precision and money

```
System.out.println(0.1 + 0.2);       // 0.30000000000000004
System.out.println(0.1 + 0.2 == 0.3);// false

import java.math.BigDecimal;
BigDecimal a = new BigDecimal("0.1");   // use the String constructor!
BigDecimal b = new BigDecimal("0.2");
System.out.println(a.add(b));           // 0.3
```

PITFALL

Never use `float` or `double` for currency. Binary floating point cannot represent most decimal fractions exactly. Use `BigDecimal` (built from a `String`, with an explicit `RoundingMode`), or store amounts as `long` minor units such as paise or cents. This is a standard banking and fintech interview question.

Integer overflow

```
int max = Integer.MAX_VALUE;          // 2147483647
System.out.println(max + 1);          // -2147483648  (silently wraps around!)
long ok = (long) max + 1;             // 2147483648   (widen BEFORE adding)
Math.addExact(max, 1);                // throws ArithmeticException: integer overflow
```

2.3 Type Conversion and Casting

Type casting converts a value from one type to another. There are two kinds for primitives.

Widening (implicit, automatic)

Smaller type → larger type. No cast is needed and the value always fits.

```
byte -> short -> int -> long -> float -> double
                  ^
                char
```

```
int i = 100;
long l = i;          // int  -> long   (automatic)
double d = l;        // long -> double (automatic)
int code = 'A';      // char -> int = 65
```

NOTE

`int → float` and `long → float/double` are "widening" but can **lose precision**, because a float has only about 7 significant digits. For example `(float) 123456789` stores 123456792 (prints `1.2345679E8`).

Narrowing (explicit, manual)

Larger type → smaller type. Requires a cast `(type)`, because data may be lost.

```
double d = 9.99;
int i = (int) d;           // 9    -> fractional part TRUNCATED, not rounded
long big = 130;
byte b = (byte) big;       // -126 -> overflow wraps around (130 - 256)
int r = (int) Math.round(9.99);   // 10 -> round when you mean round
char c = (char) 66;        // 'B'
```

Automatic type promotion in expressions

In arithmetic, `byte`, `short` and `char` are promoted to `int` first. The result type is the widest operand type (at least `int`).

```
byte a = 10, b = 20;
// byte c = a + b;      // COMPILE ERROR: a + b is an int
byte c = (byte) (a + b);// OK
int c2 = a + b;         // OK

byte x = 10;
x = x + 1;              // COMPILE ERROR (int result)
x += 1;                 // OK: compound assignment includes an implicit cast
x++;                    // OK

final byte k = 10;
byte y = k + 5;         // OK: compile-time constant that fits in a byte

long  total = 5 * 2L;   // long
double avg = 7 / 2;     // 3.0 -> integer division happens BEFORE widening!
double avg2 = 7 / 2.0;  // 3.5
```

Converting between String and primitives

Direction	Method	Example
String → int	`Integer.parseInt(s)`	`int n = Integer.parseInt("10");`
String → long / double / boolean	`Long.parseLong`, `Double.parseDouble`, `Boolean.parseBoolean`	`double d = Double.parseDouble("2.5");`
String → wrapper object	`Integer.valueOf(s)`	`Integer obj = Integer.valueOf("10");`
primitive → String	`String.valueOf(x)`	`String s = String.valueOf(10);`
primitive → String	`Integer.toString(x)` or `"" + x`	`String s = "" + 10;`
char → String	`String.valueOf(ch)` or `Character.toString(ch)`	`"A"`

```
Integer.parseInt("hello");       // NumberFormatException (unchecked)
Integer.parseInt(" 10");         // NumberFormatException: spaces are not trimmed
Integer.parseInt("ff", 16);      // 255: parse with a radix
Boolean.parseBoolean("yes");     // false: only "true" (any case) gives true
```

Command-line arguments

Arguments passed after the class name arrive in `main` as a `String[]`. They are always Strings.

```java
public class Calculator {
    public static void main(String[] args) {
        if (args.length < 2) {
            System.out.println("Usage: java Calculator <a> <b>");
            return;
        }
        System.out.println(args[0] + args[1]);          // "10" + "20" = 1020
        int a = Integer.parseInt(args[0]);
        int b = Integer.parseInt(args[1]);
        System.out.println(a + b);                      // 30
    }
}
// Run:  java Calculator 10 20
// Arguments containing spaces need quotes:  java App "New Delhi"
```

PITFALL

Accessing `args[0]` with no arguments throws `ArrayIndexOutOfBoundsException`. Always check `args.length` first.

2.4 Reading Console Input

```java
import java.util.Scanner;

public class InputDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter age: ");
        int age = sc.nextInt();
        sc.nextLine();                    // consume the leftover newline!
        System.out.print("Enter name: ");
        String name = sc.nextLine();      // reads the whole line
        System.out.println(name + " is " + age);
        sc.close();
    }
}
```

Method	Reads
`nextInt()`, `nextLong()`, `nextDouble()`, `nextBoolean()`	One token converted to that type
`next()`	One word (stops at whitespace)
`nextLine()`	Rest of the current line
`hasNextInt()`, `hasNext()`	Checks before reading (validation)

PITFALL

After `nextInt()`, the newline stays in the buffer, so the next `nextLine()` returns an empty string. Add an extra `sc.nextLine()` as shown above.

2.5 Comments

Type	Syntax	Use
Single-line	`// comment`	Short notes; everything after `//` on that line is ignored
Multi-line	`/* ... */`	Longer explanations; cannot be nested
Documentation (Javadoc)	`/** ... */`	API docs placed before classes, methods, fields and constructors

```java
/**
 * Transfers money between two accounts.
 *
 * @param from   source account number
 * @param to     destination account number
 * @param amount amount to transfer; must be positive
 * @return the transaction reference id
 * @throws InsufficientFundsException if the balance is too low
 * @since 2.1
 */
public String transfer(String from, String to, BigDecimal amount) { ... }
```

Javadoc tag	Purpose
`@param`	Describes a parameter
`@return`	Describes the return value
`@throws` / `@exception`	Documents an exception
`@see` / `{@link Class#method}`	Cross-reference
`@since`, `@version`, `@author`	Version and author info
`@deprecated`	Marks as obsolete (pair with the `@Deprecated` annotation)
`{@code text}`	Shows inline code

Generate HTML documentation with `javadoc -d docs *.java`.

2.6 Identifiers and Keywords

An **identifier** is a name for a class, interface, method, variable or package.

Rules (compiler-enforced)

1. Allowed characters: letters `a-z`, `A-Z`, digits `0-9`, `_` and `$` (Unicode letters are also allowed).
2. Must **not start with a digit**.
3. Must not be a **reserved keyword** or the literals `true`, `false`, `null`.
4. **Case sensitive**: `name`, `Name` and `NAME` are three different identifiers.
5. **No spaces** or other symbols (`-`, `@`, `#`, ...).
6. No length limit. A lone `_` is not allowed since Java 9.

Identifier	Valid?	Reason
`age2`, `_bill`, `$amount`, `mobile_bill`	Valid	Allowed characters, doesn't start with a digit
`2age`	Invalid	Starts with a digit
`mobile bill`	Invalid	Contains a space
`total-amount`	Invalid	`-` is an operator
`class`, `int`	Invalid	Reserved keywords
`String`, `main`, `var`	Valid (but avoid)	Not keywords, just bad names

Reserved keywords

Category	Keywords
Primitive types & void	`byte short int long float double char boolean void`
Flow control	`if else switch case default while do for break continue return`
Modifiers	`public private protected static final abstract synchronized native transient volatile strictfp`
Exception handling	`try catch finally throw throws assert`
Class related	`class interface enum extends implements package import`
Object related	`new this super instanceof`
Reserved but unused	`const goto`
Underscore (Java 9+)	`_`

Literals (not keywords, but also reserved): `true`, `false`, `null`.

Contextual keywords have special meaning only in certain places and can still be used as identifiers elsewhere: `var`, `record`, `yield`, `sealed`, `permits`, `non-sealed`, `when`, and module words such as `module`, `requires` and `exports`.

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes list `null` as a keyword and give the count as "53". Precisely: Java has 50 classic keywords plus _, and `true`, `false`, `null` are **literals**. Interviewers usually just want to know that `true`, `false` and `null` are literals rather than keywords, and that `const` and `goto` are reserved but unused.

2.7 Naming Conventions

Conventions are not enforced by the compiler but are followed everywhere. Methods and variables use *camelCase*; classes use *PascalCase*.

Element	Convention	Examples
Class / Interface / Enum / Record	PascalCase, nouns (interfaces often adjectives)	`PaymentService`, `Runnable`, `AccountType`
Method	camelCase, starts with a verb	`calculateInterest()`, `getBalance()`, `saveAllUsers()`
Variable	camelCase, meaningful noun	`userAge`, `creditCardNumber`
Boolean variable / method	Reads as a question	`isActive`, `hasBalance`, `canWithdraw()`
Constant (`static final`)	UPPER_SNAKE_CASE	`MIN_AGE`, `MAX_RETRY_COUNT`
Enum constant	UPPER_SNAKE_CASE	`SAVINGS`, `CURRENT`, `FIXED_DEPOSIT`
Package	all lowercase, reverse domain, dot separated	`java.lang`, `com.bank.payments.service`
Generic type parameter	Single uppercase letter	`T` (type), `E` (element), `K`, `V` (key, value), `R` (result)

> **TIP**
> Getters and setters follow the JavaBeans pattern: `getName()` / `setName()`, and `isActive()` for booleans. Frameworks such as Spring, Jackson and Hibernate rely on these names.

2.8 Operators

An **operator** is a symbol that performs an operation on **operands** (`a + b`: `+` is the operator, `a` and `b` are operands).

Type	Operators	Example
Arithmetic	`+ - * / %`	`a % b`
Unary	`+ - ++ -- ! ~`	`++a`, `-x`, `!flag`
Assignment	`= += -= *= /= %= &= |= ^= <<= >>= >>>=`	`a += 5`
Relational	`== != > < >= <=`	`a >= b`
Logical	`&& || !`	`a > 5 && b < 10`
Bitwise	`& | ^ ~`	`a & b`
Shift	`<< >> >>>`	`a << 2`
Ternary (conditional)	`condition ? x : y`	`(a > b) ? a : b`
Type comparison	`instanceof`	`obj instanceof String`
Member access	`.`	`System.out.println()`
Lambda / method reference	`-> ::`	`x -> x * 2`, `String::length`

CORRECTED FROM ORIGINAL NOTES

The original notes call `.` (dot) a relational operator. It is the **member access** operator, used to reach fields, methods and nested types.

Arithmetic surprises

```
System.out.println(7 / 2);      // 3      integer division truncates
System.out.println(7 % 3);      // 1
System.out.println(-7 % 3);     // -1     sign follows the LEFT operand
System.out.println(7.5 % 2);    // 1.5    % works on doubles too
System.out.println(10 / 0);     // ArithmeticException: / by zero
System.out.println(10.0 / 0);   // Infinity
System.out.println(0.0 / 0);    // NaN
System.out.println(Double.NaN == Double.NaN);   // false! use Double.isNaN(x)
```

Increment and decrement

Pre-increment (`++a`) increments first, then uses the value. **Post-increment** (`a++`) uses the current value, then increments.

```
int a = 5;
System.out.println(a++);  // 5   (prints, then a = 6)
a++;                      //     a = 7
System.out.println(a++);  // 7   (prints, then a = 8)
System.out.println(a);    // 8

int b = 8;
System.out.println(++b);  // 9
++b;                      //     b = 10
System.out.println(++b);  // 11
System.out.println(b);    // 11

int c = 3;
int d = ++c + c++;        // 4 + 4 = 8  (++c makes c 4; c++ uses 4 then c becomes 5)
System.out.println(d + " " + c);   // 8 5

int i = 5;
i = i++;                  // i stays 5! (old value is assigned back over the increment)
```

Relational and logical operators

```
int age = 25; boolean hasId = true;
boolean canEnter = age >= 18 && hasId;    // true
boolean discount = age < 12 || age > 60;  // false
boolean notAdult = !(age >= 18);          // false
```

Short-circuit evaluation: `&&` skips the right side if the left is `false`; `||` skips it if the left is `true`. The non-short-circuit `&` and `|` always evaluate both sides.

```
String s = null;
if (s != null && s.length() > 0) { }   // safe: length() is never called
if (s != null &  s.length() > 0) { }   // NullPointerException: both sides evaluated
```

> **TIP**
> Put the cheap or null-guarding condition first. Writing `"YES".equals(input)` instead of `input.equals("YES")` also avoids a `NullPointerException` when `input` is null.

Bitwise and shift operators

```
int a = 5;   // 0101
int b = 3;   // 0011
a & b   // 0001 = 1   AND: 1 only if both bits are 1
a | b   // 0111 = 7   OR:  1 if either bit is 1
a ^ b   // 0110 = 6   XOR: 1 if the bits differ
~a      // -6         NOT: flips every bit (~x == -x - 1)

8 << 1     // 16   shift left by n = multiply by 2^n
8 >> 1     // 4    shift right by n = divide by 2^n (keeps sign)
-8 >> 1    // -4   arithmetic shift: sign bit copied
-8 >>> 28  // 15   unsigned shift: fills with 0s

(n & 1) == 0          // even-number check
(x ^ y ^ y) == x      // XOR twice restores the value
(flags & MASK) != 0   // checks a permission/feature flag
```

Ternary operator

```
int max = (a > b) ? a : b;
String status = balance >= 0 ? "ACTIVE" : "OVERDRAWN";
String grade = marks >= 90 ? "A" : marks >= 75 ? "B" : "C";   // nested: keep it short
```

instanceof (with pattern matching, Java 16+)

```
Object obj = "rohit";
if (obj instanceof String) {               // classic: check, then cast
    String s = (String) obj;
    System.out.println(s.toUpperCase());
}
if (obj instanceof String s && s.length() > 3) {   // pattern matching: check + bind
    System.out.println(s.toUpperCase());
}
null instanceof String   // false: never throws
```

String concatenation with +

`+` is evaluated **left to right**. Once one operand is a `String`, it becomes concatenation.

```
System.out.println(1 + 2 + "3");     // "33"   (1+2 = 3, then "3"+"3")
System.out.println("1" + 2 + 3);     // "123"
System.out.println("1" + (2 + 3));   // "15"
System.out.println('a' + 'b');       // 195    (char + char = int!)
System.out.println("" + 'a' + 'b');  // "ab"
```

2.9 Operator Precedence

From highest to lowest. When in doubt, add parentheses: it costs nothing and helps readers.

Level	Operators	Associativity
1	Postfix `a++ a--`, member access `.`, `[]`, method call `()`	Left to right
2	Unary `++a --a + - ! ~`, cast `(type)`	Right to left
3	Multiplicative `* / %`	Left to right
4	Additive `+ -`	Left to right
5	Shift `<< >> >>>`	Left to right
6	Relational `< > <= >= instanceof`	Left to right
7	Equality `== !=`	Left to right
8	Bitwise AND `&`	Left to right
9	Bitwise XOR `^`	Left to right
10	Bitwise OR `|`	Left to right
11	Logical AND `&&`	Left to right
12	Logical OR `||`	Left to right
13	Ternary `? :`	Right to left
14	Assignment `= += -=` ... and lambda `->`	Right to left

```java
int r = 2 + 3 * 4;        // 14, not 20
boolean t = 5 > 3 == true; // (5 > 3) == true -> true
int x, y, z;
x = y = z = 10;           // assignment is right-associative
```

2.10 Predict the Output

```java
System.out.println(10 + 20 + "Java" + 10 + 20);   // 30Java1020
System.out.println('A' + 1);                      // 66
System.out.println((char) ('A' + 1));             // B
System.out.println(5 / 2 * 2.0);                  // 4.0  (5/2 = 2 first)
System.out.println(1 / 2 + 0.5);                  // 0.5
System.out.println((int) 3.99 + (int) -3.99);     // 0    (3 + -3, truncation toward 0)
System.out.println(Integer.MAX_VALUE + 1 < 0);    // true (overflow)
byte b = (byte) 200;  System.out.println(b);      // -56
int i = 0; i = i++ + ++i; System.out.println(i);  // 2    (0 + 2)
System.out.println(0.1f == 0.1);                  // false (float vs double precision)
long l = 1_000 * 1_000 * 1_000 * 10;              // overflow in int BEFORE widening!
System.out.println(l);                            // 1410065408 (use 10L)
```

2.11 Interview Questions

Q. What are primitive and reference data types?

Java has 8 primitives (`byte`, `short`, `int`, `long`, `float`, `double`, `char`, `boolean`) that hold the value directly. Reference types (classes, interfaces, arrays, `String`, enums, records) hold the address of an object on the heap; their default value is `null`.

Q. Why is `char` 2 bytes in Java but 1 byte in C?

Java uses Unicode (UTF-16) so it can represent characters from all languages, not just ASCII.

Q. What is the difference between widening and narrowing?
Widening converts a smaller type to a larger one automatically with no data loss (int → long). Narrowing converts a larger type to a smaller one, needs an explicit cast, and may truncate or overflow (`(int) 9.99` gives 9; `(byte) 130` gives -126).

Q. Why does `byte b = 10; b = b + 1;` fail but `b += 1;` compile?
`b + 1` is promoted to `int`, and assigning an `int` to a `byte` needs a cast. Compound assignment operators include an implicit cast back to the variable's type.

Q. Why shouldn't you use double for money?
Binary floating point can't exactly represent most decimal fractions (`0.1 + 0.2` gives `0.30000000000000004`). Use `BigDecimal` with explicit rounding, or `long` in the smallest currency unit.

Q. What is the difference between `&&` and `&`?
`&&` is logical AND with short-circuiting: the right side is skipped if the left is false. `&` always evaluates both sides and also works as bitwise AND on integers.

Q. What is the difference between `>>` and `>>>`?
`>>` is an arithmetic right shift that preserves the sign bit. `>>>` is an unsigned right shift that fills with zeros, so negative numbers become large positive ones.

Q. Are `true`, `false` and `null` keywords?
No, they are literals. They still cannot be used as identifiers.

Q. Do local variables get default values?
No. Instance and static variables get defaults; local variables must be explicitly initialised before use, otherwise it is a compile error.

Q. What is `var`? Does it make Java dynamically typed?
`var` (Java 10) lets the compiler infer a local variable's type from its initializer. The type is still fixed at compile time, so Java remains statically typed.

2.12 Quick Revision

- Declaration + initialization: `int a = 10;`. Locals have **no defaults**; fields do.
- Sizes in bytes: byte 1, short 2, int 4, long 8, float 4, double 8, char 2; boolean is JVM-dependent.
- Integer literals default to `int`; decimal literals default to `double`. Use the `L` and `f` suffixes.
- `0x` hex, `0b` binary, leading `0` octal; underscores allowed for readability.
- Widening: byte → short → int → long → float → double (char → int). Narrowing needs `(type)`.
- `byte`/`short`/`char` are promoted to `int` in expressions; compound assignment hides a cast.
- Integer `/` truncates; `/ 0` on ints throws; on doubles gives `Infinity`/`NaN`.
- Money: **BigDecimal**, never `double`.
- `a++` uses then increments; `++a` increments then uses.
- `&&`/`||` short-circuit; `&`/`|` don't.
- `+` evaluates left to right: `1 + 2 + "3"` is `"33"`.
- Naming: `PascalCase` classes, `camelCase` methods and variables, `UPPER_SNAKE` constants, lowercase packages.

PHASE 3 OF 18

Control Flow & Arrays

if/else, classic and modern switch, loops, break/continue/labels, keyboard input, arrays, 2D/jagged arrays, the Arrays class and classic array programs

IN THIS PHASE

3.1 Control Statements Overview
3.2 if, if-else and the else-if Ladder
3.3 The switch Statement (classic)
3.4 Modern switch: Expressions and Pattern Matching
3.5 Loops
3.6 break, continue, return and Labels
3.7 Nested Loops and Pattern Programs
3.8 Reading Input from the Keyboard
3.9 Arrays: Basics
3.10 Iterating and Printing Arrays
3.11 The java.util.Arrays Utility Class
3.12 Copying Arrays
3.13 Multidimensional and Jagged Arrays
3.14 Arrays vs ArrayList
3.15 Classic Array Programs
3.16 Predict the Output
3.17 Interview Questions
3.18 Quick Revision

3.1 Control Statements Overview

By default Java executes statements **top to bottom, one after another**. Control statements change that flow based on conditions or repetition.

Category	Statements	Purpose
Decision making (conditional)	`if`, `if-else`, `if-else-if` ladder, nested `if`, `switch`	Choose which block runs
Looping (iteration)	`while`, `do-while`, `for`, enhanced `for` (for-each)	Repeat a block
Branching (jump / transfer)	`break`, `continue`, `return` (+ labels)	Leave or skip part of a loop or method

3.2 if, if-else and the else-if Ladder

```
int age = 20;
if (age >= 18) {                      // 1. if: runs only when the condition is true
    System.out.println("Eligible to vote");
}

int number = 5;
if (number % 2 == 0) {                // 2. if-else: one of two blocks
    System.out.println("Even");
} else {
    System.out.println("Odd");
}

int marks = 75;
if (marks >= 90) {                    // 3. ladder: the FIRST true condition wins
    System.out.println("Grade A");
} else if (marks >= 75) {
    System.out.println("Grade B");    // printed; later branches are skipped
} else if (marks >= 60) {
    System.out.println("Grade C");
} else {
    System.out.println("Fail");
}

if (age >= 18) {                      // 4. nested if
    if (hasLicense) {
        System.out.println("Can drive");
    }
}
```

Rules and traps

```
int a = 10;
if (a) { }          // COMPILE ERROR: int cannot be converted to boolean (no 0/1 truthiness)
if (a = 20) { }     // COMPILE ERROR: assignment yields int, not boolean

boolean flag = false;
if (flag = true) {  // COMPILES! assigns true, then tests it -> always runs (a classic bug)
    System.out.println("Oops");
}

if (a > 5)
    System.out.println("A");
    System.out.println("B");   // NOT part of the if: braces were omitted -> always prints
```

REMEMBER

The `if` condition must be a `boolean` or a boolean expression. Always use braces, even for one-line bodies; it prevents the "second line looks indented but isn't inside" bug.

TIP

Order ladder conditions from most specific to least specific. With `marks >= 60` first, a score of 95 would wrongly get grade C.

3.3 The switch Statement (classic)

`switch` compares one value against several **constant** cases. It is more readable than a long `if-else-if` ladder of equality checks.

```java
int day = 3;
switch (day) {
    case 1:
        System.out.println("Monday");
        break;
    case 2:
        System.out.println("Tuesday");
        break;
    case 3:
        System.out.println("Wednesday");
        break;
    case 6:
    case 7:                                  // grouping: several labels share one body
        System.out.println("Weekend");
        break;
    default:                                 // optional; runs when nothing matches
        System.out.println("Invalid day");
}
```

Fall-through

Without `break`, execution **falls through** into the following cases until a `break` or the end of the switch.

```java
int a = 1;
switch (a) {
    case 1: System.out.println("Hi");     // matches
    case 2: System.out.println("Hello");  // falls through
    default: System.out.println("Bye");   // falls through
}
// Output: Hi  Hello  Bye
```

What types can a switch use?

Allowed	Not allowed
`byte`, `short`, `char`, `int` and their wrappers (`Byte`, `Short`, `Character`, `Integer`)	`long`, `float`, `double`, `boolean`
`String` (Java 7+), compared with `equals()`	Ranges such as `case > 10:` (classic switch)
`enum` (Java 5+)	Non-constant case labels (a normal variable)
Any reference type with **pattern matching** (Java 21+)	Duplicate case labels

Rules

1. Case labels must be **compile-time constants** (literals or `final` constants) of a type compatible with the switch value.
2. Case labels must be unique.
3. `default` is optional and can appear anywhere, but put it last by convention.
4. `break` is optional (needed to stop fall-through).
5. Switching on a `null` String, wrapper or enum throws `NullPointerException` in a classic switch.

CORRECTED FROM ORIGINAL NOTES

Two examples in the original notes are wrong:

- `switch (2.1f)` with `case 1.1f:` does **not** fix the error. `float` and `double` are never allowed in a switch.
- `switch ('a') { case 1: ... }` does **not** print "Hi". `'a'` is **97**, not 1, so no case matches and nothing prints. `case 97:` or `case 'a':` would match.

```
String status = "ACTIVE";
switch (status) {                       // String switch (case sensitive)
    case "ACTIVE":   System.out.println("Allow login"); break;
    case "BLOCKED":  System.out.println("Deny");        break;
    default:         System.out.println("Unknown");
}

enum AccountType { SAVINGS, CURRENT, FD }
AccountType type = AccountType.SAVINGS;
switch (type) {
    case SAVINGS: System.out.println("4% interest"); break;   // no "AccountType." prefix
    case CURRENT: System.out.println("0% interest"); break;
    default:      System.out.println("Check FD rates");
}
```

3.4 Modern switch: Expressions and Pattern Matching

MODERN JAVA

Switch expressions (Java 14) use `->` arrows. There is **no fall-through**, no `break`, several labels can share a case, and the switch can **return a value**.

```
int day = 6;
String type = switch (day) {
    case 1, 2, 3, 4, 5 -> "Weekday";
    case 6, 7          -> "Weekend";
    default            -> "Invalid";
};

int interest = switch (accountType) {   // over an enum, every constant covered:
    case SAVINGS -> 4;                  // no default needed (exhaustive)
    case CURRENT -> 0;
    case FD -> {                        // a block needs 'yield' to return its value
        int base = 6;
        yield base + 1;
    }
};
```

Pattern matching for switch (Java 21)

```
static String describe(Object obj) {
    return switch (obj) {
        case null              -> "null value";            // null can now be handled
        case Integer i when i > 100 -> "large int " + i;   // guard with 'when'
        case Integer i         -> "int " + i;
        case String s          -> "string of length " + s.length();
        case int[] arr         -> "int array of " + arr.length;
        default                -> "other: " + obj.getClass().getSimpleName();
    };
}
```

Feature	Classic `case X:`	Arrow `case X ->`
Fall-through	Yes (needs `break`)	No
Multiple labels	`case 1: case 2:`	`case 1, 2 ->`
Returns a value	No	Yes, as an expression (`yield` inside blocks)
Exhaustiveness checked	No	Yes, when used as an expression
Handles `null`	Throws NPE	`case null ->` (Java 21)

3.5 Loops

A loop repeats a block while a condition holds.

```
int i = 1;
while (i <= 5) {                        // while: entry-controlled, 0 or more runs
    System.out.print(i + " ");
    i++;
}

int j = 1;
do {                                    // do-while: exit-controlled, runs AT LEAST once
    System.out.print(j + " ");
    j++;
} while (j <= 5);                       // note the semicolon

for (int k = 1; k <= 5; k++) {          // for: init; condition; update
    System.out.print(k + " ");
}

int[] arr = {10, 20, 30};
for (int n : arr) {                     // enhanced for (for-each): arrays & Iterables
    System.out.print(n + " ");
}
```

Loop	Control	Minimum runs	Best when
`while`	Entry-controlled	0	Number of iterations unknown (read until end of input, retry until success)
`do-while`	Exit-controlled	1	Body must run at least once (show a menu, then ask again)
`for`	Entry-controlled	0	Count is known; you need the index
for-each	Entry-controlled	0	Reading every element of an array or collection

for-loop variations

```
for (int i = 0, j = 10; i < j; i++, j--) { }   // several variables and updates
for (;;) { }                                    // all parts optional = infinite loop
for (int i = 10; i > 0; i -= 2) { }             // counting down in steps of 2
```

Limitations of for-each

- No access to the index.
- Assigning to the loop variable does **not** change the array: `for (int n : arr) n = 0;` leaves `arr` unchanged.
- Removing from a collection inside for-each throws `ConcurrentModificationException`. Use an `Iterator` or `removeIf()` (Phase 13).
- Only goes forward, over one collection at a time.

Infinite loops and unreachable code

```
while (true) { System.out.println("Hi"); }
System.out.println("Hello");   // COMPILE ERROR: unreachable statement

while (false) { System.out.println("Hi"); }   // COMPILE ERROR: unreachable statement
System.out.println("Hello");

if (false) { System.out.println("Hi"); }      // compiles: if(false) is allowed
                                              // (used for conditional compilation)
```

3.6 break, continue, return and Labels

Statement	Effect
`break`	Exits the **nearest** loop or switch immediately
`continue`	Skips the rest of the current iteration and goes to the next one
`return`	Exits the whole **method** (optionally returning a value)
`break label` / `continue label`	Applies to an **outer** labelled loop

```
for (int i = 1; i <= 10; i++) {
    if (i == 3) continue;    // skip 3
    if (i == 7) break;       // stop the loop at 7
    System.out.print(i + " ");
}
// Output: 1 2 4 5 6
```

```
outer:                                  // label: identifier followed by ':'
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) continue outer;     // jump to the next i
        if (i == 3) break outer;        // leave BOTH loops
        System.out.println(i + "," + j);
    }
}
// Output: 1,1  2,1
```

TIP

Labelled breaks are handy for searching a 2D grid, but if you need them often, extract the loops into a method and `return` instead. It reads better.

3.7 Nested Loops and Pattern Programs

```
// Multiplication table
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 5; j++) {
        System.out.print(i * j + "\t");
    }
    System.out.println();
}

int n = 4;
for (int i = 1; i <= n; i++) {          // right-angled triangle
    System.out.println("* ".repeat(i)); // *  /  * *  /  * * *  /  * * * *
}

for (int i = 1; i <= n; i++) {          // pyramid: spaces, then 2i-1 stars
    System.out.println(" ".repeat(n - i) + "*".repeat(2 * i - 1));
}
//    *
//   ***
//  *****
// *******
```

TIP

For pattern questions, the outer loop is almost always the **row** and the inner loop(s) control the **columns** (spaces first, then symbols). Work out the formula for row `i` before coding.

3.8 Reading Input from the Keyboard

```java
import java.io.*;

public class Eligibility {
    public static void main(String[] args) throws IOException {
        BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
        System.out.print("Enter age: ");
        int age = Integer.parseInt(br.readLine().trim());
        System.out.print("Enter salary: ");
        double salary = Double.parseDouble(br.readLine().trim());
        System.out.println(age >= 21 && salary > 25000 ? "Eligible" : "Not eligible");
    }
}
```

```
System.in (bytes) -> InputStreamReader (bytes -> chars) -> BufferedReader (buffered lines)
```

Option	Package	Speed	Parses numbers?	Checked exception?	Best for
`BufferedReader`	`java.io`	Fast (buffered)	No: use `parseInt`	Yes, `IOException`	Large input, competitive programming
`Scanner`	`java.util`	Slower (regex parsing)	Yes: `nextInt()`	No	Simple interactive programs
Command-line args	`String[] args`	Instant	No: use `parseInt`	No	Values known at launch
`Console`	`java.io`	n/a	No	No	Reading passwords (`readPassword()`)

3.9 Arrays: Basics

An **array** is a fixed-size container of elements of the **same type**, stored in **contiguous** memory and accessed by a zero-based **index**.

- Arrays are **objects** (a reference type), so they live on the heap.
- Size is fixed once created; use `ArrayList` when you need to grow.
- Elements are homogeneous (same type). An `Object[]` can hold anything, since everything is an `Object`.
- The `length` field gives the size. It is a **field**, not a method: `arr.length`, not `arr.length()`.

```java
int[] a;                // declaration (preferred style)
int b[];                // C-style, also valid
a = new int[5];         // creation: 5 ints, all 0
int[] c = new int[5];   // declaration + creation

int[] d = {10, 20, 30};             // array literal (only at declaration)
int[] e = new int[]{10, 20, 30};    // anonymous array: works anywhere
int[] f;
// f = {1, 2};                      // COMPILE ERROR: literal only allowed at declaration
f = new int[]{1, 2};                // OK
```

```
 Stack                Heap
+-------+        +------+------+------+------+------+
|  arr  | -----> |  100 |  101 |  102 |  103 |  104 |   int[] arr = new int[5];
+-------+        +------+------+------+------+------+
   index:             0      1      2      3      4       arr.length == 5
```

Default values of array elements

Element type	Default
`byte`, `short`, `int`, `long`	`0`
`float`, `double`	`0.0`
`char`	`'\u0000'`
`boolean`	`false`
Any reference (`String`, objects, inner arrays)	`null`

Size rules

```
int[] a = new int[0];        // OK: empty array (length 0), useful as a "no results" value
int n = 5; int[] b = new int[n * 2];   // OK: size may be any int expression
int[] c = new int['A'];      // OK: char promoted to int (65)
int[] d = new int[4.02];     // COMPILE ERROR: possible lossy conversion from double
int[] e = new int[5L];       // COMPILE ERROR: long not allowed
int[] f = new int[-4];       // compiles, but RUNTIME NegativeArraySizeException
System.out.println(c[65]);   // RUNTIME ArrayIndexOutOfBoundsException (valid: 0..64)
```

NOTE

The compiler doesn't know an array's size, because arrays are created at runtime. That is why a bad index is not a compile error: it throws `ArrayIndexOutOfBoundsException` at runtime. Valid indexes are `0` to `length - 1`.

3.10 Iterating and Printing Arrays

```
import java.util.Arrays;

int[] arr = {10, 20, 30};
for (int i = 0; i < arr.length; i++) System.out.print(arr[i] + " ");  // with index
for (int x : arr) System.out.print(x + " ");                          // for-each
System.out.println(Arrays.toString(arr));    // [10, 20, 30]
Arrays.stream(arr).forEach(System.out::println);   // streams (Phase 15)

System.out.println(arr);     // [I@15db9742  <- type code + hash, NOT the contents
```

NOTE

In `[I@15db9742`, `[` means array and `I` means int. Other codes are `[D` for double, `[Ljava.lang.String;` for String[] and `[[I` for int[][]. The part after `@` is the hash code in hex. Use `Arrays.toString()` for 1D arrays and `Arrays.deepToString()` for 2D arrays.

3.11 The java.util.Arrays Utility Class

Method	Purpose	Example
`toString(arr)`	Readable String of a 1D array	`[10, 20, 30]`
`deepToString(arr)`	Readable String of nested arrays	`[[1, 2], [3, 4]]`
`sort(arr)`	Sorts ascending (dual-pivot quicksort for primitives, TimSort for objects)	`Arrays.sort(arr);`
`sort(arr, from, to)`	Sorts a range (`to` exclusive)	`Arrays.sort(arr, 0, 3);`
`sort(objArr, comparator)`	Custom order (objects only)	`Arrays.sort(names, Comparator.reverseOrder());`
`binarySearch(arr, key)`	Index of key in a **sorted** array; negative if absent	`Arrays.binarySearch(arr, 20);`
`equals(a, b)` / `deepEquals(a, b)`	Compares contents (1D / nested)	`Arrays.equals(a1, a2)`
`fill(arr, val)`	Sets every element to `val`	`Arrays.fill(arr, -1);`
`copyOf(arr, newLen)`	New array, truncated or padded with defaults	`Arrays.copyOf(arr, 5);`
`copyOfRange(arr, from, to)`	Copies part (`to` exclusive)	`Arrays.copyOfRange(arr, 1, 4);`
`asList(...)`	Fixed-size `List` view backed by the array	`Arrays.asList("a", "b")`
`stream(arr)`	Stream of the elements	`Arrays.stream(arr).sum()`
`hashCode(arr)`	Content-based hash	`Arrays.hashCode(arr)`

```
int[] nums = {50, 10, 40, 20, 30};
Arrays.sort(nums);                                  // [10, 20, 30, 40, 50]
int pos = Arrays.binarySearch(nums, 40);            // 3
int sum = Arrays.stream(nums).sum();                // 150
int max = Arrays.stream(nums).max().getAsInt();     // 50

Integer[] boxed = {5, 1, 4};
Arrays.sort(boxed, Collections.reverseOrder());     // [5, 4, 1] (needs Integer[], not int[])

int[] a1 = {1, 2}, a2 = {1, 2};
System.out.println(a1 == a2);                 // false: different objects
System.out.println(a1.equals(a2));            // false: arrays don't override equals()!
System.out.println(Arrays.equals(a1, a2));    // true:  compares contents
```

PITFALL

`Arrays.asList()` returns a **fixed-size** list: `add()` and `remove()` throw `UnsupportedOperationException`, and changes to it write through to the array. For a real list use `new ArrayList<>(Arrays.asList(...))` or `List.of(...)` (immutable). Also, `Arrays.asList(intArray)` gives a `List<int[]>` with one element, not a list of numbers.

3.12 Copying Arrays

```
int[] src = {1, 2, 3, 4, 5};

int[] ref = src;                                // NOT a copy: both names, one array
ref[0] = 99;                                    // src[0] is now 99 too!

int[] c1 = src.clone();                         // new array, same elements
int[] c2 = Arrays.copyOf(src, src.length);      // same, can resize
int[] c3 = Arrays.copyOfRange(src, 1, 3);       // [2, 3]
int[] c4 = new int[5];
System.arraycopy(src, 0, c4, 0, src.length);    // (src, srcPos, dest, destPos, length)
```

> **NOTE**
> All of these are **shallow** copies. For an array of objects (or a 2D array), the new array points to the **same** inner objects or rows. A deep copy must copy each element or row (see object cloning in Phase 8).

3.13 Multidimensional and Jagged Arrays

A 2D array is an **array of arrays**: each row is a separate 1D array object.

```
int[][] m = {{100, 200}, {300, 400}};

 m ---> [ row0 ref | row1 ref ]
            |           |
            v           v
       [100, 200]   [300, 400]      m.length = 2 rows, m[0].length = 2 columns
```

```
int[][] m = new int[2][3];        // 2 rows x 3 columns, all 0
m[0][1] = 200;
int[][] lit = { {100, 200}, {300, 400} };

for (int i = 0; i < lit.length; i++) {             // index-based
    for (int j = 0; j < lit[i].length; j++) {      // use lit[i].length, not lit[0]
        System.out.print(lit[i][j] + " ");
    }
    System.out.println();
}
for (int[] row : lit) {                            // for-each
    System.out.println(Arrays.toString(row));
}
System.out.println(Arrays.deepToString(lit));      // [[100, 200], [300, 400]]
```

Jagged arrays (rows of different lengths)

```
int[][] jagged = new int[3][];   // only the number of rows is fixed
jagged[0] = new int[]{1};
jagged[1] = new int[]{1, 2};
jagged[2] = new int[]{1, 2, 3};
// Pascal's triangle, seating plans, monthly data with different day counts...

int[][] partial = new int[2][];
System.out.println(partial[0]);      // null: the rows were never created
System.out.println(partial[0][0]);   // NullPointerException
// int[][] bad = new int[][3];       // COMPILE ERROR: first dimension is required

int[][][] cube = new int[2][3][4];   // 3D: 2 blocks x 3 rows x 4 cols
```

3.14 Arrays vs ArrayList

Feature	Array	ArrayList
Size	Fixed at creation	Grows and shrinks automatically
Element types	Primitives **and** objects	Objects only (`Integer`, not `int`; autoboxing)
Length	`arr.length` (field)	`list.size()` (method)
Performance	Slightly faster, no boxing	Small overhead
Utilities	`Arrays` class	Rich `List` API: `add`, `remove`, `contains`, ...
Generics / type safety	Covariant (`Object[] o = new String[1]` compiles)	Invariant, checked at compile time
Multidimensional	Built in (`int[][]`)	List of lists

PITFALL

Array **covariance**: `Object[] objs = new String[2]; objs[0] = 10;` compiles but throws `ArrayStoreException` at runtime. Generic collections catch this at compile time.

3.15 Classic Array Programs

```
// 1. Reverse in place (two pointers)
int[] arr = {10, 20, 30, 40, 50};
for (int s = 0, e = arr.length - 1; s < e; s++, e--) {
    int t = arr[s]; arr[s] = arr[e]; arr[e] = t;
}
// [50, 40, 30, 20, 10]

// 2. Max, min, sum, average in one pass
int max = arr[0], min = arr[0], sum = 0;
for (int x : arr) {
    if (x > max) max = x;
    if (x < min) min = x;
    sum += x;
}
double avg = (double) sum / arr.length;   // cast first, or integer division

// 3. Second largest (single pass, handles duplicates)
int first = Integer.MIN_VALUE, second = Integer.MIN_VALUE;
for (int x : new int[]{5, 9, 9, 3, 7}) {
    if (x > first) { second = first; first = x; }
    else if (x > second && x != first) second = x;
}
// first = 9, second = 7
```

```
// 4. Non-repeated elements: O(n^2) nested loop (from the original notes)
int[] a = {1, 2, 3, 2, 4, 5, 1};
for (int i = 0; i < a.length; i++) {
    int count = 0;
    for (int j = 0; j < a.length; j++) {
        if (a[i] == a[j] && i != j) count++;
    }
    if (count == 0) System.out.print(a[i] + " ");   // 3 4 5
}

// 4b. Same in O(n) with a HashMap: the answer interviewers hope for
Map<Integer, Integer> freq = new LinkedHashMap<>();
for (int x : a) freq.merge(x, 1, Integer::sum);
freq.forEach((k, v) -> { if (v == 1) System.out.print(k + " "); });
```

```java
// 5. Binary search: O(log n), array must be sorted
static int binarySearch(int[] a, int key) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;   // avoids int overflow of (low + high)
        if (a[mid] == key) return mid;
        if (a[mid] < key) low = mid + 1; else high = mid - 1;
    }
    return -1;
}

// 6. Bubble sort: O(n^2), stops early when already sorted
static void bubbleSort(int[] a) {
    for (int i = 0; i < a.length - 1; i++) {
        boolean swapped = false;
        for (int j = 0; j < a.length - 1 - i; j++) {
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}
```

```java
// 7. Matrix addition and transpose
int[][] x = {{1, 2}, {3, 4}}, y = {{5, 6}, {7, 8}};
int[][] add = new int[2][2], tr = new int[2][2];
for (int i = 0; i < 2; i++)
    for (int j = 0; j < 2; j++) {
        add[i][j] = x[i][j] + y[i][j];   // [[6, 8], [10, 12]]
        tr[j][i] = x[i][j];              // [[1, 3], [2, 4]]
    }
```

3.16 Predict the Output

```java
int x = 5;
if (x > 2) if (x > 10) System.out.println("A"); else System.out.println("B");
// B : 'else' binds to the NEAREST if (dangling else)

int i = 0;
while (i++ < 3) System.out.print(i + " ");     // 1 2 3
int j = 0;
do { j++; } while (j < 0);  System.out.println(j);    // 1 (runs once)

for (int k = 0; k < 3; k++);                   // stray semicolon: empty loop body
{ System.out.println("once"); }                // runs ONCE, not 3 times

int[] arr = new int[3];
System.out.println(arr[1] + " " + arr.length);  // 0 3
String[] s = new String[2];
System.out.println(s[0]);                        // null
System.out.println(s[0].length());               // NullPointerException

char c = 'b';
switch (c) { case 'a': System.out.print("A"); case 'b': System.out.print("B");
             case 'c': System.out.print("C"); }  // BC (fall-through)
```

3.17 Interview Questions

Q. What is the difference between `while` and `do-while`?

`while` checks the condition before each iteration and may run zero times. `do-while` checks after, so its body runs at least once.

Q. Which types can be used in a switch?

`byte`, `short`, `char`, `int`, their wrappers, `String` (Java 7) and `enum`. From Java 21, any reference type through pattern matching. Never `long`, `float`, `double` or `boolean`.

Q. What is fall-through in a switch?

Without `break`, execution continues into the following case bodies. Arrow-style (`->`) cases never fall through.

Q. What is the difference between a switch statement and a switch expression?

A switch expression (Java 14) produces a value, uses `->` with no fall-through, allows comma-separated labels, uses `yield` inside blocks, and must be exhaustive.

Q. `break` vs `continue` vs `return`?

`break` exits the nearest loop or switch; `continue` skips to the next iteration; `return` exits the entire method. Labelled `break`/`continue` target an outer loop.

Q. Is an array a primitive or an object? Where is it stored?

Always an object, even `int[]`. It is created on the heap, and the variable holds a reference.

Q. Why is `length` not a method for arrays but `length()` is for Strings?

Array `length` is a final field set when the array is created. `String.length()` is a method of the `String` class.

Q. What happens with `int[] a = new int[-1];` and with `a[5]` on a 5-element array?

The first compiles but throws `NegativeArraySizeException` at runtime. The second throws `ArrayIndexOutOfBoundsException`. Both are runtime errors because sizes and indexes are checked when the code runs.

Q. How do you compare two arrays' contents?

`Arrays.equals(a, b)` (or `Arrays.deepEquals` for nested arrays). `==` and `a.equals(b)` both compare references.

Q. What is a jagged array?

A multidimensional array whose rows have different lengths, created by specifying only the first dimension (`new int[3][]`) and then each row separately.

Q. How do you copy an array?

`clone()`, `Arrays.copyOf()`, `Arrays.copyOfRange()` or `System.arraycopy()`. All are shallow copies; plain assignment copies only the reference.

3.18 Quick Revision

- Conditions must be `boolean`; no `if (1)`. Always use braces.
- Ladder: the first true branch wins. `else` binds to the nearest `if`.
- Switch types: byte/short/char/int (+ wrappers), String, enum, patterns (21). **No** long/float/double/boolean.
- Classic switch falls through without `break`; arrow switch doesn't, and can return a value (`yield` in blocks).
- `while` runs 0+ times, `do-while` 1+ times, `for` when the count is known, for-each for read-only traversal.
- `while (true)` / `while (false)` followed by code: unreachable statement compile error.
- Labelled `break outer;` exits nested loops.
- Arrays: fixed size, same type, heap objects, index `0..length-1`, `length` is a field.

- Element defaults: 0 / 0.0 / `'\u0000'` / false / null.
- Bad index → `ArrayIndexOutOfBoundsException`; negative size → `NegativeArraySizeException` (both at runtime).
- Print with `Arrays.toString` / `deepToString`; compare with `Arrays.equals`.
- Copies are shallow; `Arrays.asList` is fixed-size.
- 2D arrays are arrays of arrays; jagged rows are allowed; `new int[2][]` rows are `null`.

PHASE 4 OF 18

Strings, StringBuilder & Regex

String creation and the String Constant Pool, immutability, comparison, the String API, formatting, text blocks, StringBuilder/StringBuffer, regular expressions and classic string programs

IN THIS PHASE

4.1 What Is a String?
4.2 Creating Strings and the String Constant Pool
4.3 Immutability
4.4 Comparing Strings
4.5 The String API
4.6 Formatting Strings
4.7 Text Blocks (Java 15+)
4.8 StringBuilder and StringBuffer
4.9 StringJoiner and String.join
4.10 Regular Expressions (java.util.regex)
4.11 Classic String Programs
4.12 Predict the Output
4.13 Interview Questions
4.14 Quick Revision

4.1 What Is a String?

A `String` is an **object** that represents an immutable sequence of characters. Three classes in `java.lang` (no import needed) work with text:

Class	Mutable?	Thread-safe?	Since	Use when
`String`	No (immutable)	Yes (immutability makes it safe)	1.0	Text that doesn't change: names, keys, messages
`StringBuffer`	Yes	Yes (synchronized methods)	1.0	Building text shared across threads (rare today)
`StringBuilder`	Yes	No	1.5	Building or modifying text in one thread (the usual choice)

All three implement the `CharSequence` interface, which provides read-only access through `length()`, `charAt()`, `subSequence()` and `chars()`. Methods that accept a `CharSequence`, such as `contains()`, `append()` and `String.join()`, work with any of them.

```
String name = "Akshit";
String one  = "A";                                  // a single character is fine too
char[] data = {'a', 'b', 'c'};
String fromChars = new String(data);                // "abc"
String fromBytes = new String(bytes, StandardCharsets.UTF_8);  // always pass a charset
```

NOTE

Before Java 9, a String stored its text in a `char[]` (2 bytes per character). Since Java 9, **compact strings** store a `byte[]` plus an encoding flag: Latin-1 text uses 1 byte per character, otherwise UTF-16. This roughly halves memory for typical English text.

4.2 Creating Strings and the String Constant Pool

Approach	Example	Where the object goes
String literal	`String s = "Java";`	**String Constant Pool (SCP)**, reused if already present
`new` keyword	`String s = new String("Java");`	A **new** object on the heap every time (plus the literal in the SCP)

The **String Constant Pool** is a special area **inside the heap** (since Java 7; it was in PermGen before). Before creating a literal, the JVM checks the pool: if an equal String exists, the reference is reused, so the pool never holds duplicates.

```
String s1 = "Java";
String s2 = "Java";
String s3 = new String("Java");

                HEAP
   +-----------------------------------------+
   |  String Constant Pool (SCP)             |
   |  +-----------------------------------+  |
   |  |   "Java"   <----  s1, s2 (shared)  |  |
   |  +-----------------------------------+  |
   |                                         |
   |   "Java"   <----  s3 (separate object)  |
   +-----------------------------------------+

s1 == s2  -> true     s1 == s3  -> false     s1.equals(s3) -> true
```

How many objects are created?

Statement (run in order)	New objects	Why
`String s1 = "Hello";`	1	Literal placed in the SCP
`String s2 = "Hello";`	0	Reuses the SCP object
`String s3 = new String("Hello");`	1	Heap object only; `"Hello"` is already pooled
`String s4 = new String("Bye");`	2	`"Bye"` literal in the SCP + heap object
`String s5 = "Hel" + "lo";`	0	Constant expression folded at compile time to `"Hello"`

> **INTERVIEW**
> "How many objects does `new String("Hello")` create?" The standard answer is **two** (one in the pool for the literal, one on the heap) if the literal isn't pooled yet, otherwise **one**.

Benefits of the pool

- **Memory:** one shared copy of each repeated literal.
- **Performance:** fewer allocations, and `==` comparison of pooled constants is cheap.
- **Safe sharing:** possible only because Strings are immutable.

4.3 Immutability

Once created, a String object's content **can never change**. Every "modifying" method returns a **new** String.

```
String s = "Hello";
s.concat(" Java");          // creates "Hello Java" but the result is discarded
System.out.println(s);      // Hello  (unchanged!)

s = s.concat(" Java");      // reassign the REFERENCE to the new object
System.out.println(s);      // Hello Java   ("Hello" itself still exists unchanged)

s.toUpperCase();            // same mistake: result ignored
s = s.toUpperCase();        // correct
```

Why Java made String immutable

1. **String pool:** many references share one object. If one could change it, every other reference would see the change.
2. **Security:** class names, file paths, URLs, DB credentials and usernames are Strings. They can't be changed after they have been validated.
3. **Thread safety:** immutable objects can be shared between threads without synchronization.
4. **Hash code caching:** the hash is computed once and cached, which makes Strings fast and reliable as `HashMap` keys.
5. **Class loading:** class names passed to class loaders can't be tampered with.

CORRECTED FROM ORIGINAL NOTES

The original notes have the heading "Why Strings are **mutable**"; it should say **immutable**. Its example also needs clarifying: after `String s2 = s1; s1 = "User";`, `s2` is still `"Admin"` because only `s1`'s *reference* changed. Immutability matters because nobody can change the shared `"Admin"` object itself.

PITFALL

Immutability means a String's **content** can't change. The **variable** can still point to a different String unless it is `final`.

4.4 Comparing Strings

Way	Compares	Example	Result
`==`	**References** (same object?)	`"a" == new String("a")`	`false`
`equals()`	**Content**, case-sensitive	`"Java".equals("java")`	`false`
`equalsIgnoreCase()`	Content, ignoring case	`"JAVA".equalsIgnoreCase("java")`	`true`
`compareTo()`	Lexicographic (dictionary) order	`"apple".compareTo("banana")`	negative
`compareToIgnoreCase()`	Order, ignoring case	`"A".compareToIgnoreCase("a")`	`0`
`Objects.equals(a, b)`	Null-safe content check	`Objects.equals(null, null)`	`true`
`contentEquals()`	Content vs any `CharSequence`	`"ab".contentEquals(sb)`	`true` / `false`

```
String s1 = "Hello", s2 = "Hello";
String s3 = new String("Hello"), s4 = new String("Hello");
System.out.println(s1 == s2);         // true  (same pooled object)
System.out.println(s3 == s4);         // false (two heap objects)
System.out.println(s1 == s3);         // false
System.out.println(s3.equals(s4));    // true  (same content)

// compareTo: 0 if equal, negative if this < other, positive if this > other
"abc".compareTo("abc");     // 0
"a".compareTo("b");         // -1 ('a' - 'b')
"apple".compareTo("app");   // 2  (length difference when one is a prefix)
"Z".compareTo("a");         // -7 (uppercase letters sort BEFORE lowercase)
```

Compile-time constants and ==

```
String a = "Java";
String b = "Ja" + "va";        // folded at compile time -> same pooled "Java"
System.out.println(a == b);    // true

String part = "Ja";
String c = part + "va";        // built at RUNTIME -> new heap object
System.out.println(a == c);    // false

final String fpart = "Ja";
String d = fpart + "va";       // final constant -> folded at compile time
System.out.println(a == d);    // true
```

intern()

`intern()` returns the **pooled** copy of a String. If none exists yet, it adds this one to the pool first.

```
String s1 = new String("Hello");   // heap object
String s2 = s1.intern();           // reference to the pooled "Hello"
String s3 = "Hello";
System.out.println(s1 == s2);      // false (heap vs pool)
System.out.println(s2 == s3);      // true  (both pooled)
```

> **REMEMBER**
> Always use `equals()` to compare String content; `==` is almost always a bug. For null-safety, put the literal first (`"ADMIN".equals(role)`) or use `Objects.equals(a, b)`.

4.5 The String API

Indexes start at **0**. Ranges are **[begin, end)**: begin is inclusive, end is exclusive.

```
String s = "Hello";      index:  0   1   2   3   4
                         char:   H   e   l   l   o      s.length() = 5
```

Category	Method	Example	Result
Inspect	`length()`	`"Hello".length()`	`5`
Inspect	`charAt(i)`	`"Hello".charAt(1)`	`'e'`
Inspect	`isEmpty()`	`"".isEmpty()`	`true`
Inspect	`isBlank()` (Java 11)	`" ".isBlank()`	`true`
Search	`indexOf(str)` / `indexOf(str, from)`	`"banana".indexOf("an")`	`1`
Search	`lastIndexOf(str)`	`"banana".lastIndexOf('a')`	`5` (`-1` if absent)
Search	`contains(seq)`	`"Hello".contains("ell")`	`true`
Search	`startsWith()` / `endsWith()`	`"Hello".startsWith("He")`	`true`
Search	`matches(regex)`	`"12345".matches("\\d+")`	`true` (whole string)
Extract	`substring(begin)`	`"Hello".substring(2)`	`"llo"`
Extract	`substring(begin, end)`	`"Hello".substring(1, 4)`	`"ell"`
Extract	`toCharArray()`	`"abc".toCharArray()`	`['a','b','c']`
Extract	`chars()`	`"abc".chars().count()`	`3` (IntStream)
Transform	`toUpperCase()` / `toLowerCase()`	`"java".toUpperCase()`	`"JAVA"`
Transform	`trim()`	`" Java ".trim()`	`"Java"` (removes chars ≤ space)

Category	Method	Example	Result
Transform	`strip()`, `stripLeading()`, `stripTrailing()` (Java 11)	`"\u2003Java ".strip()`	`"Java"` (Unicode-aware)
Transform	`replace(old, new)`	`"Java".replace('a', 'o')`	`"Jovo"` (literal, all occurrences)
Transform	`replaceAll(regex, rep)` / `replaceFirst()`	`"a1b22".replaceAll("\\d", "")`	`"ab"`
Transform	`concat(str)`	`"Hi".concat(" there")`	`"Hi there"`
Transform	`repeat(n)` (Java 11)	`"ab".repeat(3)`	`"ababab"`
Split / join	`split(regex)`	`"A,B,C".split(",")`	`[A, B, C]`
Split / join	`String.join(delim, parts)`	`String.join("-", "a", "b")`	`"a-b"`
Split / join	`lines()` (Java 11)	`"a\nb".lines().count()`	`2`
Compare	`equals`, `equalsIgnoreCase`, `compareTo`	see the previous section	
Convert	`String.valueOf(x)`	`String.valueOf(10.5)`	`"10.5"`
Format	`String.format()` / `formatted()` (Java 15)	`"%d%%".formatted(5)`	`"5%"`
Pool	`intern()`	`s.intern()`	pooled reference

```
String str = "Hello", str1 = "Java", str2 = "JAVA", str3 = "  Java  ";
System.out.println(str.charAt(1));                    // e
System.out.println(str1.equalsIgnoreCase(str2));      // true
System.out.println(str.substring(1, 4));              // ell
System.out.println(str3.trim() + "|");                // Java|
System.out.println(Arrays.toString("A,B,C".split(","))); // [A, B, C]
System.out.println("banana".replace("an", "AN"));     // bANANa
System.out.println("  ".isEmpty() + " " + "  ".isBlank());  // false true

"Hello".charAt(10);        // StringIndexOutOfBoundsException
"Hello".substring(-1, 4);  // StringIndexOutOfBoundsException
"Hello".substring(3, 1);   // StringIndexOutOfBoundsException (begin > end)
"Hello".substring(5);      // "" (begin == length is allowed)
```

split() gotchas

```
"a.b.c".split(".");          // [] : '.' is a regex meaning "any character"!
"a.b.c".split("\\.");        // [a, b, c]   escape regex metacharacters
"a|b".split("\\|");          // [a, b]
"a.b".split(Pattern.quote(".")); // [a, b]  quote() treats the text literally

"a,b,,c,,".split(",");       // [a, b, , c]  trailing empty strings REMOVED
"a,b,,c,,".split(",", -1);   // [a, b, , c, , ]  limit -1 keeps them (CSV parsing!)
"a,b,c".split(",", 2);       // [a, b,c]     at most 2 parts
"  Hello   Java  ".trim().split("\\s+");  // [Hello, Java]  any run of whitespace
```

> **TIP**
> Regex metacharacters that need escaping in `split()`/`replaceAll()` are `. $ | ( ) [ ] { } ^ ? * + \`.
> Use `replace()` (not `replaceAll()`) for plain-text replacement: it is literal and needs no escaping.

4.6 Formatting Strings

```
String msg = String.format("Account %s has balance Rs.%,.2f", "SB-1001", 125000.5);
// Account SB-1001 has balance Rs.125,000.50
String msg2 = "Hi %s, you are %d".formatted("Rohit", 35);   // Java 15+
System.out.printf("%-10s|%5d|%n", "Item", 42);              // left/right aligned columns
```

Specifier	Meaning	Example	Output
%s	String (any object via toString())	format("%s", "hi")	hi
%d	Integer	format("%d", 42)	42
%f / %.2f	Floating point / 2 decimals	format("%.2f", 3.14159)	3.14
%,d	Thousands separator	format("%,d", 1000000)	1,000,000
%05d	Zero-padded width 5	format("%05d", 42)	00042
%10s / %-10s	Width 10, right / left aligned	format("[%-5s]", "ab")	[ab]
%x / %X	Hexadecimal	format("%x", 255)	ff
%c, %b, %e	Char, boolean, scientific	format("%e", 12345.0)	1.234500e+04
%n, %%	Platform newline, literal %	format("%d%%", 5)	5%

4.7 Text Blocks (Java 15+)

Multi-line string literals with no \n or escaped quotes. Ideal for JSON, SQL, XML and HTML in code and tests.

```
String json = """
        {
          "accountId": "SB-1001",
          "status": "ACTIVE"
        }
        """;          // the indentation of the closing """ sets the left margin

String sql = """
        SELECT id, name
        FROM customers
        WHERE city = ?
        """;
```

4.8 StringBuilder and StringBuffer

Both are **mutable** character sequences. They modify the **same object** instead of creating new ones, which is much faster when text changes repeatedly. Their APIs are identical; the difference is synchronization.

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" Java");                 // Hello Java
sb.insert(5, ",");                  // Hello, Java
sb.replace(0, 5, "Hi");             // Hi, Java
sb.delete(2, 3);                    // Hi Java
sb.deleteCharAt(0);                 // i Java
sb.setCharAt(0, 'I');               // I Java
sb.reverse();                       // avaJ I
String result = sb.toString();      // convert back to an immutable String

String csv = new StringBuilder()    // methods return 'this' -> chaining
        .append("id").append(',').append(101).append(',').append(true)
        .toString();                // "id,101,true"

// StringBuilder sb2 = "Hello";     // COMPILE ERROR: a literal is a String, not a StringBuilder
```

Method	Purpose	Returns
`append(x)`	Adds any type at the end (overloaded for all primitives and objects)	same builder
`insert(i, x)`	Inserts at index `i`	same builder
`delete(start, end)` / `deleteCharAt(i)`	Removes a range (end exclusive) / one char	same builder
`replace(start, end, str)`	Replaces a range	same builder
`reverse()`	Reverses in place	same builder
`setCharAt(i, c)` / `setLength(n)`	Changes one char / truncates or pads	`void`
`charAt`, `indexOf`, `lastIndexOf`, `substring`, `length`	Same meaning as in String	various
`capacity()` / `ensureCapacity(n)` / `trimToSize()`	Internal buffer size management	`int` / `void`
`toString()`	New immutable String with the current content	`String`

Capacity

The default capacity is **16** characters. `new StringBuilder("Hello")` has capacity `16 + 5 = 21`. When full, the builder grows to `(old capacity × 2) + 2` and copies the content. If you know the final size, pre-size it: `new StringBuilder(1024)`.

```
StringBuilder sb = new StringBuilder();
System.out.println(sb.capacity());   // 16
sb.append("12345678901234567");      // 17 chars > 16
System.out.println(sb.capacity());   // 34 = 16 * 2 + 2
```

Comparison: String vs StringBuffer vs StringBuilder

Feature	String	StringBuffer	StringBuilder
Mutability	Immutable	Mutable	Mutable
Modification	Creates a new object	Changes the same object	Changes the same object
Thread-safe	Yes (immutable)	Yes (synchronized)	No
Speed for repeated changes	Slowest (creates garbage)	Medium (lock overhead)	Fastest
`equals()`	Compares content	**Not overridden** (reference)	**Not overridden** (reference)
Storage	SCP (literals) or heap	Heap	Heap
Since	1.0	1.0	1.5

```
// Why it matters: + inside a loop creates a new String (and garbage) every iteration
String s = "";
for (int i = 0; i < 10_000; i++) s += i;          // O(n^2) copying: slow

StringBuilder sb = new StringBuilder();
for (int i = 0; i < 10_000; i++) sb.append(i);    // amortised O(n): fast
```

PITFALL

`new StringBuilder("a").equals(new StringBuilder("a"))` is **false**, because StringBuilder and StringBuffer don't override `equals()`. Compare `sb1.toString().equals(sb2.toString())`, or use `sb1.compareTo(sb2) == 0` (Java 11+).

> **NOTE**
> For a **single** expression such as `"Hi " + name + "!"`, the compiler already optimizes concatenation (StringBuilder before Java 9; `invokedynamic` string concat since Java 9), so `+` is fine there. The cost appears with `+=` inside **loops**.

4.9 StringJoiner and String.join

```
String.join(", ", "Java", "Python", "Go");          // Java, Python, Go
String.join("/", List.of("home", "rohit", "docs")); // home/rohit/docs

StringJoiner sj = new StringJoiner(", ", "[", "]"); // delimiter, prefix, suffix
sj.add("A").add("B").add("C");
System.out.println(sj);                             // [A, B, C]

List.of("x", "y").stream().collect(Collectors.joining("|"));   // x|y
```

> **NOTE**
> The original short notes list the rules for writing your own **immutable class** (final class, private final fields, no setters, defensive copies). They are covered fully, with examples, in Phase 8.

4.10 Regular Expressions (java.util.regex)

A **regular expression (regex)** is a pattern that describes text. Java provides two main classes:

- `Pattern`: a *compiled* regex. Create it once with `Pattern.compile()` and reuse it; it is thread-safe.
- `Matcher`: applies a Pattern to a specific input string and tracks the matches. It is not thread-safe.

> **CORRECTED FROM ORIGINAL NOTES**
> In the original short notes, the descriptions of the `Pattern` and `Matcher` method tables are swapped. `compile()`, `matcher()`, `matches(regex, input)`, `split()` and `quote()` belong to **Pattern**. `find()`, `matches()`, `group()`, `start()`, `end()` and `replaceAll()` belong to **Matcher**.

```
import java.util.regex.*;

Pattern p = Pattern.compile("\\d+");          // one or more digits
Matcher m = p.matcher("The car costs $500, the bike costs $200.");
while (m.find()) {                            // find the NEXT match each call
    System.out.println(m.group() + " at " + m.start() + "-" + m.end());
}
// 500 at 15-18
// 200 at 36-39

Pattern.matches("\\d{6}", "560001");          // true: the WHOLE input must match
"560001".matches("\\d{6}");                   // same, via String
```

Pattern method	Purpose
`compile(regex)` / `compile(regex, flags)`	Compile a regex (optionally with flags)
`matcher(input)`	Create a Matcher for an input
`matches(regex, input)` (static)	One-off full match
`split(input)`	Split by the pattern
`quote(text)`	Escape text so it is matched literally

Matcher method	Purpose
`matches()`	Does the **entire** input match?
`find()`	Is there a **next** match anywhere? (use in a loop)

Matcher method	Purpose
`lookingAt()`	Does the **beginning** of the input match?
`group()` / `group(n)` / `group("name")`	Text of the whole match / group n / named group
`start()`, `end()`	Index range of the current match
`replaceAll(rep)`, `replaceFirst(rep)`	Replace matches (`$1` refers to group 1)
`reset()`	Start matching again from the beginning

Regex syntax cheat sheet

Syntax	Meaning	Syntax	Meaning
`.`	Any character (except newline)	`\d` / `\D`	Digit / non-digit
`[abc]`	One of a, b or c	`\w` / `\W`	Word char `[a-zA-Z0-9_]` / non-word
`[^abc]`	Any character except a, b, c	`\s` / `\S`	Whitespace / non-whitespace
`[a-z]`, `[A-Z0-9]`	Ranges	`\b`	Word boundary
`^` / `$`	Start / end of input (or line with MULTILINE)	`\.`	A literal dot
`X*`	0 or more	`X{n}`	Exactly n
`X+`	1 or more	`X{n,}`	At least n
`X?`	0 or 1 (optional)	`X{n,m}`	Between n and m
`X*?`, `X+?`	**Lazy** (as few as possible)	`a|b`	a or b
`(X)`	Capturing group	`(?<name>X)`	Named group
`(?:X)`	Non-capturing group	`(?i)`	Inline case-insensitive flag

> **REMEMBER**
> In Java source code the backslash must be escaped, so the regex `\d` is written `"\\d"` and a literal backslash is `"\\\\"`.

Flag	Effect
`Pattern.CASE_INSENSITIVE`	Ignore case (`(?i)` inline)
`Pattern.MULTILINE`	`^` and `$` match at line breaks
`Pattern.DOTALL`	`.` also matches newline
`Pattern.UNICODE_CASE`	Unicode-aware case folding (with CASE_INSENSITIVE)
`Pattern.COMMENTS`	Allow whitespace and `#` comments in the pattern

Practical patterns (validation and extraction)

Purpose	Regex (as a Java string)
10-digit Indian mobile starting 6–9	`"[6-9]\\d{9}"`
PIN code (6 digits, not starting 0)	`"[1-9]\\d{5}"`
PAN card	`"[A-Z]{5}[0-9]{4}[A-Z]"`
IFSC code	`"[A-Z]{4}0[A-Z0-9]{6}"`
Simple email	`"^[\\w.+-]+@[\\w-]+(\\.[\\w-]+)+$"`
Date `dd-MM-yyyy` (format only)	`"\\d{2}-\\d{2}-\\d{4}"`
Strong password (8+ chars, upper, lower, digit, special)	`"^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@#$%!]).{8,}$"`

```
// Pre-compile once (static final) and reuse: compiling is the expensive part
private static final Pattern PAN = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");
boolean ok = PAN.matcher("ABCDE1234F").matches();            // true

// Capturing groups: extract day, month, year
Matcher m = Pattern.compile("(\\d{2})-(\\d{2})-(\\d{4})").matcher("Due: 15-08-2026");
if (m.find()) {
    System.out.println(m.group(1) + "/" + m.group(2) + "/" + m.group(3)); // 15/08/2026
}

// Named groups
Matcher n = Pattern.compile("(?<code>[A-Z]{3})(?<amt>\\d+)").matcher("INR500");
if (n.matches()) System.out.println(n.group("code") + " " + n.group("amt")); // INR 500

// Replace using group references: mask all but the last 4 card digits
"4111111111111111".replaceAll("\\d(?=\\d{4})", "*");   // ************1111
"2026-08-15".replaceAll("(\\d{4})-(\\d{2})-(\\d{2})", "$3/$2/$1");  // 15/08/2026

// Greedy vs lazy
"<b>bold</b>".replaceAll("<.*>", "");    // ""     greedy .* eats everything
"<b>bold</b>".replaceAll("<.*?>", "");   // "bold" lazy .*? stops at the first >
```

PITFALL

`String.matches()`, `replaceAll()` and `split()` compile the regex **on every call**. In a loop or a hot path, use a pre-compiled `static final Pattern`. Also, `matches()` needs the **whole** string to match; use `find()` to search inside it.

4.11 Classic String Programs

```
// 1. Reverse a string: three ways
String s = "Java";
String r1 = new StringBuilder(s).reverse().toString();   // "avaJ"  (best)
String r2 = "";
for (int i = s.length() - 1; i >= 0; i--) r2 += s.charAt(i);  // loop (fine for practice)
char[] c = s.toCharArray();                                   // two-pointer swap
for (int i = 0, j = c.length - 1; i < j; i++, j--) { char t = c[i]; c[i] = c[j]; c[j] = t; }
String r3 = new String(c);

// 2. Palindrome check (ignore case and non-letters)
static boolean isPalindrome(String str) {
    String t = str.toLowerCase().replaceAll("[^a-z0-9]", "");
    for (int i = 0, j = t.length() - 1; i < j; i++, j--)
        if (t.charAt(i) != t.charAt(j)) return false;
    return true;
}   // isPalindrome("Madam, I'm Adam") -> true
```

```
// 3. Anagram: sort-based O(n log n), from the original notes
static boolean isAnagram(String a, String b) {
    char[] x = a.toLowerCase().toCharArray(), y = b.toLowerCase().toCharArray();
    if (x.length != y.length) return false;
    Arrays.sort(x); Arrays.sort(y);
    return Arrays.equals(x, y);
}
// 3b. Anagram: counting O(n) for lowercase a-z
static boolean isAnagramFast(String a, String b) {
    if (a.length() != b.length()) return false;
    int[] count = new int[26];
    for (int i = 0; i < a.length(); i++) {
        count[a.charAt(i) - 'a']++;
        count[b.charAt(i) - 'a']--;
    }
    for (int n : count) if (n != 0) return false;
    return true;
}   // listen/silent, heart/earth -> true;  hello/world -> false
```

```
// 4. Count vowels / remove vowels
String text = "Hello Java Programming";
long vowels = text.toLowerCase().chars().filter(ch -> "aeiou".indexOf(ch) >= 0).count(); // 7
String noVowels = text.replaceAll("[aeiouAEIOU]", "");      // Hll Jv Prgrmmng

// 5. Word count (handles multiple spaces)
int words = text.isBlank() ? 0 : text.trim().split("\\s+").length;   // 3

// 6. Character frequency (insertion order kept)
Map<Character, Integer> freq = new LinkedHashMap<>();
for (char ch : "programming".toCharArray()) freq.merge(ch, 1, Integer::sum);
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}

// 7. First non-repeating character
for (Map.Entry<Character, Integer> e : freq.entrySet())
    if (e.getValue() == 1) { System.out.println(e.getKey()); break; }   // p
```

```
// 8. Reverse the order of words
String[] parts = "I love Java".trim().split("\\s+");
Collections.reverse(Arrays.asList(parts));
String reversed = String.join(" ", parts);          // "Java love I"

// 9. Remove duplicate characters, keeping order
StringBuilder out = new StringBuilder();
for (char ch : "banana".toCharArray())
    if (out.indexOf(String.valueOf(ch)) == -1) out.append(ch);    // "ban"

// 10. Toggle case
StringBuilder tg = new StringBuilder();
for (char ch : "HeLLo".toCharArray())
    tg.append(Character.isUpperCase(ch) ? Character.toLowerCase(ch)
                                        : Character.toUpperCase(ch));   // "hEllO"

// 11. Contains only digits?
boolean digits = "12345".chars().allMatch(Character::isDigit);  // true
```

TIP

Useful `Character` helpers: `isDigit`, `isLetter`, `isLetterOrDigit`, `isWhitespace`, `isUpperCase`, `isLowerCase`, `toUpperCase`, `toLowerCase`, `getNumericValue`.

4.12 Predict the Output

```
String s1 = "Java";
String s2 = s1;
s1 += " Rocks";
System.out.println(s2);                         // Java (s2 still refers to the old object)

String a = "Hi", b = new String("Hi");
System.out.println(a == b.intern());            // true

System.out.println("Java".substring(4));        // "" (empty, no exception)
System.out.println("abc".compareTo("abd"));     // -1
System.out.println("10" + 20 + 30);             // 102030
System.out.println(10 + 20 + "30");             // 3030
String n = null;
System.out.println(n + "x");                    // nullx (no NPE for concatenation)
System.out.println(n.length());                 // NullPointerException

StringBuilder sb = new StringBuilder("abc");
StringBuilder sb2 = sb.append("d");
System.out.println(sb == sb2);                  // true (append returns the same object)
System.out.println("a,b,,".split(",").length);  // 2 (trailing empties removed)
System.out.println("Hello".replace('l', 'L').indexOf('L'));   // 2
```

4.13 Interview Questions

Q. Why is String immutable in Java?
To make the String pool safe (shared literals can't be changed through one reference), for security (paths, URLs, class names and credentials can't be altered after validation), for thread safety, and so the hash code can be cached for fast `HashMap` lookups.

Q. What is the String Constant Pool?
A special area in the heap (since Java 7) where literals are stored without duplicates. A literal that already exists in the pool is reused instead of creating a new object.

Q. What is the difference between `==` and `equals()` for Strings?
`==` compares references (same object?). `equals()` compares character content. Use `equals()` for content.

Q. How many objects are created by `String s = new String("abc");`?
Up to two: the `"abc"` literal in the pool (if not already there) and a new String object on the heap.

Q. What does `intern()` do?
It returns the canonical pooled instance of the String, adding it to the pool if absent. Afterwards, `==` works between interned strings.

Q. What is the difference between String, StringBuffer and StringBuilder?
String is immutable. StringBuffer and StringBuilder are mutable. StringBuffer's methods are synchronized (thread-safe, slower); StringBuilder's are not (faster). Use StringBuilder for building strings in a single thread.

Q. Why should you avoid `+=` with Strings in a loop?
Each `+=` creates a new String and copies all previous characters, which is $O(n^2)$ and creates lots of garbage. StringBuilder appends into one growing buffer.

Q. Does StringBuilder override `equals()`?
No. It uses `Object.equals()` (reference comparison). Compare `toString()` results, or use `compareTo()` (Java 11+).

Q. What is the difference between `trim()` and `strip()`?
`trim()` removes characters with code ≤ U+0020 (ASCII whitespace and control characters). `strip()` (Java 11) removes all Unicode whitespace, such as U+2003 em space.

Q. What is the difference between `replace()` and `replaceAll()`?
`replace()` does literal replacement of all occurrences (char or CharSequence). `replaceAll()` treats its first argument as a **regex**.

Q. What is the difference between `Matcher.matches()` and `find()`?
`matches()` requires the entire input to match the pattern. `find()` searches for the next matching subsequence anywhere in the input and can be called repeatedly.

Q. Why is String a popular `HashMap` key?
It is immutable (its hash can't change after insertion), it caches its hash code, and it correctly overrides `equals()` and `hashCode()`.

4.14 Quick Revision

- String is an **immutable object**; StringBuilder/StringBuffer are **mutable**. All implement `CharSequence`.
- Literals go into the **String Constant Pool** (in the heap); `new String()` always creates a new heap object.
- `==` compares references; `equals()` **compares content**. Use a literal first or `Objects.equals` for null safety.

- Compile-time constants (`"Ja" + "va"`, `final` parts) are folded and pooled; runtime concatenation is not.
- String methods return **new** Strings, so assign the result: `s = s.trim();`.
- Index ranges are [begin, end); a bad index throws `StringIndexOutOfBoundsException`.
- `split`/`replaceAll`/`matches` take **regex**: escape `.` and `|`; `split` drops trailing empties unless limit is -1.
- Java 11: `isBlank`, `strip`, `repeat`, `lines`. Java 15: text blocks, `formatted()`.
- StringBuilder: default capacity 16, grows to `old*2 + 2`, does **not** override `equals()`.
- `StringBuffer` = synchronized; `StringBuilder` = fast. Never use `+=` in loops.
- Regex: `Pattern` (compiled, reusable, thread-safe) + `Matcher` (`find`, `matches`, `group`). Write `\d` as `"\\d"`.
- Pre-compile hot regexes as `static final Pattern`.

PHASE 5 OF 18

Methods, Classes & Objects

OOP vs procedural, classes and objects in memory, garbage collection basics, variable types, methods, pass-by-value, varargs, recursion, constructors, this, static and initialization order

IN THIS PHASE

5.1 Procedural vs Object-Oriented Programming
5.2 Classes and Objects
5.3 Objects in Memory
5.4 Garbage Collection Basics
5.5 Variables in Depth: Instance, Static and Local
5.6 Methods
5.7 Java Is Always Pass-by-Value
5.8 Variable Arguments (Varargs)
5.9 Recursion
5.10 Constructors
5.11 The this Keyword
5.12 The static Keyword
5.13 Initialization Blocks and Execution Order
5.14 Predict the Output
5.15 Interview Questions
5.16 Quick Revision

5.1 Procedural vs Object-Oriented Programming

Aspect	Procedural (POP)	Object-Oriented (OOP)
Program is built from	Functions / procedures	Classes and objects
Focus	Functions (steps)	Data + behaviour together
Data	Often global, freely accessible	Hidden inside objects (`private`), accessed through methods
Security	Low	High: data hiding and access control
Reuse	Copy or call functions	Inheritance, composition, polymorphism
Maintenance of large apps	Harder	Easier: modular, changes stay local
Examples	C, COBOL, Pascal	Java, C++, C#, Python, Kotlin

Simula 67 is considered the first object-oriented language. OOP rests on four pillars, which Phases 6 and 7 cover in depth:

Pillar	One-line meaning	Achieved with
Encapsulation	Bundle data and methods together and protect the data	`private` fields + getters/setters
Abstraction	Show *what* an object does, hide *how*	Abstract classes, interfaces
Inheritance	Reuse and extend an existing class (IS-A)	`extends`, `implements`
Polymorphism	One name, many forms	Overloading (compile time), overriding (runtime)

5.2 Classes and Objects

A **class** is a blueprint or template: it declares the **state** (fields), **behaviour** (methods) and **construction** (constructors) of a type. An **object** is a concrete **instance** of a class that exists in memory at runtime.

```
public class Student {                  // class: the blueprint
    int id;                             // fields = state
    String name;

    Student(int id, String name) {      // constructor = how to build one
        this.id = id;
        this.name = name;
    }

    void display() {                    // method = behaviour
        System.out.println(id + " " + name);
    }
}

Student s1 = new Student(101, "Raju");   // object 1
Student s2 = new Student(102, "Ravi");   // object 2: own copy of id and name
s1.display();                            // 101 Raju
```

```
Student s1 = new Student(101, "Raju");
  |       |    |     |
  |       |    |     +-- constructor call: initialises the new object
  |       |    +-------- new: allocates memory on the heap, returns its reference
  |       +------------- reference variable (on the stack if local)
  +--------------------- type of the reference (the class)
```

Class	Object
Blueprint / template / user-defined type	Instance of a class
Logical entity, declared once	Physical entity, created many times
Takes no heap memory for its fields (metadata lives in Metaspace)	Occupies heap memory for its own fields
Declared with the `class` keyword	Created with `new` (mostly)
Example: `Student`	Example: `s1`, `s2`

Ways to create an object

Way	Example
`new` keyword (most common)	`Student s = new Student();`
Static factory method	`List.of(1, 2)`, `LocalDate.now()`, `Integer.valueOf(5)`
`clone()`	`Student copy = (Student) s.clone();` (Phase 8)
Reflection	`Student.class.getDeclaredConstructor().newInstance();`
Deserialization	`(Student) objectInputStream.readObject();` (Phase 12)

References, null and anonymous objects

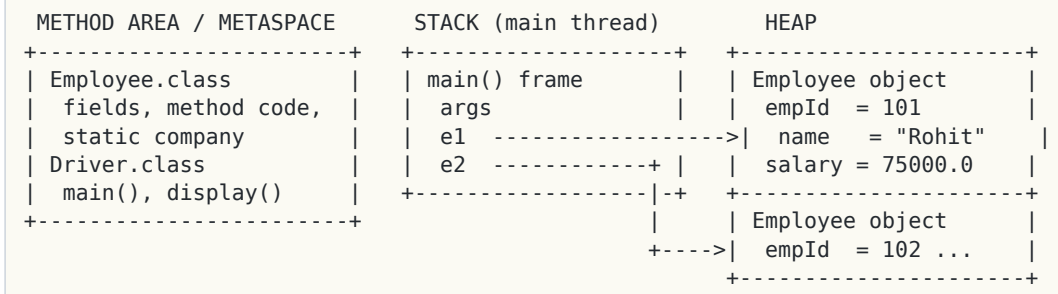
```
Student a = new Student(1, "A");
Student b = a;               // copies the REFERENCE: a and b point to the same object
b.name = "Changed";
System.out.println(a.name);  // Changed

Student c = null;            // refers to no object
c.display();                 // NullPointerException

new Student(5, "Temp").display();   // anonymous object: used once, then GC-eligible
```

5.3 Objects in Memory

```
public class Driver {
    public static void main(String[] args) {
        Employee e1 = new Employee(101, "Rohit", 75000);
        Employee e2 = new Employee(102, "Amit", 65000);
    }
}
```



What happens when a method returns an object

```
class Person { int id; String name; int age; }

class Driver {
    Person getPerson() {
        Person p = new Person();   // object on the heap, defaults: 0, null, 0
        p.id = 101; p.name = "Raju"; p.age = 20;
        return p;                  // returns the REFERENCE; the object is not copied
    }
    public static void main(String[] args) {
        Driver d = new Driver();
        Person p = d.getPerson();
        System.out.println(p.id + " " + p.name + " " + p.age);   // 101 Raju 20
    }
}
```

1. `javac` compiles both classes; the JVM loads `Driver.class` and `Person.class` into the method area.
2. The JVM calls `main()`, which pushes a **stack frame** for `main`.
3. `new Driver()` creates a Driver object on the heap; local `d` (stack) holds its reference.
4. Calling `d.getPerson()` pushes a new frame. `new Person()` allocates the object and fields get **default values**, then the explicit assignments run.
5. `return p` copies the **reference** back to `main`. The `getPerson` frame is popped, but the object survives because `main` still references it.
6. When `main` ends, its frame is popped. Objects with no remaining references become **eligible for garbage collection**.

5.4 Garbage Collection Basics

Java has **automatic memory management**: the JVM allocates objects with `new`, and the **Garbage Collector** frees heap objects that are no longer **reachable** from any live reference. There is no `delete` or `free()`.

```
Employee e = new Employee();
e = null;                        // 1. nullifying the reference

Employee a = new Employee();
a = new Employee();              // 2. reassigning: the first object is unreachable

void work() {
    Employee tmp = new Employee();
}                                // 3. local goes out of scope when the method returns

class Node { Node next; }
Node x = new Node(), y = new Node();
x.next = y; y.next = x;
x = null; y = null;              // 4. "island of isolation": they reference each other,
                                 //    but nothing live reaches them -> eligible
```

REMEMBER

`System.gc()` and `Runtime.getRuntime().gc()` only **request** a collection; the JVM may ignore them. `finalize()` is **deprecated** (Java 9) and was marked for removal in Java 18. Release resources with try-with-resources (Phase 9) instead. GC algorithms are covered in Phase 15.

CORRECTED FROM ORIGINAL NOTES

The original notes call `hashCode()` "a unique integer that identifies objects in memory". It is **not guaranteed unique** (two different objects can share a hash code), and it is **not** a memory address. It is a number used by hash-based collections. `hashCode()`, `equals()` and `toString()` are covered properly in Phase 8.

5.5 Variables in Depth: Instance, Static and Local

Feature	Instance variable	Static (class) variable	Local variable
Declared	In a class, outside methods, no `static`	In a class with `static`	Inside a method, constructor or block (incl. parameters)
Belongs to	Each object	The class (one shared copy)	The method call
Memory	Inside the object on the heap	With the class (allocated at class loading)	Stack frame
Created / destroyed	With the object / when the object is GC'd	When the class loads / when it unloads	On method entry / on method exit
Default value	Yes	Yes	**No**: must be initialised
Access	`obj.var` (directly inside instance methods)	`ClassName.var` (recommended)	Only inside its block
Modifiers allowed	access modifiers, `final`, `transient`, `volatile`	same + `static`	only `final`
Also called	Object / non-static / field	Class variable	Temporary / stack variable

Static data is shared by all objects

```
class Student {
    String name;                     // different for every student
    long phone;
    static String institute = "IIT";   // same for all students -> static
}

Student ankit = new Student();   ankit.name = "Ankit";
Student gautam = new Student();  gautam.name = "Gautam";
Student.institute = "IIT Delhi"; // change once -> every object sees it
System.out.println(ankit.institute + " " + gautam.institute);   // IIT Delhi IIT Delhi
```

```
              +----------------------------+
              | static institute = "IIT..." |   ONE copy, class level
              +-------------+--------------+
                shared by every Student object
          +-------------------+-------------------+
          v                                       v
+------------------+                  +------------------+
| Ankit object     |                  | Gautam object    |
|  name  = "Ankit" |                  |  name = "Gautam" |
|  phone = 0       |                  |  phone = 0       |
+------------------+                  +------------------+
```

> **TIP**
> Use **static** for data common to all objects (company name, interest rate, counters, constants). Use **instance** variables for data specific to each object (name, balance, account number).

Static context cannot see instance members directly

```java
class User {
    int age = 10;                          // instance
    static int count = 0;                  // static

    public static void main(String[] args) {
        // System.out.println(age);        // COMPILE ERROR: non-static from static context
        User u = new User();               // need an object first
        System.out.println(u.age);         // 10
        System.out.println(count);         // static -> accessible directly
    }

    void show() {
        System.out.println(age + count);   // instance methods can use BOTH
    }
}
```

The reason: `main` is static and runs without any object. An instance variable exists only inside an object, so the compiler can't know *which* object's `age` you mean.

5.6 Methods

A **method** is a named block of code that performs a task. Methods make code reusable, testable and readable.

```java
public static int add(int a, int b) throws IllegalArgumentException {
//  |      |    |  |    \___________/  \_________________________________/
//  |      |    |  |  name   parameters        exceptions (optional)
//  |      |    |    return type (void if nothing is returned)
//  |      |    optional modifiers (static, final, abstract, synchronized...)
//  access modifier
    return a + b;    // body / definition / implementation
}
```

Term	Meaning	Example
Method signature	**Name + parameter types** (in order). Used for overloading.	`add(int, int)`
Method declaration / header	Modifiers + return type + signature + throws	`public static int add(int a, int b)`
Method body / definition / implementation	The code inside `{ }`	`{ return a + b; }`
Parameters vs arguments	Variables in the declaration vs actual values passed	`a, b` vs `add(10, 20)`

> **NOTE**
> The return type and the `throws` clause are **not** part of the signature. Two methods that differ only in return type cannot coexist.

Rules

1. Return type and name are mandatory; parameters are optional.
2. A non-`void` method must `return` a value of a compatible type on **every** path.
3. Statements after a `return` in the same block are **unreachable** (compile error). Early `return`s inside `if` blocks are fine.
4. A `void` method may use a bare `return;` to exit early.
5. Methods cannot be nested inside other methods (use lambdas or local classes instead).

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say "return should be the last statement of a method". More precisely, a method may have several `return` statements (guard clauses are good style). Code placed *after* a `return` in the same block can never run, and the compiler rejects it.

Instance methods vs static methods

	Instance method	Static method
Belongs to	An object	The class
Called as	`obj.method()`	`ClassName.method()`
Can access instance fields / `this`	Yes	**No** (needs an object reference)
Can access static fields	Yes	Yes
Can be overridden	Yes	No: it is *hidden* (Phase 7)
Typical use	Behaviour that depends on object state	Utilities and factories (`Math.max`, `Integer.parseInt`)

```
class Calculator {
    int lastResult;                                 // instance state

    int add(int a, int b) {                         // instance method
        lastResult = a + b;
        return lastResult;
    }
    static int multiply(int a, int b) {             // static utility method
        return a * b;
    }
}

Calculator c = new Calculator();
c.add(2, 3);                    // 5   (needs an object)
Calculator.multiply(2, 3);      // 6   (no object needed)
```

Passing and returning objects

```
class Employee { int id; double salary; }

static void raise(Employee e, double pct) {       // receives a reference copy
    e.salary += e.salary * pct / 100;             // modifies the SAME object
}
static Employee find(int id) {                    // returns an object
    Employee e = new Employee();
    e.id = id;
    e.salary = id == 101 ? 50000 : 30000;
    return e;
}

Employee emp = find(101);
raise(emp, 10);
System.out.println(emp.salary);    // 55000.0
```

5.7 Java Is Always Pass-by-Value

Java passes a **copy** of every argument:

- **Primitives:** the value is copied, so the method can't change the caller's variable.
- **Objects:** the **reference** (address) is copied. The method **can modify the object's fields**, but **cannot make the caller's variable point to a different object**.

```
static void changePrimitive(int x) { x = 100; }

static void changeName(Student s) { s.name = "John"; }   // modifies the shared object

static void replace(Student s) {
    s = new Student();      // only the LOCAL copy of the reference changes
    s.name = "Amit";
}

int n = 5;
changePrimitive(n);
System.out.println(n);              // 5     (unchanged)

Student st = new Student();
st.name = "Rohit";
changeName(st);
System.out.println(st.name);        // John  (object modified)
replace(st);
System.out.println(st.name);        // John  (caller's reference not replaced)
```

```
caller:  st ---------+
                     +---->  [Student name="John"]
method:  s (copy) ---+       after s = new Student():  s ---> [Student name="Amit"]
```

> **INTERVIEW**
> "Is Java pass-by-reference for objects?" **No.** Java passes object *references by value*. Proof: a `swap(a, b)` method that swaps two object parameters has no effect on the caller.

5.8 Variable Arguments (Varargs)

Varargs (Java 5) let a method accept **zero or more** arguments of one type. Inside the method, the parameter is an **array**.

```
static int sum(int... nums) {          // int... behaves like int[]
    int total = 0;
    for (int n : nums) total += n;
    return total;
}
sum();               // 0
sum(5);              // 5
sum(1, 2, 3, 4);     // 10
sum(new int[]{1, 2});// 3  (an array can be passed directly)

static void log(String level, Object... parts) { }   // varargs must be LAST
// static void bad(int... a, String s) { }           // COMPILE ERROR
// static void bad2(int... a, String... b) { }       // COMPILE ERROR: only one varargs
```

Real-world examples include `String.format(String, Object...)`, `Arrays.asList(T...)`, `List.of(E...)` and `System.out.printf`.

5.9 Recursion

A **recursive** method calls itself to solve a smaller instance of the same problem. Every recursive method needs a **base case** that stops the recursion. Each call uses a new **stack frame**, so very deep recursion causes `StackOverflowError`.

```
static long factorial(int n) {          // n! = n * (n-1)!
    if (n <= 1) return 1;               // base case
    return n * factorial(n - 1);        // recursive case
}

static int fib(int n) {                 // 0 1 1 2 3 5 8 13 ...
    if (n < 2) return n;
    return fib(n - 1) + fib(n - 2);     // O(2^n): fine for learning, slow for n > 40
}

static int sumOfDigits(int n) {
    return n == 0 ? 0 : n % 10 + sumOfDigits(n / 10);
}

static boolean isPalindrome(String s, int i, int j) {
    if (i >= j) return true;
    return s.charAt(i) == s.charAt(j) && isPalindrome(s, i + 1, j - 1);
}
```

```
factorial(4)
 = 4 * factorial(3)              stack grows ->  [f(4)] [f(3)] [f(2)] [f(1)]
 = 4 * 3 * factorial(2)
 = 4 * 3 * 2 * factorial(1)      base case returns 1, then frames unwind:
 = 4 * 3 * 2 * 1 = 24            f(2)=2 -> f(3)=6 -> f(4)=24
```

Recursion	Iteration
Elegant for trees, divide-and-conquer, backtracking	Usually faster, constant stack usage
Uses a stack frame per call; risk of `StackOverflowError`	No stack growth
Java does **not** optimise tail calls	Preferred for simple counting loops

5.10 Constructors

A **constructor** is a special block that **initialises a new object**. It runs automatically when `new` is used.

Rules

1. The name must be **exactly the class name**.

2. There is **no return type**, not even `void`. Adding one turns it into an ordinary method!
3. It is called automatically, once per object creation.
4. It can be overloaded, and can be `public`, `protected`, default or `private`.
5. It is **not inherited**, and cannot be `static`, `final`, `abstract` or `synchronized`.
6. If you write **no** constructor, the compiler adds a **default constructor** (no-arg, calls `super()`). If you write **any** constructor, no default is added.

```
class Employee {
    String name;
    double salary;

    Employee() {                              // no-arg constructor
        this("Unknown", 0);                   // this(...) calls another constructor
    }
    Employee(String name) {
        this(name, 25000);
    }
    Employee(String name, double salary) {    // parameterized constructor
        this.name = name;                     // this.name = field, name = parameter
        this.salary = salary;
    }
    Employee(Employee other) {                // copy constructor
        this(other.name, other.salary);
    }
}

Employee e1 = new Employee();                 // Unknown 0.0
Employee e2 = new Employee("Raju");           // Raju 25000.0
Employee e3 = new Employee("Ramu", 40000);    // Ramu 40000.0
Employee e4 = new Employee(e3);               // independent copy of e3
```

```
class Product {
    Product(int id) { }
}
// new Product();        // COMPILE ERROR: no default constructor was generated

class Trap {
    void Trap() { System.out.println("I'm a METHOD, not a constructor!"); }
}
```

> **TIP**
> Run `javap Student` to see the compiler-generated default constructor in the `.class` file.

Constructor vs method

Constructor	Method
Name = class name	Any valid name
No return type	Must declare a return type (or `void`)
Called automatically by `new`	Called explicitly
Initialises object state	Performs behaviour
Not inherited, can't be overridden	Inherited and can be overridden
Compiler supplies a default if none	No default provided

Private constructors

```
public final class MathUtils {            // utility class: static methods only
    private MathUtils() { }               // prevents  new MathUtils()
    public static int square(int x) { return x * x; }
}

public class AppConfig {                   // Singleton: exactly one instance
    private static final AppConfig INSTANCE = new AppConfig();
    private AppConfig() { }                // no outside construction
    public static AppConfig getInstance() { return INSTANCE; }
}
```

5.11 The this Keyword

`this` is a reference to the **current object**: the one whose method or constructor is running.

Use	Example
1. Distinguish a field from a parameter or local with the same name	`this.name = name;`
2. Call another constructor of the same class (must be the **first** statement)	`this("Unknown", 0);`
3. Call another instance method of the current object	`this.validate();`
4. Pass the current object as an argument	`audit.record(this);`
5. Return the current object (method chaining / builders)	`return this;`
6. Refer to the outer instance from an inner class	`Outer.this.field`

```
class Account {
    private double balance;

    Account deposit(double amt) { this.balance += amt; return this; }   // chaining
    Account withdraw(double amt) { this.balance -= amt; return this; }
}
new Account().deposit(500).withdraw(200);    // fluent style
```

PITFALL

Without `this`, `name = name;` inside a constructor assigns the parameter to itself and leaves the field `null`. Also, `this` cannot be used inside a **static** method, because there is no current object.

5.12 The static Keyword

Applies to	Meaning
Variable	One copy per class, shared by all objects
Method	Called on the class; cannot use `this` or instance members directly
Block `static { }`	Runs **once**, when the class is loaded (before `main` for the main class)
Nested class	A nested class that doesn't need an outer instance (Phase 10)
Import	`import static java.lang.Math.*;` then call `sqrt(16)` without `Math.`

```java
class Account {
    private static int counter = 0;              // shared counter
    private static final double MIN_BALANCE;     // static final constant
    private final int accountNo;

    static {                                     // static initializer block
        MIN_BALANCE = 1000;                      // e.g. load from config
        System.out.println("Account class loaded");
    }

    Account() {
        accountNo = ++counter;                   // each object gets the next number
    }
    static int totalAccounts() { return counter; }
}
```

REMEMBER

Static members can be used from anywhere in the class. Instance members need an object. Accessing a static member through an object (`obj.staticVar`) compiles but is misleading; always use `ClassName.staticVar`.

5.13 Initialization Blocks and Execution Order

```java
class Demo {
    static { System.out.println("Static block"); }   // once, at class loading
    { System.out.println("Instance block"); }        // each object, before ctor body
    Demo() { System.out.println("Constructor"); }
}
```

	Static block	Instance block
Runs when	The class is loaded	Each object is created
How often	Once	Every `new`
Runs before	`main()` (for the main class)	The constructor body (after `super()`)
Can access	Static members only	Static and instance members
Used for	Class-level setup (load drivers, config, caches)	Code shared by all constructors

Complete control flow (output verified on JDK 21)

```java
public class ControlFlowDemo {
    static int a = initStatic();                 // static members: in TEXTUAL order
    static { System.out.println("Static Block 1"); }
    static { System.out.println("Static Block 2"); }

    int b = initInstance();                      // instance members: in TEXTUAL order
    { System.out.println("Instance Block 1"); }
    { System.out.println("Instance Block 2"); }

    ControlFlowDemo() { System.out.println("Constructor"); }

    static int initStatic()  { System.out.println("Static Variable Init"); return 10; }
    int initInstance()       { System.out.println("Instance Variable Init"); return 20; }

    public static void main(String[] args) {
        System.out.println("Main Started");
        new ControlFlowDemo();
        System.out.println("--- second object ---");
        new ControlFlowDemo();
    }
}
```

```
Static Variable Init        <- static phase: runs ONCE when the class loads
Static Block 1
Static Block 2
Main Started                <- then main()
Instance Variable Init      <- instance phase: runs for EVERY object
Instance Block 1
Instance Block 2
Constructor
--- second object ---
Instance Variable Init
Instance Block 1
Instance Block 2
Constructor
```

REMEMBER

Order of events:

- **Static flow (once):** static variables and static blocks, top to bottom, then `main()`.
- **Instance flow (each `new`):** `super()` → instance variable initialisers and instance blocks, top to bottom → the rest of the constructor body.

5.14 Predict the Output

```java
class Counter {
    static int s = 0;  int i = 0;
    Counter() { s++; i++; }
}
new Counter(); new Counter(); Counter c = new Counter();
System.out.println(Counter.s + " " + c.i);         // 3 1

static void inc(int[] arr) { arr[0]++; }
int[] data = {5};  inc(data);
System.out.println(data[0]);                       // 6 (array object modified)

static void reassign(int[] arr) { arr = new int[]{99}; }
reassign(data);
System.out.println(data[0]);                       // 6 (caller's reference unchanged)

class A { int x = 10; A() { x = 20; } { x = 30; } }
System.out.println(new A().x);                     // 20 (block runs before ctor body)

class B { void B() { System.out.println("method"); } }
new B();                                           // prints nothing (no constructor call)

static int f(int n) { return n == 0 ? 0 : n + f(n - 1); }
System.out.println(f(4));                          // 10
```

5.15 Interview Questions

Q. What is the difference between a class and an object?

A class is a blueprint that defines fields and methods. An object is a runtime instance of that class occupying heap memory. One class can have many objects.

Q. Where are objects and references stored?

Objects (with their instance fields) always live on the heap. Local reference variables live in the current method's stack frame. Static fields are class-level.

Q. When does an object become eligible for garbage collection?

When no live thread can reach it through any chain of references: after nullifying or reassigning references, when locals go out of scope, or when a group of objects reference only each other (island of isolation).

Q. Is Java pass-by-value or pass-by-reference?

Always pass-by-value. For objects, the value passed is a copy of the reference, so methods can change the object's state but can't reassign the caller's variable.

Q. What is a default constructor? When is it not provided?

A no-argument constructor that the compiler inserts when a class declares no constructors. As soon as you declare any constructor, the compiler doesn't add one.

Q. Can a constructor be private, static, final or have a return type?

It can be private (singletons, utility classes, factories). It cannot be static, final or abstract. If you give it a return type, it becomes a normal method.

Q. What is constructor chaining?

Calling one constructor from another: `this(...)` within the same class, or `super(...)` to the parent. The call must be the first statement, so you can't use both in one constructor.

Q. Why can't a static method access instance variables directly?

A static method runs without any object, so there is no `this` to say whose instance variable is meant. It must first obtain an object reference.

Q. What is the order of execution of static blocks, instance blocks and constructors?

Static variables and blocks run once at class loading, in textual order. For each object: the parent constructor, then instance initialisers and blocks in textual order, then the rest of the constructor body.

Q. What are varargs and their rules?

`type... name` accepts zero or more arguments as an array. It must be the last parameter, and a method can have only one.

Q. What causes `StackOverflowError` in recursion?

A missing or unreachable base case (or input that is too deep): each call adds a stack frame until the thread's stack is exhausted.

5.16 Quick Revision

- Class = blueprint; object = instance on the **heap**; reference = address held in a variable.
- `new` allocates memory → fields get defaults → constructor runs → reference returned.
- An object is GC-eligible when unreachable; `System.gc()` is only a request; `finalize()` is deprecated.
- Instance variables: per object, defaults. Static: per class, shared. Local: per call, **no defaults**.
- Static code can't touch instance members without an object; there is no `this` in static methods.
- Signature = name + parameter types (not the return type).
- **Always pass-by-value**: object methods can mutate the object but can't rebind the caller's variable.
- Varargs: `type...`, last parameter only, one per method, seen as an array.
- Recursion needs a base case; each call uses a stack frame.
- Constructor: class name, no return type, not inherited; default constructor only when none is written.
- `this(...)` / `super(...)` must be the first statement in a constructor.
- Order: static vars/blocks (once) → `main` → per object: `super()` → instance vars/blocks → constructor body.

PHASE 6 OF 18

OOP I: Encapsulation, Packages & Inheritance

Encapsulation and JavaBeans, access modifiers, non-access modifiers, packages and imports, inheritance types, constructor chaining, super, and IS-A vs HAS-A

IN THIS PHASE

6.1 Encapsulation
6.2 Access Modifiers
6.3 Non-Access Modifiers at a Glance
6.4 Packages
6.5 Inheritance
6.6 Constructors and Inheritance (Constructor Chaining)
6.7 The super Keyword
6.8 IS-A vs HAS-A (Inheritance vs Composition)
6.9 Predict the Output
6.10 Interview Questions
6.11 Quick Revision

6.1 Encapsulation

Encapsulation means bundling data (fields) and the methods that operate on it into one unit (a class), and **restricting direct access** to that data. A Java class is the textbook example.

The recipe:

1. Make fields `private`, so no outside code can read or change them directly.
2. Provide `public` **getter** methods to read and **setter** methods to update, and only where needed.
3. Put **validation and business rules** inside the setters and other methods.

```
public class BankAccount {
    private final String accountNo;     // read-only after construction: no setter
    private String holder;
    private double balance;

    public BankAccount(String accountNo, String holder) {
        this.accountNo = accountNo;
        setHolder(holder);              // reuse validation
    }

    public String getAccountNo() { return accountNo; }
    public String getHolder()    { return holder; }
    public double getBalance()   { return balance; }

    public void setHolder(String holder) {
        if (holder == null || holder.isBlank())
            throw new IllegalArgumentException("Holder name required");
        this.holder = holder.trim();
    }

    public void deposit(double amount) {         // behaviour instead of setBalance()
        if (amount <= 0) throw new IllegalArgumentException("Amount must be positive");
        balance += amount;
    }

    public void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("Insufficient funds");
        balance -= amount;
    }
}
```

Without vs with encapsulation

```
class Student { public int id; }          // WITHOUT
Student s = new Student();
s.id = 101;   s.id = -500;   s.id = 99999;   // anything goes, no control

class SafeStudent {                        // WITH
    private int id;
    public void setId(int id) {
        if (id <= 0) throw new IllegalArgumentException("id must be positive");
        this.id = id;
    }
    public int getId() { return id; }
}
SafeStudent t = new SafeStudent();
// t.id = 101;          // COMPILE ERROR: id has private access
t.setId(101);           // the only way in, and it's validated
```

INTERVIEW

"If a setter can change the field, how is that encapsulation?" Encapsulation does not forbid change. It forbids **direct, uncontrolled** change. Every change goes through code the class owns, where it can be validated, logged, made thread-safe, or reimplemented later without breaking callers.

Benefits

Benefit	Example
Data hiding / security	`balance` can't be set to a random value from outside
Validation and control	Reject negative amounts, blank names, invalid ages
Flexibility to change internals	Switch `double balance` to `BigDecimal` without touching callers
Read-only or write-only properties	Getter without setter (`accountNo`)
Easier testing and maintenance	Behaviour lives in one place

Encapsulation vs data hiding vs abstraction

Concept	Focus	How
Data hiding	Keep data inaccessible from outside	`private`
Encapsulation	Data + methods together, with controlled access	Class + private fields + public methods
Abstraction	Expose **what** an object does, hide **how**	Interfaces, abstract classes (Phase 7)

JavaBeans, POJOs and boilerplate

- **POJO** (Plain Old Java Object): an ordinary class with no framework requirements.
- **JavaBean** conventions: private fields, a public no-arg constructor, `getX()`/`setX()` (`isX()` for booleans), and often `Serializable`. Frameworks such as Spring, Hibernate/JPA and Jackson rely on these names.

> **MODERN JAVA**
> Two ways to cut getter/setter boilerplate:
> - **Lombok** annotations: `@Getter`, `@Setter`, `@NoArgsConstructor`, `@AllArgsConstructor`, `@Data`, `@Builder`.
> - **Records** (Java 16+): `public record Customer(String id, String name) { }` gives private final fields, a constructor, accessors (`id()`, `name()`), `equals`, `hashCode` and `toString` in one line. Records are covered in Phase 15.

6.2 Access Modifiers

Access modifiers control **visibility** of classes, fields, methods and constructors. Java has four levels, from most to least open:

Modifier	Same class	Same package	Subclass (other package)	Anywhere (other package)
`public`	Yes	Yes	Yes	Yes
`protected`	Yes	Yes	Yes (through inheritance)	No
default (no keyword, "package-private")	Yes	Yes	No	No
`private`	Yes	No	No	No

Where each can be used

Element	`public`	`protected`	**default**	`private`
Top-level class / interface / enum / record	Yes	**No**	Yes	**No**
Nested (member) class	Yes	Yes	Yes	Yes
Field	Yes	Yes	Yes	Yes
Method	Yes	Yes	Yes	Yes
Constructor	Yes	Yes	Yes	Yes (singleton, utility, factory)
Local variable	No access modifiers allowed (only `final`)			

```java
package com.bank.core;

public class Employee {
    public    void m1() { }   // visible everywhere
    protected void m3() { }   // package + subclasses
              void m4() { }   // package only (default)
    private   void m2() { }   // this class only

    public static void main(String[] args) {
        Employee emp = new Employee();
        emp.m1(); emp.m2(); emp.m3(); emp.m4();   // all OK inside the same class
    }
}
```

The protected subtlety

A subclass in a **different package** can use an inherited protected member only **through its own type** (`this` or a subclass reference), not through a plain parent reference.

```java
package com.bank.core;
public class Account { protected double balance; }

package com.bank.loans;
import com.bank.core.Account;

public class LoanAccount extends Account {
    void test(Account other, LoanAccount mine) {
        this.balance = 100;      // OK: inherited through own type
        mine.balance = 200;      // OK: subclass reference
        // other.balance = 300;  // COMPILE ERROR: not via a plain Account reference
    }
}
```

Rules worth remembering

1. One `.java` file can contain many top-level classes, but **at most one** `public` class, and the file must be named after it.
2. A top-level class can only be `public` or default.
3. Overriding methods **cannot reduce visibility**: a `public` method can't be overridden as `protected` (Phase 7).
4. Choose the **most restrictive** level that works: fields `private`, helper methods `private`, API methods `public`.

> **NOTE**
> Reflection (`field.setAccessible(true)`) can bypass `private`. Since Java 9, the **module system** blocks this for packages that a module doesn't open. That is why some old libraries print "illegal reflective access" warnings on newer JDKs.

6.3 Non-Access Modifiers at a Glance

Modifier	Class	Method	Field	Meaning (covered in)
`static`	nested only	Yes	Yes	Belongs to the class (Phase 5)
`final`	Yes	Yes	Yes	Can't be extended / overridden / reassigned (Phase 7)
`abstract`	Yes	Yes	No	Incomplete; must be extended / implemented (Phase 7)
`synchronized`	No	Yes (and blocks)	No	One thread at a time per lock (Phase 11)
`volatile`	No	No	Yes	Visibility of updates across threads (Phase 11)
`transient`	No	No	Yes	Skipped during serialization (Phase 12)
`native`	No	Yes	No	Implemented in C/C++ via JNI (Phase 1)
`strictfp`	Yes	Yes	No	Strict IEEE-754 floating point. **Obsolete since Java 17**.
`sealed` / `non-sealed`	Yes	No	No	Restricts which classes may extend (Java 17, Phase 15)

CORRECTED FROM ORIGINAL NOTES

The short notes present `strictfp` as useful for "platform-independent results". Since **Java 17** (JEP 306) all floating-point arithmetic is strict by default, so `strictfp` has no effect and the compiler warns that it's redundant.

REMEMBER

Illegal combinations: `abstract final`, `abstract private`, `abstract static` (methods). A method can't be both abstract and have a body.

6.4 Packages

A **package** is a namespace that groups related classes, interfaces, enums, records and sub-packages, much like folders on disk.

Why packages?

- **Organisation:** group by feature or layer (`controller`, `service`, `repository`).
- **Name-clash avoidance:** `java.util.Date` and `java.sql.Date` can coexist.
- **Access control:** default and `protected` visibility are package-based.
- **Reuse and distribution:** package as JARs.

Built-in packages

Package	Contains
`java.lang`	`String`, `Object`, `Math`, wrappers, `System`, `Thread`, exceptions. **Imported automatically.**
`java.util`	Collections, `Scanner`, `Optional`, `Random`, `UUID`
`java.util.concurrent`	Executors, concurrent collections, locks, atomics
`java.io` / `java.nio.file`	Streams, readers, writers, `Files`, `Path`
`java.time`	`LocalDate`, `LocalDateTime`, `Duration`, `ZonedDateTime`
`java.math`	`BigDecimal`, `BigInteger`
`java.net` / `java.net.http`	Networking, URLs, `HttpClient`
`java.sql`	JDBC: `Connection`, `PreparedStatement`, `ResultSet`
`java.text`	`NumberFormat`, `SimpleDateFormat` (legacy)

Creating and using a package

```
// File: src/com/bank/payments/PaymentService.java
package com.bank.payments;          // must be the FIRST statement (comments may precede it)

import java.util.List;              // single-type import (preferred)
import java.util.*;                 // on-demand (wildcard) import
import static java.lang.Math.max;   // static import of one member
import static java.lang.Math.*;     // static import of all static members

public class PaymentService { }
```

```
src/
 └── com/bank/payments/PaymentService.java       folder structure mirrors the package

javac -d out src/com/bank/payments/PaymentService.java   # -d creates package folders
java -cp out com.bank.payments.PaymentService            # run by FULLY QUALIFIED name
```

Naming convention

All lowercase, using the reverse internet domain plus project and module: `com.companyname.projectname.module`, for example `com.icici.netbanking.transfer` or `org.apache.commons.lang3`.

Import rules

1. `java.lang` is always imported implicitly.
2. A wildcard import (`java.util.*`) does **not** import sub-packages: `java.util.function.Function` needs its own import.
3. Imports only save typing. They don't load or copy code; the bytecode always uses fully qualified names. (Unlike C's `#include`, nothing is copied.)
4. When two imported packages have the same simple name, use the **fully qualified name** for at least one.
5. Classes in the **default (unnamed) package** can't be imported by classes in named packages, so always use packages in real code.

```
import java.util.Date;
public class Report {
    Date utilDate = new Date();                          // java.util.Date
    java.sql.Date sqlDate = java.sql.Date.valueOf("2026-01-31");  // fully qualified
}
```

TIP

In Eclipse, **Ctrl + Shift + O** organises imports (adds missing ones, removes unused ones). The IntelliJ equivalent is **Ctrl + Alt + O**.

MODERN JAVA

Java 9 modules (`module-info.java`) group packages and declare what a module `requires` and `exports`. Unexported packages are hidden even if their classes are public. Most applications still use the classpath, but the JDK itself is modular (`java.base`, `java.sql`, ...).

6.5 Inheritance

Inheritance lets a class (the **child**: subclass or derived class) acquire the fields and methods of another class (the **parent**: superclass or base class) using `extends`. It models an **IS-A** relationship: a `SavingsAccount` IS-A `Account`.

```
class Account {                                  // parent / superclass / base class
    protected String accountNo;
    protected double balance;
    void deposit(double amt) { balance += amt; }
    String summary() { return accountNo + " : " + balance; }
}

class SavingsAccount extends Account {           // child / subclass / derived class
    private double interestRate = 4.0;           // child adds new state
    void addInterest() {                         // ...and new behaviour
        deposit(balance * interestRate / 100);   // reuses the inherited method
    }
}

SavingsAccount sa = new SavingsAccount();
sa.accountNo = "SB-1";      // inherited field
sa.deposit(1000);           // inherited method
sa.addInterest();           // own method
System.out.println(sa.summary());   // SB-1 : 1040.0
```

Advantages: code reuse, less duplication, easier maintenance, a natural type hierarchy, and it enables **runtime polymorphism** (Phase 7).

What is (and isn't) inherited

Member	Inherited?
`public` / `protected` fields and methods	Yes
Default (package-private) members	Only if the child is in the **same package**
`private` members	**No direct access**. They exist inside the child object but are reachable only through inherited public/protected methods.
Constructors	**No**. They are invoked through `super(...)`, never inherited.
Static members	Accessible through the child, but **hidden** rather than overridden
Instance/static initializer blocks	No (they run as part of the parent's construction)

Reference type vs object type

```
class Parent { void m1() { System.out.println("Parent m1"); } }
class Child extends Parent {
    @Override void m1() { System.out.println("Child m1"); }
    void m2() { System.out.println("Child m2"); }
}

Parent p = new Child();   // parent reference, child object (upcasting: automatic)
p.m1();                   // Child m1  -> the OBJECT type decides which override runs
// p.m2();                // COMPILE ERROR -> the REFERENCE type decides what is visible

Child c = new Child();
c.m1();  c.m2();          // Child m1, Child m2

// Child bad = new Parent();   // COMPILE ERROR: a Parent is not necessarily a Child
```

> **REMEMBER**
> The **compiler** checks member access using the **reference type**. The **JVM** picks the overridden method using the **object type** at runtime (dynamic dispatch). Method lookup starts in the object's own class, then moves up the parents to `Object`.

Types of inheritance

```
 Single          Multilevel       Hierarchical          Multiple (classes: NOT allowed)
   A                 A                 A                    A       B
   ^                 ^               / | \                   \     /
   B                 B              B  C  D                   \   /
                     ^                                          C
                     C                      Hybrid = mix including multiple: NOT via classes
```

Type	Supported with classes?	Example
Single	Yes	`class B extends A`
Multilevel	Yes	`A ← B ← C` (`Object ← Account ← SavingsAccount`)
Hierarchical	Yes	`Savings`, `Current`, `FixedDeposit` all extend `Account`
Multiple	**No** (for classes)	`class C extends A, B` is a compile error
Hybrid	**No** (for classes)	A mix that includes multiple inheritance

Multiple inheritance **of type** (and of default behaviour) **is** possible with **interfaces**: `class C implements A, B` (Phase 7).

Why no multiple inheritance of classes? The diamond problem

```
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } }
class C extends A { void show() { System.out.println("C"); } }
// class D extends B, C { }     // INVALID. new D().show() -> B's or C's version?
```

If one class could extend two parents that both define `show()` (or both declare a field `id`), the call would be **ambiguous**. Java avoids the problem for classes; for interfaces with conflicting default methods, it forces you to override and choose (Phase 7).

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say the lack of multiple inheritance "makes Java a partial OOP language". That is not the reason. Java is called not *purely* object-oriented because of **primitive types** and **static members**. Lack of multiple class inheritance is a deliberate design choice.

Cyclic inheritance is illegal

```
// class User extends User { }                                  // COMPILE ERROR
// class A extends B { }  class B extends A { }                 // COMPILE ERROR: cyclic
```

> **NOTE**
> Every class has exactly one direct superclass. If none is written, it implicitly `extends Object`. So `java.lang.Object` is the root of every hierarchy, and all classes can use its 11 methods: `toString`, `equals`, `hashCode`, `getClass`, `clone`, `finalize`, `wait` (×3), `notify` and `notifyAll`.

6.6 Constructors and Inheritance (Constructor Chaining)

Creating a child object **always runs the parent's constructor first**, all the way up to `Object`. The parent part of the object must be initialised before the child part.

```
class User {
    User() { System.out.println("Parent Constructor"); }
}
class Employee extends User {
    Employee() {
        // super();   <- inserted automatically by the compiler if you don't write it
        System.out.println("Child Constructor");
    }
}
new Employee();
// Parent Constructor
// Child Constructor
```

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes say "if we remove `super()`, the child class will not compile". In fact, if a constructor doesn't start with `this(...)` or `super(...)`, the compiler **inserts** `super()` automatically. Compilation fails only when the parent has **no no-arg constructor**, as shown below.

```
class User {
    User(int id, String name) { System.out.println("User(" + id + ", " + name + ")"); }
}
class Employee extends User {
    Employee() {
        super(101, "Raju");           // REQUIRED: User has no no-arg constructor
        System.out.println("Employee()");
    }
}
// Without the super(...) line -> COMPILE ERROR:
//   constructor User in class User cannot be applied to given types
```

Rules for super() and this()

1. `super(...)` calls a parent constructor; `this(...)` calls another constructor of the same class.
2. Either must be the **first statement** in the constructor (Java 25 relaxes this slightly with *flexible constructor bodies*: statements that don't touch `this` may come first).
3. You can't have both `this(...)` and `super(...)` in the same constructor, but a `this(...)` chain eventually reaches a `super(...)`.
4. They can be used only inside constructors.

Full order with static and instance blocks (verified on JDK 21)

```
class Parent {
    static { System.out.println("1. Parent static block"); }
    { System.out.println("3. Parent instance block"); }
    Parent() { System.out.println("4. Parent constructor"); }
}
class Child extends Parent {
    static { System.out.println("2. Child static block"); }
    { System.out.println("5. Child instance block"); }
    Child() { System.out.println("6. Child constructor"); }
}
new Child();      // prints 1 to 6 in order
new Child();      // prints only 3, 4, 5, 6 (static blocks ran once)
```

6.7 The super Keyword

Use	Syntax	Purpose
Parent constructor	`super(args)`	Initialise the parent part (first statement)
Parent method	`super.method()`	Call the parent's version from an override, to extend rather than replace it
Parent field	`super.field`	Access a parent field hidden by a child field with the same name

```
class Parent {
    int x = 10;
    void m1() { System.out.println("Parent m1()"); }
}
class Child extends Parent {
    int x = 20;                                  // HIDES Parent.x (avoid in real code)
    @Override
    void m1() {
        super.m1();                              // reuse the parent behaviour first
        System.out.println("Child m1()");
    }
    void display() {
        System.out.println("Parent x = " + super.x);   // 10
        System.out.println("Child x = " + this.x);     // 20
    }
}
```

PITFALL

Fields are not polymorphic. With `Parent p = new Child();`, `p.x` gives **10** (decided by the reference type), while `p.m1()` runs Child's override. Keep fields `private` and never redeclare a parent's field name in a child.

`this` vs `super`:

this	super
Refers to the current object	Refers to the parent part of the current object
`this(...)` calls a constructor of the same class	`super(...)` calls a parent constructor
`this.field` / `this.method()`	`super.field` / `super.method()`
Not usable in static context	Not usable in static context

6.8 IS-A vs HAS-A (Inheritance vs Composition)

Relationship	Meaning	Java mechanism	Example
IS-A	Specialisation	`extends` / `implements`	`SavingsAccount` IS-A `Account`
HAS-A (association)	One object uses or holds another	A field referencing another object	`Customer` HAS-A `Address`
Aggregation (weak HAS-A)	The part can exist without the whole	Object passed in from outside	`Department` has `Employee`s (employees outlive the department)
Composition (strong HAS-A)	The part's lifecycle is owned by the whole	Object created inside, not shared	`Car` has an `Engine` created with it

```
class Engine { void start() { System.out.println("Engine started"); } }

class Car {                                     // composition: Car HAS-A Engine
    private final Engine engine = new Engine();
    void drive() { engine.start(); System.out.println("Driving"); }
}

class Department {                              // aggregation: employees supplied from outside
    private final List<Employee> staff;
    Department(List<Employee> staff) { this.staff = staff; }
}
```

TIP

Favour composition over inheritance. Use `extends` only for a true IS-A relationship where the child can be used anywhere the parent is expected (the Liskov Substitution Principle). Composition is more flexible: you can swap parts at runtime and avoid fragile deep hierarchies. Frameworks like Spring wire HAS-A dependencies through dependency injection.

6.9 Predict the Output

```
class P { P() { System.out.print("P "); } }
class Q extends P { Q() { System.out.print("Q "); } }
class R extends Q { R() { super(); System.out.print("R "); } }
new R();                                        // P Q R

class X { int v = 1; int get() { return v; } }
class Y extends X { int v = 2; int get() { return v; } }
X obj = new Y();
System.out.println(obj.v + " " + obj.get());  // 1 2  (fields: reference type; methods: object)

class M { M(int a) { } }
// class N extends M { }       // COMPILE ERROR: implicit super() finds no M()

class S { static void hi() { System.out.println("S"); } }
class T extends S { static void hi() { System.out.println("T"); } }
S s = new T();
s.hi();                                         // S  (static methods are hidden, not overridden)
```

6.10 Interview Questions

Q. What is encapsulation and how is it achieved?

Bundling data with the methods that operate on it and restricting direct access. Achieved by `private` fields plus public methods (getters/setters or behaviour methods) that enforce validation and invariants.

Q. What is the difference between encapsulation and abstraction?
Encapsulation hides **data** and controls access to it (the how of protection). Abstraction hides **implementation complexity** behind a simple interface (the what an object does). Encapsulation is implemented with access modifiers; abstraction with interfaces and abstract classes.

Q. Explain the four access modifiers.
`private`: same class only. Default: same package. `protected`: same package plus subclasses in other packages (through inheritance). `public`: everywhere. Top-level classes can only be public or default.

Q. Can a top-level class be private or protected?
No. Only nested classes can use those modifiers.

Q. What is the use of packages?
To organise related types, avoid naming conflicts, control access (package-private and protected) and support reuse and distribution as JARs.

Q. Does `import java.util.*` import `java.util.function.*`?
No. Wildcard imports cover a single package only, not sub-packages.

Q. Why doesn't Java support multiple inheritance with classes?
To avoid the diamond problem: ambiguity when two parents provide the same method or field. Java allows multiple inheritance of type through interfaces, and resolves default-method conflicts by forcing an override.

Q. Are constructors inherited? Are private members inherited?
Constructors are never inherited; they are chained via `super(...)`. Private members are not accessible in the subclass, although they exist inside the object and can be reached through inherited non-private methods.

Q. In what order are constructors called in a hierarchy?
Top-down: `Object` first, then each parent, then the child. Each constructor's first action is `super(...)` (explicit or implicit).

Q. What happens if the parent class has only a parameterized constructor?
Every child constructor must explicitly call `super(args)`. Otherwise the implicit `super()` fails to compile.

Q. What is the difference between `this` and `super`?
`this` refers to the current object (its fields, methods, other constructors). `super` refers to the parent part (parent constructor, overridden methods, hidden fields). Neither is allowed in a static context.

Q. What is the difference between IS-A and HAS-A? Which should you prefer?
IS-A is inheritance (`extends`/`implements`); HAS-A is composition or aggregation (an object holds a reference to another). Prefer composition unless a genuine substitutable IS-A relationship exists.

6.11 Quick Revision

- **Encapsulation** = private fields + controlled public methods (validation inside). It controls change; it doesn't forbid it.
- Access levels: `private` < default (package) < `protected` (package + subclasses) < `public`.
- Top-level types are only `public` or default; one public class per file.
- `strictfp` is obsolete since Java 17; `abstract` can't combine with `final`, `private` or `static`.
- Packages: lowercase reverse domain; `package` is the first statement; `java.lang` is auto-imported; wildcards skip sub-packages; use FQNs for clashes.
- `extends` = IS-A. Inherited: public/protected (+ default in the same package). Not inherited: constructors, private access.
- Class inheritance types: single, multilevel, hierarchical. Multiple and hybrid only via interfaces.
- The compiler checks the **reference** type; the JVM runs the **object** type's override. Fields and static methods use the reference type.

- Parent constructors run first; `super()` is implicit unless the parent lacks a no-arg constructor.
- `this(...)` / `super(...)`: first statement, never both.
- Order: parent static → child static → parent instance blocks + constructor → child instance blocks + constructor.
- Prefer **composition (HAS-A)** over inheritance unless IS-A truly holds.

PHASE 7 OF 18

OOP II: Polymorphism, Abstraction & Interfaces

Overloading and its resolution rules, overriding rules, covariant returns, dynamic binding, method hiding, reference casting, final, abstract classes, interfaces (default/static/private), marker and functional interfaces

IN THIS PHASE

7.1 Polymorphism
7.2 Method Overloading
7.3 Method Overriding
7.4 Dynamic Binding (Runtime Polymorphism)
7.5 Reference Casting: Upcasting and Downcasting
7.6 The final Keyword
7.7 Abstraction and Abstract Classes
7.8 Interfaces
7.9 Abstract Class vs Interface
7.10 Marker and Functional Interfaces
7.11 Predict the Output
7.12 Interview Questions
7.13 Quick Revision

7.1 Polymorphism

Polymorphism ("many forms") means one name or one reference can behave differently depending on the situation. Even operators show it: `10 + 20` adds, `"Hi" + "Hello"` concatenates.

	Compile-time (static) polymorphism	Runtime (dynamic) polymorphism
Achieved by	Method **overloading** (and constructor overloading)	Method **overriding**
Decided by	The **compiler**, from argument types	The **JVM**, from the actual object type
Binding	Static (early) binding	Dynamic (late) binding
Needs inheritance?	No (same class is enough)	Yes (parent–child or interface–class)
Example	`add(int, int)` vs `add(double, double)`	`Payment p = new UpiPayment(); p.pay();`

7.2 Method Overloading

Overloading means several methods in the same class (or inherited into it) share a **name** but have **different parameter lists**.

```
class Calculator {
    int add(int a, int b)             { return a + b; }
    int add(int a, int b, int c)      { return a + b + c; }   // different count
    double add(double a, double b)    { return a + b; }       // different types
    String add(String a, int b)       { return a + b; }       // different order...
    String add(int b, String a)       { return b + a; }       // ...also counts
    // long add(int a, int b) { }     // COMPILE ERROR: only the return type differs
}
```

Rules

1. Parameter lists **must differ** in number, type or order of types.

2. Return type, access modifier, `static` and the `throws` clause **may** differ but don't count on their own.
3. Overloading works for static methods, constructors and `main` (the JVM only calls `main(String[])`).
4. The compiler decides which overload to call **at compile time**, based on the **declared (static) types** of the arguments.

How the compiler picks an overload

When there is no exact match, the compiler tries each phase in order and stops at the first phase that finds a candidate:

Phase	Allows	Example: call `m(5)` with an `int`
1. Exact match	Identical types	`m(int)`
2. Widening	Primitive widening, subtype → supertype	`m(long)`, `m(double)`
3. Boxing / unboxing	`int` ↔ `Integer`	`m(Integer)`, `m(Object)`
4. Varargs	`int...`	`m(int...)`

Within a phase, the **most specific** candidate wins (for example `m(long)` beats `m(double)`, and `m(String)` beats `m(Object)`).

```
static void m(long x)    { System.out.println("long"); }
static void m(Integer x) { System.out.println("Integer"); }
static void m(int... x)  { System.out.println("varargs"); }
m(5);                    // long    (widening beats boxing)

static void t(Object o)  { System.out.println("Object"); }
static void t(String s)  { System.out.println("String"); }
t(null);                 // String  (most specific type)
t("hi");                 // String
Object o = "hi";
t(o);                    // Object  (decided by the DECLARED type, not the runtime type!)

static void u(String s)        { }
static void u(StringBuilder s) { }
// u(null);              // COMPILE ERROR: ambiguous (neither type is more specific)
```

PITFALL

`List<Integer> list` has both `remove(int index)` and `remove(Object o)`. `list.remove(1)` removes the element **at index 1**. To remove the *value* 1, call `list.remove(Integer.valueOf(1))`.

7.3 Method Overriding

Overriding means a subclass provides its **own implementation** of an inherited instance method with the **same signature**. At runtime, the JVM calls the version belonging to the **actual object**.

```
class Account {
    double interestRate() { return 0.0; }
    String type() { return "Generic"; }
}
class SavingsAccount extends Account {
    @Override double interestRate() { return 4.0; }
    @Override String type() { return "Savings"; }
}
class FixedDeposit extends Account {
    @Override double interestRate() { return 7.1; }
}

Account a = new SavingsAccount();
System.out.println(a.type() + " " + a.interestRate());   // Savings 4.0
```

Overriding rules

Rule	Detail
Same name and same parameter list	Otherwise it is overloading, not overriding
Return type	Same, or a **subtype** (covariant return) for objects. Primitives must match exactly.
Access modifier	**Same or wider**, never narrower (`protected` → `public` is OK; `public` → `protected` is an error)
Checked exceptions	May throw **fewer, narrower or no** checked exceptions, never new or broader ones. Unchecked exceptions are unrestricted.
`final` methods	Cannot be overridden
`static` methods	Cannot be overridden; a same-signature static method in the child **hides** the parent's
`private` methods	Not inherited, so not overridden (a same-name child method is a new method)
Constructors	Not inherited, so never overridden
Instance relationship	Requires inheritance (`extends` or `implements`)

```
class Parent {
    protected Number value() throws java.io.IOException { return 1; }
}
class Child extends Parent {
    @Override
    public Integer value() { return 2; }   // wider access, covariant, no checked exc.
}

class Bad extends Parent {
    // @Override private Number value() { }                        // ERROR: weaker access
    // @Override Number value() throws Exception { return 0; }     // ERROR: broader checked
}
```

> **TIP**
> Always use `@Override`. If the method doesn't actually override anything (typo in the name, wrong parameter type such as `equals(Student)` instead of `equals(Object)`), the compiler reports an error instead of silently creating a new method.

Covariant return types (Java 5)

An overriding method may return a **subtype** of the parent method's return type. This avoids casts for callers.

```
class Animal { Animal reproduce() { return new Animal(); } }
class Dog extends Animal {
    @Override Dog reproduce() { return new Dog(); }   // Dog is-a Animal
}
Dog puppy = new Dog().reproduce();                    // no cast needed
```

Overloading vs overriding

Feature	Overloading	Overriding
Purpose	Same operation, different inputs	Change inherited behaviour
Where	Same class (or inherited)	Parent ↔ child
Parameters	**Must differ**	**Must be the same**
Return type	Free to differ	Same or covariant
Access modifier	Free	Same or wider
Exceptions	Free	Not broader checked exceptions
`static` / `private` / `final`	Can be overloaded	Cannot be overridden
Resolved	Compile time (static binding)	Runtime (dynamic binding)
Polymorphism type	Compile-time	Runtime

7.4 Dynamic Binding (Runtime Polymorphism)

Binding links a method call to a method body.

- **Static (early) binding** happens at compile time. It applies to `private`, `static` and `final` methods, constructors, overloaded method selection, and **all field access**.
- **Dynamic (late) binding** happens at runtime. It applies to overridable **instance** methods: the JVM looks at the object's real class and calls its version (HotSpot uses a per-class method table, the *vtable*).

```
interface Payment { void pay(double amount); }

class UpiPayment implements Payment {
    public void pay(double amt) { System.out.println("UPI: " + amt); }
}
class CardPayment implements Payment {
    public void pay(double amt) { System.out.println("Card: " + amt); }
}
class NetBanking implements Payment {
    public void pay(double amt) { System.out.println("NetBanking: " + amt); }
}

Payment[] payments = { new UpiPayment(), new CardPayment(), new NetBanking() };
for (Payment p : payments) {
    p.pay(500);      // ONE call site, THREE behaviours -> chosen at runtime
}
```

> **NOTE**
> This is why polymorphism matters: code written against `Payment` keeps working when a new payment type (for example `WalletPayment`) is added, with **no changes** to existing callers. This is the *Open/Closed Principle*.

Method hiding (static methods)

```
class Parent { static void info() { System.out.println("Parent static"); } }
class Child extends Parent { static void info() { System.out.println("Child static"); } }

Parent p = new Child();
p.info();          // Parent static -> chosen by REFERENCE type (hiding, not overriding)
Child.info();      // Child static
```

Overriding (instance methods)	Hiding (static methods)
Chosen by **object** type at runtime	Chosen by **reference** type at compile time

Overriding (instance methods)	Hiding (static methods)
Polymorphic	Not polymorphic
`@Override` allowed	`@Override` not allowed (compile error)

7.5 Reference Casting: Upcasting and Downcasting

	Upcasting	Downcasting
Direction	Child → parent / interface type	Parent → child type
Syntax	Implicit: `Account a = new SavingsAccount();`	Explicit: `SavingsAccount s = (SavingsAccount) a;`
Safe?	Always	Only if the object really is that type; otherwise `ClassCastException` at runtime
Why	Generic code, polymorphism	Access child-only members

```
Account a = new SavingsAccount();        // upcast (automatic)
// a.addInterest();                      // COMPILE ERROR: not visible via Account

SavingsAccount s = (SavingsAccount) a;   // downcast: OK, the object IS a SavingsAccount
s.addInterest();

Account b = new FixedDeposit();
SavingsAccount bad = (SavingsAccount) b; // compiles, but ClassCastException at RUNTIME

// String str = (String) new Integer(5); // COMPILE ERROR: unrelated types can't be cast
```

Safe downcasting with instanceof

```
if (b instanceof SavingsAccount) {               // classic check, then cast
    SavingsAccount sv = (SavingsAccount) b;
    sv.addInterest();
}

if (b instanceof SavingsAccount sv) {            // Java 16 pattern matching: check + cast
    sv.addInterest();
}

String label = switch (b) {                      // Java 21 pattern matching for switch
    case SavingsAccount sv -> "Savings @" + sv.interestRate();
    case FixedDeposit fd   -> "FD @" + fd.interestRate();
    default                -> "Other";
};
```

REMEMBER

`obj instanceof Type` is `true` for that type **and all its supertypes**, and always `false` for `null`. `obj.getClass() == Type.class` is true only for the **exact** class.

TIP

Frequent `instanceof` + downcast chains are a design smell. Usually the behaviour belongs in an overridden method on the type itself.

7.6 The final Keyword

Applied to	Effect	Example
Local variable / parameter	Assigned once	`final int max = 10;`
Instance variable	Assigned once: at declaration, in an instance block, or in **every** constructor (a *blank final*)	`private final String accountNo;`
Static variable	Assigned once: at declaration or in a static block (a constant)	`static final double PI = 3.14159;`
Method	Cannot be overridden	`public final void audit() { }`
Class	Cannot be extended	`final class String`, `Integer`, `Math`, `LocalDate`

```
class Customer {
    private final String id;                  // blank final
    private static final int MAX_CARDS;       // static blank final
    static { MAX_CARDS = 3; }
    Customer(String id) { this.id = id; }     // must be assigned in every constructor
}

final List<String> names = new ArrayList<>();
names.add("Rohit");          // ALLOWED: the list object itself is mutable
// names = new ArrayList<>(); // ERROR: the REFERENCE can't be reassigned
```

CORRECTED FROM ORIGINAL NOTES

The original notes say "**final classes are immutable**". That's incorrect. `final` on a class only **prevents subclassing**. A final class can still have setters and mutable fields. Immutability requires `private final` fields, no mutators and defensive copies (Phase 8). `String` happens to be both final *and* immutable; being final is just one part of what makes it immutable.

NOTE

Effectively final: a local variable that is never reassigned, even without the `final` keyword. Lambdas and anonymous/local classes may only capture final or effectively final locals.

7.7 Abstraction and Abstract Classes

Abstraction means exposing **what** an object does while hiding **how** it does it. You use an ATM's Withdraw, Deposit and Balance operations without knowing the internal logic. In Java, abstraction is achieved with **abstract classes** (partial abstraction) and **interfaces** (full abstraction of behaviour).

- A **concrete method** has a body: `void display() { ... }`.
- An **abstract method** has no body and ends with `;`: `abstract double emi();`. It may appear only in an abstract class or an interface.

```
abstract class Loan {                                    // can't be instantiated
    protected final double principal;
    protected final int months;

    protected Loan(double principal, int months) {       // abstract classes HAVE constructors
        this.principal = principal;
        this.months = months;
    }

    abstract double annualRate();                        // subclasses must implement

    public final double emi() {                          // template method: fixed algorithm
        double r = annualRate() / 12 / 100;
        return principal * r * Math.pow(1 + r, months) / (Math.pow(1 + r, months) - 1);
    }
}

class HomeLoan extends Loan {
    HomeLoan(double p, int m) { super(p, m); }
    @Override double annualRate() { return 8.5; }
}
class CarLoan extends Loan {
    CarLoan(double p, int m) { super(p, m); }
    @Override double annualRate() { return 9.2; }
}

Loan loan = new HomeLoan(5_000_000, 240);                // Loan l = new Loan(...) won't compile
System.out.printf("EMI = %.2f%n", loan.emi());           // EMI = 43391.16
```

Abstract class rules

1. Declared with `abstract`; **cannot be instantiated** with `new` (anonymous subclasses are possible).
2. Can contain abstract **and** concrete methods, constructors, instance and static fields, static methods, blocks, even `main`.
3. Its constructor runs when a **subclass** object is created, through `super(...)`.
4. A concrete subclass must implement **all** inherited abstract methods; otherwise it must itself be declared `abstract`.
5. A class may be abstract with **zero** abstract methods (to prevent direct instantiation).
6. Illegal combinations: `abstract final` (class or method), `abstract private`, `abstract static` and `abstract synchronized` (methods).

7.8 Interfaces

An **interface** is a pure **contract**: it states *what* operations a type supports, not *how*. Classes **implement** interfaces; one class can implement **many** of them.

```
public interface Bank {
    double MIN_BALANCE = 1000;              // implicitly public static final
    void transfer(String to, double amt);   // implicitly public abstract
    double checkBalance();
}

class HdfcBank implements Bank {
    public void transfer(String to, double amt) { System.out.println("HDFC transfer"); }
    public double checkBalance() { return 5000; }
}
class AxisBank implements Bank {
    public void transfer(String to, double amt) { System.out.println("Axis transfer"); }
    public double checkBalance() { return 8000; }
}

Bank b = new HdfcBank();       // interface reference -> any implementation
b = new AxisBank();            // swap implementations freely
// Bank x = new Bank();        // COMPILE ERROR: interfaces can't be instantiated
```

Implicit modifiers

Member	What you write	What the compiler means
Field	`int LIMIT = 5;`	`public static final int LIMIT = 5;` (a constant; must be initialised)
Abstract method	`void pay();`	`public abstract void pay();`
Default method (Java 8)	`default void log() { }`	`public` instance method with a body
Static method (Java 8)	`static Bank of() { }`	`public static`, called **only** as `Bank.of()`
Private method (Java 9)	`private void helper() { }`	Shared helper for default/static methods
Constructor	not allowed	n/a
Instance fields	not allowed	n/a

> **REMEMBER**
> Implementing methods must be declared `public`, because interface methods are implicitly public and you can't reduce visibility.

How interfaces evolved

```
public interface PaymentGateway {
    void pay(double amount);                                  // abstract (Java 1.0+)

    default void payWithReceipt(double amount) {              // default (Java 8)
        pay(amount);
        log("Receipt generated for " + amount);
    }

    static PaymentGateway noop() {                            // static (Java 8)
        return amount -> { };                                 // lambda: one abstract method
    }

    private void log(String msg) {                            // private (Java 9)
        System.out.println("[gateway] " + msg);
    }
}
```

Default methods were added in Java 8 so that existing interfaces (such as `Collection`) could gain new methods (`stream()`, `forEach()`, `removeIf()`) **without breaking** every class that already implemented them.

Interface inheritance and the default-method diamond

```
interface RbiRules { void kyc(); }
interface Bank extends RbiRules { void openAccount(); }  // interfaces EXTEND interfaces
interface Insurance { }

class HdfcBank extends Object implements Bank, Insurance {   // extends first, then implements
    public void kyc() { }
    public void openAccount() { }
}

interface Camera { default String click() { return "Camera"; } }
interface Phone  { default String click() { return "Phone"; } }

class SmartPhone implements Camera, Phone {
    @Override
    public String click() {                      // MUST override (else compile error)
        return Camera.super.click() + "+" + Phone.super.click();   // InterfaceName.super.m()
    }
}
```

Relationship	Keyword	Multiple allowed?
class → class	`extends`	No (exactly one)
class → interface	`implements`	Yes
interface → interface	`extends`	Yes
interface → class	not possible	n/a

> **NOTE**
> Conflict resolution rules: (1) a method in a **class** (or superclass) always wins over interface defaults; (2) a more specific sub-interface wins over its parent; (3) otherwise the class must override and choose with `X.super.m()`.

Interfaces as types: loose coupling

```
class PaymentService {
    private final PaymentGateway gateway;         // depends on the CONTRACT
    PaymentService(PaymentGateway gateway) {      // implementation injected (e.g. by Spring)
        this.gateway = gateway;
    }
    void checkout(double amt) { gateway.pay(amt); }
}
```

```
Tight coupling:   PaymentService ----------> RazorpayGateway       (change = edit service)
Loose coupling:   PaymentService ---> <<PaymentGateway>> <--- RazorpayGateway
                                                          <--- StripeGateway
                                                          <--- MockGateway (unit tests!)
```

> **TIP**
> For a test engineer, this is the key benefit: code that depends on interfaces can receive **mocks or stubs** (with Mockito, for example) in unit tests, with no real payment gateway, database or network.

7.9 Abstract Class vs Interface

Feature	Abstract class	Interface
Instantiation	No	No
Methods	Abstract + concrete (any access)	Abstract, `default`, `static`, `private`
Fields	Any: instance, static, any access, mutable	Only `public static final` constants
Constructors	Yes	No
State per object	Yes	No
Inheritance	`extends` **one** class	A class `implements` **many**; an interface `extends` many
Access of methods	Any	Public (except private helpers)
Use for	**Is-a** base with shared state and code (a skeleton)	**Can-do** capability or contract (`Comparable`, `Runnable`)
Speed of adding methods	Add a concrete method safely	Add a `default` method safely

CORRECTED FROM ORIGINAL NOTES

The original notes say "from Java 8, there is no demand for abstract classes". Interfaces still **cannot hold instance state or constructors**, so abstract classes remain the right tool for a common base with shared fields and partially implemented logic (for example `AbstractList`, or `HttpServlet` in Jakarta EE). A common design is an **interface for the contract plus an abstract class for a skeletal implementation**.

7.10 Marker and Functional Interfaces

Marker interfaces have **no methods**. Implementing one tags a class so that the JVM or a library treats it specially.

Marker interface	Meaning
`java.io.Serializable`	Objects may be serialized (Phase 12)
`java.lang.Cloneable`	`Object.clone()` is allowed; otherwise `CloneNotSupportedException` (Phase 8)
`java.util.RandomAccess`	The List supports fast index access (`ArrayList` yes, `LinkedList` no)

A user-defined marker interface means nothing to the JVM itself, though your own code can check it with `instanceof`. Today, **annotations** (`@Entity`, `@Test`) are the usual way to attach metadata.

Functional interfaces have **exactly one abstract method** (default and static methods are allowed). They can be implemented with a **lambda** or a **method reference** (Java 8). The optional `@FunctionalInterface` annotation makes the compiler enforce the single-abstract-method rule.

```
interface InterestCalculator {
    double calculate(double principal);
}

InterestCalculator simple = p -> p * 0.05;                    // lambda
InterestCalculator anon = new InterestCalculator() {          // pre-Java 8 equivalent
    public double calculate(double p) { return p * 0.05; }
};
System.out.println(simple.calculate(10000));                  // 500.0
```

Common built-in functional interfaces include `Runnable`, `Callable`, `Comparator`, `Function<T,R>`, `Predicate<T>`, `Supplier<T>`, `Consumer<T>` and `BiFunction`. They are covered with streams in Phase 15.

7.11 Predict the Output

```
class A { void show() { System.out.println("A"); } }
class B extends A { void show() { System.out.println("B"); } }
A obj = new B();
obj.show();                                          // B

static void print(Object o) { System.out.println("Object"); }
static void print(String s) { System.out.println("String"); }
A ref = new B(); Object str = "x";
print(str);                                          // Object (overload chosen by declared type)
print(null);                                         // String

static void num(double d)  { System.out.println("double"); }
static void num(Integer i) { System.out.println("Integer"); }
num(10);                                             // double (widening before boxing)

interface Greeter { default String hi() { return "Hello"; } }
class Polite implements Greeter { public String hi() { return "Namaste"; } }
System.out.println(new Polite().hi());               // Namaste (class wins over default)

A a2 = new A();
System.out.println(a2 instanceof B);                 // false
B b2 = (B) a2;                                       // ClassCastException
```

7.12 Interview Questions

Q. What is polymorphism? What are its types?

The ability of one interface or method name to take many forms. Compile-time polymorphism is achieved by overloading and resolved by the compiler. Runtime polymorphism is achieved by overriding and resolved by the JVM using the actual object type.

Q. Can we overload by changing only the return type?

No. Calls would be ambiguous because the return type isn't part of the signature. The parameter lists must differ.

Q. Can static, private or final methods be overridden?

No. Static methods are hidden (chosen by reference type). Private methods aren't inherited. Final methods are explicitly locked. All three are statically bound.

Q. What are the rules for overriding regarding access and exceptions?

Access must be the same or wider. Checked exceptions must be the same, narrower or none; unchecked exceptions are unrestricted. The return type must be the same or covariant.

Q. What is a covariant return type?

An overriding method may return a subtype of the overridden method's return type (Java 5+), for example `Dog reproduce()` overriding `Animal reproduce()`.

Q. What is dynamic method dispatch?

The mechanism by which a call through a parent or interface reference executes the overriding method of the actual runtime object.

Q. What is the difference between upcasting and downcasting?

Upcasting (child → parent) is implicit and always safe. Downcasting (parent → child) needs an explicit cast and throws `ClassCastException` if the object isn't that type. Guard it with `instanceof` (pattern matching in Java 16+).

Q. Does `final` make an object immutable?

No. A final reference can't be reassigned, but the object it points to can still change. A final class can't be subclassed but can still be mutable.

Q. Can an abstract class have a constructor? Can it be instantiated?

Yes, it can have constructors; they run when subclasses are created. It can't be instantiated directly.

Q. What are the differences between an abstract class and an interface? When would you use each?

Abstract classes can have state, constructors and any access modifiers, and a class can extend only one. Interfaces define contracts with constants plus abstract/default/static/private methods, and a class can implement many. Use an abstract class for a shared base implementation in an IS-A hierarchy; use an interface for capabilities and for decoupling.

Q. Why were default methods introduced?

To let existing interfaces evolve (for example adding `stream()` to `Collection`) without breaking existing implementing classes. They also provide multiple inheritance of behaviour.

Q. How do you resolve two interfaces with the same default method?

The implementing class must override the method. It can delegate to one or both versions with `InterfaceName.super.method()`.

Q. What are marker and functional interfaces?

A marker interface has no methods and tags a class (`Serializable`, `Cloneable`). A functional interface has exactly one abstract method and can be implemented by a lambda (`Runnable`, `Comparator`, `Function`).

7.13 Quick Revision

- **Overloading**: same name, different parameters, compile time. Resolution: exact → widening → boxing → varargs, most specific wins; based on **declared** types.
- **Overriding**: same signature, runtime. Covariant return OK; access same or wider; no broader checked exceptions; always use `@Override`.
- Static methods are **hidden**; private, static and final methods and **fields** are statically bound.
- Upcast is implicit; downcast is explicit and risks `ClassCastException`. Use `instanceof` pattern matching.
- `final`: variable → assign once; method → no override; class → no subclass. **Final is not the same as immutable.**
- Abstract class: no `new`, may have constructors, state and concrete methods; subclasses implement the abstract methods or are abstract themselves.
- Interface: constants are `public static final`; methods are public abstract, plus `default`/`static` (8) and `private` (9). A class implements many interfaces.
- Default-method conflict → override and pick with `X.super.m()`. Class methods beat interface defaults.
- Program to interfaces for loose coupling and mockable tests.
- Marker interface = no methods; functional interface = one abstract method → lambdas.

PHASE 8 OF 18

Object Class, Wrappers, Math & Immutability

toString, equals and hashCode contracts, getClass, shallow vs deep cloning, wrapper classes, autoboxing and the Integer cache, the Math class, BigDecimal for money, and designing immutable classes

IN THIS PHASE

8.1 java.lang.Object: The Root of Every Class
8.2 toString()
8.3 equals(): Logical Equality
8.4 hashCode(): Must Agree with equals()
8.5 getClass() and a Glimpse of Reflection
8.6 clone(): Shallow vs Deep Copy
8.7 finalize() and Resource Cleanup
8.8 Wrapper Classes
8.9 The Math Class
8.10 BigDecimal and BigInteger
8.11 Immutable Classes
8.12 Predict the Output
8.13 Interview Questions
8.14 Quick Revision

8.1 java.lang.Object: The Root of Every Class

Every class directly or indirectly extends `java.lang.Object`. If you write `class User { }`, the compiler treats it as `class User extends Object { }`. So every object, including every array, has these **11 methods**:

Method	Returns	Can override?	Default behaviour	Used for
`toString()`	`String`	Yes	`ClassName@hexHashCode`	Printing, logging, debugging
`equals(Object o)`	`boolean`	Yes	Reference comparison (`this == o`)	Logical equality
`hashCode()`	`int`	Yes	Identity-based number (not an address)	`HashMap`, `HashSet` buckets
`getClass()`	`Class<?>`	No (`final`)	Runtime class of the object	Reflection, type checks
`clone()`	`Object`	Yes (`protected`)	Shallow field copy, if the class is `Cloneable`	Copying objects
`finalize()`	`void`	Yes	Nothing (**deprecated** since Java 9)	Avoid it
`wait()`, `wait(long)`, `wait(long, int)`	`void`	No (`final`)	Releases the monitor and waits	Inter-thread communication (Phase 11)
`notify()`, `notifyAll()`	`void`	No (`final`)	Wakes waiting thread(s)	Inter-thread communication (Phase 11)

8.2 toString()

`System.out.println(obj)` and string concatenation (`"x" + obj`) call `obj.toString()` automatically. The default output is the class name, `@`, and the hash code in hexadecimal, which is rarely useful.

```
class Student {
    int id = 101; String name = "Rohit";
}
System.out.println(new Student());     // Student@1b6d3586 (value varies per run)
```

```
class Student {
    private final int id;
    private final String name;
    Student(int id, String name) { this.id = id; this.name = name; }

    @Override
    public String toString() {
        return "Student{id=" + id + ", name='" + name + "'}";
    }
}
System.out.println(new Student(101, "Rohit"));   // Student{id=101, name='Rohit'}
```

`String`, the wrapper classes, `StringBuilder`, all collections, `LocalDate` and records already override `toString()`.

> **TIP**
> Let the IDE generate `toString()` (Eclipse: Source → Generate toString; IntelliJ: Alt+Insert), or use Lombok's `@ToString` or a record. **Never include passwords, card numbers or PINs** in `toString()`, because it ends up in logs.

8.3 equals(): Logical Equality

The default `Object.equals()` is `==`: it compares **references**. Override it when two different objects should count as **equal by content** (for example, two `Account` objects with the same account number).

The equals contract

Property	Meaning
Reflexive	`x.equals(x)` is `true`
Symmetric	`x.equals(y)` ⇔ `y.equals(x)`
Transitive	`x.equals(y)` and `y.equals(z)` ⇒ `x.equals(z)`
Consistent	Repeated calls give the same result while the objects are unchanged
Non-null	`x.equals(null)` is `false` (never throws)

A correct implementation

```
import java.util.Objects;

public final class Account {
    private final String accountNo;
    private final String ifsc;
    private double balance;            // not part of identity

    public Account(String accountNo, String ifsc) {
        this.accountNo = accountNo; this.ifsc = ifsc;
    }

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                          // 1. same reference
        if (!(o instanceof Account other)) return false;     // 2. type + null check (Java 16)
        return accountNo.equals(other.accountNo)             // 3. compare significant fields
            && Objects.equals(ifsc, other.ifsc);             //    null-safe for nullable fields
    }

    @Override
    public int hashCode() {
        return Objects.hash(accountNo, ifsc);                // SAME fields as equals()
    }
}
```

CORRECTED FROM ORIGINAL NOTES

The original notes' `equals` casts directly with `Student s = (Student) obj;`. That throws `ClassCastException` for other types and `NullPointerException` for `null`, violating the contract. Always check `this == o`, then the type and null with `instanceof` (or `getClass()`), before comparing fields.

NOTE

`instanceof` vs `getClass()`: `instanceof` lets subclass instances compare equal to parent instances, which can break symmetry if the subclass adds fields. `getClass() != o.getClass()` requires the exact same class. Making the class `final`, as above, avoids the debate.

8.4 hashCode(): Must Agree with equals()

Hash-based collections (`HashMap`, `HashSet`, `ConcurrentHashMap`) first use `hashCode()` to choose a **bucket**, and then use `equals()` to find the match inside that bucket.

The hashCode contract

1. Calling it repeatedly on an unchanged object returns the same value (within one run).
2. **If `a.equals(b)` is true, then `a.hashCode() == b.hashCode()` must be true.**
3. Unequal objects *may* share a hash code (a collision), but fewer collisions means better performance.

```
class Emp {                                    // overrides equals but NOT hashCode
    int id;
    Emp(int id) { this.id = id; }
    @Override public boolean equals(Object o) {
        return o instanceof Emp e && e.id == id;
    }
}

Set<Emp> set = new HashSet<>();
set.add(new Emp(1));
set.add(new Emp(1));
System.out.println(set.size());                  // 2  <- duplicate stored!
System.out.println(set.contains(new Emp(1)));    // false <- can't find an "equal" object
// Add hashCode() { return Integer.hashCode(id); }  -> size 1, contains true
```

REMEMBER

Always override `equals()` **and** `hashCode()` **together**, using the **same fields**. Equal objects must have equal hash codes; the reverse doesn't need to hold.

PITFALL

Don't mutate a field used in `hashCode()` while the object is inside a `HashSet` or used as a `HashMap` key. Its bucket no longer matches, so `contains`/`get` fail and `remove` silently does nothing. Prefer **immutable keys** (String, Integer, records).

	==	equals()	hashCode()
Compares	References (or primitive values)	Logical content (if overridden)	n/a: returns a number
Default	n/a	Same as ==	Identity-based
Overridden by	n/a	String, wrappers, collections, records, `LocalDate`, `BigDecimal`	Same classes

PITFALL

`BigDecimal.equals()` also compares **scale**: `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is `false`, while `compareTo` returns `0`. Use `compareTo() == 0` for numeric equality of amounts.

8.5 getClass() and a Glimpse of Reflection

`getClass()` returns the `Class` object describing the object's **runtime** class. It is `final`, so it can't be overridden.

```
Object o = new ArrayList<String>();
Class<?> c = o.getClass();
System.out.println(c.getName());          // java.util.ArrayList
System.out.println(c.getSimpleName());    // ArrayList
System.out.println(c == ArrayList.class); // true: class literal

Object created = Student.class.getDeclaredConstructor(int.class, String.class)
                              .newInstance(1, "Dyn");   // create an object via reflection
// Class.newInstance() is deprecated (Java 9): use getDeclaredConstructor().newInstance()
```

NOTE

Frameworks such as Spring, Hibernate, JUnit and Jackson rely heavily on reflection: they discover annotations, call constructors and set fields at runtime.

8.6 clone(): Shallow vs Deep Copy

`Object.clone()` creates a field-by-field copy, but only if the class implements the `Cloneable` marker interface; otherwise it throws `CloneNotSupportedException` (a checked exception). `clone()` is `protected` in `Object`, so you override it as `public` and can use a **covariant** return type.

```
class Address {
    String city;
    Address(String city) { this.city = city; }
}

class Customer implements Cloneable {
    String name;
    Address address;                                  // reference field
    Customer(String n, Address a) { name = n; address = a; }

    @Override
    public Customer clone() {                         // public + covariant return
        try {
            return (Customer) super.clone();          // SHALLOW copy
        } catch (CloneNotSupportedException e) {
            throw new AssertionError(e);              // can't happen: we are Cloneable
        }
    }

    public Customer deepClone() {
        Customer copy = clone();
        copy.address = new Address(address.city);     // DEEP: copy nested objects too
        return copy;
    }
}

Customer c1 = new Customer("Rohit", new Address("Pune"));
Customer shallow = c1.clone();
shallow.address.city = "Mumbai";
System.out.println(c1.address.city);       // Mumbai  <- shared Address changed!

Customer c2 = new Customer("Amit", new Address("Delhi"));
Customer deep = c2.deepClone();
deep.address.city = "Chennai";
System.out.println(c2.address.city);       // Delhi   <- independent copy
```

```
SHALLOW:  c1 ----> [Customer name="Rohit", address --+]
                                                      +--> [Address city]  (ONE shared object)
          shallow -> [Customer name="Rohit", address --+]

DEEP:     c2 ----> [Customer, address ---> [Address "Delhi"]]
          deep --> [Customer, address ---> [Address "Chennai"]]   (separate objects)
```

	Shallow copy	Deep copy
Primitive fields	Copied	Copied
Immutable references (String)	Shared (safe)	Shared (safe)
Mutable references (Address, List, arrays)	**Shared**, so changes leak	Copied recursively, independent
Cost	Cheap	More work and memory

TIP

`clone()` is widely considered a flawed API: a checked exception, a marker interface, no constructor call, and trouble with final fields. In modern code prefer a **copy constructor** (`new Customer(other)`), a **static factory** (`Customer.copyOf(other)`), or **immutable objects** that never need copying. Arrays are the exception: `arr.clone()` is fine.

8.7 finalize() and Resource Cleanup

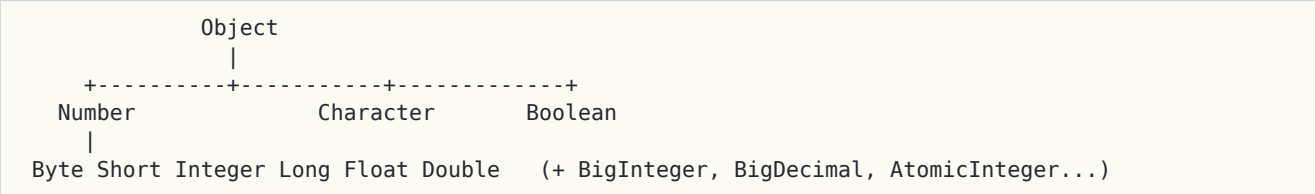
`finalize()` was meant to run before an object is garbage-collected. It is **unpredictable** (it might never run), slow and error-prone. It was deprecated in Java 9 and marked **for removal** in Java 18. Use instead:

- **try-with-resources** with `AutoCloseable` for files, sockets and DB connections (Phase 9).
- `java.lang.ref.Cleaner` (Java 9) for rare native-resource safety nets.

8.8 Wrapper Classes

Each primitive has a **wrapper class** in `java.lang` that represents the value as an **object**. All wrappers are **final** and **immutable**.

Primitive	Wrapper	Parent	Notable statics
byte	Byte	Number	parseByte, MIN_VALUE/MAX_VALUE
short	Short	Number	parseShort
int	Integer	Number	parseInt, valueOf, toBinaryString, toHexString, sum, max, compare
long	Long	Number	parseLong
float	Float	Number	parseFloat, isNaN
double	Double	Number	parseDouble, isNaN, isInfinite
char	Character	Object	isDigit, isLetter, toUpperCase, isWhitespace
boolean	Boolean	Object	parseBoolean, TRUE/FALSE



Why wrappers?

1. **Collections and generics work only with objects**: `List<Integer>`, not `List<int>`.
2. They can be `null`, which represents "no value" (for example a nullable database column).
3. They provide **utility methods**: parsing, conversion, comparison, constants.
4. Some APIs need objects: synchronization, reflection, serialization.

Boxing and unboxing

```
int i = 10;
Integer boxed = Integer.valueOf(i);      // boxing (explicit)
Integer auto = i;                        // AUTOBOXING (Java 5): Integer.valueOf(i)
int back = auto;                         // AUTO-UNBOXING: auto.intValue()

Integer a = 20, b = 30;
Integer sum = a + b;                     // unbox, add as ints, box again -> 50

List<Integer> list = new ArrayList<>();
list.add(5);                             // autoboxing
int first = list.get(0);                 // auto-unboxing
```

> **CORRECTED FROM ORIGINAL NOTES**
> The original notes create wrappers with constructors such as `new Integer(10)`, `new Byte(b)` and `new Character('A')`. These constructors are **deprecated since Java 9** and **marked for removal since Java 16**. Use `Integer.valueOf(10)`, autoboxing (`Integer x = 10;`) or `Integer.parseInt("10")`. Also, `Integer` implements `Comparable<Integer>`, not `Comparator<Integer>` as written in the original notes.

The Integer cache and the == trap

`Integer.valueOf()` (and therefore autoboxing) **caches** values from **-128 to 127**, so equal small values share the same object. Outside that range, new objects are created. The same caching applies to `Byte`, `Short`, `Long`, `Character` (0–127) and `Boolean`.

```
Integer x1 = 127, x2 = 127;
System.out.println(x1 == x2);        // true   (same cached object)

Integer y1 = 128, y2 = 128;
System.out.println(y1 == y2);        // false  (two different objects!)
System.out.println(y1.equals(y2));   // true   (compare VALUES with equals)
System.out.println(y1.intValue() == y2);   // true (one side unboxed -> numeric compare)
```

PITFALL

Never compare wrapper objects with `==`. Use `equals()`, `Integer.compare()`, or unbox. Tests that pass with small IDs and fail with large ones are a classic production bug.

Unboxing null throws NullPointerException

```
Map<String, Integer> stock = new HashMap<>();
int qty = stock.get("APPLE");     // NullPointerException: get() returned null -> unboxing
int safe = stock.getOrDefault("APPLE", 0);   // OK

Integer maybe = null;
if (maybe == 5) { }               // NullPointerException as well
```

Useful wrapper methods

Method	Example	Result
`Integer.parseInt(s)` / `parseInt(s, radix)`	`Integer.parseInt("ff", 16)`	`255` (primitive)
`Integer.valueOf(s)`	`Integer.valueOf("42")`	`Integer` object
`xxxValue()`	`Integer.valueOf(7).doubleValue()`	`7.0`
`Integer.toBinaryString` / `toHexString` / `toOctalString`	`Integer.toBinaryString(10)`	`"1010"`
`Integer.compare(a, b)`	`Integer.compare(3, 7)`	negative
`Integer.sum`, `max`, `min`	`Integer.max(3, 7)`	`7` (handy as method references)
`Integer.bitCount(n)`	`Integer.bitCount(7)`	`3`
`Character.isDigit / isLetter / isLetterOrDigit`	`Character.isDigit('7')`	`true`
`Character.getNumericValue(c)`	`Character.getNumericValue('7')`	`7`
`Double.isNaN(d)` / `Double.compare(a, b)`	`Double.isNaN(0.0 / 0)`	`true`
`Boolean.parseBoolean(s)`	`Boolean.parseBoolean("TRUE")`	`true` (anything else gives false)

TIP

Boxing has a cost. Summing a `Long` in a loop (`Long total = 0L; total += x;`) creates millions of objects. Use primitives for heavy arithmetic, and primitive streams (`IntStream`, `mapToInt`) in stream pipelines.

8.9 The Math Class

`java.lang.Math` is a `final` utility class with a private constructor and only **static** methods and constants (`Math.PI`, `Math.E`).

Method	Example	Result	Note
`abs(x)`	`Math.abs(-7.5)`	`7.5`	`Math.abs(Integer.MIN_VALUE)` is still **negative** (overflow)
`max(a, b)` / `min(a, b)`	`Math.max(10, 20)`	`20`	
`pow(a, b)`	`Math.pow(2, 10)`	`1024.0`	Always returns `double`
`sqrt(x)` / `cbrt(x)`	`Math.sqrt(25)`	`5.0`	`sqrt(-1)` is `NaN`
`ceil(x)`	`Math.ceil(4.1)`	`5.0`	Rounds up
`floor(x)`	`Math.floor(4.9)`	`4.0`	Rounds down
`round(x)`	`Math.round(4.5)`	`5` (long)	Half rounds **up** toward +∞: `round(-4.5)` is `-4`
`rint(x)`	`Math.rint(4.5)`	`4.0`	Half rounds to **even**
`random()`	`Math.random()`	`[0.0, 1.0)`	See below
`floorDiv` / `floorMod`	`Math.floorMod(-7, 3)`	`2`	vs `-7 % 3` gives `-1`
`addExact`, `subtractExact`, `multiplyExact`, `toIntExact`	`Math.addExact(Integer.MAX_VALUE, 1)`	`ArithmeticException`	Overflow-safe arithmetic
`hypot(x, y)`	`Math.hypot(3, 4)`	`5.0`	√(x² + y²)
`signum(x)`	`Math.signum(-3.2)`	`-1.0`	
`sin`, `cos`, `tan`, `log`, `log10`, `exp`	`Math.sin(Math.toRadians(30))`	`≈ 0.5`	Trig functions take **radians**
`clamp(v, min, max)` (Java 21)	`Math.clamp(150, 0, 100)`	`100`	

```java
// Random integer in [min, max] inclusive
int dice = (int) (Math.random() * 6) + 1;                    // 1..6
int r = ThreadLocalRandom.current().nextInt(100, 1000);      // 100..999 (preferred)

// Security-sensitive values (OTP, tokens, salts): use SecureRandom
SecureRandom sr = new SecureRandom();
String otp = String.format("%06d", sr.nextInt(1_000_000));   // e.g. "048213"

import static java.lang.Math.*;                              // static import
double area = PI * pow(5, 2);                                // no "Math." prefix
```

PITFALL

Don't use `Math.random()` or `java.util.Random` for OTPs, passwords or tokens, because they are predictable. Use `SecureRandom`.

8.10 BigDecimal and BigInteger

`BigInteger` holds integers of **arbitrary size** (factorials, cryptography). `BigDecimal` holds **exact decimal** numbers with controllable scale and rounding, which makes it **the** type for money in banking and fintech.

```
import java.math.*;

BigDecimal principal = new BigDecimal("100000.00");      // use the String constructor
BigDecimal rate = new BigDecimal("0.075");
BigDecimal interest = principal.multiply(rate)
                               .setScale(2, RoundingMode.HALF_UP);    // 7500.00
BigDecimal total = principal.add(interest);                           // 107500.00

new BigDecimal(0.1);         // 0.1000000000000000055511151231257827... (inexact!)
BigDecimal.valueOf(0.1);     // 0.1 (uses Double.toString, which is safe)

BigDecimal emi = total.divide(new BigDecimal("12"), 2, RoundingMode.HALF_EVEN);  // 8958.33
// total.divide(new BigDecimal("3"));   // ArithmeticException: non-terminating decimal

BigInteger big = BigInteger.valueOf(2).pow(100);   // 1267650600228229401496703205376
```

Rule	Why
Create from a `String` (or `BigDecimal.valueOf(double)`)	`new BigDecimal(0.1)` captures the binary error of the double
Always pass a scale and `RoundingMode` to `divide`	Otherwise non-terminating results throw
`HALF_EVEN` (banker's rounding) / `HALF_UP`	Choose according to your financial rules
Compare with `compareTo`, not `equals`	`equals` also compares scale (`2.0` ≠ `2.00`)
BigDecimal is **immutable**	`a.add(b)` returns a new value, so assign it

8.11 Immutable Classes

An **immutable** object's state can't change after construction. Examples in the JDK: `String`, all wrappers, `BigDecimal`, `BigInteger`, `LocalDate`/`LocalDateTime`, `List.of(...)` and records.

Rules for writing one

Rule	Why
Declare the class `final` (or give it private constructors)	Subclasses can't add mutable behaviour
Make all fields `private final`	Assigned once, not visible outside
Provide **no setters** or other mutators	Nothing can change state
Initialise everything in the constructor	The object is fully built from the start
Defensive-copy mutable inputs in the constructor	The caller can't change your internals through its own reference
Return copies or unmodifiable views of mutable fields from getters	Callers can't change your internals through the returned reference
"Modifications" return **new** objects (`withX(...)`)	Like `String.concat` and `LocalDate.plusDays`

```java
import java.time.LocalDate;
import java.util.List;

public final class Transaction {
    private final String txnId;
    private final BigDecimal amount;          // immutable type: safe to share
    private final LocalDate date;             // immutable type: safe to share
    private final List<String> tags;          // mutable type: needs defensive copying

    public Transaction(String txnId, BigDecimal amount, LocalDate date, List<String> tags) {
        this.txnId = txnId;
        this.amount = amount;
        this.date = date;
        this.tags = List.copyOf(tags);        // defensive, unmodifiable copy (Java 10)
    }

    public String getTxnId()      { return txnId; }
    public BigDecimal getAmount() { return amount; }
    public LocalDate getDate()    { return date; }
    public List<String> getTags() { return tags; }   // already unmodifiable

    public Transaction withAmount(BigDecimal newAmount) {   // "wither" returns a NEW object
        return new Transaction(txnId, newAmount, date, tags);
    }
}
```

PITFALL

Without the defensive copy, a caller can still change your "immutable" object: `List<String> t = new ArrayList<>(); var tx = new Transaction(..., t); t.add("HACK");` changes `tx`'s tags. Old mutable types such as `java.util.Date` need copying too, which is one reason to prefer `java.time`.

Benefits

- **Thread-safe** without synchronization: safe to share between threads.
- **Safe as `HashMap` keys and `HashSet` elements**: the hash never changes.
- **Cacheable and reusable** (like the String pool and the Integer cache).
- **Easier to reason about and test**: no hidden state changes.
- The trade-off is that every change creates a new object (usually negligible).

MODERN JAVA

A **record** (Java 16) is a concise immutable data carrier: `public record Money(BigDecimal amount, String currency) { }`. It generates the private final fields, constructor, accessors, `equals`, `hashCode` and `toString`. Add validation and defensive copies in a *compact constructor*. Records are covered in Phase 15.

8.12 Predict the Output

```
Integer a = 100, b = 100, c = 1000, d = 1000;
System.out.println((a == b) + " " + (c == d) + " " + c.equals(d));   // true false true

Long l = 127L;
System.out.println(l.equals(127));        // false (Integer vs Long: different types)
System.out.println(l == 127);             // true  (unboxed numeric comparison)

System.out.println(Math.round(-2.5) + " " + Math.round(2.5));        // -2 3
System.out.println(Math.floorMod(-7, 3) + " " + (-7 % 3));           // 2 -1
System.out.println(Math.abs(Integer.MIN_VALUE));                     // -2147483648

System.out.println(new BigDecimal("2.0").equals(new BigDecimal("2.00")));       // false
System.out.println(new BigDecimal("2.0").compareTo(new BigDecimal("2.00")));    // 0

Object o = new Object();
System.out.println(o.equals(o) + " " + o.equals(null));             // true false

Character ch = 'A';
System.out.println(ch + 1);               // 66 (unboxed to char, promoted to int)
```

8.13 Interview Questions

Q. Why must `equals()` and `hashCode()` be overridden together?

Hash-based collections locate the bucket by `hashCode()` and then confirm with `equals()`. If equal objects have different hash codes, they land in different buckets, so duplicates appear in a `HashSet` and `HashMap.get()` misses entries.

Q. Can two unequal objects have the same hash code?

Yes, that is a collision, and it's allowed. The reverse isn't: equal objects must have equal hash codes.

Q. What is the default `toString()` output?

`getClass().getName() + "@" + Integer.toHexString(hashCode())`, for example `Student@1b6d3586`.

Q. What is the difference between a shallow and a deep copy?

A shallow copy copies field values, so referenced objects are shared. A deep copy also copies the referenced mutable objects, so the copy is fully independent. `Object.clone()` is shallow by default.

Q. Why is `Cloneable` called a marker interface? What if you call `clone()` without it?

It has no methods; it only signals that `Object.clone()` is permitted. Without it, `super.clone()` throws `CloneNotSupportedException`.

Q. What are wrapper classes and why are they needed?

Object representations of the eight primitives. They are needed for collections and generics, for nullability, and for utility methods. They are final and immutable.

Q. What is autoboxing? Where can it cause problems?

Automatic conversion between primitives and wrappers (`valueOf` and `xxxValue` inserted by the compiler). Problems: `==` comparisons outside the -128..127 cache, `NullPointerException` when unboxing `null`, and performance cost in loops.

Q. Why does `Integer a = 127, b = 127; a == b` give true but 128 gives false?

`Integer.valueOf` caches -128 to 127, so 127 returns the same object. 128 creates distinct objects, so `==` (a reference comparison) is false.

Q. Why use BigDecimal instead of double for money?

`double` is binary floating point and can't represent many decimal fractions exactly, so rounding errors accumulate. `BigDecimal` stores exact decimals with explicit scale and rounding modes.

Q. How do you create an immutable class?

Make it final; make all fields private final; add no setters; initialise everything in the constructor; defensively copy mutable inputs and outputs; return new objects for changes. Or use a record (with defensive copies for mutable components).

Q. What is the difference between `Math.round`, `Math.ceil`, `Math.floor` and `Math.rint`?

`ceil` rounds up, `floor` rounds down, `round` rounds half toward positive infinity (and returns a long/int), and `rint` rounds half to even (and returns a double).

8.14 Quick Revision

- `Object` has 11 methods. `getClass`, `wait` and `notify`/`notifyAll` are final; `clone` is protected; `finalize` is deprecated.
- Override `toString()` for readable logs, but never print secrets.
- `equals` contract: reflexive, symmetric, transitive, consistent, `x.equals(null)` is false. Check `this == o` and `instanceof` first.
- **Equal objects ⇒ equal hash codes.** Override both with the **same fields**; `Objects.hash(...)` helps.
- Don't mutate fields of objects inside hash-based collections.
- `clone()` needs `Cloneable` and is shallow by default; prefer copy constructors or factories; deep copy nested mutables.
- Wrappers: final, immutable. `Number` is the parent of the numeric ones. Use `valueOf`, not the deprecated constructors.
- Autoboxing uses `valueOf`; unboxing uses `xxxValue` → NPE on `null`.
- Integer cache: -128..127. **Never compare wrappers with `==`.**
- Math: static utilities. `round(-2.5)` is -2; `floorMod` for positive modulo; `*Exact` methods for overflow safety; `SecureRandom` for OTPs.
- Money: `BigDecimal` from a String, explicit `RoundingMode`, compare with `compareTo`.
- Immutable class: final class, private final fields, no setters, defensive copies, withers. Records simplify this.

PHASE 9 OF 18

Exception Handling

The Throwable hierarchy, checked vs unchecked, try/catch/finally rules, multi-catch, try-with-resources, throw vs throws, propagation, custom and chained exceptions, common exceptions and errors, and best practices

IN THIS PHASE

9.1 What Is an Exception?
9.2 The Throwable Hierarchy
9.3 Checked vs Unchecked Exceptions
9.4 try and catch
9.5 The finally Block
9.6 try-with-resources (Java 7)
9.7 throw: Raising an Exception Yourself
9.8 throws: Declaring Exceptions
9.9 Exception Propagation
9.10 Useful Throwable Methods
9.11 Custom (User-Defined) Exceptions
9.12 Exception Chaining (Wrapping)
9.13 Exceptions and Overriding (Recap)
9.14 Common Exceptions: Cause and Fix
9.15 Important Errors
9.16 Best Practices
9.17 Predict the Output
9.18 Interview Questions
9.19 Quick Revision

9.1 What Is an Exception?

An **exception** is an unexpected event during program **execution** that disrupts the normal flow of instructions: dividing by zero, a missing file, a `null` reference, a lost DB connection. **Exception handling** lets the program respond and keep running (**graceful termination**) instead of crashing (**abnormal termination**).

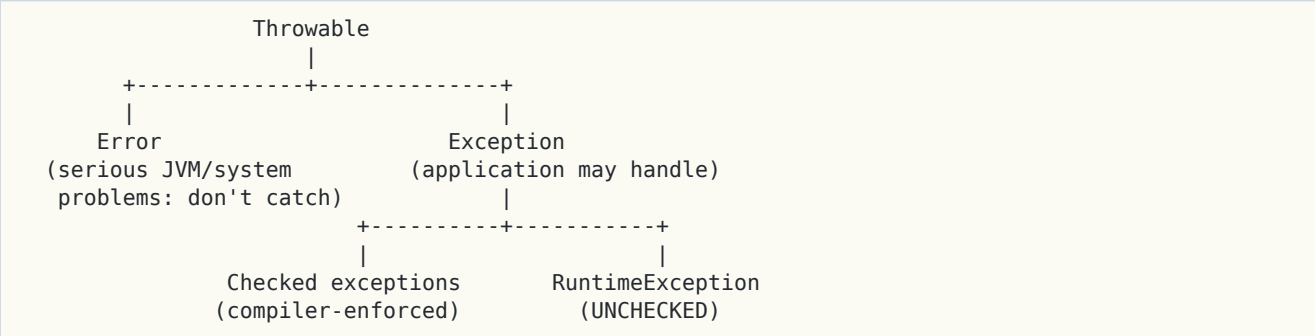
```
public class Demo {
    public static void main(String[] args) {
        System.out.println("Start");
        int a = 10 / 0;                    // ArithmeticException thrown here
        System.out.println("End");         // never reached
    }
}
```

```
Start
Exception in thread "main" java.lang.ArithmeticException: / by zero
    at Demo.main(Demo.java:4)
```

Without a handler, the **default exception handler** prints the exception's name, message and **stack trace**, and terminates the thread.

Benefit of handling	Example
Prevents abnormal termination	Show "Service unavailable, try again" instead of crashing
Separates error handling from business logic	The happy path stays readable
Propagates errors to the right layer	A DAO throws, the service translates, the controller returns HTTP 4xx/5xx
Releases resources reliably	Files, DB connections and sockets are closed in `finally` or try-with-resources

9.2 The Throwable Hierarchy



Branch	Important subclasses
`Error`	`VirtualMachineError` → `OutOfMemoryError`, `StackOverflowError`; `LinkageError` → `NoClassDefFoundError`, `ExceptionInInitializerError`; `AssertionError`
Checked `Exception`s	`IOException` → `FileNotFoundException`, `EOFException`; `SQLException`; `ClassNotFoundException`; `InterruptedException`; `ParseException`; `CloneNotSupportedException`
`RuntimeException` (unchecked)	`ArithmeticException`; `NullPointerException`; `ClassCastException`; `IndexOutOfBoundsException` → `ArrayIndexOutOfBounds…`, `StringIndexOutOfBounds…`; `IllegalArgumentException` → `NumberFormatException`; `IllegalStateException`; `UnsupportedOperationException`; `ConcurrentModificationException`; `NoSuchElementException` → `InputMismatchException`; `ArrayStoreException`; `NegativeArraySizeException`; `DateTimeException`

CORRECTED FROM ORIGINAL NOTES

The hierarchy diagram in the original notes shows `OutOfMemoryError` and `StackOverflowError` as siblings of `VirtualMachineError`; they are its **subclasses**. Likewise, `FileNotFoundException` is a subclass of `IOException`, not a sibling.

Exception vs Error

	Exception	Error
Represents	Application-level problems	Serious JVM or system failures
Recoverable?	Usually	Usually not
Should you catch it?	Yes, where you can act on it	Generally no (log it at the top level and let the process fail or restart)
Examples	`IOException`, `SQLException`, `NullPointerException`	`OutOfMemoryError`, `StackOverflowError`, `NoClassDefFoundError`
Checked?	Checked or unchecked	Always unchecked

CORRECTED FROM ORIGINAL NOTES

The original notes say errors "cannot be handled", yet a later example catches `StackOverflowError`. Errors **can** technically be caught (they are `Throwable`s), but you usually **shouldn't**, because the JVM may be in an unreliable state.

9.3 Checked vs Unchecked Exceptions

	Checked exceptions	Unchecked exceptions
Which classes	`Exception` and its subclasses, **except** `RuntimeException`	`RuntimeException`, `Error` and their subclasses
Checked by	The **compiler**: you must handle or declare them	Nobody at compile time
Typically caused by	External conditions outside the program's control (files, network, DB)	**Programming bugs** (null, bad index, bad cast, bad argument)
Handling	**Handle or declare** rule: `try-catch` or `throws`	Optional. Fix the bug or validate input.
Examples	`IOException`, `FileNotFoundException`, `SQLException`, `ClassNotFoundException`, `InterruptedException`	`NullPointerException`, `ArithmeticException`, `ArrayIndexOutOfBoundsException`, `NumberFormatException`, `IllegalArgumentException`

```
FileReader fr = new FileReader("abc.txt");   // COMPILE ERROR: unreported exception
                                             // FileNotFoundException (catch or declare)
String s = null;
s.length();                                  // compiles fine, fails at RUNTIME with NPE
```

REMEMBER

"Checked" refers to **compile time**. Every exception, checked or not, actually occurs at **runtime**. `Exception` and `Throwable` are sometimes called *partially checked* because some of their subclasses are unchecked.

9.4 try and catch

```
try {
    // risky code
} catch (ExceptionType e) {
    // handling code: runs only if a matching exception is thrown
}
```

```
System.out.println("1. start");
try {
    int[] arr = new int[3];
    System.out.println("2. before");
    arr[5] = 10;                                  // exception: jump to catch
    System.out.println("3. after");               // SKIPPED
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("4. caught: " + e.getMessage());
}
System.out.println("5. end");                     // runs: graceful continuation
// 1. start  2. before  4. caught: Index 5 out of bounds for length 3  5. end
```

NOTE

Once an exception is thrown inside `try`, the **remaining statements in the try block are skipped**. Control never returns to the point of failure. Keep try blocks small and around only the risky lines.

Multiple catch blocks: child before parent

```
try {
    String s = args.length > 0 ? args[0] : null;
    int n = Integer.parseInt(s.trim());
    System.out.println(100 / n);
} catch (NullPointerException e) {
    System.out.println("No input");
} catch (NumberFormatException e) {
    System.out.println("Not a number");
} catch (ArithmeticException e) {
    System.out.println("Zero not allowed");
} catch (Exception e) {                    // most general LAST (catch-all)
    System.out.println("Unexpected: " + e);
}
```

The JVM checks catch blocks **top to bottom** and runs only the **first** matching one.

```
try { } catch (Exception e) { } catch (ArithmeticException e) { }
// COMPILE ERROR: exception ArithmeticException has already been caught (unreachable catch)
```

Multi-catch (Java 7)

```
try {
    // ...
} catch (NumberFormatException | ArithmeticException e) {   // same handling for both
    System.out.println("Bad input: " + e.getMessage());
    // e = new ArithmeticException();   // COMPILE ERROR: e is implicitly final
}
// catch (IOException | FileNotFoundException e)  -> COMPILE ERROR: types related by subclassing
```

PITFALL

Catching a checked exception that the try block can't throw is a compile error: `try { int x = 1; } catch (IOException e) { }` fails with "exception IOException is never thrown". `Exception`, `Throwable` and unchecked exceptions are exempt from this rule.

Nested try

```
try {
    try {
        int x = 10 / 0;
    } catch (ArithmeticException e) {
        System.out.println("Inner caught");
    }
    int[] a = new int[2];
    a[3] = 1;
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Outer caught");
}
// Inner caught
// Outer caught
```

If an inner `try` has no matching catch, the exception moves to the outer `try`'s catch blocks.

9.5 The finally Block

`finally` holds **cleanup code** that runs **whether or not an exception occurs**, and even when `try` or `catch` executes `return`, `break` or `continue`.

Scenario	try	catch	finally	Code after
No exception	Completes	Skipped	**Runs**	Runs

Scenario	try	catch	finally	Code after
Exception, matching catch	Stops at the exception	Runs	**Runs**	Runs
Exception, no matching catch	Stops at the exception	n/a	**Runs**	Skipped: the exception propagates
`return` inside try	Returns...	n/a	**Runs before the method actually returns**	n/a

```
FileReader fr = null;
try {
    fr = new FileReader("abc.txt");
    System.out.println(fr.read());
} catch (IOException e) {
    System.out.println("Read failed: " + e.getMessage());
} finally {
    if (fr != null) {
        try { fr.close(); } catch (IOException e) { /* log */ }   // verbose!
    }
}
```

`try-finally` without `catch` is legal: cleanup runs, and the exception still propagates. `try` alone (with neither catch nor finally) is not legal, except in the try-with-resources form.

When finally does NOT run

1. `System.exit()` (or `Runtime.halt()`) is called in `try` or `catch`.
2. The JVM crashes, or the process is killed (`kill -9`, power loss).
3. The `try`/`catch` never finishes: an infinite loop or a deadlock.
4. The thread is a daemon thread and the JVM shuts down.

return in finally: a trap

```
static int test() {
    try {
        return 1;
    } finally {
        return 2;          // OVERRIDES the try's return value (and swallows exceptions!)
    }
}
System.out.println(test());   // 2

static int counter() {
    int x = 10;
    try {
        return x;          // value 10 is saved here
    } finally {
        x = 20;            // changing the local doesn't affect the saved return value
    }
}
System.out.println(counter()); // 10
```

PITFALL

Never `return` (or `throw`) from `finally`. It silently discards any exception thrown in `try` and replaces the return value. Most IDEs and Sonar flag it.

final vs finally vs finalize

final	finally	finalize()
Keyword (modifier)	Block used with `try`	Method of `Object`
Constant variable, no override, no subclass	Cleanup that always runs	Called by GC before reclaiming (unreliable)
Widely used	Widely used (or try-with-resources)	**Deprecated** (Java 9, for removal in 18)

9.6 try-with-resources (Java 7)

Resources declared in the `try (...)` header are **closed automatically**, in **reverse order** of declaration, whether the block completes normally or not. A resource must implement `AutoCloseable` (or its sub-interface `java.io.Closeable`).

```
try (BufferedReader br = new BufferedReader(new FileReader("abc.txt"));
     BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {
        bw.write(line.toUpperCase());
        bw.newLine();
    }
} catch (IOException e) {
    System.out.println("I/O problem: " + e.getMessage());
}   // bw closed first, then br: no finally needed

// JDBC: Connection, PreparedStatement and ResultSet are all AutoCloseable
try (Connection con = dataSource.getConnection();
     PreparedStatement ps = con.prepareStatement("SELECT balance FROM account WHERE id=?")) {
    ps.setLong(1, 1001L);
    try (ResultSet rs = ps.executeQuery()) {
        if (rs.next()) System.out.println(rs.getBigDecimal(1));
    }
}
```

```
BufferedReader reader = new BufferedReader(new FileReader("abc.txt"));
try (reader) {                      // Java 9: an existing final/effectively-final variable
    System.out.println(reader.readLine());
}
```

Suppressed exceptions (verified output)

If the body throws **and** `close()` also throws, the body's exception is the one propagated. The close exception is attached to it as **suppressed** instead of being lost, as it would be with a hand-written `finally`.

```
class Res implements AutoCloseable {
    private final String name;
    Res(String name) { this.name = name; System.out.println("open " + name); }
    @Override public void close() {
        System.out.println("close " + name);
        throw new IllegalStateException("close failed: " + name);
    }
}

try (Res a = new Res("A"); Res b = new Res("B")) {
    throw new RuntimeException("body failed");
} catch (RuntimeException e) {
    System.out.println("caught: " + e.getMessage());
    for (Throwable s : e.getSuppressed()) System.out.println("  suppressed: " + s.getMessage());
}
```

```
open A
open B
close B                          <- reverse order
close A
caught: body failed
  suppressed: close failed: B
  suppressed: close failed: A
```

TIP

Use try-with-resources for **anything** that must be closed: streams, readers, writers, `Scanner`, JDBC objects, `ExecutorService` (Java 19+), HTTP responses. It's shorter, correct by default, and keeps suppressed exceptions.

9.7 throw: Raising an Exception Yourself

`throw` creates an exception **explicitly**, usually for validation or business rules. It throws **one** object of a type that extends `Throwable`.

```java
static String getName(int id) {
    if (id <= 0) {
        throw new IllegalArgumentException("id must be positive: " + id);
    }
    if (id != 100) {
        throw new RuntimeException("Invalid Id");
        // System.out.println("x");   // COMPILE ERROR: unreachable statement after throw
    }
    return "Rohit";
}
```

```java
catch (SQLException e) {
    log.error("DB failure", e);
    throw e;                        // RETHROW to the caller after partial handling
}

throw null;                         // compiles, but throws NullPointerException at runtime
```

9.8 throws: Declaring Exceptions

`throws` in a method signature **declares** that the method may throw these (usually checked) exceptions and **delegates** handling to the caller. It satisfies the compiler's *handle or declare* rule.

```java
void loadConfig(String path) throws IOException {        // declare: caller must deal with it
    try (BufferedReader br = Files.newBufferedReader(Path.of(path))) {
        System.out.println(br.readLine());
    }
}

void start() {
    try {
        loadConfig("app.properties");
    } catch (IOException e) {                            // handled here
        System.out.println("Using defaults: " + e.getMessage());
    }
}

public static void main(String[] args) throws Exception { }   // declare everything (demos only)
```

CORRECTED FROM ORIGINAL NOTES

The original notes say `throws` lets us "ignore" checked exceptions. Nothing is ignored. `throws` **passes responsibility to the caller**. If nobody up the call stack catches the exception, it still terminates the thread at runtime.

throw vs throws

throw	throws
Actually raises an exception	Declares possible exceptions
Used **inside** a method body	Used in the method **signature**
Followed by **one** exception object: `throw new X(...)`	Followed by one or more **class names**: `throws A, B`
Mainly for unchecked/business exceptions (any `Throwable` works)	Mainly for checked exceptions (unchecked allowed, for documentation)
Execution stops at the `throw`	Has no runtime effect by itself

9.9 Exception Propagation

When a method doesn't handle an exception, the JVM **unwinds the call stack**: it pops the current frame and looks for a handler in the **caller**, then the caller's caller, and so on. If it reaches the top of the thread unhandled, the default handler prints the stack trace and the thread dies.

```
class InvalidNumberException extends Exception {         // custom CHECKED exception
    InvalidNumberException(String msg) { super(msg); }
}

public class Demo {
    void m2(int a, int b) throws InvalidNumberException {
        System.out.println("m2() started");
        try {
            int c = a / b;
            if (c < 0) throw new InvalidNumberException("Negative value not allowed");
            System.out.println("Result = " + c);
        } catch (ArithmeticException e) {
            System.out.println("Arithmetic Exception Occurred");
            throw e;                                     // rethrow
        } finally {
            System.out.println("m2() finally block executed");
        }
        System.out.println("m2() ended");
    }

    void m1(int a, int b) throws InvalidNumberException {
        System.out.println("m1() started");
        m2(a, b);
        System.out.println("m1() ended");
    }

    public static void main(String[] args) {
        System.out.println("main() started");
        try {
            new Demo().m1(10, 0);
        } catch (InvalidNumberException e) {
            System.out.println("User Defined Exception : " + e.getMessage());
        } catch (Exception e) {
            System.out.println("Exception Handled In Main: " + e);
        }
        System.out.println("main() ended");
    }
}
```

```
main() started                    output with m1(10, 0)
m1() started
m2() started
Arithmetic Exception Occurred
m2() finally block executed         <- finally runs even while rethrowing
Exception Handled In Main: java.lang.ArithmeticException: / by zero
main() ended                         <- "m2() ended" and "m1() ended" never print

With m1(-10, 2): ... m2() finally block executed
                 User Defined Exception : Negative value not allowed
```

```
call stack                        propagation (unwinding)
+-----------+                     m2()  --X-->  m1()  --X-->  main()  -> caught
|   m2()    |  <- 10 / 0 here
|   m1()    |                     java.lang.ArithmeticException: / by zero
|   main()  |                         at Demo.m2(Demo.java:9)      <- origin (top)
+-----------+                         at Demo.m1(Demo.java:23)
                                      at Demo.main(Demo.java:31)   <- entry point
```

TIP

Read a stack trace **from the top**: the first line is the exception type and message, and the first `at` line is where it was thrown. Look for the first line in **your own package**. For chained exceptions, scroll to the last **Caused by:**, which is usually the root cause.

9.10 Useful Throwable Methods

Method	Returns
`getMessage()`	The detail message (may be `null`)
`toString()`	`ClassName: message`, as printed by `System.out.println(e)`
`printStackTrace()`	Prints the type, message and full call stack to `System.err`
`getCause()`	The wrapped underlying exception (chaining)
`getStackTrace()`	`StackTraceElement[]` for programmatic inspection
`getSuppressed()` / `addSuppressed()`	Exceptions suppressed during try-with-resources
`fillInStackTrace()`	Re-records the stack (rarely needed)

TIP

In real applications, don't call `e.printStackTrace()` or `System.out.println(e)`. Use a logger, which captures the message, the stack trace and the context: `log.error("Payment failed for txn {}", txnId, e);` (SLF4J: pass `e` as the **last** argument).

9.11 Custom (User-Defined) Exceptions

Create your own exception to represent a **domain-specific** problem with a meaningful name and extra data.

Extend	Type	Caller must handle?	Typical use
`Exception`	Checked	Yes (handle or declare)	Recoverable conditions the caller should consciously handle
`RuntimeException`	Unchecked	No	Business-rule violations and invalid state. **The most common choice in Spring Boot apps.**

```
public class InsufficientFundsException extends RuntimeException {
    private final String accountNo;
    private final BigDecimal shortfall;

    public InsufficientFundsException(String accountNo, BigDecimal shortfall) {
        super("Insufficient funds in " + accountNo + ", short by " + shortfall);
        this.accountNo = accountNo;
        this.shortfall = shortfall;
    }

    public InsufficientFundsException(String message, Throwable cause) {  // chaining ctor
        super(message, cause);
        this.accountNo = null;
        this.shortfall = null;
    }

    public String getAccountNo()     { return accountNo; }
    public BigDecimal getShortfall() { return shortfall; }
}

void withdraw(BigDecimal amount) {
    if (balance.compareTo(amount) < 0) {
        throw new InsufficientFundsException(accountNo, amount.subtract(balance));
    }
    balance = balance.subtract(amount);
}
```

NOTE

In Spring Boot, custom runtime exceptions are typically mapped to HTTP responses in one place with `@RestControllerAdvice` + `@ExceptionHandler` (for example `InsufficientFundsException` → 422, `NoDataFoundException` → 404). Validate those mappings in your API tests.

9.12 Exception Chaining (Wrapping)

When translating a low-level exception into a higher-level one, **pass the original as the cause** so the root cause isn't lost.

```
public Account findAccount(long id) {
    try {
        return jdbcFind(id);
    } catch (SQLException e) {
        throw new DataAccessFailure("Could not load account " + id, e);   // keep the cause!
    }
}
```

```
Exception in thread "main" DataAccessFailure: Could not load account 1001
    at AccountRepo.findAccount(AccountRepo.java:12)
    ...
Caused by: java.sql.SQLException: Connection refused
    at ...
```

PITFALL

`throw new DataAccessFailure("failed");` without the cause, or `throw new X(e.getMessage())`, throws away the original stack trace. Debugging production issues becomes guesswork.

9.13 Exceptions and Overriding (Recap)

Parent method declares	Overriding method may declare
Nothing	No **checked** exceptions (any unchecked are fine)
`throws IOException`	Nothing, `IOException`, or a subclass such as `FileNotFoundException`
`throws IOException`	**Not** `Exception` or an unrelated checked exception such as `SQLException`
Anything	Any unchecked exceptions

Constructors are not overridden, but a subclass constructor must declare (at least) the checked exceptions of the `super(...)` constructor it calls, because it can't catch them around `super(...)`.

9.14 Common Exceptions: Cause and Fix

Exception	Type	Typical cause	Typical fix
`NullPointerException`	Unchecked	Calling a member on `null`	Validate inputs, `Objects.requireNonNull`, `Optional`, null-safe `equals`
`ArithmeticException`	Unchecked	Integer `/ 0`; non-terminating `BigDecimal.divide`	Check the divisor; pass scale and `RoundingMode`
`ArrayIndexOutOfBoundsException`	Unchecked	Index outside `0..length-1`	Loop with `i < arr.length`; check `args.length`
`StringIndexOutOfBoundsException`	Unchecked	Bad `charAt`/`substring` index	Check `length()` first
`NumberFormatException`	Unchecked	`Integer.parseInt("abc")`, spaces, empty string	`trim()`, validate with regex, catch it
`ClassCastException`	Unchecked	Invalid downcast	Check with `instanceof` first
`IllegalArgumentException`	Unchecked	Invalid argument (e.g. `Thread.sleep(-100)`)	Validate early; throw it yourself for bad args
`IllegalStateException`	Unchecked	Wrong object state (using a closed `Scanner`, `Iterator.remove()` twice)	Check the state machine
`UnsupportedOperationException`	Unchecked	Modifying `List.of(...)` or an `Arrays.asList` size	Copy into an `ArrayList`
`ConcurrentModificationException`	Unchecked	Modifying a collection while iterating over it	`Iterator.remove()`, `removeIf`, concurrent collections
`NoSuchElementException`	Unchecked	`iterator.next()` with none left, `Optional.get()` on empty	`hasNext()`, `orElse`/`orElseThrow`
`InputMismatchException`	Unchecked	`Scanner.nextInt()` reads "abc"	`hasNextInt()` first
`ArrayStoreException`	Unchecked	Wrong type stored in a covariant array	Use generics/collections
`NegativeArraySizeException`	Unchecked	`new int[-5]`	Validate the size
`FileNotFoundException`	Checked	Wrong path, or no permission	Check `Files.exists`; use absolute/config paths
`IOException`	Checked	Any I/O failure	try-with-resources, retry, report
`SQLException`	Checked	DB errors: bad SQL, constraints, connection	Wrap and translate; check the SQLState
`ClassNotFoundException`	Checked	`Class.forName("x")` can't find the class	Fix the classpath or dependency
`InterruptedException`	Checked	Thread interrupted during `sleep`/`wait`/`join`	Restore the flag: `Thread.currentThread().interrupt()`
`ParseException`	Checked	`SimpleDateFormat.parse` with bad input	Prefer `java.time` (`DateTimeParseException`, unchecked)
`CloneNotSupportedException`	Checked	`clone()` on a non-`Cloneable` class	Implement `Cloneable` or use a copy constructor

Helpful NullPointerException messages (Java 14+)

```
Customer c = new Customer();
c.getAddress().getCity().length();
// Java 8:   java.lang.NullPointerException
// Java 14+: Cannot invoke "String.length()" because the return value of
//           "Address.getCity()" is null
```

```
this.name = Objects.requireNonNull(name, "name must not be null");   // fail fast, clear message
String city = Optional.ofNullable(c.getAddress()).map(Address::getCity).orElse("Unknown");
```

9.15 Important Errors

Error	Cause	What to do
`StackOverflowError`	Infinite or very deep recursion (each call adds a stack frame)	Fix the base case, convert to iteration, or increase `-Xss`
`OutOfMemoryError: Java heap space`	Too many live objects, or a memory leak (static collections, caches, unclosed resources)	Heap dump (`-XX:+HeapDumpOnOutOfMemoryError`), analyse with Eclipse MAT, fix the leak, tune `-Xmx`
`OutOfMemoryError: Metaspace`	Too many classes loaded (class-loader leaks, heavy proxies)	Find the leak; tune `-XX:MaxMetaspaceSize`
`OutOfMemoryError: unable to create native thread`	Too many threads	Use thread pools or virtual threads
`NoClassDefFoundError`	The class was present at **compile** time but is missing or failed to initialise at **runtime**	Fix the runtime classpath, JAR conflicts, or the static-init failure
`ExceptionInInitializerError`	An exception was thrown in a static initializer or static field initialisation	Fix the static block (check its cause)
`AssertionError`	A failed `assert` (enabled with `-ea`), or thrown explicitly	Fix the violated invariant

INTERVIEW

`ClassNotFoundException` vs `NoClassDefFoundError`: the former is a **checked exception** thrown when code **dynamically** loads a class by name (`Class.forName`, `loadClass`) and it's not found. The latter is an **Error** thrown when a class that was available at **compile time** can't be found or initialised when the JVM needs it at **runtime**.

```
void m1() { m1(); }        // infinite recursion
try {
    m1();
} catch (StackOverflowError e) {
    System.out.println("StackOverflowError Handled");   // possible, but fix the recursion
}
```

9.16 Best Practices

1. **Catch specific exceptions**, not `Exception`/`Throwable`, except at a top-level boundary (controller advice, the main loop, a job runner).
2. **Never swallow exceptions**: an empty `catch {}` hides failures. At minimum, log with the stack trace.
3. **Throw early, catch late**: validate inputs at method entry; handle where you can actually do something meaningful.
4. Use **try-with-resources** for every closable resource.
5. **Preserve the cause** when wrapping (`new X(msg, e)`).
6. Use **unchecked exceptions for programming errors and business-rule violations**; checked exceptions for recoverable external conditions the caller must address.
7. Don't use exceptions for **normal control flow** (such as ending a loop), because they are slow and hard to read.
8. Write **meaningful messages** with context (IDs, values), but **never include secrets** (passwords, card numbers, tokens).
9. Don't `return` or `throw` from `finally`.
10. On `InterruptedException`, either propagate it or restore the interrupt flag with `Thread.currentThread().interrupt()`.
11. Prefer standard exceptions where they fit: `IllegalArgumentException`, `IllegalStateException`, `UnsupportedOperationException`, `NoSuchElementException`.
12. In tests, assert exceptions precisely: JUnit 5 `assertThrows(InsufficientFundsException.class, () -> acc.withdraw(big))`, plus checks on the message or fields.

9.17 Predict the Output

```
static int f() {
    try { throw new RuntimeException(); }
    catch (RuntimeException e) { return 1; }
    finally { System.out.print("F "); }
}
System.out.println(f());                          // F 1

try {
    System.out.print("A ");
    int x = 5 / 0;
    System.out.print("B ");
} catch (ArithmeticException e) {
    System.out.print("C ");
} finally {
    System.out.print("D ");
}
System.out.println("E");                          // A C D E

try {
    throw new IllegalArgumentException("bad");
} catch (RuntimeException e) {
    System.out.println(e);                        // java.lang.IllegalArgumentException: bad
}

static int g() { int x = 1; try { return x; } finally { x = 99; } }
System.out.println(g());                          // 1

try { Object o = "s"; Integer i = (Integer) o; }
catch (ClassCastException e) { System.out.println("CCE"); }      // CCE

String s = null;
try { s.length(); } catch (Exception e) { System.out.println(e.getClass().getSimpleName()); }
                                                  // NullPointerException
```

9.18 Interview Questions

Q. What is the difference between checked and unchecked exceptions?

Checked exceptions (subclasses of `Exception` excluding `RuntimeException`) are verified by the compiler and must be caught or declared. Unchecked exceptions (`RuntimeException` and `Error` subclasses) aren't enforced by the compiler and usually indicate programming bugs.

Q. What is the difference between an Exception and an Error?

Exceptions represent application-level conditions that can often be handled. Errors represent serious JVM or system problems (`OutOfMemoryError`, `StackOverflowError`) that applications generally shouldn't try to handle.

Q. Can we have try without catch?

Yes, `try-finally`, or try-with-resources, which needs neither catch nor finally. A bare `try { }` alone isn't allowed.

Q. Does finally always execute?

Almost always: after normal completion, after a caught or uncaught exception, and even after `return`. It doesn't run if `System.exit()` is called, the JVM crashes or is killed, or the try never completes (infinite loop or deadlock).

Q. What happens if both try and finally have return statements?

The `finally` return wins. It overrides the try's value and also discards any pending exception, which is why it's a bad practice.

Q. Why must catch blocks go from specific to general?

Catch blocks are matched top-down. A general type placed first would catch everything, so later specific blocks become unreachable, which is a compile error.

Q. What is try-with-resources? What are suppressed exceptions?

A Java 7 construct that automatically closes `AutoCloseable` resources in reverse order. If the body and `close()` both throw, the body's exception propagates and the close exceptions are attached to it via `getSuppressed()`.

Q. What is the difference between `throw` and `throws`?

`throw` raises a single exception object inside a method. `throws` declares in the signature which exceptions a method may pass to its caller.

Q. What is exception propagation?

When a method doesn't handle an exception, it moves up the call stack to the caller, and so on, until a matching catch is found or the thread terminates with a stack trace.

Q. How do you create a custom exception? Checked or unchecked?

Extend `Exception` for a checked exception or `RuntimeException` for an unchecked one. Provide constructors with a message and a cause, and add domain fields if useful. Business-rule exceptions in modern apps are usually unchecked.

Q. What is exception chaining?

Wrapping a lower-level exception as the cause of a higher-level one (`new ServiceException(msg, e)`), so that the full root cause and stack trace are preserved ("Caused by: ...").

Q. What is the difference between ClassNotFoundException and NoClassDefFoundError?

The first is a checked exception from dynamic loading by name when the class isn't found. The second is an Error raised when a class that existed at compile time is missing or failed to initialise at runtime.

Q. What are the exception rules for overriding methods?

An overriding method may throw the same, narrower or no checked exceptions, never broader or new ones. Unchecked exceptions are unrestricted.

9.19 Quick Revision

- `Throwable` → `Error` (don't catch) and `Exception` → checked types plus `RuntimeException` (unchecked).
- **Checked** = compiler-enforced: *handle or declare*. **Unchecked** = `RuntimeException` and `Error`, usually bugs.
- Lines after the exception point inside `try` are skipped; only the first matching catch runs.
- Order catch blocks child → parent; multi-catch `A | B` (unrelated types, `e` final).
- `finally` runs except on `System.exit`, a crash or a hang. Never `return` from `finally`.
- try-with-resources: `AutoCloseable`, closed in reverse order, suppressed exceptions kept. Java 9 allows existing effectively-final variables.
- `throw` = raise one object; `throws` = declare and delegate. Neither ignores anything.
- Propagation unwinds the stack; read stack traces top-down and look for "Caused by".
- Custom exceptions: extend `RuntimeException` (business rules) or `Exception` (recoverable); keep the cause.
- Overriding: no new or broader checked exceptions.
- `ClassNotFoundException` (checked, dynamic loading) vs `NoClassDefFoundError` (Error, missing at runtime).
- Best practice: specific catches, no empty catches, log with context (no secrets), restore the interrupt flag.

PHASE 10 OF 18

Nested Classes, Enums & Generics

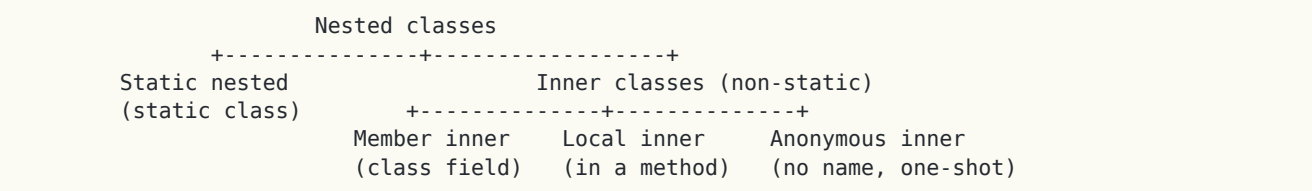
Static nested, member inner, local and anonymous classes, anonymous classes vs lambdas, enums with fields and behaviour, EnumMap/EnumSet, generic classes and methods, bounds, wildcards and PECS, and type erasure

IN THIS PHASE

10.1 Nested Classes: The Four Kinds
10.2 Member Inner Class
10.3 Static Nested Class
10.4 Local Inner Class
10.5 Anonymous Inner Class
10.6 Enums
10.7 Generics
10.8 Type Erasure
10.9 Predict the Output
10.10 Interview Questions
10.11 Quick Revision

10.1 Nested Classes: The Four Kinds

A class declared inside another class is a **nested class**. Nested classes **group** helper types with the only class that uses them, **increase encapsulation** (they can be private and can access the outer class's private members), and keep code **readable**.



Kind	Declared	Needs an outer object?	Can access outer instance members?	Created as	Compiled to
Static nested	`static class` inside a class	No	No (only static ones, or via a reference)	`new Outer.Nested()`	`Outer$Nested.class`
Member inner	Non-static class inside a class	**Yes**	**Yes**, including private	`outer.new Inner()`	`Outer$Inner.class`
Local inner	Inside a method or block	Yes, if in an instance method	Yes, plus effectively-final locals	`new Local()` within that block	`Outer$1Local.class`
Anonymous inner	`new Type() { ... }` expression	Yes, if in an instance context	Yes, plus effectively-final locals	At declaration only	`Outer$1.class`

10.2 Member Inner Class

```
class Outer {
    private int x = 10;

    class Inner {                                   // tied to an Outer instance
        private int x = 20;
        void display() {
            int x = 30;
            System.out.println(x);                  // 30 (local)
            System.out.println(this.x);             // 20 (Inner's field)
            System.out.println(Outer.this.x);       // 10 (enclosing instance's field)
        }
    }

    void useInner() {
        new Inner().display();                      // inside Outer: no prefix needed
    }
}

Outer outer = new Outer();
Outer.Inner inner = outer.new Inner();              // outside: outer.new Inner()
inner.display();
```

- An inner class can access **all** members of the outer class, including private ones (and vice versa).
- Every inner object holds a hidden reference to its **enclosing instance** (`Outer.this`).
- Since Java 16, inner classes may also declare `static` members.

> **PITFALL**
> Because every inner-class object keeps its outer object alive, a long-lived inner object (stored in a cache, or passed to another thread) can cause **memory leaks**. If the nested class doesn't need the outer instance, make it **static**.

Real-world use: `ArrayList`'s iterator (`ArrayList.Itr`) is a private inner class that reads the list's internal array directly.

10.3 Static Nested Class

A static nested class behaves like a top-level class that is **namespaced** inside another class. It has no link to an outer instance.

```
class Outer {
    static int count = 10;
    int y = 20;

    static class Nested {
        void display() {
            System.out.println(count);     // OK: static member of Outer
            // System.out.println(y);      // COMPILE ERROR: needs an Outer instance
        }
    }
}
Outer.Nested n = new Outer.Nested();       // no Outer object needed
```

Classic use: the Builder pattern

```
public final class Customer {
    private final String name;          // required
    private final String email;         // optional
    private final String city;          // optional

    private Customer(Builder b) {
        this.name = b.name; this.email = b.email; this.city = b.city;
    }

    public static class Builder {                        // static nested
        private final String name;
        private String email, city;
        public Builder(String name) { this.name = name; }
        public Builder email(String e) { this.email = e; return this; }
        public Builder city(String c)  { this.city = c;  return this; }
        public Customer build()        { return new Customer(this); }
    }
}

Customer c = new Customer.Builder("Rohit").email("r@x.com").city("Pune").build();
```

Other JDK examples: `Map.Entry`, `AbstractMap.SimpleEntry`, `Character.UnicodeBlock`, and the nodes of `HashMap` (`HashMap.Node`).

10.4 Local Inner Class

A local class is declared **inside a method or block** and is visible only there. It can use the method's **effectively final** local variables.

```
class Validator {
    private final int minLength = 8;

    boolean validate(String password) {
        String special = "@#$%";                         // effectively final

        class Rule {                                     // local class
            boolean check() {
                return password.length() >= minLength                    // outer field
                    && password.chars().anyMatch(ch -> special.indexOf(ch) >= 0);
            }
        }
        // special = "!";   // would make it NOT effectively final -> compile error above
        return new Rule().check();
    }
}
```

MODERN JAVA

Java 16+ also allows **local records, enums and interfaces** inside methods. They are handy for short-lived data shapes in a single method.

10.5 Anonymous Inner Class

An anonymous class is a **nameless** class declared and instantiated in one expression. It either **implements an interface** or **extends a class** (concrete or abstract). It is used once, on the spot.

```
interface Greeting { void greet(); }

Greeting g = new Greeting() {                       // implements an interface
    @Override public void greet() { System.out.println("Hello, Anonymous!"); }
};
g.greet();

Thread t = new Thread() {                           // extends a class
    @Override public void run() { System.out.println("worker running"); }
};
t.start();

List<String> names = new ArrayList<>(List.of("Ravi", "Amit", "Zoya"));
names.sort(new Comparator<String>() {               // classic pre-Java 8 Comparator
    @Override public int compare(String a, String b) { return b.compareTo(a); }
});
System.out.println(names);                          // [Zoya, Ravi, Amit]
```

Anonymous class vs lambda

```
Runnable r1 = new Runnable() {                      // anonymous class
    @Override public void run() { System.out.println("Task"); }
};
Runnable r2 = () -> System.out.println("Task");     // lambda (Java 8): same behaviour
names.sort((a, b) -> b.compareTo(a));               // lambda comparator
names.sort(Comparator.reverseOrder());              // or a ready-made comparator
```

	Anonymous class	Lambda
Works with	Interfaces with **any** number of methods, abstract or concrete classes	**Functional interfaces only** (one abstract method)
Meaning of `this`	The anonymous object itself	The **enclosing** instance
Can have fields and state	Yes	No
Generated `.class` file	Yes (`Outer$1.class`)	No (`invokedynamic` at runtime)
Verbosity	High	Minimal
Captured locals	Must be effectively final	Must be effectively final

> **TIP**
> Use a lambda (or method reference) whenever the target is a functional interface. Use an anonymous class when you need state, several methods, or to extend a class.

Which nested type should I use?

Need	Use
A helper that doesn't need the outer object (Builder, Node, Entry, DTO)	**Static nested** (the default choice)
A helper that must read or write the outer object's state (an iterator over its data)	Member inner
A one-method-scoped helper with a name	Local class (or local record)
A one-off implementation with state or several methods	Anonymous class
A one-off implementation of a functional interface	**Lambda**

10.6 Enums

An **enum** is a special type representing a **fixed set of named constants**: days of the week, account types, transaction statuses, card networks.

```
enum TxnStatus { PENDING, APPROVED, REJECTED }

TxnStatus s = TxnStatus.APPROVED;
System.out.println(s);                              // APPROVED

switch (s) {                                        // works in switch (no prefix in labels)
    case PENDING  -> System.out.println("Request pending");
    case APPROVED -> System.out.println("Request approved");
    case REJECTED -> System.out.println("Request rejected");
}
```

Why enums instead of int or String constants?

```
// Old style: no type safety
static final int PENDING = 0, APPROVED = 1, REJECTED = 2;
void update(int status) { }          // update(42) compiles: invalid value accepted!

// Enum: the compiler only accepts real constants
void update(TxnStatus status) { }    // update(42) and update("APPROVED") won't compile
```

Benefits: type safety, a closed set of values, readable code, compatibility with `switch` (including exhaustive switch expressions), and the ability to carry **fields and behaviour**.

Facts about enums

1. Every enum implicitly **extends** `java.lang.Enum`, so it **cannot extend another class**, but it **can implement interfaces**.
2. Each constant is implicitly `public static final`, and exactly **one instance** exists per constant.
3. Constructors are implicitly **private**: you can't call `new TxnStatus()`.
4. Enums are implicitly **final**, unless a constant has a body.
5. You can compare constants safely with `==` (singletons), and `==` is null-safe.
6. Enums are `Comparable` (by declaration order) and `Serializable`.

Built-in methods

Method	Example	Result
`values()` (static)	`TxnStatus.values()`	`[PENDING, APPROVED, REJECTED]` (a new array each call)
`valueOf(String)` (static)	`TxnStatus.valueOf("APPROVED")`	`APPROVED`. Case-sensitive; unknown names throw `IllegalArgumentException`.
`name()`	`APPROVED.name()`	`"APPROVED"` (exact declared name; final)
`toString()`	`APPROVED.toString()`	`"APPROVED"` by default (overridable)
`ordinal()`	`APPROVED.ordinal()`	`1` (zero-based position)
`compareTo(e)`	`PENDING.compareTo(REJECTED)`	negative (declaration order)
`getDeclaringClass()`	`APPROVED.getDeclaringClass()`	`TxnStatus.class`

Enums with fields, constructors and methods

```
public enum TxnType {
    UPI("U01", "Unified Payments Interface", 0.00),
    NEFT("N01", "National Electronic Funds Transfer", 2.50),
    RTGS("R01", "Real Time Gross Settlement", 25.00),
    IMPS("I01", "Immediate Payment Service", 5.00);    // semicolon before members

    private final String code;
    private final String description;
    private final double fee;

    TxnType(String code, String description, double fee) {   // implicitly private
        this.code = code; this.description = description; this.fee = fee;
    }

    public String getCode()        { return code; }
    public String getDescription() { return description; }
    public double getFee()         { return fee; }

    private static final Map<String, TxnType> BY_CODE = new HashMap<>();
    static {
        for (TxnType t : values()) BY_CODE.put(t.code, t);   // fast reverse lookup
    }
    public static TxnType fromCode(String code) {
        TxnType t = BY_CODE.get(code);
        if (t == null) throw new IllegalArgumentException("Unknown code: " + code);
        return t;
    }
}

System.out.println(TxnType.NEFT.getFee());           // 2.5
System.out.println(TxnType.fromCode("R01"));         // RTGS
```

Constant-specific behaviour

```
enum Operation {
    PLUS("+")   { int apply(int a, int b) { return a + b; } },
    MINUS("-")  { int apply(int a, int b) { return a - b; } },
    TIMES("*")  { int apply(int a, int b) { return a * b; } };

    private final String symbol;
    Operation(String symbol) { this.symbol = symbol; }
    abstract int apply(int a, int b);         // each constant must implement it
    @Override public String toString() { return symbol; }
}

for (Operation op : Operation.values())
    System.out.println("6 " + op + " 3 = " + op.apply(6, 3));
// 6 + 3 = 9    6 - 3 = 3    6 * 3 = 18
```

Enum implementing an interface, and the enum singleton

```
interface FeePolicy { double feeFor(double amount); }

enum CardType implements FeePolicy {
    DEBIT  { public double feeFor(double amt) { return 0; } },
    CREDIT { public double feeFor(double amt) { return amt * 0.018; } }
}

public enum AppConfig {          // simplest thread-safe, serialization-safe singleton
    INSTANCE;
    private final Properties props = new Properties();
    public String get(String key) { return props.getProperty(key); }
}
AppConfig.INSTANCE.get("db.url");
```

EnumSet and EnumMap

Specialised, very fast collections (backed by bit vectors and arrays) for enum keys:

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }

EnumSet<Day> weekend  = EnumSet.of(Day.SAT, Day.SUN);
EnumSet<Day> weekdays = EnumSet.complementOf(weekend);       // MON..FRI
EnumSet<Day> midweek  = EnumSet.range(Day.TUE, Day.THU);

EnumMap<TxnStatus, Long> counts = new EnumMap<>(TxnStatus.class);
counts.merge(TxnStatus.APPROVED, 1L, Long::sum);
System.out.println(counts);                                  // {APPROVED=1}
```

PITFALL

Never **persist** `ordinal()` values (in a database, a file or an API). Inserting or reordering constants silently changes their meaning. Store `name()` or an explicit code field. In JPA, use `@Enumerated(EnumType.STRING)` instead of the default `ORDINAL`.

10.7 Generics

Generics (Java 5) let classes, interfaces and methods be **parameterised by type**. You get **compile-time type safety**, **no casts**, and **reusable** code.

```
// Without generics (raw types): anything goes, errors appear at RUNTIME
List list = new ArrayList();
list.add(10);
list.add("Hello");
String v = (String) list.get(0);      // ClassCastException at runtime

// With generics: errors caught at COMPILE time, no casts
List<String> names = new ArrayList<>();   // <> = diamond operator (Java 7), type inferred
names.add("Java");
// names.add(100);                    // COMPILE ERROR
String first = names.get(0);          // no cast needed
```

Term	Meaning	Example
Type parameter	Placeholder declared on a class or method	`T` in `class Box<T>`
Type argument	The actual type supplied	`String` in `Box<String>`
Parameterized type	Generic type with arguments	`List<String>`
Raw type	Generic type used without arguments (legacy; avoid)	`List`
Diamond	Let the compiler infer the arguments	`new HashMap<>()`

Naming conventions: `T` (type), `E` (element), `K`/`V` (key/value), `N` (number), `R` (result), `S`/`U` (second and third types).

Generic classes and interfaces

```
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

Box<Integer> intBox = new Box<>();
intBox.set(10);
int n = intBox.get();                // no cast (auto-unboxing)

Box<String> strBox = new Box<>();
strBox.set("Java");

public class Pair<K, V> {            // several type parameters
    private final K key; private final V value;
    public Pair(K key, V value) { this.key = key; this.value = value; }
    public K getKey() { return key; }
    public V getValue() { return value; }
}
Pair<String, Integer> p = new Pair<>("age", 35);

public interface Repository<T, ID> {  // generic interface (like Spring Data's CrudRepository)
    T findById(ID id);
    List<T> findAll();
    T save(T entity);
}
class AccountRepo implements Repository<Account, Long> { /* T=Account, ID=Long */ }
```

> **NOTE**
> Type arguments must be **reference types**. `List<int>` is illegal; use `List<Integer>` (with autoboxing).

Generic methods

A generic method declares **its own** type parameter before the return type. It can live in any class, generic or not, and is often `static`.

```
public class Utils {
    public static <T> void printAll(T[] items) {                // <T> before return type
        for (T item : items) System.out.print(item + " ");
        System.out.println();
    }

    public static <T> void swap(T[] arr, int i, int j) {
        T tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
    }

    public static <K, V> Map<V, K> invert(Map<K, V> map) {
        Map<V, K> out = new HashMap<>();
        map.forEach((k, v) -> out.put(v, k));
        return out;
    }
}

Utils.printAll(new Integer[]{1, 2, 3});       // T inferred as Integer
Utils.printAll(new String[]{"a", "b"});       // T inferred as String
Utils.<String>printAll(new String[]{"x"});    // explicit type argument (rarely needed)
```

Generic class	Generic method
Type parameter declared on the class: `class Box<T>`	Declared on the method: `<T> void m(T t)`
Fixed when the object is created: `new Box<String>()`	Inferred for **each call** from the arguments
Shared by all instance members	Scoped to that one method; can be `static`

Bounded type parameters

`<T extends X>` restricts `T` to `X` or its subtypes, so you can call `X`'s methods on `T`. (`extends` is used for both classes and interfaces here.)

```
class Stats<T extends Number> {                        // only numeric types
    private final List<T> values = new ArrayList<>();
    void add(T v) { values.add(v); }
    double sum() {
        double s = 0;
        for (T v : values) s += v.doubleValue();       // Number's method is available
        return s;
    }
}
Stats<Integer> si = new Stats<>();                     // OK
// Stats<String> ss = new Stats<>();                   // COMPILE ERROR: String isn't a Number

static <T extends Comparable<T>> T max(List<T> list) {    // bounded generic method
    T best = list.get(0);
    for (T t : list) if (t.compareTo(best) > 0) best = t;
    return best;
}
max(List.of(3, 9, 4));            // 9
max(List.of("pear", "apple"));    // pear

class Ledger<T extends Number & Comparable<T>> { }  // class bound first, then interfaces
```

Rules for multiple bounds: at most **one class**, which must come **first**, followed by any number of interfaces, separated by `&`.

Generics are invariant

```
List<Integer> ints = new ArrayList<>();
// List<Number> nums = ints;          // COMPILE ERROR even though Integer IS-A Number
// Why: otherwise nums.add(3.14) would put a Double into a List<Integer>.

Object[] objs = new String[2];        // arrays ARE covariant (compiles)...
objs[0] = 42;                         // ...but throws ArrayStoreException at runtime
```

`List<Integer>` is **not** a subtype of `List<Number>`. To write methods that accept "a list of some kind of Number", use **wildcards**.

Wildcards: ?, ? extends, ? super

Wildcard	Accepts	Can read as	Can add
`List<?>` (unbounded)	A List of anything	`Object`	Nothing (except `null`)
`List<? extends Number>` (upper bound)	`List<Number>`, `List<Integer>`, `List<Double>`...	`Number`	Nothing (except `null`)
`List<? super Integer>` (lower bound)	`List<Integer>`, `List<Number>`, `List<Object>`	Only `Object`	`Integer` (and its subtypes)

```
static void printAll(List<?> list) {                 // read-only, any type
    for (Object o : list) System.out.print(o + " ");
}

static double total(List<? extends Number> nums) {   // PRODUCER: we read Numbers out
    double sum = 0;
    for (Number n : nums) sum += n.doubleValue();
    // nums.add(1);                                  // COMPILE ERROR: exact type unknown
    return sum;
}
total(List.of(1, 2, 3));          // List<Integer> OK
total(List.of(1.5, 2.5));         // List<Double>  OK

static void fillIds(List<? super Integer> sink) {    // CONSUMER: we put Integers in
    for (int i = 1; i <= 3; i++) sink.add(i);
    Object o = sink.get(0);                          // reading gives only Object
}
fillIds(new ArrayList<Integer>());
fillIds(new ArrayList<Number>());
fillIds(new ArrayList<Object>());
```

PECS: Producer Extends, Consumer Super

> **REMEMBER**
>
> **PECS**:
>
> - If a parameter **produces** values you **read** → `? extends T`.
> - If a parameter **consumes** values you **write** → `? super T`.
> - If it does both → use the exact type `T`, with no wildcard.

```
// Real JDK signature that follows PECS:
public static <T> void copy(List<? super T> dest, List<? extends T> src)

List<Number> dest = new ArrayList<>(List.of(0, 0, 0));
List<Integer> src = List.of(7, 8, 9);
Collections.copy(dest, src);             // dest consumes, src produces
```

10.8 Type Erasure

Generics exist **only at compile time**. The compiler checks the types, inserts casts, and then **erases** type parameters: `T` becomes `Object` (or its first bound, such as `Number`), and `List<String>` becomes plain `List` in the bytecode. This kept Java 5 generics backward-compatible with old code.

```
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println(a.getClass() == b.getClass());   // true: both are just ArrayList at runtime
```

Consequences of erasure (things you can't do)

Not allowed	Why	Workaround
`new T()`	The type is unknown at runtime	Pass a `Supplier<T>` or a `Class<T>`
`new T[10]` / `new List<String>[10]`	Arrays need the runtime element type	`(T[]) new Object[n]` with care, or use a `List<T>`
`obj instanceof List<String>`	No runtime type argument	`instanceof List<?>`
`static T field;`	The static is shared by all parameterisations	Make it non-static, or a generic method
`List<int>`	Type arguments must be objects	`List<Integer>`, or primitive streams/arrays
Overloads `m(List<String>)` and `m(List<Integer>)`	Both erase to `m(List)`: name clash	Use different method names
`catch (T e)` / generic exception classes	Catch needs the runtime type	Not possible

```
static <T> T create(Supplier<T> factory) { return factory.get(); }
StringBuilder sb = create(StringBuilder::new);            // workaround for new T()

static <T> T[] newArray(Class<T> type, int n) {
    @SuppressWarnings("unchecked")
    T[] arr = (T[]) java.lang.reflect.Array.newInstance(type, n);
    return arr;
}
```

PITFALL

Raw types (`List list = new ArrayList();`) switch off generic checks and produce "unchecked" warnings. Mixing raw and generic code lets wrong types slip in until a `ClassCastException` appears far away (*heap pollution*). Don't use raw types in new code.

10.9 Predict the Output

```
class Outer { int v = 1; class Inner { int v = 2; int get() { return Outer.this.v + v; } } }
System.out.println(new Outer().new Inner().get());          // 3

enum Size { S, M, L }
System.out.println(Size.M.ordinal() + " " + Size.valueOf("L").compareTo(Size.S));   // 1 2
System.out.println(Size.values().length);                   // 3
try { Size.valueOf("XL"); } catch (IllegalArgumentException e) { System.out.println("IAE"); }

List<String> s = new ArrayList<>(); List<Integer> i = new ArrayList<>();
System.out.println(s.getClass() == i.getClass());           // true (erasure)

Runnable r = new Runnable() {
    public void run() { System.out.println(getClass().getSimpleName().isEmpty()); }
};
r.run();                                    // true (anonymous class: empty simple name)

List<? extends Number> nums = List.of(1, 2.5);
Number first = nums.get(0);
System.out.println(first);                                  // 1
```

10.10 Interview Questions

Q. What are the types of nested classes in Java?

Static nested classes, and three kinds of inner (non-static) classes: member, local and anonymous. Inner classes hold a reference to an enclosing instance; static nested classes don't.

Q. What is the difference between a static nested class and an inner class?

A static nested class doesn't need an outer instance and can only access the outer class's static members directly. A member inner class is tied to an outer instance (`outer.new Inner()`) and can access all its members via `Outer.this`.

Q. Why must local variables used in anonymous classes or lambdas be effectively final?

The inner code captures a **copy** of the value. Allowing later changes would make the copy and the original diverge, so Java requires the variable never to be reassigned.

Q. How is an anonymous class different from a lambda?

Lambdas only implement functional interfaces, don't create a separate class file, have no state of their own, and `this` refers to the enclosing instance. Anonymous classes can implement any interface or extend any class, hold fields, and define several methods; their `this` is the anonymous object.

Q. Can an enum extend a class or implement an interface? Can it have a constructor?

It can't extend a class (it already extends `java.lang.Enum`), but it can implement interfaces. It can have fields, methods and constructors, which are implicitly private and invoked only for its constants.

Q. What is the difference between `name()`, `toString()` and `ordinal()`?

`name()` returns the declared identifier (final). `toString()` returns the same by default but can be overridden for display. `ordinal()` returns the zero-based declaration position, which shouldn't be persisted.

Q. Why is an enum the best way to implement a singleton?

The JVM guarantees one instance per constant. It is thread-safe at class initialisation, protected against reflection attacks, and handles serialization correctly without extra code.

Q. What are generics and why use them?

Parameterised types that provide compile-time type checking, remove casts, and let one class or method work with many types safely.

Q. What is type erasure?

The compiler removes generic type information after checking it, replacing type parameters with their bounds (or `Object`) and inserting casts. At runtime, `List<String>` and `List<Integer>` are both just `List`.

Q. Why is `List<Integer>` not a subtype of `List<Number>`?

Generics are invariant. If it were a subtype, you could add a `Double` through the `List<Number>` reference into a list that must only contain Integers.

Q. Explain `? extends T` vs `? super T` (PECS).

`? extends T` accepts T or its subtypes; you can safely read `T`s but not add. `? super T` accepts T or its supertypes; you can add `T`s but only read `Object`s. Producer → extends, consumer → super.

Q. Why can't we create `new T()` or `new T[]`?

The actual type of `T` is erased and unknown at runtime. Pass a `Supplier<T>` or a `Class<T>` (reflection) instead.

10.11 Quick Revision

- Nested classes: **static nested** (no outer instance, `new Outer.Nested()`), **member inner** (`outer.new Inner()`, `Outer.this`), **local** (inside a method), **anonymous** (`new X() { }`).
- Inner classes hold a reference to the outer object, so prefer static nested when possible (e.g. the Builder pattern).
- Captured locals must be **effectively final** (for local and anonymous classes and lambdas).
- Lambda for functional interfaces; anonymous class when you need state, several methods, or to extend a class.
- Enum: fixed constants, extends `Enum`, can implement interfaces, private constructors, one instance per constant, compare with `==`.
- `values()`, `valueOf()` (case-sensitive; IAE if unknown), `name()`, `ordinal()` (never persist it).
- Enums can carry fields, constructors, methods and constant-specific bodies. `EnumSet`/`EnumMap` are fast enum collections. An enum is the best singleton.
- Generics: compile-time safety, no casts. `<>` diamond; reference types only; avoid raw types.
- Generic method: `<T>` before the return type, inferred per call.
- Bounds: `<T extends Number & Comparable<T>>` (one class first, then interfaces).
- Invariance: `List<Integer>` is not a `List<Number>`. Wildcards: `?`, `? extends` (read), `? super` (write). **PECS**.
- Erasure: no `new T()`, no generic arrays, no `instanceof List<String>`, no primitive type arguments, overload clashes.

PHASE 11 OF 18

Multithreading & Concurrency

Processes vs threads, creating threads, lifecycle, sleep/join/interrupt, daemon threads, race conditions, synchronized, volatile, atomics, wait/notify, deadlock, locks, executors, Callable/Future, CompletableFuture, concurrent utilities and virtual threads

IN THIS PHASE

11.1 Multitasking, Processes and Threads
11.2 Creating Threads
11.3 Thread Lifecycle
11.4 Important Thread Methods
11.5 Race Conditions: Why Synchronization Is Needed
11.6 synchronized
11.7 volatile and Visibility
11.8 Atomic Variables
11.9 Inter-Thread Communication: wait(), notify(), notifyAll()
11.10 Deadlock, Livelock and Starvation
11.11 Explicit Locks: ReentrantLock and ReadWriteLock
11.12 The Executor Framework and Thread Pools
11.13 CompletableFuture (Java 8): Asynchronous Pipelines
11.14 Concurrent Collections and Synchronizers
11.15 Virtual Threads (Java 21)
11.16 Best Practices
11.17 Predict the Output
11.18 Interview Questions
11.19 Quick Revision

11.1 Multitasking, Processes and Threads

- **Single tasking:** one task at a time; the CPU idles while waiting for I/O.
- **Multitasking:** several tasks make progress at once. There are two kinds: **process-based** (separate programs, such as a browser and an IDE) and **thread-based** (several threads inside one program).
- **Multithreading:** one Java process runs several **threads** (independent paths of execution) at the same time. For example, a banking server handles thousands of customer requests concurrently, and a UI stays responsive while a report generates in the background.

	Process	Thread
What	An independent running program	A unit of execution **inside** a process
Memory	Own address space	**Shares** the process heap; has its **own stack** and PC register
Creation / context switch	Heavy, slow	Light, fast
Communication	Inter-process communication (sockets, files, pipes)	Shared objects (needs synchronization!)
Failure impact	Isolated	An unhandled error can affect the whole process

NOTE

Concurrency means dealing with many tasks at once (interleaving, even on one core). **Parallelism** means literally executing at the same instant on multiple cores. Java threads give you concurrency, and parallelism when cores are available.

The main thread

Every Java program starts with one thread called **main**, created by the JVM to run `main()`.

```
Thread t = Thread.currentThread();
System.out.println(t.getName());       // main
System.out.println(t.getPriority());   // 5 (NORM_PRIORITY)
System.out.println(t.isDaemon());      // false
System.out.println(t);                 // Thread[#1,main,5,main] (Java 21 format)
```

11.2 Creating Threads

1. Extend Thread

```
class Worker extends Thread {
    @Override
    public void run() {                                    // the task
        System.out.println("Running in " + getName());
    }
}
new Worker().start();                                      // start() creates the new thread
```

2. Implement Runnable (preferred)

```
class Task implements Runnable {
    @Override
    public void run() {
        System.out.println("Running in " + Thread.currentThread().getName());
    }
}
Thread t = new Thread(new Task(), "report-worker");      // task + optional name
t.start();

Thread t2 = new Thread(() -> System.out.println("lambda task"));   // Runnable as a lambda
t2.start();
```

Extending `Thread`	**Implementing** `Runnable`
Uses up the single superclass slot	The class can still extend another class
Task and thread are coupled together	**Task separated** from the thread mechanism
Can't hand the task to a thread pool	Works with `ExecutorService` and thread pools
Each object is a thread	One Runnable can be shared by several threads

3. Implement Callable (returns a result, can throw checked exceptions)

```
Callable<Integer> sumTask = () -> {
    int sum = 0;
    for (int i = 1; i <= 100; i++) sum += i;
    return sum;                                            // returns a value
};
try (ExecutorService pool = Executors.newFixedThreadPool(2)) {   // Java 19+: AutoCloseable
    Future<Integer> f = pool.submit(sumTask);
    System.out.println(f.get());                           // 5050 (blocks until done)
}
```

`Runnable`	`Callable<V>`
`void run()`	`V call() throws Exception`
No result	Returns a result through a `Future<V>`
Can't throw checked exceptions	Can throw checked exceptions
`new Thread(runnable)` or `executor.execute/submit`	`executor.submit(callable)` only (a `Thread` can't take a Callable directly)
Java 1.0	Java 5

start() vs run()

```
Thread t = new Thread(() -> System.out.println("in " + Thread.currentThread().getName()));
t.run();      // in main      <- a plain method call: NO new thread
t.start();    // in Thread-0  <- new thread created; it calls run()
// t.start(); // IllegalThreadStateException: a thread can be started only ONCE
```

> **REMEMBER**
> `start()` asks the JVM to create a new thread, which then calls `run()`. Calling `run()` yourself just runs it on the current thread. A terminated thread can never be restarted: create a new `Thread` or submit the task to a pool again.

11.3 Thread Lifecycle

Java's `Thread.State` enum has **six** states. There is no separate RUNNING state; a thread that is running is in `RUNNABLE`.

```
          start()                          run() ends / uncaught exception
  NEW  ------------>  RUNNABLE  ------------------------------------>  TERMINATED
                    (ready or running)
                      |    ^
  waiting for monitor |    |  lock acquired
  (synchronized)      v    |
                    BLOCKED
                      |    ^
  wait(), join(),     v    |  notify()/notifyAll(), joined thread ends, unpark()
  park()            WAITING
                      |    ^
  sleep(ms),          v    |  time elapses / notify / interrupt
  wait(ms), join(ms) TIMED_WAITING
```

State	Meaning	Entered by
`NEW`	Created, not started	`new Thread(...)`
`RUNNABLE`	Running, or ready and waiting for the CPU (the scheduler decides)	`start()`
`BLOCKED`	Waiting to acquire a monitor lock	Entering a `synchronized` block held by another thread
`WAITING`	Waiting indefinitely for another thread	`wait()`, `join()`, `LockSupport.park()`
`TIMED_WAITING`	Waiting with a timeout	`sleep(ms)`, `wait(ms)`, `join(ms)`
`TERMINATED`	Finished	`run()` returns or throws

```
Thread t = new Thread(() -> {
    try { Thread.sleep(200); } catch (InterruptedException e) { }
});
System.out.println(t.getState());   // NEW
t.start();
Thread.sleep(50);
System.out.println(t.getState());   // TIMED_WAITING (sleeping)
t.join();
System.out.println(t.getState());   // TERMINATED
```

11.4 Important Thread Methods

Method	Static?	Purpose
`start()`	No	Begin execution in a new thread
`run()`	No	The task body (don't call it directly)
`Thread.sleep(ms)`	**Yes**	Pause the current thread (**keeps** any locks held); throws `InterruptedException`
`join()` / `join(ms)`	No	Wait for **that** thread to finish
`Thread.yield()`	**Yes**	Hint that the current thread is willing to give up the CPU (rarely useful)
`interrupt()`	No	Set the target thread's interrupt flag; wakes it from sleep/wait/join
`isInterrupted()` / `Thread.interrupted()`	No / **Yes**	Check the flag / check **and clear** the current thread's flag
`Thread.currentThread()`	**Yes**	The thread executing this code
`getName()` / `setName()`	No	Thread name (helpful in logs and thread dumps)
`getPriority()` / `setPriority(1..10)`	No	Scheduler **hint**: MIN 1, NORM 5, MAX 10
`setDaemon(true)` / `isDaemon()`	No	Mark as a background thread (**before** `start()`)
`isAlive()` / `getState()`	No	Started and not finished? / exact state
`threadId()` (Java 19)	No	Unique id (replaces the deprecated `getId()`)

join: wait for other threads

```
Thread t1 = new Thread(() -> System.out.println("load customers"));
Thread t2 = new Thread(() -> System.out.println("load accounts"));
t1.start(); t2.start();
t1.join(); t2.join();                    // main waits for both
System.out.println("generate report");   // always runs last
```

interrupt: cooperative cancellation

Java has no safe way to *kill* a thread (`stop()` is deprecated and removed). You **request** cancellation with `interrupt()`, and the task checks the flag or reacts to `InterruptedException`.

```
Thread worker = new Thread(() -> {
    while (!Thread.currentThread().isInterrupted()) {       // check the flag
        try {
            System.out.println("polling...");
            Thread.sleep(1000);                              // blocking call
        } catch (InterruptedException e) {
            System.out.println("interrupted, cleaning up");
            Thread.currentThread().interrupt();              // RESTORE the flag
        }
    }
});
worker.start();
Thread.sleep(2500);
worker.interrupt();                                          // politely ask it to stop
```

PITFALL

When `sleep`/`wait`/`join` throws `InterruptedException`, the interrupt flag is **cleared**. Never swallow it with an empty catch. Either propagate it or call `Thread.currentThread().interrupt()` so that code further up knows a cancellation was requested.

Priorities and the scheduler

The **thread scheduler** (part of the JVM and the OS) decides which runnable thread gets the CPU and for how long. Thread execution order is **never guaranteed**, and priorities are only **hints** that many operating systems largely ignore. Never rely on priorities for correctness. `setPriority()` with a value outside 1–10 throws `IllegalArgumentException`.

Daemon threads

User (non-daemon) thread	Daemon thread
The JVM **waits** for it to finish before exiting	The JVM **does not wait**: it is abruptly stopped when the last user thread ends
Application work (main, request handlers)	Background services: GC, monitoring, housekeeping
Default for threads created by `main`	Set with `setDaemon(true)` **before** `start()` (otherwise `IllegalThreadStateException`)

PITFALL

Never use daemon threads for work that must complete (saving data, sending a payment confirmation): their `finally` blocks may never run when the JVM exits.

11.5 Race Conditions: Why Synchronization Is Needed

When several threads **read and write shared mutable data** without coordination, the result depends on timing. This is a **race condition**.

```
class Counter {
    int count = 0;
    void increment() { count++; }          // NOT atomic: read -> add 1 -> write
}

Counter c = new Counter();
Runnable job = () -> { for (int i = 0; i < 10_000; i++) c.increment(); };
Thread t1 = new Thread(job), t2 = new Thread(job);
t1.start(); t2.start(); t1.join(); t2.join();
System.out.println(c.count);               // expected 20000, often less (e.g. 13742; varies)
```

NOTE

On a multi-core machine this usually prints less than 20000, and a different number each run. On a single core (or with a lucky schedule) it may print 20000. That is exactly why race conditions are dangerous: code can pass tests on one machine and fail in production. Correctness must come from synchronization, not from observed output.

```
count = 5
Thread A: read 5 ............ add -> 6 ... write 6
Thread B:        read 5 ...... add -> 6 ............ write 6      => one increment LOST
```

The code that touches shared mutable state, here `count++`, is the **critical section**. It must be executed by only one thread at a time (**mutual exclusion**), and each thread must see the others' latest writes (**visibility**).

11.6 synchronized

Every Java object has an **intrinsic lock** (a **monitor**). A thread must **acquire** the lock to enter a `synchronized` method or block; other threads trying to enter wait in the `BLOCKED` state. The lock is **released automatically** on exit, even if an exception is thrown.

```
class Counter {
    private int count = 0;

    public synchronized void increment() { count++; }   // lock = this
    public synchronized int get() { return count; }     // reads need the lock too (visibility)
}
// With synchronized, the two-thread program above always prints 20000.
```

Synchronized method vs block

```
class BankAccount {
    private double balance;
    private final Object lock = new Object();             // private lock object (best practice)

    public void withdraw(String who, double amount) {
        audit(who, amount);                               // non-critical work: no lock needed
        synchronized (lock) {                             // lock ONLY the critical section
            if (balance >= amount) {
                balance -= amount;
                System.out.println(who + " withdrew " + amount);
            } else {
                System.out.println(who + ": insufficient funds");
            }
        }
    }
    public void deposit(double amount) {
        synchronized (lock) { balance += amount; }
    }
    private void audit(String who, double amt) { /* logging, validation */ }
}
```

Synchronized method	Synchronized block
Locks the whole method	Locks only the critical lines
Lock is `this` (or `Class` for static methods)	Any object you choose: `this`, a private lock object, `X.class`
Simpler	More granular, so better concurrency

Instance lock vs class lock

```
class Demo {
    synchronized void a() { }               // lock: the Demo object (this)
    static synchronized void b() { }        // lock: Demo.class (one per class)
    void c() { synchronized (Demo.class) { } }   // same lock as b()
}
```

Can two threads run these at the same time?	Answer
Two synchronized instance methods on the **same** object	**No** (same `this` lock)
Synchronized instance methods on **different** objects	Yes (different locks)
Two `static synchronized` methods of the same class	**No** (same class lock)
A `static synchronized` and an instance `synchronized` method	**Yes** (different locks)
A synchronized and a **non**-synchronized method on the same object	Yes (the non-synchronized method needs no lock)

> **NOTE**
> Intrinsic locks are **reentrant**: a thread holding a lock can enter another `synchronized` method on the same object without deadlocking itself. Constructors can't be `synchronized` (compile error), and synchronization gives **no ordering or fairness** guarantee.

11.7 volatile and Visibility

Each thread may cache variables (in CPU registers or caches), so one thread's write might **never become visible** to another. `volatile` guarantees that reads and writes go to main memory, and establishes a *happens-before* ordering.

```
class Poller implements Runnable {
    private volatile boolean running = true;     // without volatile, the loop may never stop
    public void run() {
        while (running) { /* poll a queue */ }
        System.out.println("stopped");
    }
    public void stop() { running = false; }      // written by another thread
}
```

Guarantee	volatile	synchronized / locks	Atomic*
Visibility	Yes	Yes	Yes
Atomic single read/write (including long/double)	Yes	Yes	Yes
Atomic compound action (count++, check-then-act)	**No**	Yes	Yes (for that variable)
Blocks other threads	No	Yes	No (lock-free, uses CAS)
Typical use	Status flags, safe publication	Multi-step invariants	Counters, sequence numbers

11.8 Atomic Variables

`java.util.concurrent.atomic` provides **lock-free**, thread-safe operations using CPU compare-and-swap (CAS) instructions.

```
AtomicInteger counter = new AtomicInteger();
counter.incrementAndGet();              // ++count, atomically
counter.addAndGet(5);
counter.compareAndSet(6, 100);          // set to 100 only if it is currently 6 (CAS)
counter.updateAndGet(x -> x * 2);

AtomicLong txnSeq = new AtomicLong(1000);
long nextId = txnSeq.getAndIncrement();  // thread-safe ID generator

LongAdder hits = new LongAdder();        // faster than AtomicLong under heavy contention
hits.increment();
long total = hits.sum();
```

11.9 Inter-Thread Communication: wait(), notify(), notifyAll()

These `Object` methods let threads **coordinate**. A thread waits until a condition becomes true; another thread changes the state and notifies it. They must be called while **holding that object's monitor** (inside `synchronized` on the same object), otherwise `IllegalMonitorStateException` is thrown.

```
class OrderQueue {
    private final Queue<String> queue = new LinkedList<>();
    private final int capacity = 5;

    public synchronized void put(String order) throws InterruptedException {
        while (queue.size() == capacity) wait();     // ALWAYS wait in a while loop
        queue.add(order);
        notifyAll();                                 // wake any waiting consumers
    }

    public synchronized String take() throws InterruptedException {
        while (queue.isEmpty()) wait();              // releases the lock while waiting
        String order = queue.poll();
        notifyAll();                                 // wake any waiting producers
        return order;
    }
}
```

> **REMEMBER**
> Use a `while`, not an `if`, around `wait()`. Threads can wake up spuriously, or another thread may consume the item first, so the condition must be re-checked. Prefer `notifyAll()` unless you are sure exactly one waiter can proceed. In modern code, a `BlockingQueue` (covered later in this phase) does all of this for you.

sleep()	wait()
`Thread` class, **static**	`Object` class, instance method
Keeps locks held	**Releases** that object's monitor while waiting
Pauses for a time	Waits for a condition: notify, timeout or interrupt
Callable anywhere	Only inside `synchronized` on the same object
Wakes after the time elapses	Wakes on `notify`/`notifyAll`, timeout, interrupt or spuriously

11.10 Deadlock, Livelock and Starvation

A **deadlock** happens when two or more threads each hold a lock the other needs, so they wait forever.

```
Object accountA = new Object(), accountB = new Object();

Thread t1 = new Thread(() -> {
    synchronized (accountA) {
        sleepQuietly(100);
        synchronized (accountB) { System.out.println("t1 transfer A->B"); }
    }
});
Thread t2 = new Thread(() -> {
    synchronized (accountB) {                    // opposite order -> DEADLOCK
        sleepQuietly(100);
        synchronized (accountA) { System.out.println("t2 transfer B->A"); }
    }
});
t1.start(); t2.start();                          // both hang forever
```

Four conditions must all hold for a deadlock (the Coffman conditions): mutual exclusion, hold-and-wait, no preemption, and circular wait. Break any one of them to prevent it:

1. **Consistent lock ordering:** always lock accounts in a global order, for example by ascending account ID. This is the most common fix.
2. `tryLock` **with a timeout** (`ReentrantLock`): back off and retry instead of waiting forever.
3. Hold locks for as short a time as possible, and never call unknown code while holding a lock.

```
void transfer(Account from, Account to, BigDecimal amt) {
    Account first  = from.getId() < to.getId() ? from : to;    // global order
    Account second = from.getId() < to.getId() ? to : from;
    synchronized (first) {
        synchronized (second) {
            from.debit(amt);
            to.credit(amt);
        }
    }
}
```

TIP

Diagnose a hung application with a **thread dump**: `jstack <pid>` or `jcmd <pid> Thread.print`. The JVM reports "Found one Java-level deadlock" along with the threads and the locks involved.

Problem	What happens
Deadlock	Threads block forever, each waiting for the other's lock
Livelock	Threads keep reacting to each other, staying busy but making no progress
Starvation	A thread never gets the CPU or a lock because others always win (unfair locks, greedy threads)

11.11 Explicit Locks: ReentrantLock and ReadWriteLock

`java.util.concurrent.locks` offers locks with more features than `synchronized`.

```
private final ReentrantLock lock = new ReentrantLock();     // new ReentrantLock(true) = fair

public void withdraw(double amt) {
    lock.lock();
    try {
        if (balance >= amt) balance -= amt;                 // critical section
    } finally {
        lock.unlock();                                      // ALWAYS unlock in finally
    }
}

public boolean tryTransfer(double amt) throws InterruptedException {
    if (lock.tryLock(500, TimeUnit.MILLISECONDS)) {         // avoid waiting forever
        try { balance -= amt; return true; }
        finally { lock.unlock(); }
    }
    return false;                                           // couldn't get the lock: back off
}

ReadWriteLock rw = new ReentrantReadWriteLock();            // many readers OR one writer
rw.readLock().lock();  try { /* read rates */ }  finally { rw.readLock().unlock(); }
rw.writeLock().lock(); try { /* update rates */ } finally { rw.writeLock().unlock(); }
```

`synchronized`	`ReentrantLock`
Keyword; released automatically	API; **must** call `unlock()` in `finally`
No timeout; can't interrupt a thread waiting for the lock	`tryLock()`, `tryLock(timeout)`, `lockInterruptibly()`
Always unfair	Optional fairness
One wait set (`wait`/`notify`)	Several `Condition` objects (`await`/`signal`)
Simpler, so prefer it for simple cases	Use when you need the extra features

11.12 The Executor Framework and Thread Pools

Creating a new thread per task is expensive, and creating them without limit can crash the JVM. An `ExecutorService` manages a **pool** of reusable worker threads and a **queue** of tasks. You submit *tasks*; the pool decides *which thread* runs them.

```
  submit(task) --> [  Task Queue  ] --> Worker-1  (reused)
  submit(task) -->  t5 t4 t3 t2 t1  --> Worker-2  (reused)
  submit(task) -->                  --> Worker-3  (reused)
```

Factory (`Executors.`)	Behaviour	Use for
`newFixedThreadPool(n)`	n threads, unbounded queue	Steady CPU-bound or limited-I/O work
`newSingleThreadExecutor()`	One thread; tasks run sequentially in order	Ordered background processing
`newCachedThreadPool()`	Creates threads as needed, reuses idle ones (60 s)	Many short tasks (can grow without limit!)
`newScheduledThreadPool(n)`	Delayed or periodic tasks	Polling, report schedules, timeouts
`newWorkStealingPool()`	`ForkJoinPool` with parallelism = cores	Divide-and-conquer parallel tasks
`newVirtualThreadPerTaskExecutor()` (Java 21)	One **virtual** thread per task	High-concurrency blocking I/O

```
ExecutorService pool = Executors.newFixedThreadPool(3);
for (int i = 1; i <= 6; i++) {
    int id = i;                                              // effectively final copy
    pool.execute(() -> System.out.println("Task " + id + " on "
            + Thread.currentThread().getName()));            // pool-1-thread-1..3 reused
}
pool.shutdown();                                             // stop accepting new tasks
if (!pool.awaitTermination(60, TimeUnit.SECONDS)) {          // wait for running tasks
    pool.shutdownNow();                                      // then interrupt stragglers
}

ScheduledExecutorService ses = Executors.newScheduledThreadPool(1);
ses.scheduleAtFixedRate(() -> System.out.println("heartbeat"), 0, 5, TimeUnit.SECONDS);
```

Method	Behaviour
`execute(Runnable)`	Fire and forget; no result; exceptions go to the thread's handler
`submit(Runnable / Callable)`	Returns a `Future`; exceptions are captured and rethrown from `get()` as `ExecutionException`
`invokeAll(tasks)`	Runs all tasks, returns a list of Futures when **all** are done
`invokeAny(tasks)`	Returns the result of the **first** task to succeed; cancels the rest
`shutdown()`	No new tasks; queued and running tasks finish; doesn't block
`shutdownNow()`	Interrupts running tasks; returns tasks never started
`awaitTermination(t, unit)`	Blocks until all tasks finish or the timeout passes
`close()` (Java 19+)	Shutdown + await, so an executor can be used in try-with-resources

Future

```
ExecutorService pool = Executors.newFixedThreadPool(2);
Future<BigDecimal> balance = pool.submit(() -> fetchBalance("SB-1001"));   // Callable
// ... do other work meanwhile ...
BigDecimal b = balance.get(2, TimeUnit.SECONDS);   // blocks; TimeoutException if too slow
balance.isDone();  balance.cancel(true);  balance.isCancelled();
```

ThreadPoolExecutor under the hood

```
ThreadPoolExecutor exec = new ThreadPoolExecutor(
        4,                                  // corePoolSize: threads kept alive
        8,                                  // maximumPoolSize: used when the queue is full
        30, TimeUnit.SECONDS,               // keepAlive for threads above core
        new ArrayBlockingQueue<>(100),      // BOUNDED queue (backpressure)
        new ThreadPoolExecutor.CallerRunsPolicy());   // what to do when saturated
```

Rejection policies are `AbortPolicy` (the default: throws `RejectedExecutionException`), `CallerRunsPolicy` (the caller runs the task, which slows producers), `DiscardPolicy` and `DiscardOldestPolicy`.

> **PITFALL**
> `newFixedThreadPool` uses an **unbounded** queue, and `newCachedThreadPool` an **unbounded number of threads**. Under heavy load they can cause `OutOfMemoryError`. Production code often builds a `ThreadPoolExecutor` with bounded queues, named threads and a deliberate rejection policy. Also: **always shut down** executors, because their non-daemon threads keep the JVM alive.

11.13 CompletableFuture (Java 8): Asynchronous Pipelines

`Future.get()` blocks. `CompletableFuture` lets you **chain**, **combine** and **handle errors** asynchronously, much like promises in JavaScript.

```
CompletableFuture<BigDecimal> balanceF =
        CompletableFuture.supplyAsync(() -> fetchBalance("SB-1001"));   // runs in a pool
CompletableFuture<BigDecimal> rateF =
        CompletableFuture.supplyAsync(() -> fetchInterestRate("SAVINGS"));

CompletableFuture<String> report = balanceF
        .thenCombine(rateF, (bal, rate) -> bal.multiply(rate))          // combine two results
        .thenApply(interest -> "Projected interest: " + interest)       // transform
        .exceptionally(ex -> "Service unavailable: " + ex.getMessage()) // fallback on error
        .orTimeout(3, TimeUnit.SECONDS);                                 // Java 9

System.out.println(report.join());          // join() = get() without checked exceptions

CompletableFuture<Void> all = CompletableFuture.allOf(balanceF, rateF);  // wait for all
CompletableFuture<Object> any = CompletableFuture.anyOf(balanceF, rateF); // first to finish
```

Method	Purpose
`supplyAsync(supplier)` / `runAsync(runnable)`	Start async work (common `ForkJoinPool`, or a pool you pass in)
`thenApply(fn)`	Transform the result (like `map`)
`thenCompose(fn)`	Chain another async call (like `flatMap`)
`thenCombine(other, fn)`	Merge two independent results
`thenAccept(consumer)` / `thenRun(r)`	Consume the result / run an action afterwards
`exceptionally(fn)` / `handle(fn)`	Recover from errors / handle the result or error
`allOf(...)` / `anyOf(...)`	Wait for all / for the first
`join()` / `get()`	Block for the result (unchecked / checked exceptions)

11.14 Concurrent Collections and Synchronizers

Utility	What it does	Typical use
`ConcurrentHashMap`	Thread-safe map with fine-grained locking; atomic `merge`, `compute`, `putIfAbsent`	Shared caches, counters by key (internals in Phase 14)
`CopyOnWriteArrayList`	Copies the array on every write; lock-free reads; iterators never throw CME	Listener lists: read often, written rarely
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`)	`put` blocks when full; `take` blocks when empty	Producer–consumer pipelines
`CountDownLatch(n)`	Threads `await()` until `countDown()` has been called n times (one-shot)	Wait for n services to start, or n tasks to finish
`CyclicBarrier(n)`	n threads wait for each other, then proceed together (reusable)	Phased parallel computations
`Semaphore(n)`	At most n permits held at once	Limit concurrent DB/API calls (throttling)
`ThreadLocal<T>`	A separate copy of a variable per thread	Per-request context, non-thread-safe formatters

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);
Thread producer = new Thread(() -> {
    try { for (int i = 1; i <= 5; i++) queue.put("TXN-" + i); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
Thread consumer = new Thread(() -> {
    try { for (int i = 1; i <= 5; i++) System.out.println("processed " + queue.take()); }
    catch (InterruptedException e) { Thread.currentThread().interrupt(); }
});
producer.start(); consumer.start();       // no wait/notify code needed

CountDownLatch ready = new CountDownLatch(3);
for (String svc : List.of("db", "cache", "mq")) {
    new Thread(() -> { System.out.println(svc + " up"); ready.countDown(); }).start();
}
ready.await();                            // main continues only after all 3 count down
System.out.println("all services ready");

Semaphore apiLimit = new Semaphore(5);    // max 5 concurrent calls to a partner API
apiLimit.acquire();
try { /* call API */ } finally { apiLimit.release(); }
```

PITFALL

With thread pools, a `ThreadLocal` value **stays on the reused thread** and can leak into the next request (or leak memory). Always call `threadLocal.remove()` in a `finally` block.

11.15 Virtual Threads (Java 21)

Classic ("platform") threads map one-to-one to OS threads, which are expensive (around 1 MB of stack each), so servers are limited to a few thousand of them. **Virtual threads** are lightweight threads managed by the JVM. Millions can exist, and when one blocks on I/O, the JVM **unmounts** it from its carrier OS thread, so that thread is freed for other work.

```
Thread vt = Thread.ofVirtual().start(() -> System.out.println("hello from a virtual thread"));
vt.join();
Thread.startVirtualThread(() -> callPaymentApi());

try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
    for (int i = 0; i < 10_000; i++) {
        int id = i;
        exec.submit(() -> {                // 10,000 concurrent blocking calls: no problem
            Thread.sleep(Duration.ofSeconds(1));
            return id;
        });
    }
}                                          // close() waits for all tasks
```

	Platform thread	Virtual thread
Backed by	One OS thread each	A small pool of carrier OS threads
Cost	Heavy (stack memory, OS scheduling)	Very cheap: create one per task
Good for	CPU-bound work	**Blocking I/O-bound** work (HTTP, DB, file)
Pooling	Yes (pools exist to reuse threads)	**Don't pool**: create a new one per task

NOTE

Virtual threads make simple blocking code (thread-per-request) scale like reactive code. Spring Boot 3.2+ can enable them with `spring.threads.virtual.enabled=true`. On Java 21–23, blocking inside `synchronized` *pins* the carrier thread (use `ReentrantLock` for long blocking sections); Java 24 (JEP 491) removed most pinning. Virtual threads don't make CPU-bound work faster.

11.16 Best Practices

1. Prefer **executors** (or virtual threads) over creating `Thread`s manually; always shut executors down.
2. **Minimise shared mutable state**: use immutable objects, confinement (local variables) and message passing (queues).
3. Keep critical sections **small**; lock on **private final** lock objects, not `this` or Strings.
4. Use **higher-level utilities** (`ConcurrentHashMap`, atomics, `BlockingQueue`, latches) before `wait`/`notify`.
5. Acquire multiple locks in a **consistent global order**; consider `tryLock` with timeouts.
6. Always `unlock()` in `finally`; use `while` loops around waits.
7. Never swallow `InterruptedException`: restore the flag or propagate it.
8. **Name your threads** (a `ThreadFactory`), so logs and thread dumps are readable.
9. Don't rely on `sleep()` for synchronization in code or **tests**. Use latches, `join` or `Awaitility`-style polling in tests.
10. Bound queues and pools in production, and choose a rejection policy deliberately.

11.17 Predict the Output

```
Thread t = new Thread(() -> System.out.print(Thread.currentThread().getName() + " "));
t.run();                                          // main   (no new thread)
t.start(); t.join();                              // Thread-0
System.out.println();

Thread d = new Thread(() -> { });
d.start();
try { d.setDaemon(true); }
catch (IllegalThreadStateException e) { System.out.println("ITSE"); }   // ITSE

Object lock = new Object();
try { lock.wait(); }
catch (IllegalMonitorStateException e) { System.out.println("IMSE"); }  // IMSE

AtomicInteger ai = new AtomicInteger(5);
System.out.println(ai.getAndIncrement() + " " + ai.incrementAndGet());  // 5 7
System.out.println(ai.compareAndSet(7, 10) + " " + ai.get());          // true 10

Thread n = new Thread(() -> { });
System.out.println(n.getState());                 // NEW
n.start(); n.join();
System.out.println(n.getState() + " " + n.isAlive());   // TERMINATED false
```

11.18 Interview Questions

Q. What is the difference between a process and a thread?

A process is an independent program with its own memory. A thread is a lightweight execution path inside a process that shares the heap with other threads but has its own stack and program counter.

Q. How do you create a thread? Which way is preferred?

Extend `Thread`, implement `Runnable` (or use a lambda), or submit a `Callable` to an executor. Implementing `Runnable`/`Callable` is preferred: it keeps inheritance free, separates the task from the thread, and works with thread pools.

Q. What happens if you call `run()` instead of `start()`? Can a thread be started twice?

`run()` executes on the current thread, so no new thread is created. Calling `start()` twice throws `IllegalThreadStateException`.

Q. What are the thread states?

NEW, RUNNABLE, BLOCKED, WAITING, TIMED_WAITING and TERMINATED. There is no RUNNING state in `Thread.State`; running threads are RUNNABLE.

Q. What is a race condition, and how do you prevent it?

Incorrect results caused by unsynchronized concurrent access to shared mutable data, where the outcome depends on timing. Prevent it with `synchronized`, locks, atomic classes, concurrent collections, or by avoiding shared mutable state.

Q. What is the difference between `synchronized` and `volatile`?

`synchronized` provides mutual exclusion **and** visibility for a block of code. `volatile` provides only visibility and ordering for a single variable; it doesn't make compound actions like `count++` atomic.

Q. What is the difference between `sleep()` and `wait()`?

`sleep` is a static Thread method that pauses for a time and keeps locks. `wait` is an Object method, must be called holding that object's monitor, releases it, and waits for notify, a timeout or an interrupt.

Q. Why must `wait()` be called in a loop?

Because of spurious wakeups, and because the condition may have changed again by the time the thread reacquires the lock. The condition must be re-checked.

Q. What is a deadlock, and how do you avoid it?
Threads waiting forever for locks held by each other. Avoid it with consistent lock ordering, `tryLock` timeouts, minimal lock scope, and by not calling external code while holding locks. Detect it with thread dumps (`jstack`).

Q. What are the advantages of `ReentrantLock` over `synchronized`?
Timed and interruptible lock attempts (`tryLock`), optional fairness, and multiple `Condition`s. The cost is that unlocking is manual and must go in `finally`.

Q. Why use thread pools? Explain `shutdown` vs `shutdownNow`.
Pools reuse threads, limit concurrency and queue tasks, which is cheaper and safer than creating threads per task. `shutdown` stops new submissions and lets queued tasks finish. `shutdownNow` interrupts running tasks and returns the tasks that never started.

Q. What is the difference between Runnable and Callable, and what is a Future?
Callable returns a value and can throw checked exceptions; Runnable can't. A `Future` represents a pending result: `get()` blocks, and you can also check `isDone()` or `cancel()`.

Q. How is CompletableFuture better than Future?
It supports non-blocking chaining (`thenApply`, `thenCompose`), combining results (`thenCombine`, `allOf`), error recovery (`exceptionally`) and timeouts, without blocking threads at every step.

Q. What are daemon threads?
Background threads that don't keep the JVM alive. The JVM exits when only daemon threads remain. Mark a thread with `setDaemon(true)` before `start()`.

Q. What are virtual threads, and when should you use them?
Lightweight JVM-managed threads (Java 21) that are cheap enough to create one per task. They are ideal for high-concurrency, blocking I/O workloads (web requests, DB and API calls), not for CPU-bound work, and they shouldn't be pooled.

11.19 Quick Revision

- Threads share the **heap**, but each has its own **stack**. Execution order is never guaranteed; priorities are hints.
- Create: `Runnable`/lambda (preferred), `extends Thread`, `Callable` via an executor. `start()` once; `run()` directly creates no thread.
- States: NEW → RUNNABLE ↔ BLOCKED / WAITING / TIMED_WAITING → TERMINATED.
- `sleep` keeps locks; `wait` releases them (needs `synchronized`; use a `while` loop). `join` waits for a thread to finish.
- Interrupt = cooperative cancellation. Restore the flag after catching `InterruptedException`.
- Daemon threads don't keep the JVM alive; call `setDaemon` before `start`.
- Race condition: `count++` isn't atomic. Fix with `synchronized`, locks or atomics.
- `synchronized` instance method → lock `this`; static → lock `Class`. Locks are reentrant; release is automatic.
- `volatile` = visibility only (flags); atomics = lock-free counters (CAS); `LongAdder` for high contention.
- Deadlock: needs all four Coffman conditions. Prevent with lock ordering or `tryLock`; diagnose with `jstack`.
- `ReentrantLock`: `lock()` / `try` / `finally unlock()`; `tryLock`, fairness, Conditions. `ReadWriteLock` for read-heavy data.
- Executors: fixed, single, cached, scheduled, virtual-per-task. `execute` vs `submit` (Future). Always shut down; bound queues in production.
- `CompletableFuture`: `supplyAsync` → `thenApply`/`thenCompose`/`thenCombine` → `exceptionally` → `join`.
- Utilities: `ConcurrentHashMap`, `CopyOnWriteArrayList`, `BlockingQueue`, `CountDownLatch`, `CyclicBarrier`, `Semaphore`, `ThreadLocal` (call `remove()`).

- Virtual threads (Java 21): a million cheap threads for blocking I/O; don't pool them.

PHASE 12 OF 18

File I/O, NIO.2 & Serialization

Streams, the File class, byte vs character streams, buffering, charsets, PrintWriter, RandomAccessFile, java.nio.file (Path & Files), properties files, serialization, transient, serialVersionUID and safe alternatives

IN THIS PHASE

12.1 Java I/O Overview
12.2 The File Class (java.io)
12.3 Byte Streams
12.4 Character Streams and Buffering
12.5 Choosing the Right Class
12.6 Other Useful java.io Classes
12.7 NIO.2: Path and Files (Java 7+)
12.8 Properties Files: External Configuration
12.9 Serialization
12.10 Best Practices
12.11 Predict the Output
12.12 Interview Questions
12.13 Quick Revision

12.1 Java I/O Overview

I/O (Input/Output) means reading data from a **source** (keyboard, file, network socket, memory) and writing it to a **destination**. Java models this as **streams**: sequences of data flowing in one direction.

```
Source --> InputStream/Reader --> Java program --> OutputStream/Writer --> Destination
(file, keyboard, socket, byte[])                    (file, console, socket, byte[])
```

Package	Contents
`java.io` (Java 1.0)	Classic stream-based I/O: `File`, `InputStream`/`OutputStream`, `Reader`/`Writer`, serialization
`java.nio` (Java 1.4)	Buffers, channels, selectors: non-blocking, high-performance I/O
`java.nio.file` (NIO.2, Java 7)	Modern file API: `Path`, `Files`, `FileSystems`, file walking and watching. **Preferred for files today.**

Byte streams vs character streams

	Byte streams	Character streams
Unit	8-bit bytes	16-bit chars (text), decoded using a **charset**
Abstract base classes	`InputStream`, `OutputStream`	`Reader`, `Writer`
Use for	**Binary data**: images, PDFs, ZIPs, audio, serialized objects	**Text**: .txt, .csv, .json, .xml, logs
File classes	`FileInputStream`, `FileOutputStream`	`FileReader`, `FileWriter`
Buffered versions	`BufferedInputStream`, `BufferedOutputStream`	`BufferedReader`, `BufferedWriter`
Bridge between them	n/a	`InputStreamReader` (bytes → chars), `OutputStreamWriter` (chars → bytes)

```
InputStream (abstract)                      Reader (abstract)
 +- FileInputStream                          +- InputStreamReader
 +- ByteArrayInputStream                     |   +- FileReader
 +- FilterInputStream                        +- BufferedReader
 |   +- BufferedInputStream                  +- StringReader, CharArrayReader
 |   +- DataInputStream
 +- ObjectInputStream                       Writer (abstract)
                                             +- OutputStreamWriter
OutputStream (abstract)                      |   +- FileWriter
 +- FileOutputStream                         +- BufferedWriter
 +- ByteArrayOutputStream                    +- PrintWriter
 +- FilterOutputStream                       +- StringWriter, CharArrayWriter
 |   +- BufferedOutputStream
 |   +- DataOutputStream
 |   +- PrintStream   (System.out, System.err)
 +- ObjectOutputStream
```

> **NOTE**
> Most of these classes follow the **Decorator pattern**: you wrap one stream in another to add features. `new BufferedReader(new InputStreamReader(new FileInputStream(f), UTF_8))` adds decoding and then buffering on top of raw bytes. Closing the **outermost** wrapper closes the whole chain.

12.2 The File Class (java.io)

A `File` object represents a **path** to a file or directory. It **does not read or write content**; it handles names, existence checks, metadata, and creating or deleting entries.

```
File f = new File("abc.txt");                  // logical object; nothing is created yet
System.out.println(f.exists());                // false
System.out.println(f.createNewFile());         // true  -> empty file created
System.out.println(f.createNewFile());         // false -> it already exists

File dir = new File("data");
dir.mkdir();                                   // one level; false if exists/no parent
new File("data/2026/reports").mkdirs();        // creates all missing parent directories
File inDir = new File(dir, "test.txt");        // parent + child
inDir.createNewFile();

for (File item : dir.listFiles()) {            // list(): names; listFiles(): File objects
    System.out.println((item.isFile() ? "File : " : "Directory : ") + item.getName());
}
```

Method	Returns	Purpose
`exists()`, `isFile()`, `isDirectory()`	`boolean`	Existence and type checks
`createNewFile()`	`boolean`	Create an empty file; `false` if it exists; throws `IOException`
`mkdir()` / `mkdirs()`	`boolean`	Create a directory / with parents
`delete()`	`boolean`	Delete a file or an **empty** directory (`false` otherwise)
`list()` / `listFiles()`	`String[]` / `File[]`	Directory contents (`null` if not a directory!)
`length()`	`long`	Size in bytes
`getName()`, `getPath()`, `getAbsolutePath()`, `getParent()`	`String`	Path information
`renameTo(File)`	`boolean`	Rename or move (platform-dependent)
`canRead()`, `canWrite()`, `lastModified()`	various	Permissions and metadata

TIP

`File` methods return `boolean` instead of throwing a meaningful exception, so a failed `delete()` doesn't tell you *why*. NIO.2 `Files` methods throw descriptive exceptions (`NoSuchFileException`, `DirectoryNotEmptyException`), which is one reason to prefer them.

12.3 Byte Streams

```
// Copy any file (binary-safe) using a buffer
try (InputStream in = new FileInputStream("photo.jpg");
     OutputStream out = new FileOutputStream("photo-copy.jpg")) {
    byte[] buffer = new byte[8192];
    int bytesRead;
    while ((bytesRead = in.read(buffer)) != -1) {     // read() returns -1 at end of stream
        out.write(buffer, 0, bytesRead);              // write only the bytes actually read
    }
}
// Java 9+: in.transferTo(out) does the same loop for you

try (FileOutputStream log = new FileOutputStream("audit.log", true)) {   // true = APPEND
    log.write("login ok\n".getBytes(StandardCharsets.UTF_8));
}
```

PITFALL

`read()` (no arguments) returns **one byte as an** `int` (0–255), or **-1** at end of stream. Reading and writing one byte at a time without buffering is extremely slow, because each call can hit the OS. Use a `byte[]` buffer or `Buffered*` streams.

12.4 Character Streams and Buffering

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter("customers.txt"))) {
    bw.write("101,Rohit,Pune");
    bw.newLine();                                  // platform line separator
    bw.write("102,Amit,Delhi");
}                                                  // close() flushes the buffer

try (BufferedReader br = new BufferedReader(new FileReader("customers.txt"))) {
    String line;
    while ((line = br.readLine()) != null) {       // readLine() returns null at end of file
        String[] parts = line.split(",");
        System.out.println(parts[1] + " lives in " + parts[2]);
    }
}
```

Character encodings (charsets)

Text is stored as bytes, and a **charset** maps between characters and bytes. Reading with the wrong charset produces garbled text: ₹ or é shows up as â‚¹ or Ã©.

```
// Explicit charset (Java 11+ constructors): always predictable
var cs = StandardCharsets.UTF_8;
try (BufferedReader br = new BufferedReader(new FileReader("in.txt", cs));
     BufferedWriter bw = new BufferedWriter(new FileWriter("out.txt", cs))) {
    bw.write(br.readLine());
}

// Bridge classes: bytes <-> chars with a chosen charset (works on any InputStream)
Reader r = new InputStreamReader(socket.getInputStream(), StandardCharsets.UTF_8);
Writer w = new OutputStreamWriter(new FileOutputStream("out.txt"), StandardCharsets.UTF_8);
```

> **NOTE**
> Before **Java 18**, `FileReader`/`FileWriter` used the **platform default** charset (often windows-1252 on Windows), a classic "works on my machine" bug. Since Java 18 (JEP 400), the default is **UTF-8** everywhere. Specifying `StandardCharsets.UTF_8` explicitly is still best practice.

PrintWriter and PrintStream

```
try (PrintWriter pw = new PrintWriter(new FileWriter("statement.txt"))) {
    pw.println("ACCOUNT STATEMENT");
    pw.printf("%-12s %10s%n", "Date", "Amount");
    pw.printf("%-12s %,10.2f%n", "2026-08-01", 25000.0);
}
System.out.println("...");      // System.out is a PrintStream
System.err.println("error!");   // so is System.err
```

> **PITFALL**
> `PrintWriter` and `PrintStream` **never throw** `IOException`; they set an internal error flag instead. Call `checkError()` if you need to know whether writes failed. Forgetting to flush or close is the usual cause of an "empty file".

12.5 Choosing the Right Class

Comparison	Key difference
`Reader` vs `InputStream`	Characters (decoded text) vs raw bytes
`Writer` vs `OutputStream`	Characters vs raw bytes
`BufferedReader` vs `FileReader`	`BufferedReader` buffers (fast) and has `readLine()`. `FileReader` reads unbuffered, one character or array at a time. Wrap one in the other.
`BufferedWriter` vs `FileWriter`	Buffered (fewer system calls) and has `newLine()`; must be flushed or closed
`BufferedReader` vs `Scanner`	`BufferedReader`: fast, 8 KB buffer, lines only, throws `IOException`. `Scanner`: parses tokens and primitives (`nextInt`), slower, 1 KB buffer, swallows `IOException`.
`PrintWriter` vs `PrintStream`	Character stream (Writer family) vs byte stream (`System.out`). Both have `printf`/`println` and neither throws `IOException`.
`java.io.File` vs `java.nio.file.Files`	Old path object with boolean results vs modern utilities with clear exceptions, charsets, streams and walking

12.6 Other Useful java.io Classes

Data streams: primitives in binary

```
try (DataOutputStream out = new DataOutputStream(
        new BufferedOutputStream(new FileOutputStream("rates.bin")))) {
    out.writeInt(3); out.writeDouble(7.25); out.writeUTF("FD");
}
try (DataInputStream in = new DataInputStream(
        new BufferedInputStream(new FileInputStream("rates.bin")))) {
    int n = in.readInt(); double rate = in.readDouble(); String type = in.readUTF();
    // must read in EXACTLY the same order and types as written
}
```

RandomAccessFile: jump to any position

```
try (RandomAccessFile raf = new RandomAccessFile("records.dat", "rw")) {  // r, rw, rws, rwd
    raf.writeInt(100); raf.writeInt(200); raf.writeInt(300);   // 4 bytes each
    raf.seek(4);                              // move the file pointer to the 2nd int
    System.out.println(raf.readInt());        // 200
    raf.seek(4); raf.writeInt(250);           // update in place
    System.out.println(raf.length() + " bytes, pointer at " + raf.getFilePointer());
}
```

Memory streams and Console

```
ByteArrayOutputStream bos = new ByteArrayOutputStream();   // write into memory
bos.write("hello".getBytes());
byte[] bytes = bos.toByteArray();
InputStream in = new ByteArrayInputStream(bytes);         // great for unit tests

Console console = System.console();                       // null inside most IDEs!
if (console != null) {
    String user = console.readLine("User: ");
    char[] pwd = console.readPassword("Password: ");      // input not echoed
    java.util.Arrays.fill(pwd, ' ');                      // wipe the secret after use
}
```

12.7 NIO.2: Path and Files (Java 7+)

`java.nio.file` is the modern, recommended API for working with files: concise, charset-aware, with clear exceptions and stream support.

```
import java.nio.file.*;
import static java.nio.charset.StandardCharsets.UTF_8;

Path dir  = Path.of("data", "reports");              // Java 11 (Paths.get(...) before that)
Path file = dir.resolve("summary.txt");              // data/reports/summary.txt

Files.createDirectories(dir);                        // like mkdirs; no error if it exists
Files.writeString(file, "Total: 1,25,000\n", UTF_8);                    // Java 11
Files.writeString(file, "Count: 42\n", UTF_8, StandardOpenOption.APPEND);
String content = Files.readString(file, UTF_8);                         // whole file
List<String> lines = Files.readAllLines(file, UTF_8);                   // all lines

try (Stream<String> stream = Files.lines(file, UTF_8)) {   // lazy: for HUGE files
    long count = stream.filter(l -> l.startsWith("Total")).count();
}                                                          // Files.lines must be closed!

Files.copy(file, dir.resolve("backup.txt"), StandardCopyOption.REPLACE_EXISTING);
Files.move(dir.resolve("backup.txt"), dir.resolve("old.txt"), StandardCopyOption.ATOMIC_MOVE);
Files.deleteIfExists(dir.resolve("old.txt"));
System.out.println(Files.exists(file) + " " + Files.size(file) + " bytes");
```

`Files` method	Purpose
`readString` / `writeString` (11)	Whole file as one String
`readAllLines` / `write(path, lines)`	All lines as a `List<String>`
`readAllBytes` / `write(path, bytes)`	Binary content
`lines(path)`	Lazy `Stream<String>` (close it: try-with-resources)
`newBufferedReader` / `newBufferedWriter`	Buffered character streams with a charset
`newInputStream` / `newOutputStream`	Byte streams
`exists`, `notExists`, `isDirectory`, `isRegularFile`, `isReadable`, `size`	Checks and metadata

Files method	Purpose
createFile, createDirectory, createDirectories, createTempFile	Creation
copy, move, delete, deleteIfExists	Manipulation (StandardCopyOption.REPLACE_EXISTING, ATOMIC_MOVE)
list(dir), walk(dir), find(dir, depth, matcher)	Directory listing and recursive traversal (Streams: close them)
getLastModifiedTime, getAttribute, probeContentType	Attributes

StandardOpenOption	Meaning
CREATE / CREATE_NEW	Create if missing / fail if it already exists
APPEND	Write at the end
TRUNCATE_EXISTING	Clear the existing content (the default for write methods along with CREATE, WRITE)
READ / WRITE	Access mode

Path operations

```
Path p = Path.of("/home/rohit/projects/../docs/./notes.txt");
p.getFileName();                 // notes.txt
p.getParent();                   // /home/rohit/projects/../docs/.
p.normalize();                   // /home/rohit/docs/notes.txt
p.isAbsolute();                  // true
Path base = Path.of("/home/rohit");
base.resolve("docs/a.txt");      // /home/rohit/docs/a.txt
base.relativize(Path.of("/home/rohit/docs/a.txt"));   // docs/a.txt
Path.of("report.pdf").toAbsolutePath();               // resolved against the working directory
p.toFile();  new File("x").toPath();                  // convert between old and new APIs
```

Practical example: summarise a transactions CSV

```
// transactions.csv:  txnId,account,type,amount
//                    T1,SB-1001,CREDIT,5000.00
//                    T2,SB-1001,DEBIT,1200.50
Path csv = Path.of("transactions.csv");
BigDecimal balance = BigDecimal.ZERO;
try (BufferedReader br = Files.newBufferedReader(csv, UTF_8)) {
    br.readLine();                                                  // skip the header
    String line;
    while ((line = br.readLine()) != null) {
        String[] f = line.split(",", -1);
        BigDecimal amt = new BigDecimal(f[3].trim());
        balance = f[2].equals("CREDIT") ? balance.add(amt) : balance.subtract(amt);
    }
}
System.out.println("Net: " + balance);                           // Net: 3799.50

try (Stream<Path> files = Files.walk(Path.of("logs"))) {         // recursive search
    files.filter(f -> f.toString().endsWith(".log"))
         .filter(f -> f.toFile().length() > 1_000_000)
         .forEach(System.out::println);                         // large log files
}
```

12.8 Properties Files: External Configuration

`.properties` files hold `key=value` configuration. Test frameworks (Appium/Selenium settings), Spring Boot's `application.properties` and JDBC setups all use them.

```
# config.properties
app.env=QA
device.name=Pixel_7
platform.version=14
api.base.url=https://qa.bank.example.com
```

```
Properties props = new Properties();
try (Reader r = Files.newBufferedReader(Path.of("config.properties"), UTF_8)) {
    props.load(r);
}
String env = props.getProperty("app.env");                    // QA
String timeout = props.getProperty("timeout.seconds", "30");   // default if missing

// Load from the classpath (src/main/resources or src/test/resources)
try (InputStream in = Config.class.getClassLoader().getResourceAsStream("config.properties")) {
    props.load(in);
}
```

TIP

Load config from the **classpath** (`getResourceAsStream`) rather than a hard-coded path like `C:\Users\...`. It then works the same in your IDE, in Maven/Gradle and in CI. Keep secrets out of committed properties files; use environment variables or a vault.

12.9 Serialization

Serialization converts an object's state into a **byte stream** (to save to a file, send over a network, or cache). **Deserialization** rebuilds the object from those bytes.

```
 Object (heap) --ObjectOutputStream.writeObject()--> bytes --> file / network / cache
 Object (heap) <--ObjectInputStream.readObject()---- bytes <--
```

```
import java.io.*;

public class Customer implements Serializable {        // marker interface: no methods
    private static final long serialVersionUID = 1L;   // version id: declare it explicitly!
    private String id;
    private String name;
    private transient String password;                 // transient: NOT serialized
    private static String bank = "ABC";                // static: class-level, NOT serialized

    public Customer(String id, String name, String password) {
        this.id = id; this.name = name; this.password = password;
    }
    @Override public String toString() { return id + " " + name + " " + password; }
}

Customer c = new Customer("C1", "Rohit", "secret");
try (ObjectOutputStream oos = new ObjectOutputStream(new FileOutputStream("customer.ser"))) {
    oos.writeObject(c);                                // serialize
}
try (ObjectInputStream ois = new ObjectInputStream(new FileInputStream("customer.ser"))) {
    Customer copy = (Customer) ois.readObject();       // deserialize (ClassNotFoundException)
    System.out.println(copy);                          // C1 Rohit null  <- transient lost
}
```

Rules

Rule	Detail
Must implement `Serializable`	Otherwise `NotSerializableException`
Whole object graph	Every non-transient field's object must also be serializable (a `Customer` with an `Address` field needs `Address implements Serializable`)
`transient` fields	Skipped. They get default values (null, 0, false) after deserialization. Use for passwords, caches, derived data and non-serializable helpers.
`static` fields	Not serialized: they belong to the class, not the object
Constructor	**Not called** for the serializable class when deserializing. Fields come from the stream.
Non-serializable parent	Its **no-arg constructor runs** during deserialization, and its fields are re-initialised rather than restored. The parent **must** have an accessible no-arg constructor.
Arrays and collections	Serializable if their elements are (`ArrayList`, `HashMap` and `String` are)

serialVersionUID

Each serializable class has a version number. During deserialization the JVM compares the stream's UID with the current class's UID; if they differ, it throws `InvalidClassException`. If you don't declare it, the compiler **computes** one from the class structure, so even adding a method can break compatibility with previously saved data. **Always declare** `private static final long serialVersionUID = 1L;` and change it deliberately.

Customising serialization

```
public class Account implements Serializable {
    private static final long serialVersionUID = 1L;
    private String accountNo;
    private transient String pin;                    // never write the raw PIN

    @Serial
    private void writeObject(ObjectOutputStream out) throws IOException {
        out.defaultWriteObject();                    // normal fields
        out.writeObject(mask(pin));                  // custom extra data
    }
    @Serial
    private void readObject(ObjectInputStream in) throws IOException, ClassNotFoundException {
        in.defaultReadObject();
        String masked = (String) in.readObject();
        // validate invariants here; re-create transient helpers
    }
    private static String mask(String p) { return p == null ? null : "****"; }
}
```

`Externalizable` (with `writeExternal`/`readExternal` and a **mandatory public no-arg constructor**) gives you full manual control. A singleton class should implement `readResolve()` to return its instance, otherwise deserialization creates a second object (enum singletons handle this automatically).

PITFALL

Never deserialize untrusted data with `ObjectInputStream`. Crafted byte streams can trigger remote code execution through "gadget" classes on the classpath. This has caused major real-world vulnerabilities (Apache Commons Collections, CVE-2015-7501). If you must, use an `ObjectInputFilter` (Java 9+) allow-list. For data exchange, prefer **JSON** (Jackson, Gson), XML, or Protocol Buffers: they are language-neutral, human-readable or compact, and safer.

```
// Typical modern approach (Jackson library, used by Spring Boot)
ObjectMapper mapper = new ObjectMapper();
String json = mapper.writeValueAsString(customer);          // {"id":"C1","name":"Rohit"}
Customer back = mapper.readValue(json, Customer.class);
```

12.10 Best Practices

1. Always use **try-with-resources** for streams, readers, writers and `Files.lines`/`walk`/`list` streams.
2. Prefer `java.nio.file.Files` for everyday file work; fall back to `java.io` streams for streaming or legacy APIs.
3. **Buffer** I/O (`Buffered*`, `Files.newBufferedReader`), never byte-by-byte.
4. **Specify the charset** (`UTF_8`) whenever converting between bytes and text.
5. Stream large files line by line (`readLine`, `Files.lines`) instead of loading them whole.
6. Build paths with `Path.of(a, b)` / `resolve` instead of concatenating `"\\"` or `"/"`, and load config from the classpath.
7. Handle `NoSuchFileException`, `AccessDeniedException` and `FileAlreadyExistsException` explicitly where it matters.
8. In tests, use **temporary files and folders** (`Files.createTempFile`, JUnit 5 `@TempDir`) and clean up.
9. Declare `serialVersionUID`, mark secrets `transient`, and **avoid Java serialization for untrusted input**; use JSON.

12.11 Predict the Output

```
File f = new File("demo-dir");
System.out.println(f.mkdir() + " " + f.mkdir());      // true false
System.out.println(new File("missing-dir").list());   // null (not a directory)

Path p = Files.writeString(Path.of("t.txt"), "a\nb\nc");
System.out.println(Files.readAllLines(p).size());     // 3
System.out.println(Files.size(p));                    // 5 (bytes: a \n b \n c)

try (BufferedReader br = Files.newBufferedReader(p)) {
    br.readLine(); br.readLine(); br.readLine();
    System.out.println(br.readLine());                // null (end of file)
}

ByteArrayInputStream in = new ByteArrayInputStream(new byte[]{65, 66});
System.out.println(in.read() + " " + in.read() + " " + in.read());   // 65 66 -1

Path base = Path.of("/a/b");
System.out.println(base.resolve("c/../d").normalize());   // /a/b/d
```

12.12 Interview Questions

Q. What is the difference between byte streams and character streams?

Byte streams (`InputStream`/`OutputStream`) handle raw 8-bit data and suit binary files. Character streams (`Reader`/`Writer`) handle text, decoding bytes into characters with a charset.

Q. Why use BufferedReader and BufferedWriter?

They reduce expensive system calls by reading and writing large chunks into an in-memory buffer. They also add `readLine()` and `newLine()`.

Q. What is the difference between FileReader and InputStreamReader?

`InputStreamReader` is a bridge that converts **any** byte stream to characters with a chosen charset. `FileReader` is a convenience subclass for files; before Java 11 it couldn't take a charset and used the platform default.

Q. What is the difference between BufferedReader and Scanner?
`BufferedReader` is faster with a bigger buffer and reads lines only. `Scanner` parses tokens and primitives with regex, is slower, and hides `IOException`. Use `BufferedReader` for large input and `Scanner` for convenient parsing of small input.

Q. What is the difference between `java.io.File` and `java.nio.file.Files`/`Path`?
`File` is the legacy path abstraction with boolean-returning methods and limited features. NIO.2 offers `Path` plus `Files` utilities with clear exceptions, charsets, symbolic links, attributes, stream-based reading and directory walking.

Q. What is serialization? How do you make a class serializable?
Converting an object to a byte stream for storage or transfer. Implement the marker interface `Serializable` (all non-transient fields must be serializable too) and use `ObjectOutputStream.writeObject()`.

Q. What is `transient`? Are static fields serialized?
`transient` fields are excluded from serialization and get default values when restored. Static fields belong to the class, so they are never serialized.

Q. What is serialVersionUID, and why should you declare it?
A version identifier checked during deserialization; a mismatch throws `InvalidClassException`. If you don't declare it, it is computed from the class structure, so harmless changes can break compatibility.

Q. Is a constructor called during deserialization?
Not for the serializable class itself. The no-arg constructor of the first **non-serializable** superclass is called, so that superclass's fields are re-initialised rather than restored.

Q. What are the risks of Java serialization?
Deserializing untrusted data can execute malicious gadget chains (RCE). Serialization also creates fragile versioning, bypasses constructors and invariants, and is Java-only. Prefer JSON or Protobuf, or use `ObjectInputFilter`.

Q. How do you read a very large file efficiently?
Stream it line by line with `BufferedReader.readLine()` or `Files.lines()` inside try-with-resources, rather than `readAllLines`/`readString`, which load everything into memory.

12.13 Quick Revision

- Streams flow one way. Bytes: `InputStream`/`OutputStream`; text: `Reader`/`Writer`; bridges: `InputStreamReader`/`OutputStreamWriter`.
- Decorator pattern: wrap streams to add buffering or encoding; closing the outer one closes the chain.
- `File` = a path plus metadata (no content). `createNewFile`/`mkdir(s)`/`delete`/`listFiles` return booleans or `null`.
- `read()` returns -1 at EOF; `readLine()` returns `null` at EOF. Always buffer.
- Specify **UTF-8**. Java 18+ defaults to UTF-8, but older platforms didn't.
- `PrintWriter`/`PrintStream` never throw `IOException` (use `checkError`); flush or close!
- NIO.2: `Path.of`, `resolve`, `normalize`; `Files.readString`/`writeString`/`readAllLines`/`lines`/`copy`/`move`/`walk`. Close `lines`, `walk` and `list` streams.
- Properties: `props.load(reader)`, `getProperty(key, default)`; load from the classpath.
- Serialization: `Serializable` (marker), `ObjectOutputStream.writeObject`, `ObjectInputStream.readObject`.
- `transient` and `static` are not serialized. Declare `serialVersionUID`. Mismatch → `InvalidClassException`.

- No constructor runs for the serializable class on read; a non-serializable parent needs a no-arg constructor.
- Never deserialize untrusted bytes; prefer JSON (Jackson).

PHASE 13 OF 18

Collections I: List, Set, Queue & Sorting

Why collections, the hierarchy, ArrayList/LinkedList/Vector/Stack internals, immutable lists, iterators and fail-fast behaviour, HashSet/LinkedHashSet/TreeSet, Queue/Deque/PriorityQueue, Comparable vs Comparator, and the Collections utility class

IN THIS PHASE

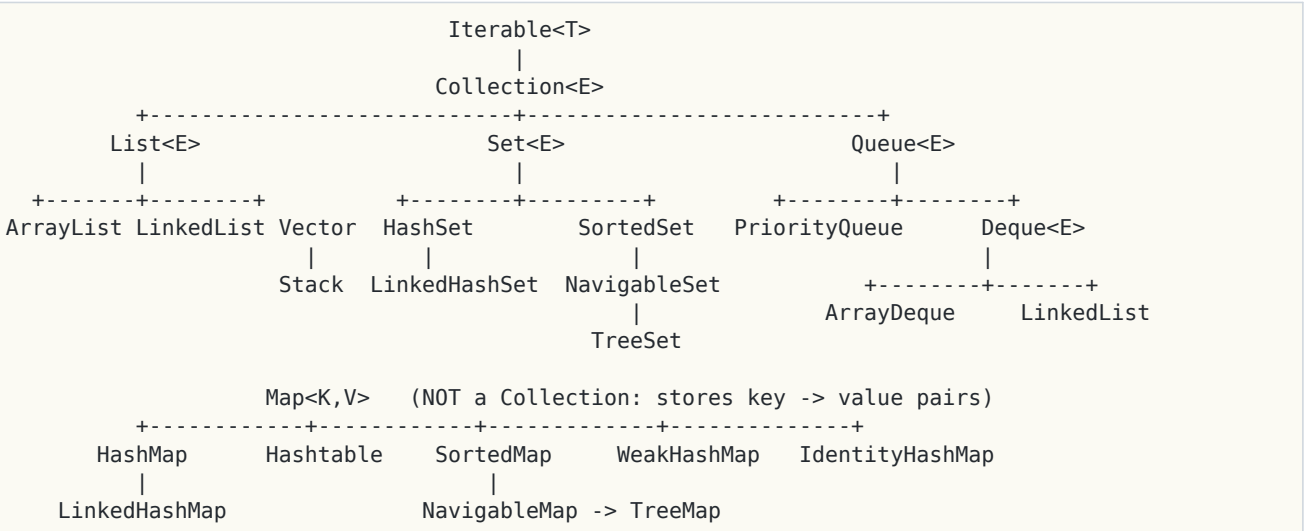
13.1 Why Collections?
13.2 The Collection Hierarchy
13.3 List: ArrayList
13.4 List: LinkedList, Vector and Stack
13.5 Iterating Collections
13.6 Set: HashSet, LinkedHashSet, TreeSet
13.7 Queue and Deque
13.8 Comparable vs Comparator
13.9 The Collections Utility Class
13.10 Predict the Output
13.11 Interview Questions
13.12 Quick Revision

13.1 Why Collections?

Arrays have limitations: **fixed size**, no built-in add/remove/search/sort operations, and shifting is up to you. The **Java Collections Framework** (JCF, `java.util`, Java 1.2) provides **interfaces** (List, Set, Queue, Map) with **ready-made implementations** and **algorithms** (sorting, searching).

Array	Collection
Fixed size	Grows and shrinks dynamically
Primitives and objects	Objects only (primitives are autoboxed)
`length` field, manual algorithms	Rich API: `add`, `remove`, `contains`, `sort`, streams
Homogeneous by declared type	Generic, type-safe (`List<String>`)
Slightly faster, less memory	Small overhead; many optimised data structures to choose from

13.2 The Collection Hierarchy



Interface	Ordering	Duplicates	Key idea
`List`	Insertion order, **index-based**	Allowed	Sequence: access by position
`Set`	Depends (none / insertion / sorted)	**Not allowed**	Uniqueness by `equals`/`hashCode` (or `compareTo`)
`Queue` / `Deque`	FIFO, priority, or both ends	Allowed	Processing order: offer, poll, peek
`Map`	Depends	Keys unique, values may repeat	Lookup by key

MODERN JAVA

Java 21 added **sequenced collections**: `SequencedCollection` (`getFirst()`, `getLast()`, `addFirst()`, `reversed()`, ...) implemented by `List`, `Deque` and `LinkedHashSet`, plus `SequencedMap` for `LinkedHashMap` and `TreeMap` (covered in Phase 14).

Core Collection methods

Method	Purpose
`add(e)`, `addAll(c)`	Insert (returns `false` if a Set already contains it)
`remove(o)`, `removeAll(c)`, `retainAll(c)`, `removeIf(pred)`, `clear()`	Remove
`contains(o)`, `containsAll(c)`	Membership (uses `equals`)
`size()`, `isEmpty()`	Counting
`iterator()`, `forEach(action)`	Traversal
`toArray()`, `toArray(new T[0])`, `toArray(String[]::new)` (11)	Convert to an array
`stream()`, `parallelStream()`	Stream pipelines (Phase 15)

13.3 List: ArrayList

`ArrayList` is a **resizable array**. It is the default choice for lists.

- Ordered by insertion; index-based; **duplicates** and **multiple nulls** allowed.
- **Fast random access** `get(i)`: O(1). Adding at the end: amortised O(1).
- Inserting or removing in the **middle** is O(n), because elements must shift.
- Not synchronized.

```
List<String> names = new ArrayList<>();       // program to the interface
names.add("Ravi");                            // [Ravi]
names.add("Amit");                            // [Ravi, Amit]
names.add(0, "Zoya");                         // [Zoya, Ravi, Amit]  (shift right)
names.set(1, "Rohit");                        // [Zoya, Rohit, Amit] (replace)
names.get(2);                                 // Amit
names.indexOf("Amit");                        // 2  (-1 if absent; uses equals)
names.contains("Zoya");                       // true
names.remove("Zoya");                         // by object -> [Rohit, Amit]
names.remove(0);                              // by INDEX  -> [Amit]
names.addAll(List.of("B", "C"));              // [Amit, B, C]
List<String> sub = names.subList(1, 3);       // [B, C]: a VIEW (changes write through)
```

Internal working

```
new ArrayList<>()       -> elementData = {} (empty; the array is allocated lazily)
first add()             -> capacity 10:   [A][ ][ ][ ][ ][ ][ ][ ][ ][ ]
11th add()              -> grow by 50%: newCapacity = old + (old >> 1) = 15
                           copy everything with Arrays.copyOf -> O(n), but rare
add(0, X) / remove(0)   -> System.arraycopy shifts every later element: O(n)
```

Constructor	Purpose
`ArrayList()`	Empty; capacity 10 allocated on the first add
`ArrayList(int initialCapacity)`	Pre-size when you know the approximate count (avoids repeated growth)
`ArrayList(Collection c)`	Copy another collection: `new ArrayList<>(set)`

TIP

`size()` is the number of elements; **capacity** is the internal array length, which you can't query. `ensureCapacity(n)` and `trimToSize()` manage it. Prefer `List<String> x = new ArrayList<>()` (interface type) so implementations can be swapped easily.

13.4 List: LinkedList, Vector and Stack

LinkedList

A **doubly linked list**: each node holds `prev`, `item` and `next`. It implements both **List** and **Deque**.

```
 first                                                  last
   |                                                     |
   v                                                     v
[null|Ravi|*] <--> [*|Amit|*] <--> [*|Zoya|null]
```

```
LinkedList<Integer> ll = new LinkedList<>(List.of(20, 30));
ll.addFirst(10);  ll.addLast(40);          // O(1) at the ends: [10, 20, 30, 40]
ll.removeFirst(); ll.peekLast();           // Deque operations
ll.get(2);                                 // O(n): walks from the nearer end
```

NOTE

LinkedList's middle insertion is O(1) **only once you're at the node** (with a `ListIterator`). Finding the position is O(n). Its nodes are scattered in memory and each carries 2 extra references, so in practice **ArrayList is faster for almost everything**, and `ArrayDeque` is faster for queues and stacks.

Vector and Stack (legacy)

- `Vector`: a synchronized growable array (Java 1.0). It **doubles** its capacity when full.
- `Stack`: extends `Vector`, LIFO: `push`, `pop`, `peek`, `empty`, `search`.

Both are **legacy**. Use `ArrayList` (with external synchronization or concurrent collections when needed) and `ArrayDeque` for stacks.

```
Deque<Integer> stack = new ArrayDeque<>();   // preferred stack
stack.push(1); stack.push(2); stack.push(3);
stack.pop();     // 3   (LIFO)
stack.peek();    // 2
```

ArrayList vs LinkedList vs Vector vs Stack

Feature	ArrayList	LinkedList	Vector	Stack
Structure	Resizable array	Doubly linked list	Resizable array	Vector subclass
`get(i)`	**O(1)**	O(n)	O(1)	O(1)
Add/remove at end	Amortised O(1)	O(1)	O(1)	O(1) push/pop
Add/remove at start or middle	O(n) shift	O(1) once positioned (finding it is O(n))	O(n)	n/a
Memory	Compact	Extra 2 refs per node	Compact	Compact
Growth	+50%	n/a (per node)	×2	×2
Thread-safe	No	No	Yes (synchronized)	Yes
Interfaces	List, RandomAccess	List, **Deque**	List, RandomAccess	List (LIFO methods)
Status	**Default choice**	Rarely the best	Legacy	Legacy → use `ArrayDeque`

Unmodifiable and immutable lists

Creation	Add/remove	Set an element	Nulls	Backed by the source?
`List.of(a, b)` (9)	✗ UOE	✗ UOE	✗ NPE	No: truly immutable
`List.copyOf(coll)` (10)	✗	✗	✗	No: an immutable copy
`Arrays.asList(a, b)`	✗ UOE	✓ (writes through to the array)	✓	**Yes**: a fixed-size view of the array
`Collections.unmodifiableList(list)`	✗	✗	✓	**Yes**: a read-only *view*; the original can still change
`stream().toList()` (16)	✗	✗	✓	No

(UOE = `UnsupportedOperationException`.)

13.5 Iterating Collections

```
List<String> list = new ArrayList<>(List.of("A", "B", "C"));

for (int i = 0; i < list.size(); i++) System.out.print(list.get(i));   // index loop (Lists)
for (String s : list) System.out.print(s);                             // for-each
Iterator<String> it = list.iterator();                                 // explicit Iterator
while (it.hasNext()) System.out.print(it.next());
list.forEach(s -> System.out.print(s));                                // Java 8 forEach
list.forEach(System.out::print);                                       // method reference
list.stream().filter(s -> !s.equals("B")).forEach(System.out::print); // streams

ListIterator<String> li = list.listIterator(list.size());              // start at the end
while (li.hasPrevious()) System.out.print(li.previous());              // CBA (backwards)
```

Cursors: Iterator vs ListIterator vs Enumeration

Feature	Enumeration (legacy)	Iterator	ListIterator
Applies to	Vector, Hashtable	**All** collections	List only
Direction	Forward	Forward	**Forward and backward**
Methods	hasMoreElements, nextElement	hasNext, next, remove	+ hasPrevious, previous, nextIndex, add, set
Modify while iterating	No	Remove	Remove, **add**, **set**
Obtained via	vector.elements()	coll.iterator()	list.listIterator()

Fail-fast iterators and ConcurrentModificationException

Most java.util collections (ArrayList, HashMap, HashSet...) have **fail-fast** iterators. Each collection keeps a modCount; if the collection is **structurally modified** during iteration by anything other than the iterator itself, the next next() call throws ConcurrentModificationException (CME). This happens even in a **single** thread.

```
List<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));

for (Integer n : nums) {
    if (n % 2 == 0) nums.remove(n);          // ConcurrentModificationException!
}

Iterator<Integer> it = nums.iterator();      // FIX 1: remove via the iterator
while (it.hasNext()) {
    if (it.next() % 2 == 0) it.remove();
}
nums.removeIf(n -> n % 2 == 0);              // FIX 2 (Java 8): simplest
List<Integer> odds = nums.stream()           // FIX 3: build a new list
        .filter(n -> n % 2 != 0).toList();
for (Integer n : new ArrayList<>(nums)) { }  // FIX 4: iterate over a copy
```

Fail-safe (more precisely, *snapshot* or *weakly consistent*) iterators come from concurrent collections. CopyOnWriteArrayList iterates over a **snapshot** taken when the iterator was created; ConcurrentHashMap iterators are **weakly consistent**: they never throw CME and may or may not reflect later changes.

```
List<String> cow = new CopyOnWriteArrayList<>(List.of("A", "B"));
for (String s : cow) cow.add("X");     // no CME: the loop sees only the snapshot [A, B]
System.out.println(cow);               // [A, B, X, X]
```

	Fail-fast	Snapshot / weakly consistent ("fail-safe")
Collections	ArrayList, LinkedList, HashMap, HashSet, TreeMap	CopyOnWriteArrayList/Set, ConcurrentHashMap, ConcurrentLinkedQueue
On concurrent modification	Throws CME (best effort)	No exception
Sees changes made during iteration	n/a	Snapshot: never; weakly consistent: maybe
Memory / cost	No extra cost	Copy-on-write cost, or concurrent design

13.6 Set: HashSet, LinkedHashSet, TreeSet

A `Set` stores **unique** elements, so `add` returns `false` for a duplicate.

```
Set<String> hs = new HashSet<>(List.of("Pune", "Delhi", "Mumbai"));
hs.add("Delhi");                    // false: already present
hs.add(null);                       // one null allowed
System.out.println(hs);             // order NOT guaranteed, e.g. [null, Delhi, Pune, Mumbai]

Set<String> lhs = new LinkedHashSet<>(List.of("Pune", "Delhi", "Mumbai"));
System.out.println(lhs);            // [Pune, Delhi, Mumbai]  insertion order

Set<String> ts = new TreeSet<>(List.of("Pune", "Delhi", "Mumbai"));
System.out.println(ts);             // [Delhi, Mumbai, Pune]  sorted
// ts.add(null);                    // NullPointerException
```

Feature	HashSet	LinkedHashSet	TreeSet
Backed by	`HashMap` (elements stored as keys)	`LinkedHashMap`	`TreeMap` (red-black tree)
Order	None (hash order)	**Insertion** order	**Sorted** (natural or Comparator)
`add` / `contains` / `remove`	O(1) average	O(1) average	**O(log n)**
Null	One null	One null	**No** (NPE)
Uniqueness decided by	`hashCode()` + `equals()`	`hashCode()` + `equals()`	`compareTo()`/`compare()` **== 0**
Extra interfaces	n/a	`SequencedSet` (21)	`NavigableSet`, `SortedSet`

PITFALL

Custom objects in a `HashSet` **must** override `equals()` **and** `hashCode()` (Phase 8), otherwise "duplicates" get in. In a `TreeSet` they must be `Comparable` (or you must supply a `Comparator`), otherwise you get a `ClassCastException` on the first add. A `TreeSet` treats elements with `compare == 0` as **duplicates**, even if `equals` says they differ.

TreeSet navigation (NavigableSet)

```
TreeSet<Integer> scores = new TreeSet<>(List.of(45, 72, 88, 91, 60));   // [45, 60, 72, 88, 91]
scores.first();        // 45            scores.last();          // 91
scores.floor(70);      // 60  (<= 70)   scores.ceiling(70);     // 72 (>= 70)
scores.lower(72);      // 60  (< 72)    scores.higher(72);      // 88 (> 72)
scores.headSet(72);    // [45, 60]      (< 72)
scores.tailSet(72);    // [72, 88, 91]  (>= 72)
scores.subSet(60, 90); // [60, 72, 88]  [60, 90)
scores.descendingSet();// [91, 88, 72, 60, 45]
scores.pollFirst();    // removes and returns 45
```

Set algebra

```
Set<Integer> a = new HashSet<>(Set.of(1, 2, 3, 4));
Set<Integer> b = Set.of(3, 4, 5);
Set<Integer> union = new HashSet<>(a);        union.addAll(b);       // [1, 2, 3, 4, 5]
Set<Integer> inter = new HashSet<>(a);        inter.retainAll(b);    // [3, 4]
Set<Integer> diff  = new HashSet<>(a);        diff.removeAll(b);     // [1, 2]
Collections.disjoint(a, Set.of(9));           // true: no common elements

List<String> dupes = List.of("a", "b", "a", "c", "b");
List<String> unique = new ArrayList<>(new LinkedHashSet<>(dupes));   // [a, b, c] order kept
```

13.7 Queue and Deque

Operation	Throws an exception on failure	Returns a special value
Insert	`add(e)`: `IllegalStateException` if full	`offer(e)`: `false`
Remove head	`remove()`: `NoSuchElementException`	`poll()`: `null`
Examine head	`element()`: `NoSuchElementException`	`peek()`: `null`

```
Queue<String> q = new ArrayDeque<>();      // FIFO
q.offer("T1"); q.offer("T2"); q.offer("T3");
q.peek();      // T1 (not removed)
q.poll();      // T1 (removed)
q.poll(); q.poll();
q.poll();      // null (empty)
// q.remove(); // NoSuchElementException

Deque<String> dq = new ArrayDeque<>();     // double-ended: queue AND stack
dq.offerFirst("B"); dq.offerFirst("A"); dq.offerLast("C");   // [A, B, C]
dq.pollFirst(); dq.pollLast();                               // A, C
```

TIP

`ArrayDeque` is the recommended general-purpose **queue and stack**: faster than `LinkedList` and `Stack`, resizable, and with no capacity limit. It does **not** accept `null`.

PriorityQueue: a binary heap

Elements come out in **priority order** (natural order = smallest first, a min-heap), not insertion order.

```
PriorityQueue<Integer> pq = new PriorityQueue<>();
for (int x : new int[]{50, 20, 40, 10, 30}) pq.offer(x);
System.out.println(pq);            // [10, 20, 40, 50, 30]  heap layout, NOT sorted order!
while (!pq.isEmpty()) System.out.print(pq.poll() + " ");   // 10 20 30 40 50 (sorted removal)

PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Comparator.reverseOrder());

record Txn(String id, int amount) { }
PriorityQueue<Txn> highValueFirst =
        new PriorityQueue<>(Comparator.comparingInt(Txn::amount).reversed());
highValueFirst.add(new Txn("T1", 500));
highValueFirst.add(new Txn("T2", 90_000));
highValueFirst.poll();             // Txn[id=T2, amount=90000] -> review big ones first
```

	Complexity
`offer` / `poll`	O(log n)
`peek`	O(1)
`contains` / `remove(Object)`	O(n)

PITFALL

Printing or iterating a `PriorityQueue` does **not** show sorted order. Only repeated `poll()` returns elements in priority order. `PriorityQueue` doesn't allow `null` and isn't thread-safe (use `PriorityBlockingQueue`).

13.8 Comparable vs Comparator

Comparable: natural ordering, defined inside the class

```
public class Employee implements Comparable<Employee> {
    private final int id; private final String name; private final double salary;
    public Employee(int id, String name, double salary) {
        this.id = id; this.name = name; this.salary = salary;
    }
    public int getId() { return id; }
    public String getName() { return name; }
    public double getSalary() { return salary; }

    @Override
    public int compareTo(Employee other) {           // natural order: by id
        return Integer.compare(this.id, other.id);   // negative / 0 / positive
    }
    @Override public String toString() { return id + ":" + name + ":" + salary; }
}

List<Employee> emps = new ArrayList<>(List.of(
        new Employee(3, "Ravi", 50000), new Employee(1, "Zoya", 70000),
        new Employee(2, "Amit", 50000)));
Collections.sort(emps);          // uses compareTo -> by id: 1:Zoya, 2:Amit, 3:Ravi
```

Comparator: external, any number of orderings

```
emps.sort(Comparator.comparing(Employee::getName));                    // by name
emps.sort(Comparator.comparingDouble(Employee::getSalary).reversed()); // salary desc
emps.sort(Comparator.comparingDouble(Employee::getSalary)              // salary asc,
        .thenComparing(Employee::getName));                            // then name
emps.sort((a, b) -> a.getName().compareToIgnoreCase(b.getName()));    // lambda

Comparator<Employee> byNameNullsLast = Comparator.comparing(
        Employee::getName, Comparator.nullsLast(Comparator.naturalOrder()));

Comparator<Employee> legacy = new Comparator<>() {                     // pre-Java 8 style
    @Override public int compare(Employee a, Employee b) {
        return Double.compare(a.getSalary(), b.getSalary());
    }
};
TreeSet<Employee> bySalary = new TreeSet<>(legacy);   // sorted collections accept comparators
```

	Comparable<T>	Comparator<T>
Package	java.lang	java.util
Method	int compareTo(T other)	int compare(T a, T b)
Defined	**Inside** the class being sorted	**Outside**: a separate class, lambda or method reference
How many orderings	One ("natural" ordering)	Unlimited
Modifying the class	Required	Not required (works for classes you don't own)
Used by	Collections.sort(list), TreeSet, TreeMap, Arrays.sort	list.sort(cmp), new TreeSet<>(cmp), stream.sorted(cmp)
Examples	String, Integer, LocalDate, BigDecimal	Comparator.comparing(...), String.CASE_INSENSITIVE_ORDER

PITFALL

Don't write comparators as `return a.id - b.id;`. Subtraction can **overflow** for large or negative values and give the wrong sign. Use `Integer.compare(a, b)`, `Long.compare` or `Comparator.comparingInt`.

> **NOTE**
> Try to keep `compareTo` **consistent with equals** (`compareTo == 0` exactly when `equals` is true). `BigDecimal` is a known exception: `new BigDecimal("2.0")` and `"2.00"` compare equal but aren't `equals`, so a `TreeSet` keeps one of them and a `HashSet` keeps both.

13.9 The Collections Utility Class

`java.util.Collections` (note the **s**) contains static algorithms and wrappers. `Collection` (no s) is the **interface**.

Method	Purpose
`sort(list)` / `sort(list, cmp)`	Stable sort (TimSort), O(n log n)
`reverse(list)`, `shuffle(list)`, `swap(list, i, j)`, `rotate(list, d)`	Reorder
`max(coll)`, `min(coll)` (optionally with a comparator)	Extremes
`frequency(coll, o)`	Count occurrences
`binarySearch(list, key)`	O(log n) search on a **sorted** list
`reverseOrder()` / `reverseOrder(cmp)`	Descending comparator
`fill(list, v)`, `nCopies(n, v)`, `addAll(coll, a, b, c)`	Bulk helpers
`unmodifiableList/Set/Map(...)`	Read-only views
`synchronizedList/Set/Map(...)`	Thread-safe wrappers (lock around each call)
`emptyList()`, `singletonList(x)`, `disjoint(c1, c2)`	Small helpers

```
List<Integer> marks = new ArrayList<>(List.of(70, 95, 82, 95, 60));
Collections.sort(marks);                              // [60, 70, 82, 95, 95]
Collections.reverse(marks);                           // [95, 95, 82, 70, 60]
Collections.max(marks);                               // 95
Collections.frequency(marks, 95);                     // 2
Collections.sort(marks);
Collections.binarySearch(marks, 82);                  // 2
marks.sort(Collections.reverseOrder());               // [95, 95, 82, 70, 60]

List<String> safe = Collections.synchronizedList(new ArrayList<>());
synchronized (safe) {                                 // iteration still needs MANUAL locking
    for (String s : safe) { }
}
```

13.10 Predict the Output

```
List<Integer> l = new ArrayList<>(List.of(10, 20, 30));
l.remove(1);                        // removes the element AT INDEX 1
l.remove(Integer.valueOf(10));      // removes the VALUE 10
System.out.println(l);              // [30]

Set<String> s = new TreeSet<>(List.of("b", "B", "a"));
System.out.println(s);              // [B, a, b]  (uppercase sorts first)

Set<String> ci = new TreeSet<>(String.CASE_INSENSITIVE_ORDER);
ci.addAll(List.of("b", "B", "a"));
System.out.println(ci);             // [a, b]  ("B" equals "b" by compare: a duplicate)

Queue<Integer> pq = new PriorityQueue<>(List.of(5, 1, 3));
System.out.println(pq.poll() + " " + pq.peek());   // 1 3

Deque<Integer> st = new ArrayDeque<>();
st.push(1); st.push(2); st.push(3);
System.out.println(st.pop() + " " + st);           // 3 [2, 1]

List<String> fixed = Arrays.asList("x", "y");
fixed.set(0, "z");
System.out.println(fixed);                          // [z, y]
try { fixed.add("w"); }
catch (UnsupportedOperationException e) { System.out.println("UOE"); }   // UOE
```

13.11 Interview Questions

Q. What is the difference between Collection and Collections?

`Collection` is the root interface of List, Set and Queue. `Collections` is a utility class of static methods: sort, reverse, max, unmodifiable and synchronized wrappers.

Q. What is the difference between ArrayList and LinkedList?

ArrayList is a resizable array: O(1) random access, O(n) middle insertions or removals, compact memory. LinkedList is a doubly linked list: O(n) access, O(1) insert or remove at the ends or at an iterator position, more memory. ArrayList is the better default.

Q. How does ArrayList grow?

The default capacity of 10 is allocated on the first add. When full, a new array about 1.5× the size (`old + old/2`) is created and the elements are copied.

Q. What is the difference between ArrayList and Vector?

Vector is synchronized (thread-safe but slower), legacy, and doubles in size. ArrayList is unsynchronized and grows by 50%. Use concurrent collections when you need thread safety.

Q. What is the difference between Iterator and ListIterator?

Iterator works with any collection, forward only, and can remove. ListIterator is List-only, moves both directions, and can also `add`, `set` and report indexes.

Q. What is ConcurrentModificationException? How do you avoid it?

It is thrown by fail-fast iterators when a collection is structurally modified during iteration by anything other than the iterator. Avoid it with `iterator.remove()`, `removeIf()`, collecting to a new list, iterating over a copy, or concurrent collections.

Q. What is the difference between fail-fast and fail-safe iterators?

Fail-fast iterators (ArrayList, HashMap) throw CME on concurrent modification. Fail-safe iterators (CopyOnWriteArrayList's snapshot, ConcurrentHashMap's weakly consistent iterators) don't throw, and work on a snapshot or a best-effort live view.

Q. How does HashSet ensure uniqueness?

It is backed by a HashMap: elements are stored as keys with a dummy value, and uniqueness is decided by `hashCode()` then `equals()`.

Q. What is the difference between HashSet, LinkedHashSet and TreeSet?
No order with O(1) operations; insertion order with O(1) operations; sorted order with O(log n) operations. HashSet and LinkedHashSet allow one null; TreeSet doesn't allow null and uses `compareTo`/`compare` for uniqueness.

Q. What is the difference between Comparable and Comparator?
Comparable defines one natural ordering inside the class (`compareTo`). Comparator defines any number of external orderings (`compare`), and works for classes you can't modify. Java 8 adds `comparing`, `thenComparing` and `reversed`.

Q. What is the difference between `poll()` and `remove()` on a Queue?
Both remove the head. `poll()` returns `null` if the queue is empty; `remove()` throws `NoSuchElementException`. The same pattern holds for `peek()`/`element()` and `offer()`/`add()`.

Q. Why prefer ArrayDeque over Stack?
Stack is a legacy synchronized Vector subclass. ArrayDeque is faster, unsynchronized, and offers both stack (`push`/`pop`) and queue operations through the Deque interface.

Q. What is the difference between `List.of`, `Arrays.asList` and `Collections.unmodifiableList`?
`List.of` is truly immutable and rejects nulls. `Arrays.asList` is a fixed-size view of an array: `set` works and writes through, but add/remove don't. `unmodifiableList` is a read-only view of another list that still changes if the original changes.

13.12 Quick Revision

- JCF = interfaces (List, Set, Queue/Deque, Map) + implementations + algorithms. Map is **not** a Collection.
- **List**: ordered, indexed, duplicates. ArrayList (default, O(1) get, +50% growth); LinkedList (also a Deque); Vector/Stack are legacy.
- Immutable: `List.of`/`copyOf` (no nulls). `Arrays.asList` is fixed-size and writes through. `unmodifiableList` is a view.
- Iteration: for, for-each, Iterator, ListIterator (both directions; add/set), `forEach`, streams.
- Fail-fast → CME. Fix with `it.remove()`, `removeIf`, a copy, or concurrent collections (snapshot / weakly consistent).
- **Set**: unique. HashSet (no order, O(1)), LinkedHashSet (insertion order), TreeSet (sorted, O(log n), no null, uniqueness by compare).
- Custom elements: `equals` + `hashCode` for hash sets; `Comparable`/`Comparator` for TreeSet.
- **Queue**: `offer`/`poll`/`peek` (return special values) vs `add`/`remove`/`element` (throw). `ArrayDeque` for queue and stack.
- PriorityQueue = min-heap; only `poll()` order is sorted; use a reversed comparator for a max-heap.
- `Comparable.compareTo` (one natural order) vs `Comparator.compare` (many). Use `Integer.compare`, never subtraction.
- `Collections`: sort, reverse, shuffle, max, frequency, binarySearch (on sorted lists), unmodifiable and synchronized wrappers.

PHASE 14 OF 18

Collections II: Maps & Concurrent Collections

The Map API, HashMap internals (hashing, buckets, collisions, treeification, resizing), custom keys, LinkedHashMap and an LRU cache, TreeMap navigation, Hashtable, IdentityHashMap, WeakHashMap, ConcurrentHashMap, sequenced collections, choosing a collection, and Big-O cheat sheets

IN THIS PHASE

14.1 The Map Interface
14.2 HashMap Internals
14.3 Custom Objects as Keys
14.4 LinkedHashMap: Predictable Order and an LRU Cache
14.5 TreeMap: Sorted Keys and Range Queries
14.6 Other Map Implementations
14.7 Map Implementations Compared
14.8 Thread-Safe Collections
14.9 Sequenced Collections (Java 21)
14.10 Choosing the Right Collection
14.11 Big-O Cheat Sheet
14.12 Classic Map Programs
14.13 Predict the Output
14.14 Interview Questions
14.15 Quick Revision

14.1 The Map Interface

A **Map** stores **key → value** pairs (entries). Keys are **unique**; values may repeat. `Map` is part of the Collections Framework but does **not** extend `Collection`.

```
Map<String, Integer> stock = new HashMap<>();
stock.put("APPLE", 50);                // add
stock.put("MANGO", 20);
stock.put("APPLE", 70);                // same key -> value REPLACED (returns old value 50)
stock.get("APPLE");                    // 70
stock.get("KIWI");                     // null (absent)
stock.getOrDefault("KIWI", 0);         // 0
stock.containsKey("MANGO");            // true
stock.containsValue(20);               // true (O(n) scan)
stock.remove("MANGO");                 // returns 20
stock.size();                          // 1
```

Map methods (including the Java 8 additions)

Method	Purpose
`put(k, v)`	Insert or replace; returns the previous value or `null`
`get(k)`, `getOrDefault(k, def)`	Lookup
`containsKey(k)`, `containsValue(v)`	Checks (key O(1) in a HashMap; value O(n))
`remove(k)`, `remove(k, v)`	Delete (the second only if mapped to v)
`putIfAbsent(k, v)`	Insert only if the key is missing or null
`computeIfAbsent(k, k -> newV)`	Create the value lazily if missing (great for multimaps)
`computeIfPresent(k, (k, v) -> ...)`	Update only if present (returning null removes the entry)
`compute(k, (k, v) -> ...)`	Always recompute
`merge(k, v, (old, v) -> ...)`	Insert v, or combine with the existing value (counters, sums)
`replace(k, v)`, `replaceAll((k, v) -> ...)`	Update values
`keySet()`, `values()`, `entrySet()`	**Views** of keys, values and entries (removing from a view removes from the map)
`forEach((k, v) -> ...)`	Iterate
`Map.of(...)`, `Map.entry(k, v)`, `Map.copyOf(m)`, `Map.ofEntries(...)`	Immutable maps (Java 9/10); no null keys or values

```
Map<String, Integer> wordCount = new HashMap<>();
for (String w : "to be or not to be".split(" ")) {
    wordCount.merge(w, 1, Integer::sum);              // counting in one line
}
// {not=1, or=1, be=2, to=2}  (HashMap order is not guaranteed)

Map<String, List<String>> branchCustomers = new HashMap<>();
branchCustomers.computeIfAbsent("PUNE", k -> new ArrayList<>()).add("Rohit");   // multimap
branchCustomers.computeIfAbsent("PUNE", k -> new ArrayList<>()).add("Amit");
// {PUNE=[Rohit, Amit]}
```

Iterating a Map

```
Map<String, Double> balances = new LinkedHashMap<>();
balances.put("SB-1", 2500.0); balances.put("SB-2", 9100.0);

for (Map.Entry<String, Double> e : balances.entrySet()) {     // BEST: key + value together
    System.out.println(e.getKey() + " -> " + e.getValue());
}
balances.forEach((acct, bal) -> System.out.println(acct + " = " + bal));   // Java 8
for (String key : balances.keySet()) { balances.get(key); }  // works, extra lookup
balances.entrySet().removeIf(e -> e.getValue() < 5000);       // safe removal while iterating
balances.replaceAll((k, v) -> v * 1.04);                      // bulk update
```

`Map.Entry<K,V>` is the nested interface representing one pair: `getKey()`, `getValue()`, `setValue(v)` (writes through to the map, for mutable maps).

14.2 HashMap Internals

`HashMap` is an **array of buckets** (`Node<K,V>[] table`). Each node stores `hash`, `key`, `value` and `next`.

```
table (capacity 16)             a bucket holds a linked list (or a red-black tree)
+----+
| 0  | -> [hash|null|V] (the null key always lives in bucket 0)
| 1  |
| 2  | -> [hash|"Ravi"|101] -> [hash|"Amit"|104]     <- collision: same bucket index
| 3  |
| .. |
| 9  | -> [hash|"Zoya"|102]
| 15 |
+----+
```

put(key, value), step by step

1. **Hash the key**: `h = key.hashCode(); hash = h ^ (h >>> 16)`. This spreads the high bits into the low bits so that more of the hash affects the index.
2. **Find the bucket**: `index = (n - 1) & hash`, where `n` is the table size (always a power of 2). This is a fast equivalent of `hash % n`.
3. **Empty bucket** → place a new node there.
4. **Occupied bucket** → walk the nodes. If a node has the **same hash and** `equals()` key, **replace its value**. Otherwise append a new node (a **collision**).
5. If a bucket's list reaches **8 nodes** (`TREEIFY_THRESHOLD`) **and** the table capacity is **≥ 64**, the list becomes a **red-black tree** (O(log n) lookup). If capacity is below 64, the table resizes instead. A tree shrinks back to a list at **6** nodes.
6. Increment the size. If `size > capacity × loadFactor` (16 × 0.75 = **12**), **resize**: double the capacity and redistribute the nodes. Each node either stays at index `i` or moves to `i + oldCapacity`.

`get(key)` repeats steps 1–2, then checks nodes in that bucket using `hash` and `equals()`.

Constant	Default	Meaning
Initial capacity	**16** (allocated on the first `put`)	Number of buckets (always a power of 2)
Load factor	**0.75**	Resize when size exceeds capacity × 0.75
Threshold	12	First resize happens at the 13th entry
`TREEIFY_THRESHOLD`	8	List → tree in one bucket
`UNTREEIFY_THRESHOLD`	6	Tree → list
`MIN_TREEIFY_CAPACITY`	64	Below this, resize instead of treeifying

Operation	Average	Worst case
`put`, `get`, `remove`, `containsKey`	**O(1)**	O(log n) since Java 8 (tree bins); O(n) before Java 8
Iteration	O(capacity + size)	n/a

Key properties

- One `null` key (stored in bucket 0) and any number of `null` values.
- **No ordering guarantee**: the order can even change after a resize.
- Not synchronized. Concurrent modification can corrupt it or lose updates; use `ConcurrentHashMap`.
- Pre-size big maps to avoid repeated resizing: `new HashMap<>(expected * 4 / 3 + 1)`, or `HashMap.newHashMap(expected)` in Java 19+.

> **REMEMBER**
> A HashMap's correctness depends entirely on the key's `hashCode()` **and** `equals()`. Equal keys must have equal hash codes. A bad `hashCode` (for example `return 1;`) still works correctly but piles every entry into one bucket, which makes the map slow.

14.3 Custom Objects as Keys

```
final class AccountKey {                          // immutable key class
    private final String ifsc, accountNo;
    AccountKey(String ifsc, String accountNo) { this.ifsc = ifsc; this.accountNo = accountNo; }
    @Override public boolean equals(Object o) {
        return o instanceof AccountKey k
            && ifsc.equals(k.ifsc) && accountNo.equals(k.accountNo);
    }
    @Override public int hashCode() { return Objects.hash(ifsc, accountNo); }
}

Map<AccountKey, BigDecimal> ledger = new HashMap<>();
ledger.put(new AccountKey("HDFC0001", "1001"), new BigDecimal("500.00"));
ledger.get(new AccountKey("HDFC0001", "1001"));    // 500.00: found via equal key
// Without equals/hashCode -> null, because a new key object is a "different" key

record CardKey(String network, String last4) { }   // records generate equals/hashCode
Map<CardKey, String> owners = Map.of(new CardKey("VISA", "4417"), "Rohit");
```

PITFALL

If you **mutate** a key's fields after inserting it, its hash changes, so `get` looks in the wrong bucket and returns `null`, and the entry becomes unreachable (a memory leak). Use **immutable keys**: `String`, wrappers, enums, records, or final classes.

14.4 LinkedHashMap: Predictable Order and an LRU Cache

`LinkedHashMap` extends `HashMap` and adds a **doubly linked list** through all entries, so it keeps:

- **Insertion order** (the default), or
- **Access order** (`new LinkedHashMap<>(cap, 0.75f, true)`): each `get`/`put` moves the entry to the end.

```
Map<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("C", 3); lhm.put("A", 1); lhm.put("B", 2);
System.out.println(lhm);                  // {C=3, A=1, B=2}  insertion order

class LruCache<K, V> extends LinkedHashMap<K, V> {          // classic interview question
    private final int capacity;
    LruCache(int capacity) {
        super(16, 0.75f, true);                              // accessOrder = true
        this.capacity = capacity;
    }
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;                            // evict the least recently used
    }
}

LruCache<String, String> cache = new LruCache<>(2);
cache.put("A", "1"); cache.put("B", "2");
cache.get("A");                           // A becomes most recently used
cache.put("C", "3");                      // evicts B (least recently used)
System.out.println(cache.keySet());       // [A, C]
```

14.5 TreeMap: Sorted Keys and Range Queries

`TreeMap` stores entries in a **red-black tree** sorted by key (natural order or a `Comparator`). Operations are **O(log n)**. Null keys aren't allowed (null values are). It implements `NavigableMap`.

```
TreeMap<Integer, String> tm = new TreeMap<>(Map.of(30, "C", 10, "A", 20, "B"));
System.out.println(tm);                    // {10=A, 20=B, 30=C}  sorted by key
tm.firstKey();  tm.lastEntry();            // 10, 30=C
tm.headMap(20); tm.tailMap(20);            // {10=A}, {20=B, 30=C}
tm.subMap(10, 30);                         // {10=A, 20=B}  [from, to)
tm.descendingMap();                        // {30=C, 20=B, 10=A}
tm.floorKey(25); tm.ceilingKey(25);        // 20, 30

// Real use: slab lookup (fees or interest by amount slab)
TreeMap<Integer, Double> fdRates = new TreeMap<>();
fdRates.put(0, 3.5);          // 0 to 99,999      -> 3.5%
fdRates.put(100_000, 6.8);    // 1 L to 9.99 L    -> 6.8%
fdRates.put(1_000_000, 7.25); // 10 L and above   -> 7.25%
double rate = fdRates.floorEntry(250_000).getValue();   // 6.8 (greatest key <= 2.5 L)

TreeMap<String, Integer> byName = new TreeMap<>(String.CASE_INSENSITIVE_ORDER);
```

14.6 Other Map Implementations

Hashtable (legacy)

A synchronized hash table from Java 1.0. Every method is `synchronized` (one lock for the whole table), and it allows **no null key and no null value**.

Feature	HashMap	Hashtable
Since	1.2 (Collections Framework)	1.0 (legacy)
Thread-safe	No	Yes (method-level `synchronized`)
Performance	Faster	Slower (contention)
Null keys / values	One null key, many null values	**None** (NPE)
Iteration	Fail-fast `Iterator`	`Enumeration` + Iterator
Superclass	`AbstractMap`	`Dictionary` (obsolete)
Recommendation	Single-threaded use	Use `ConcurrentHashMap` instead

IdentityHashMap

Compares keys with `==` (reference identity) and `System.identityHashCode()` instead of `equals`/`hashCode`.

```
Map<String, Integer> hm = new HashMap<>();
Map<String, Integer> ihm = new IdentityHashMap<>();
String k1 = new String("Ashok"), k2 = new String("Ashok");    // equal, different objects
hm.put(k1, 101);  hm.put(k2, 102);
ihm.put(k1, 101); ihm.put(k2, 102);
System.out.println(hm.size() + " " + ihm.size());           // 1 2
```

It is used for object-graph algorithms (serialization, deep copy) where *identity* matters.

WeakHashMap

Holds keys through **weak references**. When a key has no other strong reference, the GC may collect it, and its entry **disappears** from the map. It is used for caches and metadata attached to objects with an independent lifetime.

```
Map<Object, String> meta = new WeakHashMap<>();
Object session = new Object();
meta.put(session, "user=rohit");
session = null;                  // no strong reference left
System.gc();                     // only a REQUEST: removal is likely but not guaranteed
System.out.println(meta.size()); // usually 0 after GC (not guaranteed)
```

> **NOTE**
> `EnumMap` (Phase 10) is the fastest map when keys are enum constants. `Properties` (Phase 12) is a legacy `Hashtable<Object,Object>` subclass for String configuration.

14.7 Map Implementations Compared

Map	Ordering	Null key / values	Thread-safe	get/put	Underlying structure
`HashMap`	None	1 / many	No	O(1)	Hash table + tree bins
`LinkedHashMap`	Insertion or access	1 / many	No	O(1)	Hash table + linked list
`TreeMap`	**Sorted** by key	**No** / many	No	O(log n)	Red-black tree
`Hashtable`	None	No / No	Yes (coarse lock)	O(1)	Hash table
`ConcurrentHashMap`	None	**No / No**	**Yes** (fine-grained)	O(1)	Hash table + CAS + per-bin locks
`ConcurrentSkipListMap`	**Sorted**	No / No	Yes	O(log n)	Skip list
`EnumMap`	Enum declaration order	No / many	No	O(1)	Array indexed by ordinal
`IdentityHashMap`	None	1 / many	No	O(1)	Linear-probe table, `==` keys
`WeakHashMap`	None	1 / many	No	O(1)	Hash table with weak keys

14.8 Thread-Safe Collections

Option	How it's thread-safe	Notes
`Collections.synchronizedMap/List/Set(x)`	One lock around every method	Iteration still needs a manual `synchronized(x)` block; compound actions aren't atomic
`Hashtable`, `Vector`	Synchronized methods	Legacy; same limitations
`ConcurrentHashMap`	CAS + locking per bucket (bin)	High concurrency; atomic `merge`/`compute`; weakly consistent iterators
`CopyOnWriteArrayList` / `CopyOnWriteArraySet`	Copies the whole array on each write	Very fast reads, slow writes; snapshot iterators
`ConcurrentLinkedQueue` / `ConcurrentLinkedDeque`	Lock-free (CAS)	Non-blocking queues
`BlockingQueue` (`ArrayBlockingQueue`, `LinkedBlockingQueue`, `PriorityBlockingQueue`, `DelayQueue`)	Locks + conditions	Producer–consumer (Phase 11)
`ConcurrentSkipListMap` / `Set`	Lock-free skip list	Concurrent **sorted** collections

ConcurrentHashMap in depth

- **Java 7**: the table was split into 16 **segments**, each with its own lock (so up to 16 concurrent writers).
- **Java 8+**: segments are gone. An empty bin is filled with **CAS** (no lock); otherwise the thread does `synchronized` on that **bin's first node** only. Reads are **lock-free** (volatile reads). Resizing is done

cooperatively by several threads. Tree bins exist here too.

- **No null keys or values**: `get(k)` returning `null` must unambiguously mean "absent". In a concurrent map you can't safely follow it with a `containsKey` check, because another thread may change the map in between.
- Iterators are **weakly consistent**: no CME, and they may or may not show concurrent updates.
- `size()` is an estimate under concurrent updates; `mappingCount()` returns a `long`.

```
ConcurrentHashMap<String, Long> hits = new ConcurrentHashMap<>();

// WRONG under concurrency: check-then-act race between get and put
Long cur = hits.get("/login");
hits.put("/login", cur == null ? 1 : cur + 1);

// RIGHT: atomic per-key operations
hits.merge("/login", 1L, Long::sum);
hits.compute("/pay", (k, v) -> v == null ? 1 : v + 1);
hits.computeIfAbsent("/home", k -> 0L);
hits.putIfAbsent("/logout", 0L);
```

PITFALL

Even with a thread-safe map, `if (!map.containsKey(k)) map.put(k, v);` is a **race** (two separate calls). Use the atomic methods `putIfAbsent`, `computeIfAbsent` and `merge`. The same applies to `Collections.synchronizedMap`.

Fail-fast vs fail-safe (recap)

	Fail-fast	Fail-safe (snapshot / weakly consistent)
Examples	`HashMap`, `ArrayList`, `HashSet`, `TreeMap`	`ConcurrentHashMap`, `CopyOnWriteArrayList`, `ConcurrentSkipListMap`
Modify during iteration	`ConcurrentModificationException`	Allowed, no exception
Mechanism	`modCount` check	Snapshot copy (COW) or concurrent design

14.9 Sequenced Collections (Java 21)

New interfaces give a **uniform API for the first and last elements** of ordered collections.

```
List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
list.getFirst(); list.getLast();         // 1, 3   (instead of get(0), get(size()-1))
list.addFirst(0); list.removeLast();     // [0, 1, 2]
list.reversed();                         // [2, 1, 0]: a reversed VIEW

LinkedHashMap<String, Integer> lhm = new LinkedHashMap<>();
lhm.put("A", 1); lhm.put("B", 2); lhm.put("C", 3);
lhm.firstEntry(); lhm.lastEntry();       // A=1, C=3
lhm.pollFirstEntry();                    // removes A=1
lhm.putFirst("Z", 26);                   // {Z=26, B=2, C=3}
lhm.sequencedKeySet().reversed();        // [C, B, Z]
```

Interface	Implemented by
`SequencedCollection`	`List`, `Deque`, `LinkedHashSet`, `SortedSet`
`SequencedSet`	`LinkedHashSet`, `SortedSet`/`TreeSet`
`SequencedMap`	`LinkedHashMap`, `SortedMap`/`TreeMap`

14.10 Choosing the Right Collection

Need	Use
Ordered list, fast index access (the default list)	`ArrayList`
Queue or stack	`ArrayDeque`
Process by priority (top-k, scheduling)	`PriorityQueue`
Unique elements, fast lookup	`HashSet`
Unique + insertion order (dedupe while keeping order)	`LinkedHashSet`
Unique + sorted + range queries	`TreeSet`
Key-value lookup (the default map)	`HashMap`
Key-value + insertion order / LRU cache	`LinkedHashMap`
Key-value + sorted keys / slabs / ranges	`TreeMap`
Enum keys or elements	`EnumMap` / `EnumSet`
Multi-threaded map, counters, caches	`ConcurrentHashMap`
Read-mostly list shared across threads	`CopyOnWriteArrayList`
Producer–consumer hand-off	`ArrayBlockingQueue` / `LinkedBlockingQueue`
Fixed constants that must not change	`List.of` / `Set.of` / `Map.of`

14.11 Big-O Cheat Sheet

Collection	add / put	get / lookup	contains	remove	Ordered?
ArrayList	O(1)* end, O(n) middle	O(1) by index	O(n)	O(n)	Insertion
LinkedList	O(1) ends	O(n)	O(n)	O(1) at ends / iterator	Insertion
ArrayDeque	O(1)* ends	O(1) ends	O(n)	O(1) ends	Insertion
PriorityQueue	O(log n)	O(1) peek	O(n)	O(log n) poll	Heap order
HashSet / HashMap	O(1)	O(1)	O(1)	O(1)	No
LinkedHashSet / LinkedHashMap	O(1)	O(1)	O(1)	O(1)	Insertion / access
TreeSet / TreeMap	O(log n)	O(log n)	O(log n)	O(log n)	Sorted
ConcurrentHashMap	O(1)	O(1)	O(1)	O(1)	No

(* = amortised. Hash-based worst case is O(log n) with tree bins.)

14.12 Classic Map Programs

```
// 1. Character frequency, sorted by character
Map<Character, Integer> freq = new TreeMap<>();
for (char c : "banking".toCharArray()) freq.merge(c, 1, Integer::sum);
System.out.println(freq);                       // {a=1, b=1, g=1, i=1, k=1, n=2}

// 2. First non-repeating character (LinkedHashMap keeps the order)
Map<Character, Integer> lf = new LinkedHashMap<>();
for (char c : "swiss".toCharArray()) lf.merge(c, 1, Integer::sum);
lf.entrySet().stream().filter(e -> e.getValue() == 1)
  .findFirst().ifPresent(e -> System.out.println(e.getKey()));   // w

// 3. Two-sum in O(n): indices of two numbers adding up to the target
int[] nums = {2, 7, 11, 15}; int target = 9;
Map<Integer, Integer> seen = new HashMap<>();
for (int i = 0; i < nums.length; i++) {
    Integer j = seen.get(target - nums[i]);
    if (j != null) { System.out.println(j + "," + i); break; }   // 0,1
    seen.put(nums[i], i);
}

// 4. Group anagrams
Map<String, List<String>> groups = new HashMap<>();
for (String w : List.of("eat", "tea", "tan", "ate", "nat")) {
    char[] k = w.toCharArray(); Arrays.sort(k);
    groups.computeIfAbsent(new String(k), x -> new ArrayList<>()).add(w);
}
// {aet=[eat, tea, ate], ant=[tan, nat]}

// 5. Sort a map by value (descending)
Map<String, Integer> sales = Map.of("Pune", 40, "Delhi", 75, "Goa", 20);
sales.entrySet().stream()
     .sorted(Map.Entry.<String, Integer>comparingByValue().reversed())
     .forEach(e -> System.out.print(e.getKey() + " "));       // Delhi Pune Goa
```

14.13 Predict the Output

```
Map<String, Integer> m = new HashMap<>();
System.out.println(m.put("a", 1) + " " + m.put("a", 2) + " " + m.get("a"));  // null 1 2

Map<String, Integer> tm = new TreeMap<>(Map.of("b", 2, "a", 1, "c", 3));
System.out.println(tm.keySet());                    // [a, b, c]

Map<Integer, String> h = new HashMap<>();
h.put(null, "x"); h.put(1, null);
System.out.println(h.get(null) + " " + h.containsKey(1) + " " + h.get(1));  // x true null

try { new ConcurrentHashMap<String, String>().put("k", null); }
catch (NullPointerException e) { System.out.println("CHM null NPE"); }      // CHM null NPE

try { new Hashtable<String, String>().put(null, "v"); }
catch (NullPointerException e) { System.out.println("Hashtable null NPE"); }

Map<String, Integer> counts = new HashMap<>();
counts.merge("x", 5, Integer::sum);
counts.merge("x", 5, Integer::sum);
System.out.println(counts);                         // {x=10}

try {
    Map<String, Integer> map = new HashMap<>(Map.of("a", 1, "b", 2));
    for (String k : map.keySet()) map.remove(k);
} catch (ConcurrentModificationException e) { System.out.println("CME"); }  // CME
```

14.14 Interview Questions

Q. How does HashMap work internally?

It keeps an array of buckets. `put` computes `hash = h ^ (h >>> 16)` from `hashCode()`, picks the bucket `(n-1) & hash`, and either inserts a node or walks the bucket's chain comparing hash and `equals()` to replace an existing key's value. Chains become red-black trees at 8 nodes (when capacity ≥ 64). When size exceeds capacity × 0.75, it doubles and redistributes.

Q. What is a collision, and how does HashMap handle it?

Different keys landing in the same bucket. HashMap chains them in a linked list, and converts long chains to balanced trees (Java 8+), so the worst-case lookup is O(log n) instead of O(n).

Q. What are the load factor and the initial capacity? What happens on resize?

The defaults are 16 buckets and a load factor of 0.75 (threshold 12). Exceeding the threshold doubles the table; each node either stays at its index or moves to index + oldCapacity.

Q. Why must keys override `equals()` and `hashCode()`? Why should keys be immutable?

Lookups find the bucket with `hashCode()` and match keys with `equals()`. If a key's fields change after insertion, its hash no longer matches its bucket, so the entry can't be found.

Q. Can HashMap have a null key? What about ConcurrentHashMap and Hashtable?

HashMap allows one null key and many null values. ConcurrentHashMap and Hashtable allow neither. For ConcurrentHashMap this prevents ambiguity of `get` returning null under concurrency.

Q. What is the difference between HashMap, LinkedHashMap and TreeMap?

No order with O(1) operations; insertion (or access) order with O(1) operations; sorted keys with O(log n) operations and navigation methods.

Q. How would you implement an LRU cache?

Extend `LinkedHashMap` with `accessOrder = true` and override `removeEldestEntry` to return `size() > capacity`. (Without that class: a HashMap plus a doubly linked list.)

Q. What is the difference between HashMap and Hashtable?

Hashtable is legacy and synchronized on every method, and allows no nulls. HashMap is unsynchronized, faster, allows one null key, and has fail-fast iterators. Use ConcurrentHashMap for thread safety.

Q. How is ConcurrentHashMap thread-safe? How did it change in Java 8?

Java 7 used segment locks. Java 8 uses CAS for empty bins and synchronizes only on the head node of an affected bin; reads are lock-free and resizing is cooperative. Compound operations like `merge`/`compute` are atomic per key.

Q. What is the difference between `Collections.synchronizedMap` and ConcurrentHashMap?

`synchronizedMap` wraps every call in one lock, so throughput is low and iteration requires manual locking. ConcurrentHashMap allows concurrent reads and writes to different bins, has weakly consistent iterators, and offers atomic compound methods.

Q. What are IdentityHashMap and WeakHashMap used for?

IdentityHashMap compares keys by reference (`==`), which suits identity-sensitive algorithms. WeakHashMap holds weak keys, so entries vanish once a key is otherwise unreachable, which suits metadata caches.

Q. What is the difference between `putIfAbsent` and `computeIfAbsent`?

`putIfAbsent(k, v)` takes an already-created value (it is evaluated even if unused). `computeIfAbsent(k, fn)` creates the value lazily, only when the key is missing, which is ideal for expensive objects and multimaps.

14.15 Quick Revision

- Map = unique keys → values; not a Collection. `put` replaces and returns the old value.
- Java 8 methods: `getOrDefault`, `putIfAbsent`, `computeIfAbsent` (multimap), `merge` (counters), `compute`, `replaceAll`.
- Iterate with `entrySet()`; remove safely with `entrySet().removeIf(...)`.
- HashMap: Node[] buckets; `hash ^ (hash >>> 16)`; index `(n-1) & hash`; collisions chain → tree at 8 (capacity ≥ 64); load factor 0.75; resize ×2; one null key.
- Keys: correct `equals` + `hashCode`, immutable (String, records, enums).
- LinkedHashMap: insertion or access order; LRU = access order + `removeEldestEntry`.
- TreeMap: sorted keys, O(log n), `floorEntry`/`ceilingKey`/`headMap`/`tailMap`/`subMap`; no null keys.
- Hashtable: legacy, synchronized, no nulls → use ConcurrentHashMap.
- IdentityHashMap uses `==`; WeakHashMap entries disappear after the key is GC'd.
- ConcurrentHashMap (Java 8): CAS + per-bin locks, lock-free reads, no nulls, atomic `merge`/`compute`, weakly consistent iterators.
- Check-then-act on any map is a race → use `putIfAbsent`, `computeIfAbsent`, `merge`.
- Java 21 sequenced collections: `getFirst`/`getLast`/`reversed`, `firstEntry`/`putFirst` on LinkedHashMap.
- Defaults: `ArrayList`, `HashMap`, `HashSet`, `ArrayDeque`; sorted → Tree*; ordered → Linked*; concurrent → `ConcurrentHashMap`.

PHASE 15 OF 18

JVM Deep Dive & Modern Java (8 → 25)

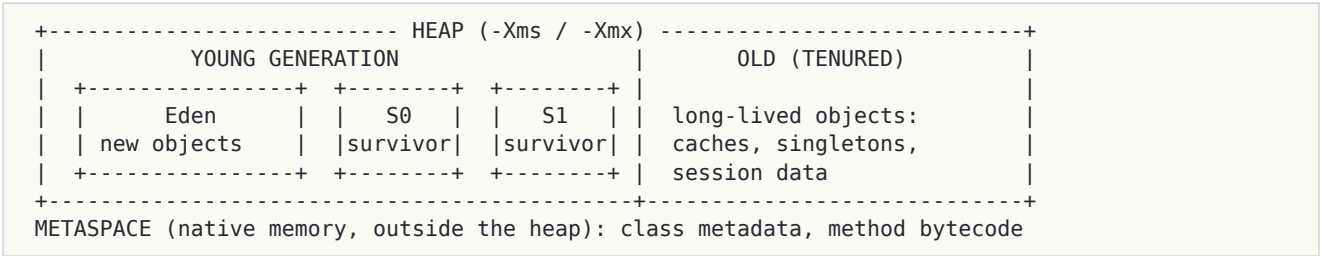
Heap generations and garbage collection, GC roots, collectors and tuning, reference types, class loading in depth, lambdas, functional interfaces, method references, Streams and Collectors, Optional, java.time, records, sealed classes, a version feature map and a rapid-fire interview round

IN THIS PHASE

15.1 Heap Structure and Object Lifecycle
15.2 How Garbage Collection Works
15.3 Class Loading in Depth
15.4 Lambda Expressions (Java 8)
15.5 Built-in Functional Interfaces (java.util.function)
15.6 Method References
15.7 Streams API
15.8 Optional (Java 8)
15.9 Date and Time API
15.10 Records (Java 16)
15.11 Sealed Classes (Java 17) and Pattern Matching
15.12 Modern Java Feature Map (8 → 25)
15.13 Rapid-Fire Interview Round (all phases)
15.14 Quick Revision

15.1 Heap Structure and Object Lifecycle

Most objects **die young** (temporary strings, DTOs, iterators). The HotSpot heap is therefore split into **generations**, so the GC can clean short-lived objects cheaply and often.



1. New objects are allocated in **Eden**.
2. When Eden fills up, a **Minor GC** runs. Live objects are copied to a Survivor space (S0 or S1), and Eden is wiped in one go.
3. Survivors bounce between S0 and S1. Each GC they survive increases their **age**.
4. Objects that reach the tenuring threshold (age up to 15) are **promoted** to the **Old generation**. Very large objects may go there directly.
5. When the Old generation fills, a **Major/Full GC** runs. It is slower, with longer pauses.

GC event	Cleans	Typical cost
Minor / Young GC	Young generation	Frequent, fast (copying live objects only)
Major / Old GC	Old generation	Less frequent, slower
Full GC	Whole heap (+ Metaspace)	Slowest; long pauses: a red flag in production

15.2 How Garbage Collection Works

GC roots and reachability

An object is **alive** if it is **reachable** from a **GC root** through a chain of strong references. Everything else is garbage, including cycles (the **island of isolation**).

GC roots include: local variables and parameters in active thread stacks, static fields of loaded classes, active threads, JNI references, and objects used as locks by running code.

Mark, Sweep, Compact

```
Before GC:      [A] [B] [C] [D] [E]          A, C, E reachable; B, D unreachable
1. Mark:        [A*][B ][C*][D ][E*]         walk from GC roots, mark live objects
2. Sweep:       [A ][  ][C ][  ][E ]         reclaim the memory of unmarked objects
3. Compact:     [A ][C ][E ][  free  ]       slide survivors together, no fragmentation
```

- **Marking** traverses the object graph from the GC roots.
- **Sweeping** reclaims unmarked memory, which can leave the heap **fragmented**.
- **Compaction** moves live objects together so large allocations still fit, updating references.
- **Copying** (used in the young generation) copies live objects to a fresh space and implicitly compacts.

Not every collector does these steps the same way; modern collectors do most of the marking **concurrently**, while your application keeps running.

Stop-the-World pauses

During some GC phases, the JVM pauses **all application threads** (a **Stop-the-World**, or STW, pause) so object references stay consistent. Long STW pauses cause latency spikes and timeouts, for example a payment API that suddenly takes 2 seconds. Choosing and tuning the collector is mostly about **pause time vs throughput**.

Garbage collectors in HotSpot

Collector	Flag	Style	Best for
Serial	`-XX:+UseSerialGC`	Single-threaded, STW	Small heaps, containers with 1 CPU, client tools
Parallel (Throughput)	`-XX:+UseParallelGC`	Multi-threaded, STW	Batch jobs where total throughput matters more than pauses
G1 (Garbage-First)	`-XX:+UseG1GC` (**default since Java 9**)	Region-based, mostly concurrent marking, pause-time target	General-purpose servers (`-XX:MaxGCPauseMillis=200`)
ZGC	`-XX:+UseZGC` (generational in 21+)	Concurrent, sub-millisecond pauses, huge heaps	Low-latency services (trading, payments)
Shenandoah	`-XX:+UseShenandoahGC`	Concurrent compaction, low pause	Low-latency (in OpenJDK builds)
Epsilon	`-XX:+UseEpsilonGC`	No-op: never collects	Performance testing, very short-lived jobs

> **NOTE**
> The CMS collector (Concurrent Mark Sweep) was deprecated in Java 9 and **removed in Java 14**. If you see `-XX:+UseConcMarkSweepGC` in old scripts, it must be migrated.

Diagnosing memory problems

```
-Xms2g -Xmx2g                          # equal min/max avoids resize pauses (common in prod)
-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/var/dumps
-Xlog:gc*:file=gc.log:time,uptime      # unified GC logging (Java 9+)

jcmd <pid> GC.heap_info                # heap usage
jcmd <pid> GC.heap_dump /tmp/h.hprof   # heap dump -> analyse in Eclipse MAT / VisualVM
jstat -gcutil <pid> 1000               # GC stats every second
jcmd <pid> JFR.start duration=60s filename=rec.jfr   # Java Flight Recorder profiling
```

Common memory leaks in Java: ever-growing `static` collections or caches with no eviction, listeners that are never removed, `ThreadLocal` values left on pool threads, unclosed resources, inner-class instances holding outer objects, and mutable `HashMap` keys.

Reference types

Reference	Collected when	Use
Strong (`Obj o = new Obj()`)	Never, while reachable	Normal code
`SoftReference`	Only when the JVM is low on memory	Memory-sensitive caches
`WeakReference`	At the next GC once weakly reachable	`WeakHashMap`, canonicalising maps, listener registries
`PhantomReference`	After finalization; `get()` always returns `null`	Post-mortem cleanup (`Cleaner`)

15.3 Class Loading in Depth

The class loader hierarchy

Loader	Loads	`getClassLoader()` **returns**
Bootstrap (native code)	Core modules: `java.base` (`java.lang`, `java.util`, ...)	`null`
Platform (named *Extension* before Java 9)	Other JDK modules (`java.sql`, `java.xml`)	`PlatformClassLoader`
Application / System (`AppClassLoader`)	Your classpath / module path: app classes and JARs	`AppClassLoader`
Custom loaders	Plugins, app servers, hot reloading, isolation	Your loader

```
System.out.println(String.class.getClassLoader());          // null (bootstrap)
System.out.println(java.sql.Date.class.getClassLoader());   // ...PlatformClassLoader
System.out.println(MyApp.class.getClassLoader());           // ...AppClassLoader
System.out.println(ClassLoader.getSystemClassLoader().getParent());   // PlatformClassLoader
```

The parent delegation model

```
loadClass("com.bank.Payment")
   Application loader: already loaded? no -> ask parent
      Platform loader: already loaded? no -> ask parent
         Bootstrap loader: in java.base? no -> return "not found"
      Platform loader: in its modules? no
   Application loader: search the classpath -> FOUND -> define the class
   (if no loader finds it -> ClassNotFoundException)
```

Why delegate upwards?

- **Security**: nobody can replace `java.lang.String` with a malicious copy on the classpath.
- **Uniqueness**: each class is loaded once by the right loader.
- **Consistency**: a class's identity is *its name plus its defining loader*, so the same class loaded by two loaders gives two distinct types (classic `ClassCastException: X cannot be cast to X` in app servers).

Loading → Linking → Initialization

Phase	What happens
Loading	Find the `.class` bytes, create the `Class` object in Metaspace
Linking: **Verification**	Bytecode verifier checks format and type safety (`VerifyError`)
Linking: **Preparation**	Allocate static fields and set them to **default values** (0, null, false)
Linking: **Resolution**	Symbolic references → direct references (may be lazy)
Initialization	Run static initialisers and `static { }` blocks in textual order (once, thread-safe)

A class is **initialised** on first *active use*: `new`, calling a static method, reading or writing a non-constant static field, `Class.forName(name)`, initialising a subclass, or being the main class. Merely declaring a variable of that type, using `Xyz.class`, or reading a compile-time constant (`static final int X = 5`) does **not** trigger initialisation.

```
Class<?> c1 = Class.forName("com.mysql.cj.jdbc.Driver");  // loads AND initialises
Class<?> c2 = ClassLoader.getSystemClassLoader()
                         .loadClass("com.bank.Payment");  // loads only; init deferred
```

> **TIP**
> The JIT compiler is covered in Phase 1. HotSpot interprets first, profiles, and compiles hot methods with C1 (fast) and then C2 (optimised). Microbenchmarks without a warm-up phase (JMH) are therefore misleading.

15.4 Lambda Expressions (Java 8)

A **lambda** is an anonymous function: a concise implementation of a **functional interface** (one abstract method). It enables passing **behaviour as data**.

```
// (parameters) -> expression      or      (parameters) -> { statements; }
Runnable r       = () -> System.out.println("run");
Comparator<String> byLen = (a, b) -> Integer.compare(a.length(), b.length());
Predicate<Integer> isEven = n -> n % 2 == 0;           // one parameter: parentheses optional
Function<Integer, Integer> square = (Integer x) -> x * x;   // explicit type (optional)
BinaryOperator<Integer> add = (var a, var b) -> a + b;      // var in lambdas (Java 11)
Supplier<List<String>> listMaker = () -> {               // block body needs return
    List<String> l = new ArrayList<>();
    l.add("init");
    return l;
};
```

- The **target type** (the functional interface the lambda is assigned to) determines the parameter and return types.
- Captured local variables must be **effectively final**.
- Inside a lambda, `this` refers to the **enclosing** instance (unlike in anonymous classes).
- Lambdas don't create a `.class` file; they are linked at runtime via `invokedynamic`.

15.5 Built-in Functional Interfaces (java.util.function)

Interface	Method	Takes → Returns	Example
`Predicate<T>`	`test(T)`	T → boolean	`s -> s.isBlank()`
`Function<T,R>`	`apply(T)`	T → R	`s -> s.length()`
`Consumer<T>`	`accept(T)`	T → void	`s -> System.out.println(s)`
`Supplier<T>`	`get()`	() → T	`() -> LocalDate.now()`
`UnaryOperator<T>`	`apply(T)`	T → T	`s -> s.toUpperCase()`
`BinaryOperator<T>`	`apply(T, T)`	(T, T) → T	`(a, b) -> a + b`
`BiFunction<T,U,R>`	`apply(T, U)`	(T, U) → R	`(acct, amt) -> acct + ":" + amt`
`BiPredicate<T,U>`, `BiConsumer<T,U>`	`test` / `accept`	Two inputs	`(k, v) -> ...`
Primitive versions	`IntPredicate`, `IntFunction`, `ToIntFunction`, `IntUnaryOperator`, `IntSupplier`, ...	Avoid boxing	`IntPredicate p = i -> i > 0;`

```
Predicate<String> notBlank = s -> !s.isBlank();
Predicate<String> isNumeric = s -> s.chars().allMatch(Character::isDigit);
Predicate<String> validAccount = notBlank.and(isNumeric).and(s -> s.length() == 10);
validAccount.test("1234567890");          // true
notBlank.negate().test("  ");             // true
Predicate.not(String::isBlank);           // Java 11

Function<Double, Double> addGst   = amt -> amt * 1.18;
Function<Double, Double> discount = amt -> amt - 100;
addGst.andThen(discount).apply(1000.0);   // (1000 * 1.18) - 100 = 1080.0
addGst.compose(discount).apply(1000.0);   // (1000 - 100) * 1.18 = 1062.0
Function.identity();                      // x -> x
```

15.6 Method References

A **method reference** (`Class::method`) is shorthand for a lambda that only calls an existing method.

Kind	Syntax	Equivalent lambda
Static method	`Integer::parseInt`	`s -> Integer.parseInt(s)`
Instance method of a particular object	`System.out::println`	`x -> System.out.println(x)`
Instance method of an arbitrary object of a type	`String::toUpperCase`	`s -> s.toUpperCase()`
Constructor	`ArrayList::new`, `int[]::new`	`() -> new ArrayList<>()`, `n -> new int[n]`

```
List<String> ids = List.of("3", "1", "2");
ids.stream().map(Integer::parseInt).sorted().forEach(System.out::println);   // 1 2 3
Supplier<Map<String, Integer>> mapMaker = HashMap::new;
Function<String, StringBuilder> sbMaker = StringBuilder::new;
```

15.7 Streams API

A **Stream** is a sequence of elements supporting **declarative**, **pipelined** operations (filter, map, reduce...). You describe *what* you want; the library handles the *how*.

- A stream is **not a data structure**: it doesn't store elements, it processes them from a source.
- It **doesn't modify** the source.
- It is **lazy**: intermediate operations run only when a terminal operation is invoked.
- It is **single-use**: after a terminal operation, reusing it throws `IllegalStateException`.

```
SOURCE           INTERMEDIATE OPERATIONS (lazy, return Stream)       TERMINAL (triggers work)
list.stream() -> filter(...) -> map(...) -> sorted(...) -> limit(n) -> collect / forEach / sum
```

Creating streams

```
List.of(1, 2, 3).stream();                    // from a collection
Arrays.stream(new int[]{1, 2, 3});            // IntStream from an array
Stream.of("a", "b", "c");                     // from values
IntStream.range(1, 5);                        // 1 2 3 4   (rangeClosed(1, 5) includes 5)
Stream.iterate(1, n -> n * 2).limit(5);       // 1 2 4 8 16
Stream.iterate(1, n -> n <= 100, n -> n * 3); // 1 3 9 27 81 (Java 9)
Stream.generate(UUID::randomUUID).limit(3);   // 3 random ids
Files.lines(path);  "a,b".chars();  map.entrySet().stream();
```

Intermediate operations (lazy)

Operation	Purpose
`filter(Predicate)`	Keep matching elements
`map(Function)`	Transform each element
`mapToInt/Long/Double`	To a primitive stream (`sum`, `average`, ...)
`flatMap(Function<T, Stream<R>>)`	Flatten nested structures (a list of lists → one stream)
`distinct()`	Remove duplicates (uses `equals`)
`sorted()` / `sorted(Comparator)`	Sort
`limit(n)` / `skip(n)`	Truncate / skip
`peek(Consumer)`	Look at elements (debugging)
`takeWhile` / `dropWhile` (9)	Take or drop while a condition holds (on ordered streams)
`boxed()`	`IntStream` → `Stream<Integer>`

Terminal operations

Operation	Returns
`forEach(Consumer)`	void
`collect(Collector)` / `toList()` (16)	A collection or other result
`reduce(identity, BinaryOperator)`	A single combined value
`count()`, `min(cmp)`, `max(cmp)`	long / `Optional<T>`
`sum()`, `average()`, `summaryStatistics()`	Primitive streams only
`anyMatch`, `allMatch`, `noneMatch`	boolean (short-circuit)
`findFirst()`, `findAny()`	`Optional<T>` (short-circuit)
`toArray()`	Array

Streams in practice

```
record Txn(String id, String account, String type, double amount, String city) { }

List<Txn> txns = List.of(
    new Txn("T1", "SB-1", "CREDIT", 5000, "Pune"),
    new Txn("T2", "SB-1", "DEBIT",  1200, "Pune"),
    new Txn("T3", "SB-2", "CREDIT", 9000, "Delhi"),
    new Txn("T4", "SB-3", "DEBIT",  300,  "Delhi"),
    new Txn("T5", "SB-2", "DEBIT",  7000, "Mumbai"));

// High-value transaction ids, largest first
List<String> big = txns.stream()
        .filter(t -> t.amount() >= 5000)
        .sorted(Comparator.comparingDouble(Txn::amount).reversed())
        .map(Txn::id)
        .toList();                                          // [T3, T5, T1]

double totalCredit = txns.stream()
        .filter(t -> t.type().equals("CREDIT"))
        .mapToDouble(Txn::amount)
        .sum();                                             // 14000.0

boolean anyHuge = txns.stream().anyMatch(t -> t.amount() > 8000);      // true
Optional<Txn> smallest = txns.stream().min(Comparator.comparingDouble(Txn::amount));  // T4
List<String> cities = txns.stream().map(Txn::city).distinct().sorted().toList();
// [Delhi, Mumbai, Pune]

int sum = IntStream.rangeClosed(1, 100).sum();                         // 5050
int product = Stream.of(1, 2, 3, 4).reduce(1, (a, b) -> a * b);         // 24
DoubleSummaryStatistics st = txns.stream().mapToDouble(Txn::amount).summaryStatistics();
// count=5, sum=22500.0, min=300.0, average=4500.0, max=9000.0

List<List<String>> nested = List.of(List.of("a", "b"), List.of("c"));
List<String> flat = nested.stream().flatMap(List::stream).toList();     // [a, b, c]
```

Collectors

```
import static java.util.stream.Collectors.*;

Map<String, List<Txn>> byCity = txns.stream().collect(groupingBy(Txn::city));
Map<String, Long> countByType = txns.stream().collect(groupingBy(Txn::type, counting()));
// {CREDIT=2, DEBIT=3}
Map<String, Double> sumByAccount = txns.stream()
        .collect(groupingBy(Txn::account, TreeMap::new, summingDouble(Txn::amount)));
// {SB-1=6200.0, SB-2=16000.0, SB-3=300.0}
Map<Boolean, List<String>> partition = txns.stream()
        .collect(partitioningBy(t -> t.amount() > 2000, mapping(Txn::id, toList())));
// {false=[T2, T4], true=[T1, T3, T5]}
String idsCsv = txns.stream().map(Txn::id).collect(joining(", ", "[", "]"));
// [T1, T2, T3, T4, T5]
Map<String, Double> amountById = txns.stream().collect(toMap(Txn::id, Txn::amount));
Map<String, Double> maxByCity = txns.stream()
        .collect(toMap(Txn::city, Txn::amount, Math::max));     // merge fn for duplicate keys
Map<String, Double> avgByType = txns.stream()
        .collect(groupingBy(Txn::type, averagingDouble(Txn::amount)));
// {CREDIT=7000.0, DEBIT=2833.33...}
```

Collector	Result
`toList()`, `toSet()`, `toCollection(TreeSet::new)`	Collections
`toMap(keyFn, valueFn[, mergeFn[, mapFactory]])`	Map (**duplicate keys without a merge function →** `IllegalStateException`)
`groupingBy(classifier[, mapFactory][, downstream])`	`Map<K, List<T>>` or a downstream result

Collector	Result
`partitioningBy(predicate[, downstream])`	`Map<Boolean, ...>`
`counting()`, `summingInt/Double`, `averagingX`, `minBy`, `maxBy`	Aggregates (often downstream)
`mapping(fn, downstream)`, `filtering`, `flatMapping` (9)	Transform inside a group
`joining(delim[, prefix, suffix])`	String
`collectingAndThen(c, finisher)`	Post-process (for example make it unmodifiable)
`teeing(c1, c2, merger)` (12)	Combine two collectors

Laziness and short-circuiting (verified)

```
Stream.of("a", "bb", "ccc", "dddd")
      .filter(s -> { System.out.println("filter " + s); return s.length() > 1; })
      .map(s ->    { System.out.println("map " + s);    return s.toUpperCase(); })
      .findFirst();
// filter a
// filter bb
// map bb        <- stops here: elements flow one at a time; ccc and dddd are never touched
```

Stream vs Collection, and pitfalls

Collection	Stream
Stores data	Computes over data
Eager	Lazy
Can be traversed many times	Single-use
External iteration (you write the loop)	Internal iteration (the library loops)
Mutable (for most)	Never modifies its source

PITFALL

Common stream mistakes:

- Reusing a stream → `IllegalStateException: stream has already been operated upon or closed`.
- Side effects in `map`/`filter` (modifying external lists). Use `collect` instead.
- `toMap` with duplicate keys and no merge function → `IllegalStateException`.
- `stream().toList()` is unmodifiable; `collect(Collectors.toList())` currently returns a mutable `ArrayList` (not guaranteed).
- `parallelStream()` for small or I/O-bound workloads, or with shared mutable state, is usually slower or wrong. Measure first.
- Forgetting that nothing executes without a terminal operation.

15.8 Optional (Java 8)

`Optional<T>` is a container that **may or may not hold a value**. It makes "might be absent" explicit in an API **return type**, instead of returning `null`.

```
Optional<String> some  = Optional.of("SB-1001");          // value must be non-null
Optional<String> maybe = Optional.ofNullable(lookup());   // null -> empty
Optional<String> none  = Optional.empty();

maybe.isPresent();  maybe.isEmpty();                       // checks (isEmpty: Java 11)
maybe.ifPresent(acc -> System.out.println("Found " + acc));
maybe.ifPresentOrElse(System.out::println, () -> System.out.println("none"));   // Java 9

String a = maybe.orElse("DEFAULT");                        // default (always evaluated)
String b = maybe.orElseGet(() -> expensiveDefault());      // lazy default
String c = maybe.orElseThrow(() -> new AccountNotFoundException("missing"));
String d = some.orElseThrow();                             // Java 10: NoSuchElementException

Optional<Integer> len = maybe.map(String::length);         // transform if present
Optional<String> pune = findCustomer(id).map(Customer::getAddress)   // safe navigation
        .map(Address::getCity)
        .filter(city -> city.equals("Pune"));
```

REMEMBER

Optional best practices:

- Use it mainly as a **method return type**, not for fields, parameters or collections (return an empty list instead).
- Never call `get()` without checking; prefer `orElse`/`orElseGet`/`orElseThrow`/`map`.
- `orElse(x)` always evaluates `x`; use `orElseGet` when the default is expensive.
- Never return `null` from a method declared to return `Optional`.

15.9 Date and Time API

Legacy classes: why they were replaced

- `java.util.Date` is **mutable**, its name is misleading (it is really an instant with time), and many methods are deprecated. Its `toString` uses the JVM's time zone.
- `java.sql.Date` extends `java.util.Date` and represents only a date, for JDBC.
- `Calendar` (`Calendar.getInstance()`) is mutable and clunky, and **months are 0-based** (`Calendar.MONTH` = 0 is January): a famous bug source.
- `SimpleDateFormat` is **not thread-safe**: sharing one instance between threads gives corrupted dates.

```
Calendar c = Calendar.getInstance();
int month = c.get(Calendar.MONTH) + 1;          // +1 needed: 0-based months!
int hour = c.get(Calendar.HOUR_OF_DAY);
Date now = new Date();                          // legacy: avoid in new code

StringTokenizer st = new StringTokenizer("Java,Spring,Boot", ",");   // legacy tokenizer
while (st.hasMoreTokens()) System.out.println(st.nextToken());      // prefer String.split()
```

java.time (Java 8): immutable, thread-safe, clear

Class	Represents	Example
`LocalDate`	Date only, no time zone	`2026-08-15` (birth date, EMI due date)
`LocalTime`	Time only	`10:30:00` (branch opening time)
`LocalDateTime`	Date + time, no zone	`2026-08-15T10:30`
`ZonedDateTime`	Date + time + zone	`2026-08-15T10:30+05:30[Asia/Kolkata]`
`Instant`	A point on the UTC timeline	`2026-08-15T05:00:00Z` (timestamps, logs, DB)
`Duration`	Time-based amount (seconds, nanos)	`PT2H30M`
`Period`	Date-based amount (years, months, days)	`P1Y2M3D`
`DateTimeFormatter`	Formatting and parsing (thread-safe)	`dd-MM-yyyy`
`ChronoUnit`	Units for arithmetic and differences	`ChronoUnit.DAYS.between(a, b)`

```
LocalDate today = LocalDate.of(2026, 8, 15);                // months are 1-based (August = 8)
LocalDate due = today.plusMonths(1).withDayOfMonth(5);       // 2026-09-05
LocalDate monthEnd = today.with(TemporalAdjusters.lastDayOfMonth());   // 2026-08-31
today.getDayOfWeek();                                        // SATURDAY
today.isLeapYear();                                          // false
today.isBefore(due);                                         // true
// today.plusDays(1) returns a NEW object; 'today' itself never changes (immutable)

LocalDate dob = LocalDate.of(1990, 3, 25);
Period age = Period.between(dob, today);                     // P36Y4M21D
age.getYears();                                              // 36
long days = ChronoUnit.DAYS.between(today, due);             // 21

LocalTime open = LocalTime.of(9, 30);
Duration shift = Duration.between(open, LocalTime.of(17, 45));   // PT8H15M
LocalDateTime ldt = LocalDateTime.of(today, open);           // 2026-08-15T09:30

DateTimeFormatter fmt = DateTimeFormatter.ofPattern("dd-MM-yyyy");
String text = today.format(fmt);                             // 15-08-2026
LocalDate parsed = LocalDate.parse("05-09-2026", fmt);       // DateTimeParseException if invalid
LocalDate iso = LocalDate.parse("2026-08-15");               // ISO format by default

ZonedDateTime ist = ZonedDateTime.of(ldt, ZoneId.of("Asia/Kolkata"));
ZonedDateTime ny  = ist.withZoneSameInstant(ZoneId.of("America/New_York"));   // 00:00 same day
Instant stamp = ist.toInstant();                             // 2026-08-15T04:00:00Z

Date legacy = Date.from(stamp);                              // bridge legacy <-> modern
Instant back = legacy.toInstant();
```

Pattern letter	Meaning	Pattern letter	Meaning
`yyyy` / `uuuu`	Year	`HH` / `hh`	Hour 0–23 / 1–12
`MM` / `MMM` / `MMMM`	Month 08 / Aug / August	`mm`	**Minutes** (not months!)
`dd`	Day of month	`ss` / `SSS`	Seconds / milliseconds
`EEE` / `EEEE`	Mon / Monday	`a`	AM/PM

TIP

Store timestamps as `Instant` (UTC) in databases and logs; convert to `ZonedDateTime` only for display. Use `LocalDate` for calendar concepts (DOB, due dates). In tests, inject a `java.time.Clock` (`LocalDate.now(clock)`) so "today" is controllable and tests are deterministic.

15.10 Records (Java 16)

A **record** is a concise, **immutable data carrier**. The compiler generates private final fields, a canonical constructor, accessor methods (`name()`, not `getName()`), and `equals`, `hashCode` and `toString`.

```
public record Money(BigDecimal amount, String currency) {
    public Money {                                          // compact constructor: validation
        Objects.requireNonNull(amount, "amount");
        if (amount.signum() < 0) throw new IllegalArgumentException("negative amount");
        currency = currency.toUpperCase();                  // normalise a parameter
    }
    public static Money inr(String amt) { return new Money(new BigDecimal(amt), "INR"); }
    public Money plus(Money other) {                        // methods allowed
        return new Money(amount.add(other.amount), currency);
    }
}

Money m = Money.inr("250.00");
System.out.println(m);                                      // Money[amount=250.00, currency=INR]
System.out.println(m.amount());                             // 250.00
System.out.println(m.equals(Money.inr("250.00")));          // true (value-based equality)

record Customer(String id, List<String> phones) {
    Customer { phones = List.copyOf(phones); }              // defensive copy: truly immutable
}
```

Can	Cannot
Implement interfaces	Extend another class (records implicitly extend `java.lang.Record`)
Have static fields and methods, instance methods, nested types	Declare instance fields beyond the components
Customise the constructor (compact or canonical)	Have setters (fields are `final`)
Be generic: `record Pair<A,B>(A first, B second)`	Be extended (records are implicitly `final`)

> **NOTE**
> Records are ideal for DTOs, API request/response bodies (Jackson supports them), map keys, value objects (Money, AccountId) and multiple return values. JPA **entities** shouldn't be records, because entities need mutability and a no-arg constructor.

15.11 Sealed Classes (Java 17) and Pattern Matching

A **sealed** class or interface restricts **which classes may extend or implement it**, using `permits`. Each permitted subclass must be `final`, `sealed` or `non-sealed`. The compiler then knows **every possible subtype**, so `switch` can be checked for exhaustiveness.

```
public sealed interface PaymentResult permits Success, Failure, Pending { }
public record Success(String txnId, BigDecimal amount) implements PaymentResult { }
public record Failure(String code, String reason)      implements PaymentResult { }
public record Pending(String txnId)                    implements PaymentResult { }

static String describe(PaymentResult r) {
    return switch (r) {                                    // no default needed: exhaustive
        case Success s when s.amount().compareTo(new BigDecimal("100000")) > 0
                -> "Large payment " + s.txnId() + " flagged for review";
        case Success(String id, BigDecimal amt)            // record pattern (Java 21)
                -> "Paid " + amt + " (" + id + ")";
        case Failure(var code, var reason)
                -> "Failed [" + code + "]: " + reason;
        case Pending p -> "Awaiting confirmation for " + p.txnId();
    };
}
```

Adding a new permitted type (for example `Reversed`) makes every non-exhaustive `switch` **fail to compile**, so the compiler finds every place that must handle the new case.

15.12 Modern Java Feature Map (8 → 25)

Version	Year	Key language and API features
8 (LTS)	2014	Lambdas, functional interfaces, method references, Streams, `Optional`, `java.time`, default/static interface methods, `CompletableFuture`
9	2017	Modules (JPMS), JShell, `List.of`/`Set.of`/`Map.of`, private interface methods, `Optional`/Stream improvements, G1 default
10	2018	`var` for local variables, `List.copyOf`, `orElseThrow()`
11 (LTS)	2018	`String.isBlank`/`strip`/`lines`/`repeat`, `Files.readString`/`writeString`, standard `HttpClient`, `var` in lambdas, single-file launch
12–13	2019	`Collectors.teeing`, switch-expression and text-block previews
14	2020	**Switch expressions** (final), helpful NullPointerException messages
15	2020	**Text blocks** (final), ZGC production-ready
16	2021	**Records**, **pattern matching for** `instanceof`, `Stream.toList()`
17 (LTS)	2021	**Sealed classes**, strict floating point everywhere, new random generators
18	2022	**UTF-8 by default**, simple web server (`jwebserver`), `finalize` deprecated for removal
19–20	2022–23	Virtual threads and structured concurrency (preview), `HashMap.newHashMap(n)`
21 (LTS)	2023	**Virtual threads**, **pattern matching for switch**, **record patterns**, **sequenced collections**, generational ZGC
22	2024	**Unnamed variables and patterns** (_), FFM API (native interop), launching multi-file source programs
23–24	2024–25	Markdown doc comments, **Stream gatherers** (24), less virtual-thread pinning with `synchronized` (24)
25 (LTS)	2025	**Compact source files & instance** `main`, **flexible constructor bodies**, **scoped values**, **module import declarations**

```
// Java 22+: unnamed variable for things you must declare but don't use
try { process(); } catch (IOException _) { log("ignored"); }
map.forEach((_, value) -> total += value);
```

15.13 Rapid-Fire Interview Round (all phases)

Question	Short answer
JDK vs JRE vs JVM?	Dev kit ⊃ runtime ⊃ virtual machine
Is Java pass-by-reference?	No: always pass-by-value (references are copied)
`==` vs `equals()`?	Reference vs logical content
Why is String immutable?	Pool safety, security, thread safety, hash caching
String vs StringBuilder vs StringBuffer?	Immutable / mutable and fast / mutable and synchronized
Can we override a static method?	No: it is hidden
Can we overload `main`?	Yes; the JVM calls only `main(String[])`
Abstract class vs interface?	State and constructors, single inheritance / contract, multiple inheritance
Why no multiple inheritance of classes?	Diamond problem
`final` vs `finally` vs `finalize`?	Modifier / cleanup block / deprecated GC hook
Checked vs unchecked exceptions?	Compiler-enforced / RuntimeException and Error
`throw` vs `throws`?	Raise one / declare possible
Does finally always run?	Except `System.exit`, a crash, or a hang
`equals`/`hashCode` contract?	Equal objects ⇒ equal hash codes
HashMap internal working?	Hash → bucket → equals; tree bins at 8; resize at 0.75
HashMap vs ConcurrentHashMap?	Unsafe, 1 null key / thread-safe, no nulls, atomic ops
ArrayList vs LinkedList?	Array, O(1) get / nodes, O(n) get
Fail-fast vs fail-safe?	Throws CME / snapshot or weakly consistent
Comparable vs Comparator?	Natural order inside the class / external, many orders
`sleep()` vs `wait()`?	Keeps lock / releases monitor
`synchronized` vs `volatile`?	Mutual exclusion + visibility / visibility only
How do you prevent deadlock?	Consistent lock ordering, `tryLock` with timeout
Runnable vs Callable?	No result / result + checked exceptions
What is a daemon thread?	Background thread that doesn't keep the JVM alive
Platform vs virtual thread?	OS thread / cheap JVM thread for blocking I/O
What is `transient`?	Field skipped by serialization
Why `serialVersionUID`?	Version check during deserialization
Stack vs heap?	Per-thread frames / shared objects managed by GC
When is an object GC-eligible?	When unreachable from GC roots
Default GC since Java 9?	G1
Parent delegation?	Ask the parent loader first: security and uniqueness
`Class.forName` vs `loadClass`?	Loads and initialises / loads only
Intermediate vs terminal stream ops?	Lazy, return a Stream / trigger execution
`map` vs `flatMap`?	One-to-one / one-to-many, flattened
`orElse` vs `orElseGet`?	Eager default / lazy default
Record vs class?	Immutable data carrier with generated members
Why sealed classes?	Restricted hierarchy plus exhaustive switch
`LocalDateTime` vs `ZonedDateTime` vs `Instant`?	No zone / with zone / UTC point
Why BigDecimal for money?	Exact decimal arithmetic with rounding control

Question	Short answer
Integer cache range?	-128 to 127, so never compare wrappers with ==

15.14 Quick Revision

- Heap: **Eden → Survivors → Old**; Metaspace is outside the heap. Minor GC is frequent and cheap; Full GC is a warning sign.
- GC = reachability from **GC roots**; mark → sweep → compact; STW pauses; **G1** is the default, **ZGC** for low latency; CMS is removed.
- Tools: `-Xlog:gc`, heap dumps on OOM, `jcmd`, `jstat`, JFR, Eclipse MAT. Leaks: static caches, listeners, `ThreadLocal`, mutable keys.
- References: strong > soft (low memory) > weak (next GC) > phantom.
- Class loaders: Bootstrap (`null`) → Platform → Application; parent delegation. Load → Verify/Prepare/Resolve → Initialise (static blocks) on first active use.
- Lambdas implement functional interfaces; captured locals are effectively final; `this` = enclosing instance.
- `java.util.function`: Predicate, Function, Consumer, Supplier, Unary/BinaryOperator, Bi-versions, primitive versions; compose with `and`/`or`/`negate`/`andThen`/`compose`.
- Method references: static, bound instance, unbound instance, constructor.
- Streams: source → lazy intermediate ops → one terminal op; single-use; no source mutation. Collectors: `groupingBy`, `counting`, `summingX`, `partitioningBy`, `toMap` (with a merge function), `joining`.
- Optional: return types only; `map`/`filter`/`orElseGet`/`orElseThrow`; never a bare `get()`.
- java.time: immutable, thread-safe; `LocalDate`/`LocalTime`/`LocalDateTime`/`ZonedDateTime`/`Instant`; `Period` (dates) vs `Duration` (time); `DateTimeFormatter` (`MM` months, `mm` minutes).
- Legacy `Date`/`Calendar` (0-based months) and `SimpleDateFormat` (not thread-safe): avoid in new code.
- Records: immutable carriers, compact constructors, accessors without the `get` prefix. Sealed + records + pattern switch = exhaustive, type-safe domain modelling.
- LTS versions: **8, 11, 17, 21, 25**.

PHASE 16 OF 18

Annotations & Reflection

Built-in and meta-annotations, retention and targets, writing custom annotations, processing them at runtime, the Reflection API (classes, fields, methods, constructors), dynamic proxies, costs and risks, and how Spring, JUnit and Appium use all of this

IN THIS PHASE

16.1 What Are Annotations?
16.2 Built-in Annotations
16.3 Meta-Annotations
16.4 Writing Custom Annotations
16.5 Processing Annotations at Runtime
16.6 The Reflection API
16.7 Costs and Risks of Reflection
16.8 Interview Questions
16.9 Quick Revision

16.1 What Are Annotations?

An **annotation** is **metadata**: a label attached to code (classes, methods, fields, parameters...) that doesn't change what the code does by itself. **Tools read it**: the compiler, build-time processors, or frameworks at runtime through reflection.

```
public String toString() { return "..."; }

void shouldTransferFunds() { }

private WebElement loginButton;

public class AccountController { }
```

Who reads annotations	When	Examples
Compiler	Compile time	`@Override`, `@FunctionalInterface`, `@SuppressWarnings`
Annotation processors	Compile time (generate code)	Lombok `@Data`, MapStruct, Dagger
Frameworks via reflection	Runtime	JUnit `@Test`, Spring `@Autowired`, JPA `@Entity`, Jackson `@JsonProperty`, TestNG `@DataProvider`

16.2 Built-in Annotations

Annotation	Purpose
`@Override`	Compile error if the method doesn't actually override or implement anything
`@Deprecated(since = "2.0", forRemoval = true)`	Marks an API as obsolete; callers get warnings
`@SuppressWarnings("unchecked")`	Silences specific compiler warnings (`"deprecation"`, `"rawtypes"`, `"unused"`...)
`@FunctionalInterface`	Compile error unless the interface has exactly one abstract method
`@SafeVarargs`	Promises a generic varargs method doesn't pollute the heap
`@Serial` (Java 14)	Marks serialization members (`serialVersionUID`, `writeObject`...) for compiler checks

```
public void legacyTransfer() { }

List<String> names = (List<String>) rawObject;     // suppress only where you're sure
```

16.3 Meta-Annotations

Annotations applied to **annotation definitions** to control how they behave.

@Retention: how long the annotation survives

RetentionPolicy	Kept in	Visible at runtime via reflection?	Example
`SOURCE`	Source code only (discarded by `javac`)	No	`@Override`, `@SuppressWarnings`, Lombok
`CLASS` (**default**)	The `.class` file	No	Bytecode tools
`RUNTIME`	The `.class` file **and** loaded into the JVM	**Yes**	`@Test`, `@Autowired`, `@Entity`, `@FindBy`

@Target: where it may be placed

ElementType	Allowed on
`TYPE`	Classes, interfaces, enums, records
`FIELD`	Fields (including enum constants)
`METHOD`	Methods
`PARAMETER`	Method parameters
`CONSTRUCTOR`	Constructors
`LOCAL_VARIABLE`	Local variables
`ANNOTATION_TYPE`	Other annotations (to build meta-annotations)
`PACKAGE`, `MODULE`, `RECORD_COMPONENT`	Packages, modules, record components
`TYPE_USE`, `TYPE_PARAMETER` (8)	Any use of a type: `List<@NonNull String>`

Other meta-annotation	Effect
`@Documented`	Include the annotation in Javadoc
`@Inherited`	A **class**-level annotation is inherited by subclasses
`@Repeatable(Container.class)` (8)	Allow the same annotation several times on one element

16.4 Writing Custom Annotations

```
import java.lang.annotation.*;

public @interface NotBlank {                // @interface declares an annotation type
    String message() default "must not be blank";
}

public @interface Range {
    long min() default 0;
    long max() default Long.MAX_VALUE;
}

public @interface Audited { }               // marker annotation: no elements

public @interface Smoke {
    String value();                         // element named 'value' -> @Smoke("login")
    int priority() default 3;
}
```

Rules for annotation elements

1. Elements look like methods with **no parameters** and **no** `throws`.
2. Allowed return types: primitives, `String`, `Class<?>`, enums, other annotations, and **arrays** of these. Not `Integer`, `List` or arbitrary objects.
3. `default` values must be **compile-time constants**; `null` is never allowed as a value.
4. If the only element you set is named `value`, you can omit the name: `@Smoke("login")`.
5. Array elements accept a single value without braces: `@SuppressWarnings("unused")` equals `@SuppressWarnings({"unused"})`.

16.5 Processing Annotations at Runtime

Reflection can read `RUNTIME` annotations. The examples below are tiny versions of what Bean Validation (`@NotBlank`, `@Size`) and JUnit do internally.

A mini validation framework

```
class Customer {
    @NotBlank(message = "name is required") private String name;
    @Range(min = 18, max = 120)                private int age;
    Customer(String name, int age) { this.name = name; this.age = age; }
}

static List<String> validate(Object obj) throws IllegalAccessException {
    List<String> errors = new ArrayList<>();
    for (Field f : obj.getClass().getDeclaredFields()) {
        f.setAccessible(true);                                  // read private fields
        Object value = f.get(obj);
        NotBlank nb = f.getAnnotation(NotBlank.class);
        if (nb != null && (value == null || value.toString().isBlank())) {
            errors.add(f.getName() + ": " + nb.message());
        }
        Range r = f.getAnnotation(Range.class);
        if (r != null) {
            long v = ((Number) value).longValue();
            if (v < r.min() || v > r.max())
                errors.add(f.getName() + ": must be between " + r.min() + " and " + r.max());
        }
    }
    return errors;
}

System.out.println(validate(new Customer(" ", 15)));
// [name: name is required, age: must be between 18 and 120]
System.out.println(Customer.class.isAnnotationPresent(Audited.class));   // true
```

A mini test runner (how JUnit/TestNG find tests)

```
class LoginTests {
    @Smoke(value = "valid login", priority = 1) public void validLogin()  { }
    @Smoke("locked account")
    public void lockedUser() { throw new IllegalStateException("locked"); }
    public void helper() { }                                            // not a test
}

Object suite = LoginTests.class.getDeclaredConstructor().newInstance();
for (Method m : LoginTests.class.getDeclaredMethods()) {
    Smoke s = m.getAnnotation(Smoke.class);
    if (s == null) continue;                                            // skip non-tests
    try {
        m.invoke(suite);
        System.out.println("PASS " + s.value());
    } catch (InvocationTargetException e) {                             // wraps the real failure
        System.out.println("FAIL " + s.value() + " -> " + e.getCause().getMessage());
    }
}
// PASS valid login
// FAIL locked account -> locked     (method order from reflection is NOT guaranteed)
```

> **NOTE**
> Compile-time **annotation processors** (`javax.annotation.processing`) read `SOURCE`/`CLASS` annotations during compilation and can **generate code**. That's how Lombok produces getters and setters, and how MapStruct generates mappers, with no runtime reflection cost.

16.6 The Reflection API

Reflection (`java.lang.reflect`) lets a program **inspect and use classes at runtime**: discover fields, methods, constructors and annotations, create objects, call methods and read or write fields, even ones it didn't know about at compile time.

Getting a Class object

```
Class<?> c1 = Customer.class;                       // class literal (compile-time known)
Class<?> c2 = new Customer("A", 30).getClass();     // from an instance (runtime type)
Class<?> c3 = Class.forName("com.bank.Customer");   // from a name (loads + initialises)
```

Inspecting a class

```
Class<?> c = ArrayList.class;
c.getName();                    // java.util.ArrayList
c.getSimpleName();              // ArrayList
c.getPackageName();             // java.util
c.getSuperclass();              // class java.util.AbstractList
c.getInterfaces();              // [List, RandomAccess, Cloneable, Serializable]
Modifier.isPublic(c.getModifiers());    // true
c.isInterface(); c.isEnum(); c.isRecord(); c.isArray();
```

Method	Returns
`getFields()` / `getMethods()` / `getConstructors()`	**Public** members, **including inherited** ones
`getDeclaredFields()` / `getDeclaredMethods()` / `getDeclaredConstructors()`	**All** members (any access) declared **in this class only**
`getField(name)` / `getDeclaredField(name)`	One field (`NoSuchFieldException` if missing)
`getMethod(name, paramTypes...)` / `getDeclaredMethod(...)`	One method
`getAnnotation(A.class)`, `isAnnotationPresent(A.class)`, `getAnnotations()`	Annotations

Creating objects, calling methods, accessing fields

```
class Account {
    private String accountNo;
    private double balance;
    public Account() { }
    public Account(String no) { this.accountNo = no; }
    private double applyFee(double amt) { return amt - 10; }
}

Class<Account> ac = Account.class;
Account a1 = ac.getDeclaredConstructor().newInstance();                  // no-arg
Account a2 = ac.getDeclaredConstructor(String.class).newInstance("SB-1"); // with args

Field bal = ac.getDeclaredField("balance");
bal.setAccessible(true);                          // bypass 'private' (same module/opened only)
bal.set(a2, 5000.0);
System.out.println(bal.get(a2));                  // 5000.0

Method fee = ac.getDeclaredMethod("applyFee", double.class);
fee.setAccessible(true);
System.out.println(fee.invoke(a2, 500.0));        // 490.0 (returns Object; boxed)
// Method.invoke(null, args) for static methods
```

Exception	When
`ClassNotFoundException`	`Class.forName` can't find the class

Exception	When
`NoSuchMethodException` / `NoSuchFieldException`	Wrong name or parameter types
`IllegalAccessException`	Accessing an inaccessible member without `setAccessible(true)`
`InvocationTargetException`	The **invoked method itself threw**: inspect `getCause()`
`InstantiationException`	Trying to instantiate an abstract class or interface
`InaccessibleObjectException` (Java 9+)	`setAccessible(true)` on a member of a module that doesn't `open` the package

Dynamic proxies: how Spring AOP and mocks work

`java.lang.reflect.Proxy` creates, **at runtime**, an object that implements given interfaces and routes every call to an `InvocationHandler`. This lets you add logging, timing, transactions, security or caching **without changing the real class**.

```
interface PaymentService { String pay(String acct, double amt); }

class RealPaymentService implements PaymentService {
    public String pay(String acct, double amt) { return "PAID " + amt + " to " + acct; }
}

PaymentService real = new RealPaymentService();
PaymentService logged = (PaymentService) Proxy.newProxyInstance(
        PaymentService.class.getClassLoader(),
        new Class<?>[]{PaymentService.class},
        (proxy, method, args) -> {                          // InvocationHandler
            System.out.println("-> " + method.getName() + Arrays.toString(args));
            long start = System.nanoTime();
            Object result = method.invoke(real, args);      // delegate to the real object
            long us = (System.nanoTime() - start) / 1000;
            System.out.println("<- " + result + " in " + us + "us");
            return result;
        });

logged.pay("SB-1001", 2500);
// -> pay[SB-1001, 2500.0]
// <- PAID 2500.0 to SB-1001 in 85us   (time varies)
```

> **NOTE**
> JDK proxies work only with **interfaces**. Spring uses them for interface-based beans and switches to CGLIB/ByteBuddy (subclass generation) for classes. That's why `@Transactional` doesn't apply when a method calls another method of the **same** object: the call bypasses the proxy.

16.7 Costs and Risks of Reflection

Concern	Detail
Performance	Reflective calls are slower than direct calls (lookups, boxing, access checks). Cache `Method`/`Field` objects; consider `MethodHandle`s for hot paths.
Breaks encapsulation	`setAccessible(true)` reaches private state and can violate class invariants
No compile-time safety	Typos in names fail only at **runtime** (`NoSuchMethodException`); refactoring tools may miss string names
Module restrictions	Since Java 9/16/17, deep reflection into JDK internals is blocked unless the module `opens` the package (`--add-opens` on the command line)
Security	Untrusted input deciding which class or method to call is dangerous (similar to deserialization gadgets)

> **TIP**
> Application code rarely needs reflection directly. Prefer interfaces, lambdas and polymorphism. Reflection belongs inside **frameworks and tools**: DI containers, test runners, serializers, ORMs and mocking libraries.

Where you meet annotations and reflection every day

Tool	Annotations	Reflection does...
JUnit 5	`@Test`, `@BeforeEach`, `@ParameterizedTest`, `@DisplayName`	Finds annotated methods, instantiates the test class, invokes the methods
TestNG	`@Test(groups, priority, dataProvider)`, `@DataProvider`	Same, plus grouping and ordering
Appium / Selenium PageFactory	`@FindBy`, `@AndroidFindBy`, `@iOSXCUITFindBy`	Creates element proxies and **injects** them into private fields
Spring	`@Component`, `@Autowired`, `@Value`, `@Transactional`	Scans classes, creates beans, injects dependencies, wraps them in proxies
Jackson	`@JsonProperty`, `@JsonIgnore`	Reads and writes fields and getters to map JSON
JPA / Hibernate	`@Entity`, `@Id`, `@Column`	Maps fields to columns and builds objects from rows
Mockito	`@Mock`, `@InjectMocks`	Generates mock subclasses and injects them into fields

16.8 Interview Questions

Q. What are annotations, and what are they used for?

Metadata attached to program elements. They are used by the compiler (`@Override`), by code generators at build time (Lombok), and by frameworks at runtime through reflection (`@Test`, `@Autowired`, `@Entity`).

Q. What are the retention policies?

`SOURCE` is discarded by the compiler. `CLASS` (the default) is stored in the `.class` file but not available at runtime. `RUNTIME` is available through reflection, which is needed for framework annotations.

Q. How do you create a custom annotation?

Declare it with `@interface`, add `@Retention` and `@Target`, and define elements as parameterless methods with optional `default` values (primitives, String, Class, enums, annotations or arrays of these). Read it with `getAnnotation()` at runtime.

Q. What is `@Inherited`?

A meta-annotation that makes a **class-level** annotation apply to subclasses too. It doesn't affect method or field annotations.

Q. What is reflection, and when is it used?

An API to inspect and manipulate classes, methods, fields and constructors at runtime. Frameworks use it for dependency injection, test discovery, serialization, ORM mapping, proxies and mocking.

Q. What is the difference between `getMethods()` and `getDeclaredMethods()`?

`getMethods()` returns all **public** methods including inherited ones. `getDeclaredMethods()` returns all methods of **any** access declared in that class only (not inherited).

Q. How can you access a private field via reflection? Is it always possible?

Use `getDeclaredField(name)`, then `setAccessible(true)`, then `get`/`set`. Since Java 9, this fails with `InaccessibleObjectException` when the target package belongs to a module that doesn't open it to your code.

Q. What is `InvocationTargetException`?

The wrapper exception thrown by `Method.invoke`/`Constructor.newInstance` when the invoked code itself throws. The real exception is available from `getCause()`.

Q. What is a dynamic proxy?

A runtime-generated class implementing given interfaces whose calls go to an `InvocationHandler`. It enables cross-cutting concerns (logging, transactions, security) without modifying the target. JDK proxies need interfaces.

Q. What are the drawbacks of reflection?

Slower calls, broken encapsulation, no compile-time checking, fragility under refactoring, module access restrictions, and security risks if inputs are untrusted.

16.9 Quick Revision

- Annotations = metadata read by the compiler, by processors (code generation) or by frameworks via reflection.
- Built-ins: `@Override`, `@Deprecated(since, forRemoval)`, `@SuppressWarnings`, `@FunctionalInterface`, `@SafeVarargs`, `@Serial`.
- Meta: `@Retention` (SOURCE / CLASS default / **RUNTIME**), `@Target` (TYPE, FIELD, METHOD, ...), `@Documented`, `@Inherited`, `@Repeatable`.
- Custom: `@interface`; elements are parameterless methods; types are primitives, String, Class, enum, annotation or arrays; defaults are constants; `value` shorthand.
- Read at runtime: `getAnnotation`, `isAnnotationPresent` (needs RUNTIME retention).
- Reflection: get a `Class` via `.class` / `getClass()` / `forName`. `getXxx` = public + inherited; `getDeclaredXxx` = all access, this class only.
- Create: `getDeclaredConstructor(...).newInstance(...)`. Call: `method.invoke(obj, args)`. Private: `setAccessible(true)` (subject to modules).
- `InvocationTargetException.getCause()` is the real error.
- `Proxy.newProxyInstance` + `InvocationHandler` = interface proxies (the basis of AOP and mocking).
- Reflection is powerful but slow and unsafe for app code; it belongs in frameworks.

PHASE 17 OF 18

SOLID Principles & Design Patterns

SOLID with before/after banking examples, DRY/KISS/YAGNI, creational patterns (Singleton variants, Factory, Builder), structural patterns (Adapter, Decorator, Proxy, Facade), behavioural patterns (Strategy, Observer, Template Method, Chain of Responsibility, Command), patterns in the JDK and in test automation

IN THIS PHASE

17.1 Why Design Principles?
17.2 SOLID Principles
17.3 Design Pattern Categories
17.4 Creational Patterns
17.5 Structural Patterns
17.6 Behavioural Patterns
17.7 Patterns in the JDK and Frameworks
17.8 Patterns in Test Automation (Appium / Selenium)
17.9 Interview Questions
17.10 Quick Revision

17.1 Why Design Principles?

Code that merely works becomes hard to change: one new payment type touches ten classes, a fix in one place breaks another, tests need a real database. **Design principles** keep code **easy to change, test and understand**. **Design patterns** are proven, named solutions to recurring design problems (from the 1994 "Gang of Four" book), so "use a Strategy here" communicates a whole design in three words.

17.2 SOLID Principles

Letter	Principle	One line
S	Single Responsibility	A class should have **one reason to change**
O	Open/Closed	Open for **extension**, closed for **modification**
L	Liskov Substitution	Subtypes must be usable **anywhere** their parent is expected
I	Interface Segregation	Many small, focused interfaces beat one fat interface
D	Dependency Inversion	Depend on **abstractions**, not concrete classes

S: Single Responsibility Principle

```
// BEFORE: one class, four reasons to change (business rules, SQL, PDF layout, SMTP)
class AccountService {
    void transfer(...) { /* rules */ }
    void saveToDb(...) { /* JDBC */ }
    byte[] statementPdf(...) { /* layout */ }
    void emailCustomer(...) { /* SMTP */ }
}

// AFTER: each class has one job and can change or be tested independently
class TransferService { void transfer(...) { } }
class AccountRepository { void save(Account a) { } }
class StatementGenerator { byte[] toPdf(Statement s) { return null; } }
class NotificationService { void email(String to, String msg) { } }
```

O: Open/Closed Principle

```
// BEFORE: every new account type means EDITING this method (risk of breaking others)
double interest(Account a) {
    if (a.type().equals("SAVINGS")) return a.balance() * 0.04;
    else if (a.type().equals("FD")) return a.balance() * 0.07;
    // add RD? NRE? -> keep editing...
    return 0;
}

// AFTER: add a new type by adding a NEW class; existing code stays untouched
interface InterestPolicy { double interest(double balance); }
class SavingsPolicy implements InterestPolicy {
    public double interest(double b) { return b * 0.04; }
}
class FdPolicy implements InterestPolicy {
    public double interest(double b) { return b * 0.07; }
}
class RdPolicy implements InterestPolicy {                  // NEW type: nothing else edited
    public double interest(double b) { return b * 0.065; }
}
```

L: Liskov Substitution Principle

```
class Account {
    void withdraw(double amt) { /* debit */ }
}
class FixedDeposit extends Account {
    @Override void withdraw(double amt) {
        throw new UnsupportedOperationException("FD can't withdraw");   // VIOLATES LSP
    }
}
// Code written for Account ("any account can withdraw") breaks when given an FD.

// FIX: model capabilities, so only withdrawable accounts promise withdraw()
interface Account { double balance(); }
interface Withdrawable extends Account { void withdraw(double amt); }
class SavingsAccount implements Withdrawable { /* ... */ }
class FixedDepositAccount implements Account { /* no withdraw promised */ }
```

> **NOTE**
> Classic LSP smells: a subclass that throws `UnsupportedOperationException`, ignores a method, strengthens preconditions (accepts fewer inputs) or weakens postconditions (guarantees less). The textbook example is `Square extends Rectangle`: setting a square's width also changes its height, breaking code that expects a rectangle.

I: Interface Segregation Principle

```
// BEFORE: every device is forced to implement everything
interface BankingDevice {
    void withdrawCash(); void depositCheque(); void printPassbook(); void scanFingerprint();
}

// AFTER: small role interfaces; classes implement only what they support
interface CashDispenser   { void withdrawCash(); }
interface ChequeDeposit   { void depositCheque(); }
interface PassbookPrinter { void printPassbook(); }
class Atm implements CashDispenser, ChequeDeposit { /* ... */ }
class PassbookKiosk implements PassbookPrinter { /* ... */ }
```

D: Dependency Inversion Principle

```
// BEFORE: high-level policy hard-wired to a low-level detail
class PaymentService {
    private final RazorpayGateway gateway = new RazorpayGateway();   // can't swap, can't mock
}

// AFTER: both depend on an abstraction; the concrete class is INJECTED
interface PaymentGateway { String charge(String acct, double amt); }
class PaymentService {
    private final PaymentGateway gateway;
    PaymentService(PaymentGateway gateway) { this.gateway = gateway; }   // injected
    String pay(String acct, double amt) { return gateway.charge(acct, amt); }
}
new PaymentService(new RazorpayGateway());          // production
new PaymentService((a, amt) -> "MOCK-OK");          // unit test: no network needed
```

> **TIP**
>
> **Dependency Injection** (DI) is the technique; **Inversion of Control** (IoC) is the idea (a container creates and wires objects). Spring's `@Autowired`/constructor injection is DIP in practice. Prefer **constructor injection**: dependencies are explicit, can be `final`, and tests can pass fakes.

Other principles

Principle	Meaning
DRY: Don't Repeat Yourself	One source of truth for each piece of knowledge (validation rules, constants, SQL)
KISS: Keep It Simple	The simplest design that works; avoid cleverness
YAGNI: You Aren't Gonna Need It	Don't build for imagined future requirements
Composition over inheritance	Prefer HAS-A plus interfaces to deep `extends` chains (Phase 6)
Program to an interface	Declare variables, parameters and returns as interfaces (`List`, `PaymentGateway`)
Law of Demeter	Talk to friends, not strangers: avoid `a.getB().getC().doX()` chains
High cohesion, low coupling	Related things together, few dependencies between modules

17.3 Design Pattern Categories

Category	Concern	Patterns covered here
Creational	How objects are created	Singleton, Factory Method, Abstract Factory, Builder, Prototype
Structural	How classes and objects are composed	Adapter, Decorator, Proxy, Facade, Composite
Behavioural	How objects interact and share responsibility	Strategy, Observer, Template Method, Chain of Responsibility, Command, Iterator, State

17.4 Creational Patterns

Singleton: exactly one instance

Use it for genuinely single shared resources: configuration, a cache, a registry. Overuse creates hidden global state that is hard to test.

```java
// 1. Eager: simple, thread-safe (created at class loading)
public final class ConfigEager {
    private static final ConfigEager INSTANCE = new ConfigEager();
    private ConfigEager() { }
    public static ConfigEager getInstance() { return INSTANCE; }
}

// 2. Lazy with synchronized method: thread-safe but locks on EVERY call
public final class ConfigSync {
    private static ConfigSync instance;
    private ConfigSync() { }
    public static synchronized ConfigSync getInstance() {
        if (instance == null) instance = new ConfigSync();
        return instance;
    }
}

// 3. Double-checked locking: lazy + fast; 'volatile' is REQUIRED
public final class ConfigDcl {
    private static volatile ConfigDcl instance;
    private ConfigDcl() { }
    public static ConfigDcl getInstance() {
        if (instance == null) {                          // 1st check: no lock
            synchronized (ConfigDcl.class) {
                if (instance == null) {                  // 2nd check: with lock
                    instance = new ConfigDcl();
                }
            }
        }
        return instance;
    }
}

// 4. Bill Pugh holder idiom: lazy, thread-safe, no locking (JVM class-init guarantees)
public final class ConfigHolder {
    private ConfigHolder() { }
    private static class Holder { static final ConfigHolder INSTANCE = new ConfigHolder(); }
    public static ConfigHolder getInstance() { return Holder.INSTANCE; }
}

// 5. Enum: simplest; safe against reflection and serialization (Phase 10)
public enum AppConfig { INSTANCE; }
```

Way to break a singleton	Defence
Reflection (`setAccessible` on a private constructor)	Throw from the constructor if an instance exists, or use an **enum**
Serialization (deserializing creates a new object)	Implement `readResolve()` returning `INSTANCE`, or use an enum
Cloning	Don't implement `Cloneable`, or throw `CloneNotSupportedException`
Multiple class loaders	One instance **per class loader** (relevant in app servers)

PITFALL

Without `volatile` in double-checked locking, another thread may see a **partially constructed** object, because instruction reordering can publish the reference before the constructor finishes.

Factory Method / Simple Factory: hide which class gets created

```
interface PaymentProcessor { String process(double amt); }
class UpiProcessor implements PaymentProcessor {
    public String process(double a) { return "UPI " + a; }
}
class CardProcessor implements PaymentProcessor {
    public String process(double a) { return "CARD " + a; }
}
class NetBankingProcessor implements PaymentProcessor {
    public String process(double a) { return "NB " + a; }
}

final class PaymentProcessorFactory {
    private PaymentProcessorFactory() { }
    static PaymentProcessor of(String mode) {
        return switch (mode.toUpperCase()) {
            case "UPI"  -> new UpiProcessor();
            case "CARD" -> new CardProcessor();
            case "NB"   -> new NetBankingProcessor();
            default -> throw new IllegalArgumentException("Unsupported mode: " + mode);
        };
    }
}

PaymentProcessor p = PaymentProcessorFactory.of("upi");   // no 'new UpiProcessor' here
System.out.println(p.process(499));                       // UPI 499.0
```

Callers depend only on the interface, and creation logic lives in one place. JDK examples: `List.of()`, `Calendar.getInstance()`, `NumberFormat.getCurrencyInstance()`, `Executors.newFixedThreadPool()`.

Abstract Factory creates **families** of related objects. For example a `UiFactory` with `AndroidUiFactory`/`IosUiFactory`, each producing a matching `Button` and `TextField`.

Builder: readable construction of complex objects

Use it when a class has **many parameters**, especially optional ones (avoiding "telescoping constructors"). Phase 10 shows a full implementation with a static nested `Builder`.

```
HttpRequest req = HttpRequest.newBuilder()                 // JDK builder (Phase 18)
        .uri(URI.create("https://api.bank.example/v1/accounts"))
        .header("Accept", "application/json")
        .timeout(Duration.ofSeconds(5))
        .GET()
        .build();
```

Other builders: `StringBuilder`, `Stream.builder()`, `Locale.Builder`, and Lombok's `@Builder`.

Prototype

Create new objects by **copying an existing one** (`clone()` or a copy constructor; Phase 8). This is useful when construction is expensive, such as a pre-configured template object.

17.5 Structural Patterns

Adapter: make incompatible interfaces work together

```
interface PaymentGateway { String charge(String acct, double amt); }    // what our code expects

class LegacySoapBankClient {                        // third-party, can't change
    int debitAmountInPaise(String accountNumber, long paise) { return 0; }   // 0 = OK
}

class LegacyBankAdapter implements PaymentGateway {                    // the adapter
    private final LegacySoapBankClient client = new LegacySoapBankClient();
    @Override public String charge(String acct, double amt) {
        int code = client.debitAmountInPaise(acct, Math.round(amt * 100));   // translate
        return code == 0 ? "SUCCESS" : "FAILED:" + code;
    }
}
```

JDK examples: `Arrays.asList()` (array → List), `InputStreamReader` (bytes → chars).

Decorator: add behaviour by wrapping, at runtime

```
interface Notifier { void send(String msg); }
class EmailNotifier implements Notifier {
    public void send(String m) { System.out.println("EMAIL: " + m); }
}

abstract class NotifierDecorator implements Notifier {
    protected final Notifier inner;
    NotifierDecorator(Notifier inner) { this.inner = inner; }
}
class SmsDecorator extends NotifierDecorator {
    SmsDecorator(Notifier n) { super(n); }
    public void send(String m) { inner.send(m); System.out.println("SMS: " + m); }
}
class MaskingDecorator extends NotifierDecorator {
    MaskingDecorator(Notifier n) { super(n); }
    public void send(String m) {
        inner.send(m.replaceAll("\\d(?=\\d{4})", "*"));                 // mask digits
    }
}

Notifier n = new MaskingDecorator(new SmsDecorator(new EmailNotifier()));
n.send("Debited from card 4111222233334444");
// EMAIL: Debited from card ************4444
// SMS: Debited from card ************4444
```

JDK example: the whole `java.io` stack, such as `new BufferedReader(new InputStreamReader(in))` (Phase 12), and `Collections.unmodifiableList(list)`.

Proxy: control access to an object

The proxy has the **same interface** as the real object and adds caching, lazy loading, logging, security checks or remote access. Phase 16 shows a JDK dynamic proxy; Spring's `@Transactional` and `@Cacheable` and Hibernate lazy loading are proxies.

```
class CachedRateProxy implements RateService {
    private final RateService real;
    private final Map<String, Double> cache = new ConcurrentHashMap<>();
    CachedRateProxy(RateService real) { this.real = real; }
    public double rate(String currency) {
        return cache.computeIfAbsent(currency, real::rate);   // hit the slow service once
    }
}
```

Decorator vs Proxy	
Decorator	**Adds** responsibilities; often stacked; the client chooses the combination
Proxy	**Controls access** (caching, security, laziness); usually one; often created by a framework

Facade: one simple entry point to a complex subsystem

```
class FundTransferFacade {
    private final AccountService accounts; private final FraudCheck fraud;
    private final LedgerService ledger;    private final NotificationService notify;
    // constructor omitted
    public String transfer(String from, String to, BigDecimal amt) {
        fraud.verify(from, amt);
        accounts.debit(from, amt);
        accounts.credit(to, amt);
        String ref = ledger.record(from, to, amt);
        notify.send(from, "Transferred " + amt + " ref " + ref);
        return ref;                          // callers make ONE call instead of five
    }
}
```

Composite treats individual objects and groups uniformly through a tree (a `Portfolio` contains `Holding`s and sub-`Portfolio`s, all with `value()`).

17.6 Behavioural Patterns

Strategy: interchangeable algorithms

```
interface FeeStrategy { BigDecimal fee(BigDecimal amount); }

class Payment {
    private final FeeStrategy feeStrategy;
    Payment(FeeStrategy s) { this.feeStrategy = s; }
    BigDecimal total(BigDecimal amt) { return amt.add(feeStrategy.fee(amt)); }
}

FeeStrategy upi  = amt -> BigDecimal.ZERO;                 // lambdas as strategies
FeeStrategy neft = amt -> new BigDecimal("2.50");
FeeStrategy card = amt -> amt.multiply(new BigDecimal("0.018"))
                             .setScale(2, RoundingMode.HALF_UP);

new Payment(card).total(new BigDecimal("1000"));    // 1018.00
new Payment(upi).total(new BigDecimal("1000"));     // 1000
```

A Strategy replaces `if/else` or `switch` on a type with polymorphism. JDK examples: `Comparator` passed to `sort`, `ThreadPoolExecutor` rejection policies.

Observer: notify many listeners of an event

```
interface TxnListener { void onTransaction(String acct, BigDecimal amt); }

class TransactionPublisher {
    private final List<TxnListener> listeners = new CopyOnWriteArrayList<>();
    void subscribe(TxnListener l)   { listeners.add(l); }
    void unsubscribe(TxnListener l) { listeners.remove(l); }      // avoid listener leaks!
    void publish(String acct, BigDecimal amt) {
        listeners.forEach(l -> l.onTransaction(acct, amt));
    }
}

TransactionPublisher pub = new TransactionPublisher();
pub.subscribe((a, amt) -> System.out.println("SMS to " + a + ": debited " + amt));
pub.subscribe((a, amt) -> { if (amt.compareTo(new BigDecimal("50000")) > 0)
                              System.out.println("FRAUD CHECK for " + a); });
pub.publish("SB-1001", new BigDecimal("75000"));
// SMS to SB-1001: debited 75000
// FRAUD CHECK for SB-1001
```

Seen in GUI event listeners, Spring's `ApplicationEventPublisher`/`@EventListener`, Kafka consumers (distributed pub-sub), and the reactive `Flow` API.

Template Method: fixed skeleton, customisable steps

Phase 7's `Loan.emi()` (a `final` algorithm calling the abstract `annualRate()`) is a Template Method. Another example:

```
abstract class StatementJob {
    public final void run() {                // the template: order is fixed
        var data = fetch();
        var doc = format(data);
        deliver(doc);
    }
    protected abstract List<String> fetch();
    protected abstract String format(List<String> rows);
    protected void deliver(String doc) { System.out.println("Emailing statement"); }   // hook
}
```

JDK and frameworks: `AbstractList`, `HttpServlet.service()` → `doGet`/`doPost`, Spring's `JdbcTemplate`, JUnit's lifecycle.

Chain of Responsibility: pass a request along handlers

```
interface TxnValidator { void validate(Txn t); }        // each throws if invalid

List<TxnValidator> chain = List.of(
        t -> { if (!t.kycDone()) throw new IllegalStateException("KYC pending"); },
        t -> { if (t.amount() > 200_000) throw new IllegalStateException("Limit exceeded"); },
        t -> { if (t.isBlacklisted()) throw new IllegalStateException("Fraud risk"); });

chain.forEach(v -> v.validate(txn));    // stops at first failure; easy to add/reorder
```

Seen in servlet filters, Spring Security's filter chain, logging handlers, and exception propagation itself.

Command, Iterator, State (brief)

Pattern	Idea	Example
Command	Wrap a request as an object (queue, log, undo, retry)	`Runnable`/`Callable` submitted to an executor; scheduled jobs; undo stacks
Iterator	Traverse a collection without exposing its internals	`java.util.Iterator`, for-each
State	An object changes behaviour when its internal state changes	Loan status `APPLIED → APPROVED → DISBURSED → CLOSED` modelled as an enum/classes with allowed transitions

17.7 Patterns in the JDK and Frameworks

Pattern	JDK / framework examples
Singleton	`Runtime.getRuntime()`, Spring beans (singleton scope by default)
Factory	`List.of`, `Calendar.getInstance`, `NumberFormat.getInstance`, `Executors.*`, Spring `BeanFactory`
Builder	`StringBuilder`, `HttpRequest.newBuilder()`, `Stream.builder()`, Lombok `@Builder`
Prototype	`Object.clone()`, Spring prototype scope
Adapter	`Arrays.asList`, `InputStreamReader`, Spring MVC `HandlerAdapter`
Decorator	`BufferedInputStream`, `Collections.synchronizedList`/`unmodifiableList`
Proxy	`java.lang.reflect.Proxy`, Spring AOP, Hibernate lazy entities, Mockito mocks
Facade	`javax.faces.context.FacesContext`, Spring `JdbcTemplate` over raw JDBC
Strategy	`Comparator`, `RejectedExecutionHandler`, Spring `Resource` loaders
Observer	`java.util.EventListener`, `Flow.Subscriber`, Spring events
Template Method	`AbstractList`, `HttpServlet`, `JdbcTemplate`, `RestTemplate` callbacks
Chain of Responsibility	Servlet `FilterChain`, Spring Security filters, `java.util.logging` handlers
Iterator	`Iterator`, `Spliterator`

17.8 Patterns in Test Automation (Appium / Selenium)

Pattern	How it appears in a test framework
Page Object Model	One class per screen, holding locators and actions (`LoginPage.login(user, pwd)`); tests read like business steps
Factory	`DriverFactory.create(platform)` returns an `AndroidDriver`, `IOSDriver` or `ChromeDriver` from config
Singleton + ThreadLocal	`DriverManager` holds one driver **per thread** for parallel runs: `ThreadLocal<AppiumDriver>` (call `remove()` after each test)
Builder	Fluent test data: `new CustomerBuilder().withKyc(true).withBalance(5000).build()`
Strategy	Platform-specific gestures or locator strategies chosen at runtime
Decorator / Proxy	`@FindBy` element proxies (PageFactory); wrapping a driver to add logging and screenshots
Facade	`CheckoutFlow` composes several page objects into one business step
Observer	TestNG/JUnit listeners (`ITestListener`, `TestWatcher`) reacting to pass/fail events for reporting

```
public final class DriverManager {
    private static final ThreadLocal<AppiumDriver> DRIVER = new ThreadLocal<>();
    private DriverManager() { }
    public static AppiumDriver get() { return DRIVER.get(); }
    public static void set(AppiumDriver d) { DRIVER.set(d); }
    public static void quit() {
        AppiumDriver d = DRIVER.get();
        if (d != null) { d.quit(); DRIVER.remove(); }   // prevent leaks across pooled threads
    }
}
```

17.9 Interview Questions

Q. Explain SOLID with examples.

S: one reason to change (split transfer logic from persistence and notifications). O: extend through new classes rather than editing switch statements (an InterestPolicy per account type). L: subclasses must honour the parent's contract (an FD shouldn't inherit `withdraw`). I: small role interfaces (CashDispenser vs PassbookPrinter). D: depend on abstractions and inject implementations (a PaymentGateway interface plus constructor injection).

Q. What is the difference between Dependency Inversion, Dependency Injection and IoC?

DIP is the design principle (depend on abstractions). DI is the technique of supplying dependencies from outside (constructor or setter). IoC is the broader idea that a framework controls object creation and the flow (Spring's container).

Q. How do you implement a thread-safe singleton? Which is best?

Eager initialisation, a synchronized accessor, double-checked locking with `volatile`, the Bill Pugh holder idiom, or an enum. An enum is simplest and safe against reflection and serialization; the holder idiom is best for lazy non-enum singletons.

Q. Why is `volatile` needed in double-checked locking?

To prevent reordering that could publish a reference to a not-yet-fully-constructed object to other threads, and to guarantee visibility of the write.

Q. What is the difference between Factory and Builder?

Factory decides **which** class to instantiate, usually in one call. Builder constructs **one complex object step by step**, with many optional parts.

Q. What is the difference between Adapter, Decorator, Proxy and Facade?

Adapter converts one interface to another. Decorator adds behaviour while keeping the interface (stackable). Proxy keeps the interface but controls access (caching, laziness, security). Facade provides a simpler interface over a whole subsystem.

Q. What is the difference between Strategy and Template Method?

Strategy uses **composition**: a whole algorithm is swapped at runtime through an interface or lambda. Template Method uses **inheritance**: a fixed algorithm skeleton in a base class with steps overridden by subclasses.

Q. Where is the Observer pattern used, and what is its risk?

In event listeners, Spring events and pub-sub messaging. The main risk is memory leaks from listeners that are never unsubscribed, plus ordering or exception issues in synchronous notification.

Q. Which design patterns do you use in your automation framework?

Page Object Model, a Factory for drivers, a ThreadLocal-based singleton DriverManager for parallel runs, Builders for test data, Strategy for platform differences, listeners (Observer) for reporting, and PageFactory element proxies.

17.10 Quick Revision

- **SOLID**: SRP (one reason to change), OCP (extend, don't modify), LSP (substitutable subtypes), ISP (small interfaces), DIP (depend on abstractions; inject).
- DRY, KISS, YAGNI, composition over inheritance, program to interfaces, Law of Demeter.
- **Creational**: Singleton (eager / synchronized / DCL + volatile / holder / **enum**), Factory (which class), Abstract Factory (families), Builder (step by step), Prototype (copy).
- Singleton breakers: reflection, serialization (`readResolve`), cloning, multiple class loaders.
- **Structural**: Adapter (convert), Decorator (add by wrapping; java.io), Proxy (control access; Spring AOP), Facade (simplify), Composite (trees).
- **Behavioural**: Strategy (swap algorithm; lambdas), Observer (publish/subscribe), Template Method (final skeleton + hooks), Chain of Responsibility (validators/filters), Command (requests as objects), Iterator, State.
- Automation: POM, DriverFactory, ThreadLocal DriverManager, test-data Builders, listeners.

PHASE 18 OF 18

HTTP Client, Networking & Internationalization

Networking basics, URI/URL and encoding, TCP sockets, the Java 11 HttpClient (sync, async, JSON, timeouts, retries), a local HTTP server for API mocks, and i18n with Locale, NumberFormat (₹ and lakh grouping), localized dates, MessageFormat, ResourceBundle and Collator

IN THIS PHASE

18.1 Networking Basics
18.2 TCP Sockets (java.net)
18.3 The Java 11 HttpClient (java.net.http)
18.4 A Local HTTP Server for Mocks and Tests
18.5 Internationalization (i18n) and Localization (l10n)
18.6 Interview Questions
18.7 Quick Revision

18.1 Networking Basics

Term	Meaning
IP address	Identifies a machine on a network (`192.168.1.10`, IPv6 `::1`); `localhost` = `127.0.0.1`
Port	Identifies a process on that machine (0–65535): HTTP 80, HTTPS 443, Appium 4723, PostgreSQL 5432
Socket	One endpoint of a two-way connection: IP + port
TCP	Connection-oriented, reliable, ordered (HTTP, DB connections)
UDP	Connectionless, fast, no delivery guarantee (DNS, streaming)
HTTP / HTTPS	Request–response protocol over TCP (HTTPS = HTTP + TLS encryption)

URL anatomy, URI vs URL

```
https://api.bank.example:8443/v1/accounts/SB-1001?type=savings&page=2#top
\___/   \_____________/ \__/\____________________/\_________________/\__/
scheme        host       port        path               query      fragment
```

```
URI u = URI.create("https://api.bank.example:8443/v1/accounts/SB-1001?type=savings&page=2#top");
u.getScheme();   // https          u.getHost();   // api.bank.example
u.getPort();     // 8443           u.getPath();   // /v1/accounts/SB-1001
u.getQuery();    // type=savings&page=2            u.getFragment(); // top
URI.create("https://a.com/x/y/").resolve("../z?q=1");   // https://a.com/x/z?q=1
URL url = u.toURL();              // URL constructors are deprecated (Java 20): go via URI
```

A **URI** identifies a resource (syntax only, no network access). A **URL** is a URI that also says how to locate it. In modern Java, build a `URI`, and convert with `toURL()` only for legacy APIs.

URL encoding

Query parameters must be **percent-encoded**: spaces, `&`, `=`, `/` and non-ASCII characters would otherwise break the URL.

```
String q = URLEncoder.encode("name=Rohit Kumar&city=Pune/MH", StandardCharsets.UTF_8);
// name%3DRohit+Kumar%26city%3DPune%2FMH
URLDecoder.decode("Rohit+Kumar%26co", StandardCharsets.UTF_8);    // Rohit Kumar&co

InetAddress addr = InetAddress.getByName("localhost");            // DNS lookup
System.out.println(addr.getHostAddress());                        // 127.0.0.1
```

18.2 TCP Sockets (java.net)

Raw sockets are the foundation under HTTP, JDBC drivers and messaging clients. You'll rarely write them yourself, but understanding them helps with debugging (timeouts, connection refused, ports).

```
// SERVER: listens on a port, echoes one line back in upper case
try (ServerSocket server = new ServerSocket(5050)) {
    try (Socket client = server.accept();                // blocks until a client connects
         BufferedReader in = new BufferedReader(
                 new InputStreamReader(client.getInputStream(), UTF_8));
         PrintWriter out = new PrintWriter(client.getOutputStream(), true, UTF_8)) {
        out.println("ECHO: " + in.readLine().toUpperCase());
    }
}

// CLIENT
try (Socket socket = new Socket()) {
    socket.connect(new InetSocketAddress("localhost", 5050), 2000);    // connect timeout 2 s
    socket.setSoTimeout(3000);                                          // read timeout 3 s
    PrintWriter out = new PrintWriter(socket.getOutputStream(), true, UTF_8);
    BufferedReader in = new BufferedReader(
            new InputStreamReader(socket.getInputStream(), UTF_8));
    out.println("balance enquiry");
    System.out.println(in.readLine());                                  // ECHO: BALANCE ENQUIRY
}
```

Exception	Usual meaning
`ConnectException: Connection refused`	Nothing is listening on that host and port (service down, wrong port)
`SocketTimeoutException`	Connect or read took longer than the timeout
`UnknownHostException`	DNS couldn't resolve the host name
`SSLHandshakeException`	TLS problem: untrusted certificate, wrong protocol, missing CA in the truststore
`BindException: Address already in use`	The server port is already taken

> **TIP**
> **Always set timeouts.** By default, a socket read can block forever, and a hung downstream service then freezes your threads. This is one of the most common production incidents.

18.3 The Java 11 HttpClient (java.net.http)

The modern, built-in HTTP client replaces the old `HttpURLConnection`. It supports HTTP/1.1 and **HTTP/2**, **synchronous and asynchronous** calls (`CompletableFuture`), builders, timeouts, redirects, authentication, and WebSockets.

```
HttpClient (reusable, thread-safe: create ONCE)
   +- send(request, bodyHandler)       -> HttpResponse<T>                    (blocking)
   +- sendAsync(request, bodyHandler)  -> CompletableFuture<HttpResponse<T>>  (async)
```

Creating the client

```
HttpClient client = HttpClient.newBuilder()
        .version(HttpClient.Version.HTTP_2)              // falls back to HTTP/1.1 if needed
        .connectTimeout(Duration.ofSeconds(5))
        .followRedirects(HttpClient.Redirect.NORMAL)     // follow 3xx (except HTTPS -> HTTP)
        .build();                                        // reuse this instance everywhere
```

GET with query parameters and headers

```
String accountId = "SB-1001";
String type = URLEncoder.encode("savings & current", StandardCharsets.UTF_8);
HttpRequest get = HttpRequest.newBuilder()
        .uri(URI.create(BASE + "/v1/accounts/" + accountId + "?type=" + type))
        .header("Accept", "application/json")
        .header("Authorization", "Bearer " + token)      // never log this header!
        .timeout(Duration.ofSeconds(10))                 // per-request response timeout
        .GET()                                           // the default method
        .build();

HttpResponse<String> res = client.send(get, HttpResponse.BodyHandlers.ofString());
System.out.println(res.statusCode());                    // 200
System.out.println(res.headers().firstValue("content-type").orElse("?"));
System.out.println(res.body());                          // {"id":"SB-1001","balance":2500.50}
```

POST, PUT and DELETE with a JSON body

```
String json = """
        {"from":"SB-1001","to":"SB-2002","amount":1500.00,"currency":"INR"}
        """;
HttpRequest post = HttpRequest.newBuilder()
        .uri(URI.create(BASE + "/v1/transfers"))
        .header("Content-Type", "application/json")
        .header("Idempotency-Key", UUID.randomUUID().toString())   // safe retries for payments
        .POST(HttpRequest.BodyPublishers.ofString(json))
        .build();
HttpResponse<String> created = client.send(post, HttpResponse.BodyHandlers.ofString());
if (created.statusCode() != 201) {
    throw new IllegalStateException(
            "Transfer failed: " + created.statusCode() + " " + created.body());
}

HttpRequest put = HttpRequest.newBuilder(URI.create(BASE + "/v1/accounts/SB-1001/nickname"))
        .PUT(HttpRequest.BodyPublishers.ofString("{\"nickname\":\"Salary\"}"))
        .header("Content-Type", "application/json").build();
HttpRequest del = HttpRequest.newBuilder(URI.create(BASE + "/v1/beneficiaries/77"))
        .DELETE().build();
HttpRequest patch = HttpRequest.newBuilder(URI.create(BASE + "/v1/cards/9"))
        .method("PATCH", HttpRequest.BodyPublishers.ofString("{\"blocked\":true}")).build();
```

`BodyPublishers` **(request body)**	`BodyHandlers` **(response body)**
`ofString(s)` / `ofString(s, charset)`	`ofString()`: the whole body as a String
`ofFile(path)`	`ofFile(path)`: stream to a file (downloads)
`ofByteArray(bytes)`	`ofByteArray()`, `ofInputStream()`
`ofInputStream(supplier)`	`ofLines()`: `Stream<String>` (large text responses)
`noBody()`	`discarding()`: ignore the body

> **NOTE**
> `HttpClient` does **not** throw for 4xx or 5xx statuses. A 404 or 500 is a normal `HttpResponse`, so **always check** `statusCode()`. Exceptions are thrown only for transport problems: `HttpConnectTimeoutException`, `HttpTimeoutException`, `ConnectException`, `IOException`, and `InterruptedException` for `send`.

Asynchronous and parallel calls

```
CompletableFuture<String> balance = client
        .sendAsync(get, HttpResponse.BodyHandlers.ofString())
        .thenApply(HttpResponse::body);                          // no thread blocked meanwhile

List<URI> uris = List.of(URI.create(BASE + "/v1/accounts/SB-1"),
                         URI.create(BASE + "/v1/accounts/SB-2"),
                         URI.create(BASE + "/v1/accounts/SB-3"));
List<CompletableFuture<String>> calls = uris.stream()
        .map(u -> client.sendAsync(HttpRequest.newBuilder(u).build(),
                                   HttpResponse.BodyHandlers.ofString())
                        .thenApply(HttpResponse::body))
        .toList();
CompletableFuture.allOf(calls.toArray(CompletableFuture[]::new)).join();   // all 3 in parallel
calls.forEach(c -> System.out.println(c.join()));
```

A simple retry with backoff (for idempotent or keyed requests)

```
static HttpResponse<String> sendWithRetry(HttpClient client, HttpRequest req, int maxAttempts)
        throws IOException, InterruptedException {
    IOException last = null;
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        try {
            HttpResponse<String> r = client.send(req, HttpResponse.BodyHandlers.ofString());
            if (r.statusCode() < 500 && r.statusCode() != 429) return r;   // done
        } catch (IOException e) {
            last = e;                                          // network glitch: retry
        }
        Thread.sleep(200L * (1L << (attempt - 1)));                        // 200, 400, 800 ms
    }
    throw new IOException("Gave up after " + maxAttempts + " attempts", last);
}
```

> **PITFALL**
> Only retry **idempotent** operations (GET, PUT, DELETE), or POSTs protected by an **idempotency key**. Blindly retrying a payment POST after a timeout can debit the customer twice.

JSON mapping

The JDK has no JSON parser. In practice, combine `HttpClient` with **Jackson** (or Gson) and records:

```
record AccountDto(String id, BigDecimal balance) { }
ObjectMapper mapper = new ObjectMapper();                          // com.fasterxml.jackson
AccountDto acc = mapper.readValue(res.body(), AccountDto.class);
String body = mapper.writeValueAsString(new AccountDto("SB-9", new BigDecimal("10.00")));
```

> **TIP**
> For API **test suites**, REST Assured (built on the same ideas) gives fluent given/when/then assertions. The plain `HttpClient` is ideal for utilities, health checks, test-data setup and Appium-adjacent backend calls without extra dependencies.

HttpClient vs HttpURLConnection

`HttpURLConnection` **(Java 1.1)**	`HttpClient` **(Java 11)**
Clunky, mutable API; manual streams	Fluent builders; immutable requests
HTTP/1.1 only	HTTP/2 and HTTP/1.1, WebSockets
Blocking only	Synchronous **and** asynchronous (`CompletableFuture`)
Easy to leak connections	Connection pooling built in

18.4 A Local HTTP Server for Mocks and Tests

The JDK ships a lightweight server (`com.sun.net.httpserver`, module `jdk.httpserver`). It's handy for **stubbing a downstream API** in tests or demos. For full-featured mocking, teams use WireMock or MockWebServer.

```
HttpServer server = HttpServer.create(new InetSocketAddress(8089), 0);
server.createContext("/v1/accounts/SB-1001", exchange -> {
    byte[] body = "{\"id\":\"SB-1001\",\"balance\":2500.50}".getBytes(UTF_8);
    exchange.getResponseHeaders().add("Content-Type", "application/json");
    exchange.sendResponseHeaders(200, body.length);
    try (OutputStream os = exchange.getResponseBody()) { os.write(body); }
});
server.createContext("/v1/transfers", exchange -> {
    String req = new String(exchange.getRequestBody().readAllBytes(), UTF_8);
    int status = req.contains("\"amount\":0") ? 422 : 201;
    exchange.sendResponseHeaders(status, -1);                    // -1 = no body
    exchange.close();
});
server.setExecutor(Executors.newVirtualThreadPerTaskExecutor()); // Java 21
server.start();
// ... run client calls against http://localhost:8089 ...
server.stop(0);
```

```
> jwebserver -p 8000 -d C:\reports      (Java 18+) serve static files, e.g. test reports
```

18.5 Internationalization (i18n) and Localization (l10n)

i18n means designing software so that it can adapt to languages and regions. **l10n** means providing the actual translations and formats for a locale. In banking apps this shows up in currency formats, dates, number separators and multi-language messages (English, Hindi, Marathi...).

Locale

```
Locale enIN = Locale.of("en", "IN");             // Java 19+ (new Locale(...) is deprecated)
Locale hiIN = Locale.of("hi", "IN");
Locale tag  = Locale.forLanguageTag("mr-IN");    // BCP 47 tag: Marathi (India)
Locale.US;  Locale.GERMANY;  Locale.JAPAN;       // constants
Locale.getDefault();                             // JVM default (OS or -Duser.language)
enIN.getDisplayCountry();                        // India
hiIN.getDisplayLanguage(hiIN);                   // [Hindi] in Devanagari script
```

PITFALL

Calls without a locale (`toUpperCase()`, `String.format`, `NumberFormat.getInstance()`) use the **JVM default**, which differs between your laptop, the CI server and production. The classic bug: `"TITLE".toLowerCase()` in a Turkish locale yields a dotless `ı`. Pass `Locale.ROOT` for machine-readable text, and an explicit locale for user-facing text.

Numbers, currency and percentages (verified on JDK 21)

```
double amt = 1234567.891;
NumberFormat.getCurrencyInstance(Locale.of("en", "IN")).format(amt);  // ₹1,234,567.89
NumberFormat.getCurrencyInstance(Locale.US).format(amt);              // $1,234,567.89
NumberFormat.getCurrencyInstance(Locale.GERMANY).format(amt);         // 1.234.567,89 €
NumberFormat.getCurrencyInstance(Locale.JAPAN).format(amt);           // ¥1,234,568 (no decimals)
NumberFormat.getNumberInstance(Locale.GERMANY).format(amt);           // 1.234.567,891

NumberFormat pct = NumberFormat.getPercentInstance(Locale.of("en", "IN"));
pct.setMinimumFractionDigits(2);
pct.format(0.0725);                                       // 7.25% (default rounds: 7%)

NumberFormat compact = NumberFormat.getCompactNumberInstance(
        Locale.of("en", "IN"), NumberFormat.Style.SHORT);             // Java 12
compact.setMaximumFractionDigits(1);
compact.format(150_000);                                              // 1.5L
compact.format(12_500_000);                                           // 1.2Cr

Currency inr = Currency.getInstance(Locale.of("en", "IN"));
inr.getCurrencyCode();  inr.getDefaultFractionDigits();               // INR, 2

NumberFormat.getNumberInstance(Locale.GERMANY).parse("1.234,5");    // 1234.5 (parsing too)
```

PITFALL

A common assumption is that the `en-IN` locale, or the pattern `#,##,##0.00`, produces Indian grouping (`12,34,567.89`). On the standard JDK it **doesn't**: `DecimalFormat` supports only one grouping size, so both give `1,234,567.89`. Use ICU4J (`com.ibm.icu.text.NumberFormat`), or a small helper like the one below, when statements must show lakh/crore grouping.

```
static String indianFormat(BigDecimal amount) {             // 1234567.891 -> ₹12,34,567.89
    String plain = amount.setScale(2, RoundingMode.HALF_UP).abs().toPlainString();
    String intPart = plain.substring(0, plain.indexOf('.'));
    String decPart = plain.substring(plain.indexOf('.'));
    String last3 = intPart.length() > 3 ? intPart.substring(intPart.length() - 3) : intPart;
    String rest  = intPart.length() > 3 ? intPart.substring(0, intPart.length() - 3) : "";
    rest = rest.replaceAll("\\B(?=(\\d{2})+$)", ",");   // pairs before last three
    String sign = amount.signum() < 0 ? "-" : "";
    return sign + "₹" + (rest.isEmpty() ? "" : rest + ",") + last3 + decPart;
}
indianFormat(new BigDecimal("1234567.891"));   // ₹12,34,567.89
indianFormat(new BigDecimal("950.5"));         // ₹950.50
```

Localized dates

```
LocalDate d = LocalDate.of(2026, 8, 15);
DateTimeFormatter longFmt = DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG);
d.format(longFmt.withLocale(Locale.US));                      // August 15, 2026
d.format(longFmt.withLocale(Locale.of("en", "IN")));          // 15 August 2026
d.format(DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM)
        .withLocale(Locale.GERMANY));                         // 15.08.2026
d.format(DateTimeFormatter.ofPattern("EEEE, d MMMM yyyy", Locale.of("hi", "IN")));
                                      // [Shanivaar, 15 Agast 2026] in Devanagari
```

ResourceBundle: externalised, translated messages

NOTE

The PDF font can't draw Devanagari, so Hindi and Marathi examples are shown **transliterated in [brackets]**. In your `.properties` files, IDE and console they appear in the real script (verified on JDK 21 with UTF-8 output).

```
src/main/resources/
  messages.properties          greeting=Welcome, {0}!           (default / fallback)
  messages_hi.properties       greeting=[Swagat hai], {0}!
  messages_mr.properties       greeting=[Swagat aahe], {0}!
```

```
ResourceBundle rb = ResourceBundle.getBundle("messages", Locale.of("hi", "IN"));
String text = MessageFormat.format(rb.getString("greeting"), "Rohit");   // [Swagat hai], Rohit!
// Lookup order: messages_hi_IN -> messages_hi -> messages_<default locale> -> messages
```

> **NOTE**
> Since Java 9, `.properties` resource bundles are read as **UTF-8**, so Hindi and Marathi text can be written directly (no more `\u0938` escapes). Spring's `MessageSource` builds on the same idea.

MessageFormat: placeholders with formatting

```
String msg = MessageFormat.format(
        "Dear {0}, your account {1} was debited with {2,number,#,##0.00} on {3}.",
        "Rohit", "SB-1001", 2500.5, "15-08-2026");
// Dear Rohit, your account SB-1001 was debited with 2,500.50 on 15-08-2026.
```

> **PITFALL**
> In `MessageFormat`, a single quote starts a quoted (literal) section. `MessageFormat.format("Don't {0}", "x")` prints `Dont {0}`: the quote disappears **and** the placeholder isn't replaced. Write `"Don''t {0}"` to get `Don't x`.

Collator: locale-aware sorting

```
List<String> words = new ArrayList<>(List.of("zebra", "Apple", "éclair", "banana"));
Collections.sort(words);                              // [Apple, banana, zebra, éclair]
                                                      //   (by code point)
words.sort(Collator.getInstance(Locale.ENGLISH));  // [Apple, banana, éclair, zebra]
                                                      //   (human order)
```

18.6 Interview Questions

Q. What is the difference between TCP and UDP?

TCP is connection-oriented, reliable and ordered (HTTP, databases). UDP is connectionless and faster but has no delivery or order guarantees (DNS, media streaming).

Q. What is the difference between URI and URL in Java?

A URI is a syntactic identifier (parsing, resolving, encoding) with no network access. A URL can be used to open connections. Build URIs (`URI.create`) and convert with `toURL()` when needed; URL constructors are deprecated since Java 20.

Q. How do you make an HTTP call in modern Java?

Create one reusable `HttpClient` (with a connect timeout), build an `HttpRequest` (URI, headers, method, body publisher, timeout), then call `send` with a `BodyHandler` for a blocking call, or `sendAsync` to get a `CompletableFuture`.

Q. Does HttpClient throw an exception for a 404 or 500 response?

No. HTTP error statuses are normal responses, so check `statusCode()`. Exceptions only signal transport failures such as timeouts, refused connections or I/O errors.

Q. Why must you set timeouts, and which ones exist?

Without them, calls to a slow or hung service can block threads indefinitely. `HttpClient` has `connectTimeout` (on the builder) and a per-request `timeout`; sockets have a connect timeout and `setSoTimeout` (read timeout).

Q. When is it safe to retry an HTTP request?
For idempotent methods (GET, PUT, DELETE), or non-idempotent ones like payment POSTs protected by an idempotency key. Use limited attempts with exponential backoff, and don't retry 4xx client errors (except 429).

Q. What is the difference between i18n and l10n?
Internationalization designs software to support many locales (externalised strings, locale-aware formatting). Localization supplies the translations and regional formats for a specific locale.

Q. How do you format currency for a locale?
Use `NumberFormat.getCurrencyInstance(locale)`, or `BigDecimal` with an explicit scale for calculations. Note that the standard JDK doesn't apply Indian lakh grouping; use ICU4J or a custom formatter.

Q. What is a ResourceBundle?
A locale-specific set of key–value messages (usually `.properties` files such as `messages_hi.properties`), resolved with a fallback chain from the most specific to the default bundle.

Q. Why is relying on the default locale risky?
Output changes with machine settings, so formats, case conversion and parsing can break in CI or production. Pass explicit locales (`Locale.ROOT` for machine text).

18.7 Quick Revision

- TCP = reliable connection; UDP = fast datagrams. A socket = IP + port. Always set **connect and read timeouts**.
- Build `URI`s (not `new URL`); encode query values with `URLEncoder.encode(v, UTF_8)`.
- Socket errors: refused (nothing listening), timeout, unknown host, SSL handshake.
- `HttpClient`: create once (thread-safe), HTTP/2, `connectTimeout`; `HttpRequest.newBuilder().uri().header().timeout().GET()/POST(BodyPublishers.ofString(json))`.
- `send` (blocking) vs `sendAsync` (CompletableFuture); `BodyHandlers.ofString`/`ofFile`/`ofLines`.
- **4xx/5xx don't throw**: check `statusCode()`. Retry only idempotent or keyed requests, with backoff.
- JSON: Jackson + records. Local mocks: `com.sun.net.httpserver.HttpServer`; `jwebserver` for static files.
- Locale: `Locale.of("en","IN")`, `forLanguageTag`; avoid hidden default-locale behaviour.
- `NumberFormat`: currency, percent, compact (`1.5L`, `1.2Cr`); the JDK has **no lakh grouping**: use ICU4J or a helper.
- Dates: `DateTimeFormatter.ofLocalizedDate(FormatStyle.X).withLocale(l)`.
- `ResourceBundle` + `MessageFormat` for translated messages (UTF-8 `.properties` since Java 9); `Collator` for human sorting.

APPENDIX

Appendix: Leftovers, Corrections Log & Revision Plan

Small topics from the original notes not covered elsewhere, a consolidated log of every correction made to the original notes, and a suggested way to revise this book before interviews

IN THIS PHASE

A.1 Leftover Topics from the Original Notes
A.2 Corrections Log
A.3 How to Revise This Book

A.1 Leftover Topics from the Original Notes

Nested interfaces

An interface declared inside a class or another interface. It is implicitly `static`, and it groups a contract with its owner (like `Map.Entry`).

```
interface Bank {
    interface Rules { int MAX_TXN = 200_000; }      // nested interface: implicitly static
    void open();
}
class RbiPolicy implements Bank.Rules { }          // implement as Outer.Inner

class Card {
    public interface Validator { boolean ok(String number); }
}
Card.Validator v = n -> n.length() == 16;          // functional, so a lambda works
System.out.println(Bank.Rules.MAX_TXN);            // 200000
```

SequenceInputStream: read several streams as one

```
try (SequenceInputStream sis = new SequenceInputStream(
        new FileInputStream("header.txt"), new FileInputStream("body.txt"))) {
    System.out.print(new String(sis.readAllBytes()));   // header content, then body content
}
```

It is useful for concatenating file parts. With NIO.2 you would more often write the parts to one `OutputStream`, or use `Files.write` with `StandardOpenOption.APPEND`.

FileDescriptor: the OS-level handle

A `FileDescriptor` is **not** a file or a stream. It is the low-level handle the operating system uses for an open file, socket or device.

Member	Meaning
`FileDescriptor.in` / `out` / `err`	Standard input, output and error (what `System.in/out/err` wrap)
`stream.getFD()`	Handle of an open `FileInputStream`/`FileOutputStream`/`RandomAccessFile`
`fd.sync()`	Force buffered OS data to physical storage (durability for critical writes)
`fd.valid()`	Whether the handle is still open

```
try (FileOutputStream fos = new FileOutputStream("ledger.dat")) {
    fos.write(bytes);
    fos.getFD().sync();        // make sure the ledger entry really hit the disk
}
```

FilePermission

`java.io.FilePermission` represents access rights (`read`, `write`, `execute`, `delete`, `readlink`) on a path. A path ending in `-` means "recursively everything below"; `*` means "files directly in the folder".

```
FilePermission rw = new FilePermission("/data/-", "read,write");
rw.implies(new FilePermission("/data/reports/a.txt", "read"));   // true
rw.implies(new FilePermission("/etc/passwd", "read"));           // false
```

> **NOTE**
> Permissions were enforced by the `SecurityManager`, which is **deprecated for removal since Java 17** (JEP 411) and permanently disabled in Java 24. Today, file access is controlled by OS permissions, containers and application-level checks, so `FilePermission` is mostly of historical interest.

URL-related exceptions

Exception	Cause	Example
`URISyntaxException` (checked)	Invalid URI syntax	`new URI("http://bad host")` (space in host)
`MalformedURLException` (checked, an `IOException`)	Unknown protocol or bad URL form	`URI.create("http://abc").toURL()` → "unknown protocol: http"
`IllegalArgumentException`	`URI.create` with bad syntax, or `toURL()` on a relative URI	`URI.create("relative/path").toURL()` → "URI is not absolute"

A.2 Corrections Log

Every statement from the two original note files that was corrected in this book, in one place, so you don't carry a wrong idea into an interview.

Phase	Original notes said	Correct statement
1	Bytecode is secure because viruses attack object code, not bytecode	Security comes from the bytecode verifier, no pointer arithmetic, class-loader isolation and runtime checks
1	Distributed means code need not run sequentially	That describes multithreading. Distributed means built-in networking support.
1	Java is an interpreted language	Java is compiled (to bytecode) **and** interpreted/JIT-compiled at runtime
2	`int MIN_AGE = 21;` is a constant	Only `final` makes a constant; capital letters are just a convention
2	float/double hold 6/15 "decimal places"	About 6–7 / 15–16 **significant digits** in total
2	`boolean` occupies 1 bit	Its size is JVM-dependent (usually 1 byte as a field)
2	`char` stores an ASCII value	It stores a UTF-16 code unit (Unicode); ASCII is just the first 128 values
2	53 keywords, including `null`	50 keywords + _; `true`, `false`, `null` are **literals**
2	`.` is a relational operator	`.` is the member-access operator
3	`switch (2.1f)` with float cases works	float, double, long and boolean are never allowed in a switch

Phase	Original notes said	Correct statement
3	`switch ('a')` matches `case 1`	`'a'` is 97, so nothing matches
4	"Why Strings are mutable"	Strings are **immutable**; reassigning a variable doesn't change the shared object
4	Pattern and Matcher method descriptions	They were swapped. `compile`/`matcher`/`split`/`quote` belong to Pattern; `find`/`matches`/`group` to Matcher.
5	`hashCode()` is a unique id or memory address	Not unique, and not an address: just a number for hash-based collections
5	`return` must be the last statement	Several returns are fine; only code after a return in the same block is unreachable
6	`strictfp` gives platform-independent results	Obsolete since Java 17: all floating point is strict by default
6	No multiple inheritance makes Java "partial OOP"	Primitives and static members are why Java isn't *purely* OOP
6	Removing `super()` breaks compilation	The compiler inserts `super()`; it fails only if the parent lacks a no-arg constructor
7	Final classes are immutable	`final` only prevents subclassing; immutability needs private final fields, no setters and defensive copies
7	Since Java 8 there is no need for abstract classes	Interfaces still can't hold instance state or constructors; abstract classes remain useful
8	`equals()` with a direct cast	Check `this == o` and `instanceof` (type + null) before casting
8	Wrappers via `new Integer(10)` etc.	Those constructors are deprecated for removal; use `valueOf` or autoboxing
8	Integer implements `Comparator<Integer>`	It implements `Comparable<Integer>`
9	OOM/StackOverflow are siblings of VirtualMachineError	They are its **subclasses**; FileNotFoundException is a subclass of IOException
9	Errors cannot be handled	They can be caught but usually shouldn't be
9	`throws` lets us ignore checked exceptions	It delegates them to the caller; nothing is ignored
9	finally always executes	Except on `System.exit`, a JVM crash/kill, or a try that never finishes

A.3 How to Revise This Book

When	What to do
First pass (1–2 weeks)	Read one phase per day; type out and run every code block in JShell or your IDE; note anything surprising
Daily (15 minutes)	Read one phase's **Quick Revision** and **Predict the Output** sections without looking at the answers
Weekly	Re-answer every **Interview Questions** block aloud, in under 60 seconds each, as you would in an interview
Before an interview	Phase 15's rapid-fire table, this corrections log, the Big-O tables (Phase 14), the `equals`/`hashCode` and HashMap internals (Phases 8 and 14), and the concurrency basics (Phase 11)
To go deeper	Spring Boot (DI, REST, JPA), testing (JUnit 5, Mockito, REST Assured), SQL and JDBC, and system design for backend services

TIP

A good habit for every topic: **explain it, show it in code, state the pitfall.** For example: "HashMap uses hashCode to pick a bucket and equals to find the key; here's an immutable key class; mutating a key after insertion makes the entry unreachable."